Bachelor of Science in Information Technology

DATABASE SYSTEMS

Designing, Querying and Trusting a Relational Database

Unified Lecture Handout

Complete Course in One Volume • Fall 2026

A database is not a place to put data. It is a set of promises about data.

This course is how each promise is made — and, every time, what the database does when you do not ask it to keep one.

What this volume covers. Every clause of the HEC outline for *Database Systems*, traced chapter by chapter in the Course Specification overleaf.

That outline is published in the **2017** curriculum (p. 88) and reprinted *word for word* in the **2023** curriculum (p. 58) — the two differ by a single added full stop across ninety words, six years apart. The **2025** curriculum deletes it: it covers only BS Computer Science, publishes no course content for any major course at all, and supplies five rewritten learning outcomes with no Bloom levels. All of that is recorded in the Course Specification rather than smoothed over.

The course is taught on a **running database**. One schema is designed in Chapter 6, created in Chapter 10, and every chapter afterwards adds to it or asks something of it; by Chapter 23 it is the semester project. The tool is **SQLite**, which you already have.

Six Parts • Twenty-Three Chapters • Seven Appendices

How to Use This Handout

There are two ways to teach this subject badly.

The first is to teach it as theory. Relational algebra on a whiteboard, normal forms as definitions to be recited, ACID as four words beginning with the right letters. A student who has learned it this way can answer an examination question about third normal form and cannot look at a table and say whether it is in it.

The second is to teach it as a tool. Open the software, click here, type a SELECT, admire the rows. A student who has learned it this way can produce an answer and has no way of knowing whether it is the right one, because nothing in the course ever told them what the database was promising.

This handout does neither, and the reason is in the title. A database makes promises — that this attribute is a number, that this row refers to a row that exists, that either all of this happened or none of it did — and every one of those promises is *checkable*. So the method of this book is to state the promise, then ask the database to break it, and see what happens. Eleven chapters end with a figure produced that way on a real machine. They are marked, and the file that produces each one is named.

The furniture

Every chapter has the same parts, in the same order, so that you can find what you need without reading from the beginning.

You will see	It means
Learning Outcomes	What you should be able to *do* afterwards. Five per chapter, each with a verb you can be examined on
Before You Start	What this chapter assumes, and where it was taught
Key Terms	Every term defined here. If one is unfamiliar at the end, the chapter did not work
Definition	A term being given its precise meaning. These are the sentences to learn exactly
Concept	An idea, a rule of thumb, or a way of deciding
Watch Out	Something that is true, surprising, and will cost you if you do not know it
Example	A worked case, usually from the running database
Worked Example	Arithmetic, every step shown. Nothing is left "by inspection"
Measured	A figure produced by running the named file in `code/`, on the machine in Appendix A. You can reproduce every one
Common Pitfalls	What goes wrong, collected, so that you meet it here rather than in the laboratory
Try It Yourself	The laboratory. Numbered steps, run in order
Checkpoint	Three questions. Answers in Appendix C, and they are full answers rather than a line each
Chapter Summary	The chapter in eight or nine lines
Review Questions	Seven: two multiple choice, three short, two applied

The laboratory, and what it needs

Nothing to install. The whole course runs on **SQLite**, which is already on every machine you are likely to use: it ships inside Python, it ships with most operating systems, and a database is a single file you can copy, email or delete. Appendix A sets it up in one sitting, which for most readers means checking that it is already there.

That choice is not a compromise with the curriculum. **HEC names no database product in any edition** — the 2017 and 2023 outlines say only "in any DBMS", and 2025 says "popular database management systems". What SQLite can and cannot show you is stated plainly in Chapter 1 and again in Chapter 23, because a tool's limits are part of what you are learning.

The database we build

One schema, carried through the whole course. You design it in Chapter 5 and Chapter 6, map it to tables and create it in Chapter 9, tighten its types in Chapter 10, migrate it while it is full in Chapter 11, and query it for five chapters. In Chapter 18 you build the wide single-table copy that every department builds, watch it break three ways, decompose it — and arrive back at exactly the tables you already had. Then you index it, put its writes in transactions, and defend it. It is a university enrolment database — students, courses, sections, instructors, enrolments, grades — chosen because you already know how it is supposed to behave, so no lecture time goes on explaining the subject matter.

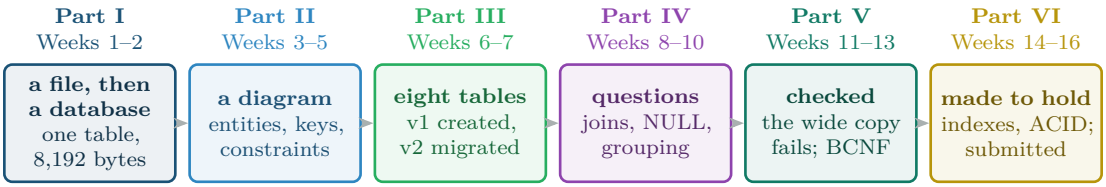


Figure F.1: The course on one line: one database, `university.db`, and what each part does to it. It exists from Week 2, and the file you submit in Week 16 is the same file.

If you read nothing else

- Chapter 8, on the difference between *declaring* a key and *having* one.
- Chapter 18, which is the chapter this course exists for.
- Chapter 21, where the rule everybody repeats stops being true and you can measure the share of a table at which it does.

Course Specification

This chapter exists so that a student, a lecturer or a reviewer can see exactly what this handout claims to cover and on what authority. Everything in it is traceable to a published document, and where those documents are wrong, unclear or contradict one another, that is recorded here rather than quietly corrected.

Particulars

Item	Value
Course title	*Database Systems*. HEC 2023's description page calls it *Database System*, singular, while every table in the same booklet says *Systems*. The plural is used here
Course code	**CS-103**. Unlike every sibling course in this series, there is no `INFT-` equivalent: the registrar uses CS-103 and nothing else
Programme	BS Information Technology, **2nd semester**. This matches HEC 2023, which places the course in semester 2 of every computing degree. HEC 2017 placed it at BSIT semester V; the discrepancy is recorded, not resolved
Credit hours	**4 (3+1)** — three lecture credits and **one credited laboratory**. The laboratory is paid for, which is why every chapter has one and why they are long enough to be worth the hour
Prerequisite	**None.** HEC 2017's own booklet gives two different answers — see defect 4
Duration	Sixteen teaching weeks, midterm in Week 8, project due Week 16 (Appendix G)
Teaching methodology	As published (HEC 2017): lectures, written assignments, **practical labs**, semester project, presentations
Assessment	As published (HEC 2017): sessional exam, home assignments, quizzes, project, presentations, final exam
Set texts	As published: Connolly and Begg; Garcia-Molina, Ullman and Widom; Silberschatz, Korth and Sudarshan; Ramakrishnan and Gehrke. No publication years are given for any of the four, in any edition — see Appendix E

Where the outline comes from

Edition	Status	Semester	Course content published?
2017	Computing Core — compulsory in BSCS, BSSE *and* BSIT	IV, IV, **V**	**Yes** — p. 88

Edition	Status	Semester	Course content published?
2023	Computing Core — compulsory in all ten computing degrees and the Associate Degree	**2**	**Yes** — p. 58, *the 2017 text reprinted*
2025	Major Compulsory — but the edition covers only BS Computer Science and the Associate Degree. **There is no BSIT in it at all**	III	**No** — deleted; five CLO bullets only

⚠ Six defects in the published documents

Recorded rather than inherited. Each is stated with the correction this handout uses.

1. **The syllabus is frozen.** The 2023 course outline is the 2017 course outline word for word, with **one added full stop** ("vs" becomes "vs.") across ninety words — six years apart. Then 2025 removes it entirely. A field that changed a great deal in that period is described by a text that did not change at all.
2. **The outline normalises before it designs.** The clause list reaches *normalization, functional dependencies, normal forms* **before** *entity relationship model . . . entity-relationship diagrams*. You cannot normalise a schema you have not designed. All four of HEC's own reference books teach the other order. **This handout reorders**: design in Part II, map in Part III, normalise in Part V. The clause numbers in the table below show exactly where each one moved to.
3. **One clause is printed twice.** "attributes" appears in the relational-model run and again in the entity-relationship run. Thirty three clauses, thirty two distinct. Both occurrences are traced below and the second is marked as a duplicate.
4. **The prerequisite contradicts itself inside one booklet.** HEC 2017's course description page says `Prerequisites: None`. All three of its scheme-of-studies tables — BSCS p. 25, BSIT p. 31, BSSE p. 39 — say **Data Structures & Algorithms**. At 2nd semester the department cannot assume Data Structures, so **this handout assumes none**, and Chapter 21 and Chapter 22 are written to be self-contained because of it.
5. **The 2023 learning-outcome table is broken in print.** CLO-3's text stops mid-sentence; its remainder ("anomalies by normalizing database tables.") is printed as CLO-4's text; and the real CLO-4 is left in a trailing row with *no number and no Bloom level*. This is a typesetting fault in the published national curriculum, not a misreading. The outcomes are reconstructed below from the 2017 text, which is intact.
6. **A Bloom level is wrong.** CLO-3 reads "Identify functional dependencies and *resolve* database anomalies by normalizing database tables" and is tagged **C2**. Identifying is C2; resolving and normalising are C3 at least. Every sibling course in this series complains that HEC left the Bloom column blank. Here it is filled in — and filled in incorrectly, which is harder to notice.

Course content, traced clause by clause

The outline is reproduced below exactly as published, in its published order, split into its thirty-three clauses. The right-hand column is where each is taught. Clauses that move are the subject of defect 2.

#	Clause, as published	Taught in
1	Basic database concepts	Chapter 1
2	Database approach vs file based system	Chapter 1
3	database architecture	Chapter 3
4	three level schema architecture	Chapter 2
5	data independence	Chapter 2
6	relational data model	Chapter 7
7	attributes	Chapter 7
8	schemas	Chapter 7
9	tuples	Chapter 7
10	domains	Chapter 7
11	relation instances	Chapter 7
12	keys of relations	Chapter 8
13	integrity constraints	Chapter 8
14	relational algebra	Chapter 12
15	selection	Chapter 12
16	projection	Chapter 12
17	Cartesian product	Chapter 12
18	types of joins	Chapter 13
19	**normalization**	Chapter 18 *(moved later)*
20	**functional dependencies**	Chapter 17 *(moved later)*
21	**normal forms**	Chapter 18 *(moved later)*
22	**entity relationship model**	Chapter 5 *(moved earlier)*
23	entity sets	Chapter 5
24	attributes	**duplicate of 7**
25	relationship	Chapter 5, Chapter 6
26	entity-relationship diagrams	Chapter 6, Chapter 9
27	Structured Query Language (SQL)	Chapter 10, Chapter 14
28	Joins and sub-queries in SQL	Chapter 13, Chapter 15
29	Grouping and aggregation in SQL	Chapter 16
30	concurrency control	Chapter 22
31	database backup and recovery	Chapter 22
32	indexes	Chapter 21
33	NoSQL systems	Chapter 23

Three topics taught that the outline does not name

A reviewer will ask, so the justification is here rather than implied.

- **Chapter 11, changing a schema that is already full.** The outline never mentions ALTER. A student who cannot change a schema cannot carry one database through sixteen weeks, and the project requires exactly that.
- **Chapter 20, views and triggers.** Absent from the clause list. Required by CLO-2, which asks for *logical and physical* schemas, and assumed by the department's own *E-Commerce Application Development* (INFT-4041), which lists this course as its prerequisite — see Appendix F.
- **Chapter 23, security and authorisation.** Absent from 2017 and 2023 entirely. **HEC 2025 requires it by name**: its fifth learning outcome is "Analyze the role of databases in information systems and their security considerations". The newest edition justifies the addition.

Course Learning Outcomes

HEC 2017's four, verbatim, with the Domain and Bloom levels exactly as published — including the one that is wrong.

CLO	As published	Level	Taught in
CLO-1	"Explain fundamental database concepts."	C2	Part I
CLO-2	"Design conceptual, logical and physical database schemas using different data models."	C5	Parts II, III
CLO-3	"Identify functional dependencies and resolve database anomalies by normalizing database tables."	**C2** †	Part V
CLO-4	"Use Structured Query Language (SQL) for database definition and manipulation in any DBMS."	C4	Parts III, IV

† **As published, and incorrect** — see defect 6. Assessed in this course at **C3**.

Four added by the department, each discharging one of HEC 2025's five outcomes, which the 2017 clause list does not reach.

CLO	Level	Outcome	Discharges
CLO-5	C3	Write SQL for retrieval, insertion, update and deletion against a schema you did not design	HEC 2025 outcome 2
CLO-6	C2	Explain transactions, concurrency and recovery, and demonstrate what a transaction prevents	HEC 2025 outcome 3
CLO-7	C3	Implement and administer a working database, including backup and restore	HEC 2025 outcome 4
CLO-8	C4	Analyse a database's role in a wider system and its security posture, and justify the analysis from evidence	HEC 2025 outcome 5

Prerequisite Self-Check

This is a **second-semester** course. It assumes one semester of programming and nothing else — in particular it does *not* assume Data Structures, whatever HEC 2017's scheme-of-studies tables say (defect 4).

Twelve questions. If you can answer ten, start at Chapter 1. If you cannot, the right-hand column says where to look — and none of it will take more than an evening.

#	Question	If not
1	What is the difference between a file and a folder?	—
2	A text file holds one record per line, fields separated by commas. How would you find every line whose third field is "2026"?	Programming Fundamentals
3	Write a loop that reads a file line by line.	Programming Fundamentals
4	What does it mean for two values to be *equal* when one is the number 7 and the other is the text "7"?	Chapter 10 returns to this
5	What is an array index? What happens if you use one that is too large?	Programming Fundamentals
6	Given sets $A = \{1, 2, 3\}$ and $B = \{2, 3, 4\}$, write $A \cup B$, $A \cap B$ and $A - B$.	Appendix B
7	What does $A \times B$ mean for two sets, and how many elements does it have?	Appendix B
8	Is the statement "$x > 5$ and $x < 3$" ever true?	Appendix B
9	What is the truth value of "not (P and Q)" when P is true and Q is false?	Appendix B
10	Two people edit the same file at the same time and both save. What happens to the first person's changes?	Chapter 22
11	Roughly how many times do you have to halve 1,000,000 to get to 1?	Appendix B; Chapter 21 needs it
12	What is a function, in the mathematical sense — how many outputs may one input have?	Appendix B; Chapter 17 depends on it

Question 12 is the one that matters

Everything in Part V rests on it. A *functional dependency* is nothing more than the statement that one set of attributes is a function of another: give me a student's roll number and there is exactly one name it can be paired with. If question 12 was uncomfortable, read Appendix B before Chapter 17, not during it.

Contents

How to Use This Handout ii

Course Specification iv

Prerequisite Self-Check viii

I What a Database Is 1

1 Why Not Just Files? 3
1.1 The Program You Have Already Written . . . 3
1.2 Six Things That Go Wrong . . . 4
1.3 What a Database Management System Is . . . 5
1.4 When a File Is Still the Right Answer . . . 6

2 Three Levels, and Why They Are Separate 9
2.1 One Database, Three Descriptions . . . 9
2.2 Schema and Instance . . . 10
2.3 Data Independence, Which Is the Point of All This . . . 11
2.4 Where SQLite's Three Levels Are . . . 11

3 What Is Inside a DBMS, and Who Does What 15
3.1 A DBMS Is Not One Program . . . 15
3.2 Following One Statement Down . . . 16
3.3 The Data Dictionary Is Just More Tables . . . 17
3.4 The Four Roles . . . 17

4 Your First Database, in One Command 21
4.1 Three Statements and a File . . . 21
4.2 Writing the Schema Down Before You Type It . . . 22
4.3 What It Actually Cost . . . 22
4.4 The Eight Dot-Commands . . . 24

II Designing It 28

5 The Entity-Relationship Model 30
5.1 Design Happens on Paper . . . 30
5.2 Entities: the Things . . . 31
5.3 Attributes, and the Four Distinctions That Matter . . . 31
5.4 Keys: Three Words, One Choice . . . 32
5.5 Relationships, and Degree . . . 33

6 Cardinality, Participation and Weak Entities **38**
6.1 A Diagram Without Cardinalities Says Almost Nothing . . . 38
6.2 Cardinality, by Asking Two Questions . . . 39
6.3 Participation: Must It Take Part at All? . . . 39
6.4 Weak Entities . . . 41

7 The Relational Model **45**
7.1 One Idea, and It Is Mathematics . . . 45
7.2 The Four Properties of a Relation . . . 46
7.3 Keys, Restated for Relations . . . 47
7.4 Null, and Why It Is the Awkward Part . . . 48

8 Keys and Integrity Constraints **52**
8.1 Three Constraints the Model Gives You Free . . . 52
8.2 Foreign Keys . . . 53
8.3 Declared Is Not Enforced . . . 54

III From Diagram to Schema **60**

9 Mapping an ER Diagram to Tables **62**
9.1 The Part of Design That Is Mechanical . . . 62
9.2 Rules 1 to 3: the Entities . . . 63
9.3 Rules 4 to 6: the Relationships . . . 64
9.4 Rule 7, and the Two Awkward Cases . . . 65

10 SQL: Creating the Schema **69**
10.1 The Statement, in Full . . . 69
10.2 Type Affinity, and What It Costs . . . 70
10.3 Three Smaller Things That Matter . . . 72

11 Changing a Schema That Is Already Full **76**
11.1 The Chapter the Outline Does Not Name . . . 76
11.2 What ALTER TABLE Can Do . . . 77
11.3 The Twelve-Step Rebuild . . . 77
11.4 Migrations as Files . . . 79

IV Asking Questions **83**

12 Relational Algebra **85**
12.1 Why a Course on SQL Starts With Algebra . . . 85
12.2 The Two Unary Operators . . . 86
12.3 The Binary Operators . . . 86
12.4 Rename, and a Question Written Three Ways . . . 87

13 Joins: the Algebra and the SQL **91**
13.1 One Operator, Three Strengths . . . 91
13.2 What a Join Drops . . . 92
13.3 Measured: Plans, Nulls and the Cost of a Missing Index . . . 92
13.4 Joining a Table to Itself . . . 94

14 Selecting Rows, and the Trouble with NULL **97**
14.1 The Statement, and the Order It Runs In . . . 97
14.2 Three-Valued Logic . . . 98
14.3 NOT IN, Which Can Return Nothing . . . 99

15 Sub-queries, EXISTS and Set Operations **103**
15.1 A Query Inside a Query . . . 103
15.2 Semi-joins and Anti-joins . . . 104
15.3 ANY, ALL, and Naming a Sub-query . . . 105

16 Grouping, Aggregation and Window Functions **110**
16.1 Collapsing Rows Into One . . . 110
16.2 GROUP BY . . . 111
16.3 Measured: Nulls, Plans, and a Rank . . . 111
16.4 Window Functions . . . 112

V Getting the Design Right **117**

17 Functional Dependencies **119**
17.1 One Table That Everyone Has Built . . . 119
17.2 Deriving New Dependencies . . . 120
17.3 Attribute Closure . . . 121
17.4 Minimal Cover . . . 122

18 Normal Forms: 1NF to BCNF **126**
18.1 Three Ways a Wide Table Goes Wrong . . . 126
18.2 The Normal Forms . . . 128

19 Decomposition, and Denormalising on Purpose **133**
19.1 Two Things a Split Must Not Lose . . . 133
19.2 When BCNF and Dependency Preservation Conflict . . . 135
19.3 Triggers, and the Summary Table Written Out . . . 136

20 Views, Triggers and Keeping Data Honest **139**
20.1 A Query With a Name . . . 139
20.2 Measured: What a View Costs, and What a Trigger Costs . . . 140
20.3 Triggers . . . 142

VI Making It Work, and What Comes Next **146**

21 Indexes, and Where They Stop Helping **148**
21.1 What an Index Is . . . 148
21.2 Measured: Where an Index Wins, and Where It Loses . . . 149
21.3 Using an Index Well . . . 151

22 Transactions, Concurrency and Recovery **154**
22.1 Promises About Groups of Changes . . . 154
22.2 How SQLite Isolates: One Lock for the Whole File . . . 156
22.3 Recovery, and Backup . . . 157

23 Security, NoSQL, and the Semester Project **160**
23.1 Security: the Component SQLite Does Not Have . . . 160
23.2 NoSQL: What It Gives Up to Get What It Offers . . . 162
23.3 The Semester Project . . . 163

Appendices **168**

A The Toolkit: Setting It Up and Keeping It **169**
A.1 What You Need . . . 169
A.2 Why SQLite, and What It Costs You . . . 169
A.3 The Ten Commands You Will Actually Use . . . 170
A.4 Resetting to a Known State . . . 171
A.5 The Machine Every Timing Was Taken On . . . 171
A.6 Scope: What You May Point Any of This At . . . 171

B Just Enough Sets, Logic and Logarithms **173**
B.1 Sets . . . 173
B.2 Relations and Functions . . . 174
B.3 Logic . . . 174
B.4 Logarithms: How Many Halvings . . . 175

C Answers to the Checkpoints **177**

D Glossary **185**

E Resources **189**
E.1 The Four Books HEC Names, and Which Editions to Use . . . 189
E.2 The Tool's Own Documentation . . . 190
E.3 The Original Papers . . . 190
E.4 Law . . . 190
E.5 This Handout's Own Code . . . 191

F The Boundary With Other Courses **192**
F.1 The Edition History 192
F.2 Programming Fundamentals — the Real Prerequisite 192
F.3 Data Structures — Not Assumed 193
F.4 E-Commerce Application Development — Downstream 193
F.5 Information Security — Later 194
F.6 Software Engineering — Later 194

G The Sixteen-Week Schedule **195**
G.1 The Schedule 195
G.2 How the Assessment Adds Up 196

List of Figures

F.1 The course on one line: one database, `university.db`, and what each part does to it. It exists from Week 2, and the file you submit in Week 16 is the same file. . . . iii

P1.1 Reading path through Part I. Chapters 1 to 3 are the argument; Chapter 4 is the first one you can run, and by the end of it there is a file on your disk that every later chapter adds to. 2

1.1 The same three programs, two ways. The difference is not that the right-hand side has fewer files — it is that the rules about what a student is have moved out of the programs and into one place. 4

2.1 The ANSI/SPARC three-level architecture. The two mappings are the whole value of the arrangement: because they exist, a change at one level can be absorbed instead of propagated. 10

3.1 The components of a DBMS, and the path a statement takes down through them. The optimiser and the parser both consult the data dictionary — the parser to check that `student` exists and has a `name`, the optimiser to find out how big it is and what indexes it has. 16

4.1 One table, with Chapter 2's schema/instance distinction drawn on it. The box above the rule is the schema and is four lines long whether the table holds one row or a million. 22

P2.1 Reading path through Part II. Chapter 6 draws the university database this whole course uses; Chapter 8 is the first measured chapter in which the database does something you did not ask for. 29

5.1 STUDENT in Chen notation. Four of the five attribute kinds are present. `phone` and `age` are the two that will not survive contact with a table, and Chapter 9 says what happens to each. 32

5.2 Two of the three degrees. ENROLS carries an attribute of its own; REQUIRES joins COURSE to itself and is unreadable without role names. The university database you are building contains both. 34

6.1 The university database, fully constrained. Only two lines in the whole diagram are double — a course must belong to a department and a section must belong to a course. Everything else is partial, which is the usual proportion. 42

7.1 One relation, with every word of the vocabulary attached to the thing it names. Degree belongs to the heading and changes only when the schema does; cardinality belongs to the body and changes all day. 46

8.1 Referential integrity. Two of the three child rows point at a course that exists; the null is permitted by the definition; the third is an orphan. The whole of this chapter's measurement is that the third row goes in. 53

P3.1 Reading path through Part III. Chapter 9 is mechanical and should be: mapping is a set of rules you apply rather than judgements you make. The judgement was Part II and the consequences are Part V. 61

9.1 Four of the seven rules on the part of the diagram that uses them. Rule 1 makes a table per box; rule 6 turns the diamond into a table of its own; rule 7 gives the weak entity its owner's key. The arrows are the whole of logical design. 65

10.1 Type affinity in one decision. The middle branch is the whole problem: it is not an error, it is not reported, and a column containing both of the top two outcomes looks exactly like a column containing only the first. 70

11.1 Two questions decide what a schema change costs. The right-hand branch is the one Part II existed to prevent: no procedure recovers information that was never stored. 79

P4.1 Reading path through Part IV. Chapter 12 is the only one here with no SQL in it, and it is the one that makes the other four make sense. 84

12.1 The two unary operators. Selection cuts horizontally and never loses a column; projection cuts vertically and can lose rows, because three `BSIT` values become one. 86

13.1 The section nobody has been assigned to teach. An inner join drops it without comment; a left join keeps it and marks what is missing. On the seeded database this is 73 sections out of 1,500. 92

14.1 The seeded *enrolment* table, drawn to scale. A condition and its negation cover the green and the blue. The grey third is UNKNOWN to both, and WHERE discards it either way. 99

15.1 The prerequisites of `CS-305` on the seeded database. An arrow means "requires". A fixed query reaches a fixed depth; the recursive one keeps going until a step adds nothing new. 106

16.1 The difference in one picture. Grouping collapses each partition into a single row; a window function keeps every row and adds a value computed from the partition it belongs to. The colours mark the two partitions. 113

P5.1 Reading path through Part V. Chapter 18 breaks the running database and rebuilds it, and Chapter 19 measures what that cost — because normalising is a trade and a course that presents it as a pure good has not finished the argument. 118

17.1 $\{course_code\}^+$, one step per row. Each step applies a dependency whose left side is already inside the set; the second only becomes applicable because the first added *dept_code*. The closure stops at five attributes — fewer than fourteen, so *course_code* is not a key of *report*. 122

18.1 *report* decomposed to BCNF. Splitting on each partial dependency gives four relations; splitting on the two transitive ones gives two more. The result is the six tables Chapter 9 derived from the ER diagram by an entirely different route. . . . 130

19.1 A lossy split on the smallest possible scale. The two halves share only *course*, which is a key of neither, so the join pairs every student in the course with every grade in it. On the seeded data this is 143,768 invented facts. 135

20.1 What a view is, in one diagram. The dictionary holds only the view's text; SQLite substitutes it into the query that names it, then compiles. By the time anything runs there is no view left to cost anything, or to be out of date. 140

P6.1 Reading path through Part VI. Chapter 21 is the measured chapter students remember, because it contradicts something they were told in their first week and shows them exactly where the contradiction begins. 147

21.1 An index on k over a million rows, three levels deep. Each page divides its range among a few hundred children, so the path from the root to any key is three pages long, and one more read fetches the row. The scan it replaces reads every page of the table. 149

22.1 Two kinds of deadlock. The left one was produced by the harness with two DEFERRED transactions. The right one needs row-level locks, which SQLite does not have, so it is drawn here rather than run — Chapter 23 is where a server's locking is compared properly. 156

23.1 One student, held two ways. The relational form stores each fact once and answers questions across students with a search; the document form stores each student once and answers questions about one student in a single read. Which is right depends on which kind of question the system is asked. 163

Part I

What a Database Is

Every program you have written so far has stored its data in a file, and that worked. Part I is about the point at which it stops working — not in theory, but on a particular afternoon when two people save at once, or a column turns out to mean two different things, or somebody deletes a row that something else was relying on.

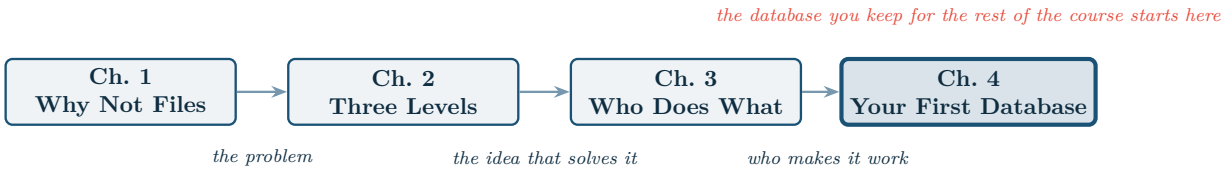


Figure P1.1: Reading path through Part I. Chapters 1 to 3 are the argument; Chapter 4 is the first one you can run, and by the end of it there is a file on your disk that every later chapter adds to.

Chapter 1

Why Not Just Files?

Learning Outcomes

By the end of this chapter you will be able to:

- **Describe** how a file-based application stores and finds data, and **state** what it costs to add one field.
- **Name** the six problems a file-based approach has, and **give** a concrete example of each.
- **Explain** what a database and a database management system are, and **distinguish** the two.
- **Judge** when a file is the right answer, which is more often than this chapter's title suggests.
- **Predict** which of the six problems a given scenario will hit first.

Before You Start

One semester of programming: reading and writing a file, loops, and what a record looks like when it is a line of comma-separated text. Nothing else. Appendix A if you have not checked that SQLite is on your machine.

Key Terms in This Chapter

data • information • file-based system • database • database management system (DBMS) • data redundancy • data inconsistency • data isolation • integrity constraint • atomicity • concurrent access • lost update • data security • application program

1.1 The Program You Have Already Written

Somewhere in your first semester you wrote a program that kept a list of things in a file. It probably looked like this, and there is nothing wrong with it:

```
# students.csv
# 2021-IT-001,Ayesha Khan,BSIT,3
# 2021-IT-002,Bilal Ahmed,BSIT,3

def find(roll_no):
    for line in open("students.csv"):
        fields = line.strip().split(",")
        if fields[0] == roll_no:
            return fields
    return None
```

It works. It is twelve lines. For a list of your own books, or a configuration file, or the output of a program that nobody else reads, it is the right answer and this chapter is not an argument against it.

This chapter is about the afternoon it stops being the right answer — and about the fact that it does not stop gradually. A file-based approach works, and works, and works, and then the department asks for one more thing and suddenly nothing works.

Data, Information, and the difference

Data are raw recorded facts: the characters `2021-IT-001` in a file.

Information is data in a context that makes it answer a question: *the roll number of the student who scored highest in Database Systems last term.*

A **file-based system** stores data and leaves every question to the application program. A **database** stores data *together with a description of what it means and what must be true of it*, so that some of the questions — and all of the guarantees — can be answered by the store itself.

That last sentence is the whole subject. Everything in the remaining twenty-two chapters is a particular case of it.

1.2 Six Things That Go Wrong

These are not six opinions. Each is a specific failure with a name, and you will meet all six in a working life.

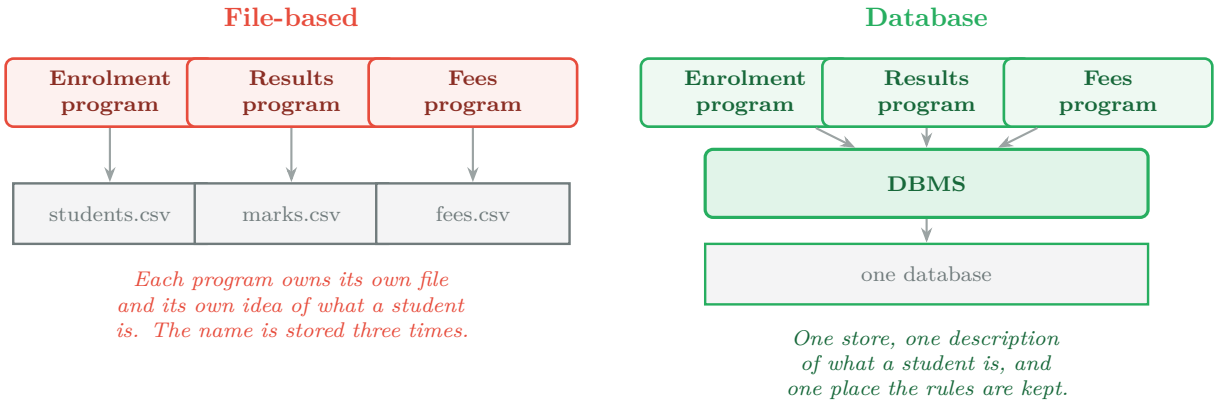


Figure 1.1: The same three programs, two ways. The difference is not that the right-hand side has fewer files — it is that the rules about what a student is have moved out of the programs and into one place.

The six, with a case for each

1. **Data redundancy.** The student's name is in `students.csv`, in `marks.csv` so the results sheet can print it, and in `fees.csv` so the receipt can. Three copies, and disk space is the least of it.
2. **Data inconsistency.** A student marries and changes her name. Two of the three files are updated. The college now holds two different answers to one question, and *nothing in the system is wrong* — each file is internally fine. This is redundancy's actual cost.
3. **Data isolation.** The question "which students owe fees and are enrolled in a course that has already started?" needs two files in different formats. There is no way to ask it; there is only a program somebody has to write.
4. **Integrity problems.** "A student's semester is between 1 and 8." Where does that rule live? In the enrolment program. And in the results program, where somebody wrote 1 to 10. The rule is a comment in three places and enforced in none.
5. **Atomicity problems.** Transferring a student between sections means removing a line from one file and adding it to another. The machine loses power in between. The student is now in no section at all, and nothing can tell you that happened.

6. **Concurrent access and security.** Two clerks open the file, both edit, both save; the second overwrites the first and neither is told — a **lost update**. And the fees clerk, who needs one column, has the whole file including every home address.

Which one bites first, and why it is never the one you expect

Students asked to rank these almost always put redundancy first, because it is the one that is visible — you can *see* the duplicated column.

In practice the order is almost exactly reversed. Redundancy costs disk, which is cheap. **Inconsistency** is what gets noticed, because somebody receives a letter addressed to a name they no longer use. **Isolation** is what gets the department to pay for a new system, because the question they want answered cannot be answered at all. And **atomicity** is the one nobody finds until the day it matters, because a half-finished transfer looks exactly like a student who was never enrolled.

Keep this ranking in mind in Chapter 23, when you have to justify your design to somebody who will ask what problem it solves.

1.3 What a Database Management System Is

Database, DBMS, Database System

A **database** is a shared, organised collection of data, together with a description of that data — what it is called, what type it is, what must be true of it, and how its parts relate.

A **database management system** (DBMS) is the *software* that creates, maintains and controls access to a database. SQLite is a DBMS; so are Oracle, PostgreSQL, MySQL and SQL Server.

A **database system** is the whole arrangement: the data, the DBMS, the application programs that use it, and the people.

These three are routinely used as if they meant the same thing, including by people who should know better. In an examination they do not, and the distinction is CLO-1's business.

The DBMS is what the right-hand side of Figure 1.1 adds. It is a single piece of software that every program goes through, and because everything goes through it, it is the one place where a rule can be stated once and be true for everybody.

What moves into the DBMS, problem by problem

Redundancy	One copy of each fact, and other tables refer to it (Chapter 9)
Inconsistency	Follows from the above — there is nothing to disagree with
Isolation	One query language over all the data (Chapter 12 onward)
Integrity	Constraints declared once, in the schema (Chapter 8)
Atomicity	Transactions: all of it or none of it (Chapter 22)
Concurrency, security	Locking, and authorisation (Chapter 22, Chapter 23)

That table is this book's table of contents, rearranged. Every chapter from here is one row of it, done properly.

1.4 When a File Is Still the Right Answer

A course called Database Systems has an obvious bias, and you should know about it.

Use a file when every one of these is true

- **One program reads it, and one writes it.** No concurrency, so no lost updates.
- **It is small enough to read entirely.** If you always want all of it, an index buys nothing.
- **The questions never change.** Isolation only hurts when somebody asks something new.
- **Losing it is survivable**, or it can be regenerated.

Configuration files, log files, a cache, the output of a nightly job — all files, all correctly so. **The failure is not using a file; it is using a file after the fourth condition stopped holding and not noticing.**

Common Pitfalls

- **Treating redundancy as the main problem.** It is the cheapest of the six and the easiest to see, which is why it gets the attention.
- **Saying "database" when you mean "DBMS".** The data is not the software.
- **Believing a DBMS removes the need to think about integrity.** It gives you somewhere to put the rules. It does not write them, and Chapter 8 measures what happens when you declare one and it is not enforced.
- **Assuming a database is always better.** Four conditions above say otherwise.
- **Calling a spreadsheet a database.** It has rows and columns and none of the six guarantees.

Try It Yourself: Break a file-based system on purpose

No database yet. This laboratory is about earning the next twenty-two chapters.

```
# --- 1. Make the files -------------------------------------------------
# Three CSV files, by hand, in a text editor. Five students each.
#   students.csv   roll_no,name,programme,semester
#   marks.csv      roll_no,name,course,marks
#   fees.csv       roll_no,name,amount,paid

# --- 2. Count the redundancy -------------------------------------------
# How many times does one student's NAME appear across the three files?
# Multiply by 4,000 students. Write the number down.

# --- 3. Create an inconsistency ----------------------------------------
# Change one student's name in students.csv and marks.csv but NOT in
# fees.csv. Now answer: what is that student's name? Justify it.

# --- 4. Hit data isolation ---------------------------------------------
# Write the program that answers: "which students owe fees AND are
# enrolled in a course where they scored below 50?"
# Time yourself. Do not finish it if it takes more than twenty minutes
# -- stopping is the result.

# --- 5. Break atomicity ------------------------------------------------
# Write a script that moves a student from one file to another, with a
# deliberate crash between the two writes. Run it. Describe the state.

# --- 6. Simulate a lost update -----------------------------------------
```

```
# Open students.csv in two editors. Change a different row in each.
# Save both. Report what survived.

# --- 7. Write the memo ------------------------------------------------
# One page to a head of department who has asked why the existing
# system should be replaced. Use YOUR numbers from steps 2 and 4.
# No jargon: the reader has not taken this course.
```

What to notice. Step 4 is the one that convinces people. Redundancy is an argument; a question that cannot be answered at all is a business problem, and it is the one that gets a project funded.

Checkpoint — Test Yourself

1. A college keeps student records in one file and library loans in another, each maintained by one clerk who never touches the other. Which of the six problems does this arrangement have, and which does it genuinely escape? Say what would have to change for your answer to change.
2. Distinguish a database from a database management system, and explain why "we bought a database" is a sentence that needs clarifying before you can act on it.
3. Your colleague argues that redundancy is harmless because disk is cheap. Give the argument you would make in reply, name the problem redundancy actually causes, and give an example in which no single file is wrong and the system still is.

Chapter Summary

- Data are recorded facts; information is data in a context that answers a question. A database stores data *with a description of what it means and what must be true of it*, which is the whole subject.
- A file-based system works until it does not, and the change is abrupt rather than gradual.
- The six problems are redundancy, inconsistency, isolation, integrity, atomicity, and concurrent access with security.
- Redundancy is the visible one and the cheapest. Inconsistency is what gets noticed, isolation is what gets a project funded, and atomicity is the one nobody finds until it matters.
- A *database* is the data; a *DBMS* is the software; a *database system* is the whole arrangement including the people.
- Everything goes through the DBMS, which is why it is the one place a rule can be stated once and be true for everybody.
- A file is still right when there is one reader, one writer, a fixed set of questions and nothing irreplaceable. The failure is not noticing when that stops being true.
- The table in Section 1.3 is this book's contents rearranged: each of the six problems is solved by a named later chapter.

Review Questions

1. (MCQ) Two files hold a student's name and only one is updated. This is an example of: (a) redundancy; (b) inconsistency; (c) isolation; (d) an integrity constraint.

2. (MCQ) The software that creates and controls access to a database is called: (a) the database; (b) the database system; (c) the DBMS; (d) the schema.

3. (Short) Give a question about a university that cannot be answered from two separate CSV files, and say which of the six problems that is.

4. **(Short)** Explain atomicity using an example that is not a bank transfer.

5. **(Short)** State two conditions under which a plain file is a better choice than a database, and give a real example of each.

6. **(Applied)** A clinic keeps appointments in a spreadsheet shared on a network drive, edited by three receptionists. Identify every one of the six problems it has, rank them by which will cause a complaint first, and justify your ranking.

7. **(Applied)** You are asked to justify replacing a working file-based system. Write the three strongest arguments in order, say what evidence you would gather for each before the meeting, and state which argument you would drop if you only had five minutes.

Chapter 2

Three Levels, and Why They Are Separate

Learning Outcomes

By the end of this chapter you will be able to:

- **Name** the three levels of the ANSI/SPARC architecture and **state** what each one describes.
- **Distinguish** a schema from an instance, and **explain** why the difference matters when something changes.
- **Define** logical and physical data independence, and **give** an example of a change that each one protects you from.
- **Identify** which level a given change belongs to.
- **Explain** why a system with no separation of levels is the file-based system of Chapter 1 again.

Before You Start

Chapter 1: the six problems, and what a DBMS is. Nothing else. This chapter is the one piece of pure architecture in the course and it is here because every later chapter quietly assumes it.

Key Terms in This Chapter

ANSI/SPARC architecture • external level • external schema • user view • conceptual level • conceptual schema • internal level • internal schema • physical level • schema • instance • mapping • logical data independence • physical data independence

2.1 One Database, Three Descriptions

In Chapter 1 the fees program and the results program each had their own file and their own idea of what a student was. Putting them behind one DBMS fixes the duplication — but it creates a new problem immediately.

The fees clerk wants a student to be a roll number, a name and an amount owing. The examinations officer wants a roll number, a name and a list of marks. The person who has to make the database fast wants to know that the student table is sorted by roll number and has an index on surname. All three are talking about the same data and none of them should have to care about the other two.

The answer, standardised in 1975 and unchanged since, is to describe the database *three times*.

The three levels

The **external level** is what a particular user or program sees: their own tailored view of the part of the database they need, in the shape they need it. There are many external schemas, one per group of users.

The **conceptual level** is the whole database, described once, logically: every entity, every attribute, every relationship and every constraint — and nothing at all about how it is stored. There is exactly one conceptual schema.

The **internal level** — also called the **physical level** — is how it is actually stored: files, pages, record layouts, indexes, compression. There is exactly one internal schema.

A **mapping** connects adjacent levels, so that the DBMS can translate a request expressed against one into an operation on the next.

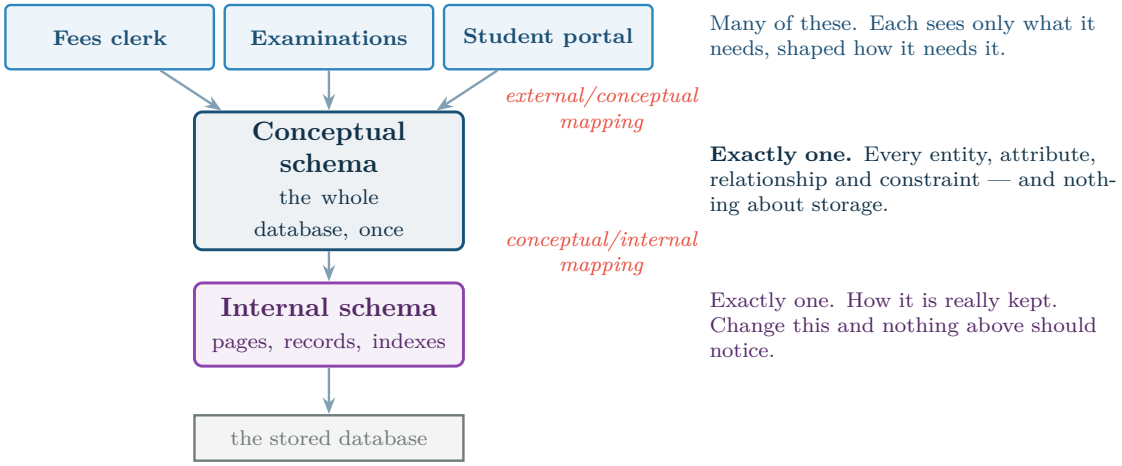


Figure 2.1: The ANSI/SPARC three-level architecture. The two mappings are the whole value of the arrangement: because they exist, a change at one level can be absorbed instead of propagated.

2.2 Schema and Instance

Two words that students use interchangeably for about a fortnight, and then cannot afford to.

Schema, Instance

A **schema** is the *description*: the names of the tables, their columns, the types, the constraints. It is set when the database is designed and changes rarely.

An **instance** (or *state*, or *extension*) is the data *in* the database at one moment. It changes every time anybody inserts a row.

The analogy worth keeping: a schema is a variable's declaration, an instance is its value. `int count;` is the schema; `7` is the instance.

The same distinction, three ways

	Schema	Instance
Programming	`int count;`	7
A database	STUDENT(roll_no, name, programme)	the 4,000 rows in it today
A form	the blank form	one filled-in copy

When somebody says "the database changed", the first question is always which of the two they mean, because the answers are days apart in effort.

2.3 Data Independence, Which Is the Point of All This

Three descriptions and two mappings are a lot of machinery. It buys exactly one thing, and the thing is worth it.

Logical and physical data independence

Physical data independence is the ability to change the *internal* schema — how data is stored — without changing the conceptual schema or any application program. Adding an index, moving to a different disk, reorganising the file: none of these should require a line of application code to change.

Logical data independence is the ability to change the *conceptual* schema — adding a table, adding a column, splitting one table into two — without changing the external schemas or the programs that use them.

Physical independence is easy and essentially achieved. **Logical independence is hard and only partly achieved**, which is why it is worth understanding where it fails.

Which level does this change belong to?

A question worth being able to answer instantly, because the cost of a change is decided by its level.

- **"Add an index on surname."** Internal. Nothing else changes. Chapter 21 does exactly this and measures it.
- **"Add a telephone number to students."** Conceptual. Existing views that never mentioned it carry on working — this is logical independence succeeding.
- **"Split the student table into student and student-contact."** Conceptual, and this is where logical independence is *tested*: Chapter 20 shows how a view can hide the split from programs that still want one table. Chapter 18 does this split for real.
- **"The fees clerk should not see home addresses."** External. A view, and in a server DBMS a privilege (Chapter 23).
- **"Store dates as text rather than numbers."** Looks internal. It is *conceptual*, because it changes what comparisons mean — and Chapter 10 measures what SQLite does about that.

The last one is the one that catches people, and it catches them because "how it is stored" sounds internal in English and is not.

2.4 Where SQLite's Three Levels Are

An architecture standardised in 1975 is worth nothing to you unless you can point at its three levels in the tool on your own machine, which at first glance appears to have only one.

All three are there; only two can be enforced

It is worth knowing how an architecture from 1975 maps onto the tool in front of you, because at first glance SQLite seems to have only one level.

- **Conceptual:** your CREATE TABLE statements. Exactly one set of them, and `.schema` prints it.
- **Internal:** the page structure, the B-trees, the indexes. Chapter 4 measures it directly and you never write it — SQLite decides, and PRAGMA is the only window onto it.

- **External:** views (Chapter 20). SQLite has them, so the level exists — but it has no users, so it cannot restrict anybody *to* a view. The level is present and the enforcement is not, which is Chapter 23's subject.

This is a real limitation and worth stating now rather than in week sixteen. You will build all three levels in this course. You will only be able to *enforce* two of them.

Common Pitfalls

- **Confusing schema with instance.** A schema change is a project; an instance change is a Tuesday.
- **Thinking there are many conceptual schemas.** There is one. Many external, one conceptual, one internal.
- **Believing physical independence means performance does not matter.** It means you can change storage *without rewriting programs*. It does not mean the choice is free — Chapter 21 measures exactly how unfree it is.
- **Calling a storage-format change internal when it changes meaning.** If a comparison starts giving different answers, it was conceptual.
- **Assuming logical independence is automatic.** It is the hard one, and Chapter 20 is how it is bought.

Try It Yourself: Find the three levels in a database you did not design

```
# --- 1. Make a database with something in it -------------------------
sqlite3 arch.db
```

then, in the shell:

```
-- --- 2. State a conceptual schema -------------------------------
CREATE TABLE student (
  roll_no   TEXT PRIMARY KEY,
  name      TEXT NOT NULL,
  programme TEXT NOT NULL,
  semester  INTEGER NOT NULL
);
INSERT INTO student VALUES
  ('2021-IT-001','Ayesha Khan','BSIT',2),
  ('2021-IT-002','Bilal Ahmed','BSIT',2),
  ('2021-CS-007','Hira Nadeem','BSCS',4);

-- --- 3. Read the conceptual schema back -------------------------
.schema student
-- That text IS the conceptual schema. Nothing in it mentions storage.

-- --- 4. Build an external schema --------------------------------
CREATE VIEW fees_view AS
  SELECT roll_no, name FROM student;
SELECT * FROM fees_view;
-- A user given only this cannot see "programme" or "semester".

-- --- 5. Look at the internal schema -----------------------------
PRAGMA page_size;
PRAGMA page_count;
SELECT name, type FROM sqlite_master;
-- You did not write any of this and cannot usefully change it.

-- --- 6. Change the internal level and prove nothing else moved --
CREATE INDEX idx_name ON student(name);
```

```
SELECT * FROM fees_view;
-- Same answer. That is physical data independence, demonstrated.

-- --- 7. Now change the conceptual level -------------------------
ALTER TABLE student ADD COLUMN email TEXT;
SELECT * FROM fees_view;
-- Still works, and still has two columns. Logical independence.

-- --- 8. Break it ------------------------------------------------
-- Try to drop the column the view depends on:
ALTER TABLE student DROP COLUMN name;
-- Write down exactly what happens, and which level failed.
```

What to notice. Steps 6 and 7 are the two kinds of independence, each demonstrated in one line. Step 8 is the limit of the second one, and the error message is worth reading twice.

Checkpoint — Test Yourself

1. Classify each of these by level, and say which kind of data independence (if any) protects the programs from it: adding an index; adding a column; renaming a column; storing the database on a faster disk; giving the library staff a view with no home addresses.
2. Explain the difference between a schema and an instance to somebody who has written programs but never used a database, using an example that is not a database.
3. SQLite has views but no users. Say precisely which of the three levels is affected by that, what you can still do, what you cannot, and how you would achieve the same effect in an application built on SQLite.

Chapter Summary

- The ANSI/SPARC architecture describes one database three times: *many* external schemas, *one* conceptual schema, *one* internal schema, joined by two mappings.
- The external level is what a user sees; the conceptual level is the whole database logically and says nothing about storage; the internal level is how it is really kept.
- A *schema* is the description and changes rarely; an *instance* is the data and changes constantly. A schema is a declaration, an instance is a value.
- **Physical data independence** lets the internal schema change without touching programs. It is essentially achieved.
- **Logical data independence** lets the conceptual schema change without touching external schemas. It is hard, partial, and bought with views (Chapter 20).
- The level a change belongs to decides what it costs. "Store dates as text" sounds internal and is conceptual, because it changes what comparisons mean.
- In SQLite the conceptual level is your CREATE TABLE statements, the internal level is pages and B-trees you never write, and the external level is views — which exist but cannot be enforced, because there are no users to restrict.

Review Questions

1. (MCQ) How many conceptual schemas does a database have? (a) one per user; (b) one per table; (c) exactly one; (d) one per application.

2. (MCQ) Adding an index without changing any program demonstrates: (a) logical data independence; (b) physical data independence; (c) an external schema; (d) an instance change.

3. (Short) Give one change that logical data independence protects you from and one that it does not, and say what makes the difference.

4. **(Short)** Why does the architecture need *mappings* as well as schemas? What would be lost without them?

5. **(Short)** A colleague says the internal level does not matter because of physical data independence. Reply in two sentences.

6. **(Applied)** A university wants three groups to use one student database: admissions, examinations, and the students themselves. Design the external schemas — what each sees and what each must not — and say which parts of your design SQLite can enforce and which it cannot.

7. **(Applied)** Your conceptual schema must change: the single `address` column becomes four columns. Describe how you would make that change without breaking existing programs, what the view would look like, what it would cost, and at what point you would decide the view is no longer worth keeping.

Chapter 3

What Is Inside a DBMS, and Who Does What

Learning Outcomes

By the end of this chapter you will be able to:

- **Name** the major components of a DBMS and **state** what each one is responsible for.
- **Trace** a single SELECT statement through those components, in order, from text to answer.
- **Explain** what the data dictionary is, and **query** it.
- **Distinguish** the four database roles — administrator, designer, application programmer, end user — by what each one decides.
- **Identify** which components SQLite has, which it lacks, and which role has to cover for the missing ones.

Before You Start

Chapter 2: the three levels and the two mappings. This chapter is largely about *who performs* those mappings.

Key Terms in This Chapter

query processor • parser • query optimiser • execution engine • storage manager • buffer manager • file manager • transaction manager • lock manager • recovery manager • authorisation manager • data dictionary • system catalog • metadata • DBA • database designer • application programmer • end user

3.1 A DBMS Is Not One Program

You type eleven words:

```
SELECT name FROM student WHERE programme = 'BSIT' ORDER BY name;
```

and rows come back. Between those two events, six or seven distinct pieces of software run, each with one job. It is worth knowing them by name, because for the rest of the course every topic is really a chapter about one of them: Chapter 21 is about the optimiser and the storage manager arguing, Chapter 22 is about the transaction and recovery managers, and Chapter 23 is about the one component SQLite does not have at all.

The components of a DBMS

The **query processor** turns your statement into an answer. It contains a **parser** (is this valid SQL, and what does it refer to?), a **query optimiser** (of the many ways to compute this, which is cheapest?) and an **execution engine** (do it).

The **storage manager** stands between the query processor and the disk. It contains a **file manager** (where on disk is this?) and a **buffer manager** (which pages are in memory, and which do I evict?).

The **transaction manager** makes groups of operations behave as one, with a **lock manager** to keep concurrent users from corrupting each other's work and a **recovery manager** to put the database back together after a crash.

The **authorisation manager** decides whether this user is allowed to do this at all.

The **data dictionary** — also called the **system catalog** — is where the DBMS keeps the schema itself. It is a database about the database: **metadata**, data about data.

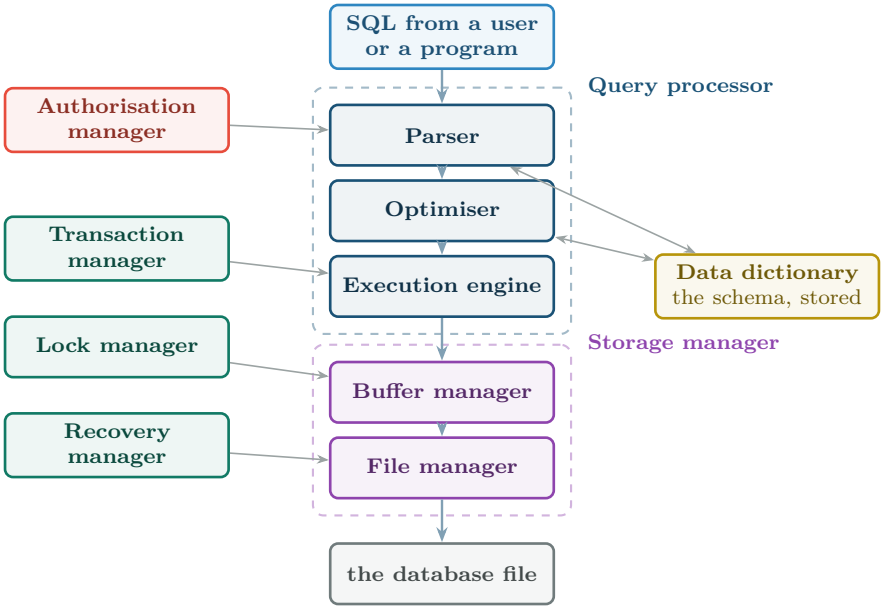


Figure 3.1: The components of a DBMS, and the path a statement takes down through them. The optimiser and the parser both consult the data dictionary — the parser to check that `student` exists and has a `name`, the optimiser to find out how big it is and what indexes it has.

3.2 Following One Statement Down

Worked Example: Eleven words, six components

```
SELECT name FROM student WHERE programme = 'BSIT' ORDER BY name;
```

1. **Authorisation manager.** May this user read `student`? In a server DBMS this is checked first and can stop everything here. *In SQLite there is no such check* — if you can open the file, you can read all of it.
2. **Parser.** Is the syntax legal? Does `student` exist? Does it have columns `name` and `programme`? Is `'BSIT'` comparable with `programme`? The answers come from the **data dictionary**, not from the data.
3. **Optimiser.** Two plans at least. Read all 4,000 rows and discard the non-BSIT ones, then sort; or, if an index on `programme` exists, use it to fetch only the BSIT rows. And if an index on `name` exists, the ORDER BY may be free — indexes are already in order. The optimiser picks by *estimated* cost, using statistics from the dictionary. Chapter 21 measures how good those estimates are.
4. **Execution engine.** Runs the chosen plan, asking the storage manager for pages as it goes.

5. **Buffer manager.** Page 7 is already in memory — no disk read. Page 112 is not, so fetch it, and evict something to make room. This is the single biggest determinant of how fast the query is, and you never see it.
6. **File manager.** Page 112 is at byte offset 111×4096 of `university.db`. Read 4,096 bytes.

The point of the exercise. You wrote *what* you wanted. Nothing in your statement said how. Steps 3 to 6 were chosen for you, and the only way to influence them is to change the schema — which is why design, and not cleverness in query writing, is most of this course.

3.3 The Data Dictionary Is Just More Tables

The claim that a DBMS stores the schema in the database is easier to believe once you have queried it.

</> The dictionary, in SQLite

Every SQLite database has a table called `sqlite_master`, and you can select from it like any other:

```
SELECT type, name, tbl_name FROM sqlite_master;
```

```
type   name          tbl_name
-----  ------------  ----------
table  student       student
index  idx_name      student
view   fees_view     student
```

The `sql` column holds the exact text of the CREATE statement that made each object — which is why `.schema` is instant: it is a SELECT, not a reconstruction.

Other systems do the same thing under other names: `information_schema` in MySQL and PostgreSQL, `ALL_TABLES` in Oracle. The idea is universal; the spelling is not.

Why this is more than a curiosity

Because the schema is data, every tool that works on data works on the schema. Three consequences you will use:

- **A program can discover a database it has never seen.** Anything that shows you a list of tables — a GUI browser, an admin page, a reporting tool — is selecting from the dictionary.
- **The schema can be backed up with the data**, because it is in the same file.
- **A dictionary query can audit a schema.** In Chapter 8 you will use one to find every table that declares a foreign key, and the point of that exercise is that the declarations are all there and none of them are being enforced.

3.4 The Four Roles

The components are software. These are people, and the distinction that matters is not job title — in a small organisation one person is all four — but *which decisions belong to which hat.*

Who decides what

The **database administrator** (DBA) owns the running system: installation, users and privileges, backups, recovery, performance, storage. The DBA decides *how it runs.*

The **database designer** owns the schema: entities, relationships, constraints, normalisation. The designer decides *what is true*.

The **application programmer** owns the programs that use the database. They decide *how it is reached*.

The **end user** owns the questions. They decide *what it is for*, and they are the only role in the list who need not know that any of the others exist.

</> One request, four reactions

"Students should be able to see their own marks on the portal, but not anybody else's."

End user	States the requirement, including the half of it that is a prohibition. Notices first when it is wrong.
Designer	Is there a reliable way to tell whose marks these are? Does ENROLMENT(roll_no, section_id, grade) tie every mark to exactly one student? If not, the requirement cannot be met at any price.
DBA	Creates the portal's database user and grants it the minimum: read on a view, and nothing on the base table.
Programmer	Writes the query with the logged-in roll number as a *parameter*, never as pasted-in text — the difference between those two is the whole of SQL injection.

Where it breaks. If the designer gets it wrong, nobody downstream can fix it: a mark not tied to one student cannot be shown to one student. This is why design comes before everything else in this course, and why Chapter 5 through Chapter 8 are four chapters in which you write almost no SQL.

⚠ In this course you are three of the four, because SQLite has no DBA

SQLite has no users, no privileges, no server to administer and no process to tune. Several things on the DBA's list simply do not exist, and the rest move onto somebody else's:

- **Users and privileges:** nobody's. The capability is absent. File permissions are the only access control there is, and Chapter 23 is about what you get back when you move to a server.
- **Backup:** yours, and it is `copy university.db` — which is genuinely all it takes, and is Chapter 22's subject once you ask what happens if you copy it mid-write.
- **Performance:** yours. `ANALYZE` and `EXPLAIN QUERY PLAN` are the whole toolkit, and Chapter 21 uses both.
- **Schema design, and writing the programs:** yours, which is the point — you will hold the designer's hat and the programmer's hat at the same time, and the labs are arranged so you can feel the conflict.

So: when this handout says "the DBA would", read it as "on a real system somebody else would, and this term it is you, with one hand tied".

! Common Pitfalls

- **Thinking the optimiser can be instructed.** In SQL you say what, never how. If the plan is bad, change the schema or the statistics — not your tone of voice.
- **Confusing the data dictionary with the data.** The dictionary holds the schema. `SELECT COUNT(*) FROM sqlite_master` counts tables, not students.

- **Assuming the roles are four people.** They are four sets of decisions. In this course they are all you.
- **Believing "the application will check it"** is equivalent to a constraint in the schema. It is not, and Chapter 8 is eight pages on why.
- **Expecting an authorisation manager in SQLite.** There is none. Open the file and you have everything.

Try It Yourself: Query the dictionary, and watch the optimiser change its mind

```
-- --- 1. Build something for the optimiser to think about -------
sqlite3 inside.db
CREATE TABLE student (
  roll_no   TEXT PRIMARY KEY,
  name      TEXT NOT NULL,
  programme TEXT NOT NULL
);
-- 300 rows, generated by a recursive CTE (explained in Ch 15):
INSERT INTO student
WITH RECURSIVE n(i) AS (SELECT 1 UNION ALL SELECT i+1 FROM n WHERE i<300)
SELECT printf('2021-IT-%03d', i),
       printf('Student %03d', i),
       CASE i%3 WHEN 0 THEN 'BSIT' WHEN 1 THEN 'BSCS' ELSE 'BSSE' END
FROM n;

-- --- 2. Read the dictionary ------------------------------------
SELECT type, name FROM sqlite_master;
SELECT sql FROM sqlite_master WHERE name = 'student';

-- --- 3. Ask the optimiser what it intends to do ----------------
EXPLAIN QUERY PLAN
  SELECT name FROM student WHERE programme = 'BSIT' ORDER BY name;
-- Write the plan down exactly. Note the words SCAN and ORDER BY.

-- --- 4. Give it an index and ask again -------------------------
CREATE INDEX idx_prog ON student(programme);
EXPLAIN QUERY PLAN
  SELECT name FROM student WHERE programme = 'BSIT' ORDER BY name;
-- The plan changed and you did not touch the query.

-- --- 5. Make the sort free too ---------------------------------
CREATE INDEX idx_name ON student(name);
EXPLAIN QUERY PLAN SELECT name FROM student ORDER BY name;
-- Does "USE TEMP B-TREE FOR ORDER BY" still appear?

-- --- 6. Look at what you have added to the dictionary ----------
SELECT type, name, tbl_name FROM sqlite_master;
-- Three new rows you never inserted.
```

What to notice. In steps 3 to 5 the statement was identical and the plan was not. Everything that changed, changed in the schema. Keep the three plans — Chapter 21 asks you to compare them against measured times, and one of them will turn out to be a worse idea than it looks.

Checkpoint — Test Yourself

1. A query runs in 40 milliseconds on Monday and 4 seconds on Friday, on the same data and with no change to the statement or the schema. Name two components that could be responsible and say how you would tell them apart.

2. The data dictionary is stored as tables in the database it describes. Give one real advantage of that and one problem it creates.
3. For each of the four roles, state one decision that is theirs alone, and then say what happens to that decision in this course — where SQLite provides no DBA at all.

Chapter Summary

- A DBMS is several cooperating components: a **query processor** (parser, optimiser, execution engine), a **storage manager** (buffer manager, file manager), a **transaction manager** with lock and recovery managers, an **authorisation manager**, and the **data dictionary**.
- A statement travels down through them. You specify *what*; steps 3 onward — plan, pages, bytes — are chosen for you.
- The parser consults the dictionary to check names and types; the optimiser consults it for sizes and indexes.
- The **data dictionary** is ordinary tables. In SQLite it is `sqlite_master`, and `.schema` is a SELECT over it.
- Because the schema is data, programs can discover a database they have never seen, and a query can audit a schema.
- Four roles, defined by their decisions: the **DBA** decides how it runs, the **designer** decides what is true, the **programmer** decides how it is reached, the **end user** decides what it is for.
- A designer's error cannot be repaired downstream, which is why design precedes SQL in this course.
- SQLite has no authorisation manager and no DBA role. Backup, performance and design all land on you, and Chapter 23 is the chapter about what a server gives back.

Review Questions

1. (MCQ) Which component decides whether to use an index? (a) the parser; (b) the buffer manager; (c) the query optimiser; (d) the application programmer.

2. (MCQ) `sqlite_master` contains: (a) the rows of every table; (b) the schema of the database; (c) the query log; (d) the list of users.

3. (Short) Explain why the parser needs the data dictionary, with a specific example of a statement it would reject because of something it found there.

4. (Short) What does the buffer manager do, and why can it make the same query take very different times on identical data?

5. (Short) Which of the four roles is responsible when a database permits two students to have the same roll number? Justify your answer in two sentences.

6. (Applied) Take the statement `SELECT s.name, e.grade FROM student s JOIN enrolment e ON s.roll_no = e.roll_no WHERE e.grade = 'A';` and walk it through all six components, saying at each step what information that component needs and where it gets it.

7. (Applied) A college wants one database used by the examinations office, the fees office and 4,000 students. Divide the work among the four roles: list what each must do before the system opens, then mark each item that SQLite cannot support and say what you would do about it.

Chapter 4

Your First Database, in One Command

Learning Outcomes

By the end of this chapter you will be able to:

- **Create** a database, a table and a row, and **read** the row back, without installing anything.
- **Read** a relation schema written in the standard notation and **write** the CREATE TABLE that implements it.
- **State** what a database costs on disk, and **explain** why the cost goes up in jumps rather than smoothly.
- **Use** the eight dot-commands you will need every week.
- **Account for** the gap between an empty database at 0 bytes and one table at 8,192 bytes.

Before You Start

Chapter 3: the components, and the data dictionary. Appendix A for the toolkit — if `sqlite3` does not start, read that first and then come back.

Key Terms in This Chapter

CREATE TABLE • INSERT • SELECT • relation schema • column type • PRIMARY KEY • NOT NULL • dot-command • page • page size • VACUUM • database file

4.1 Three Statements and a File

Everything in Chapter 1 through Chapter 3 was description. This is the chapter where you have a database.

```
sqlite3 university.db
```

That is the installation, the server start-up, the account creation and the connection, all of it. There was no download. Then:

```
CREATE TABLE student (
  roll_no   TEXT    PRIMARY KEY,
  name      TEXT    NOT NULL,
  programme TEXT    NOT NULL,
  semester  INTEGER NOT NULL
);

INSERT INTO student VALUES ('2021-IT-001','Ayesha Khan','BSIT',2);

SELECT name FROM student WHERE roll_no = '2021-IT-001';
```

```
Ayesha Khan
```

You have now used all three levels of Chapter 2 — you wrote the conceptual schema, SQLite built the internal one, and the SELECT was an external view of one column of one row — and all six components of Chapter 3 ran. It took under a tenth of a second, and the rest of this chapter is about what that tenth of a second and that file actually cost.

4.2 Writing the Schema Down Before You Type It

The CREATE TABLE above is SQL. The thing it implements has a shorter notation, used in every textbook and in the rest of this handout:

Relation schema notation

A **relation schema** is written as the relation name followed by its attributes in brackets, with the primary key underlined:

STUDENT(roll_no, name, programme, semester)

It records the *names* and the *key* and nothing else — no types, no lengths, no storage. It is the conceptual level in one line, and it is what you write on paper while designing, before any SQL exists.

From notation to SQL, and the three things the notation loses

STUDENT(roll_no, name, programme, semester) becomes the CREATE TABLE on the previous page. Going in that direction you must *add* three decisions the notation does not carry:

- **A type for each attribute.** `roll_no` is `TEXT`, not `INTEGER`, because `2021-IT-001` is not a number — it has hyphens and leading zeros, and you will never do arithmetic on it. Choosing `INTEGER` here is the single most common first-week mistake.
- **Which attributes may be absent.** NOT NULL on three of them. Chapter 14 is about how much trouble the fourth kind — the ones that may be missing — causes.
- **The key, enforced.** The underline is a claim; PRIMARY KEY is the enforcement. Chapter 8 is about the difference between those two, and about how often the gap between them is where the bugs are.

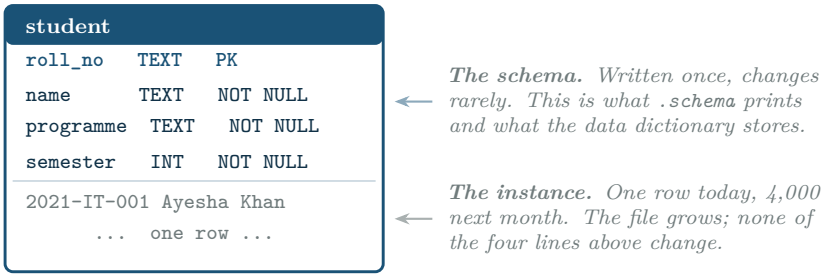


Figure 4.1: One table, with Chapter 2's schema/instance distinction drawn on it. The box above the rule is the schema and is four lines long whether the table holds one row or a million.

4.3 What It Actually Cost

A claim that a tool is lightweight is worth nothing unless somebody weighs it. The harness `code/m04_firstdatabase.py` does the three statements above and measures them.

Measured: From nothing to a row, and what the file weighs

SQLite 3.51.2, Python 3.13.13, Intel i5-6600 (4 cores, 3.30 GHz), 16 GB RAM, Windows 11, mechanical disk.

1. Time from no database at all to a row read back

`connect`	0.52 ms	the file does not exist yet
`create`	25.27 ms	one CREATE TABLE
`insert`	23.80 ms	one row
`select`	0.16 ms	read it back
TOTAL	**49.75 ms**	→ `'Ayesha Khan'`

No server was started, no port opened, no password set, nothing installed.

2. Bytes on disk

an empty database, just connected	0
after one CREATE TABLE, no rows	8,192
1 row	8,192
100 rows	8,192
1,000 rows	32,768
10,000 rows	262,144
the same 10,000 rows after VACUUM	262,144

3. Where the growth happens (page size 4,096 bytes)

at row	bytes	pages	bytes/row so far
1	8,192	2	8192
181	16,384	4	91
351	20,480	5	58

Read the three results, in order.

- **An empty database is 0 bytes.** Connecting does not write anything. The file appears when the first statement that changes something commits — which is already Chapter 22's subject showing through.
- **The first table costs 8,192 bytes, and so does the hundredth row.** Two pages: one for SQLite's own header and `sqlite_master` — the data dictionary from Chapter 3, which has to live somewhere — and one for your table. That second page then holds about 180 of these rows for nothing.
- **The file grows in jumps of exactly 4,096 bytes**, at rows 1, 181 and 351. It does not creep up per row. The cost of storage is quantised, and the unit is the page.
- VACUUM **recovered nothing**, because nothing had been deleted. It is not a compressor; it reclaims space that deletions freed. Students run it hopefully and it is worth knowing now that hope is not the mechanism.

On re-running this. The total time varies: repeated runs on this machine gave 49.75 ms and 70.38 ms. Nothing changed but the operating system's disk cache — which is the buffer manager's neighbour from Chapter 3 making itself felt in week four. **The bytes do not vary at all.** When you report a measurement, say which kind you have: the timings are a range, the page counts are a fact.

Why 8,192 bytes for an empty table is the right answer

It looks like waste — 8 KB to store nothing. It is the price of being a database rather than a file, and it buys exactly two things:

- **The dictionary.** Page 1 holds the schema. Because of it, any program can open `university.db` and discover what is in it. The three CSV files in Chapter 1 had no such page, which is precisely why a program could not tell what they meant.

- **Room to grow in place.** Page 2 is mostly empty on purpose. Rows 2 through 180 cost nothing because the space is already there, and this is why inserting is fast.

At 10,000 rows the overhead has stopped mattering: 262,144 bytes is about 26 bytes a row, and the rows themselves are about that size. **The fixed cost is paid once and the marginal cost is the data.** That is the shape you want from a storage system, and Chapter 21 is the chapter where you deliberately pay the fixed cost a second time to make reads fast.

⚠ The comparison that makes "lightweight" mean something

"Lightweight" is only a claim about something else. Counting what each arrangement requires before the first row exists:

	SQLite	**a server**	
installation steps	0	several	download, install, configure
background processes	0	≥ 1	a service that must be running
network ports open	0	≥ 1	3306, 5432, 1433 …
accounts to create	0	≥ 1	starting with the superuser
config files to edit	0	≥ 1	before anything is tuned
things that can be **misconfigured**	**0**	**all of the above**	

Every zero in the left column is a thing that cannot go wrong in week one — and also **a thing you will not learn to do this term**. That is the trade, stated plainly: you get sixteen weeks on data and none on administration. Chapter 23 is where the right-hand column is taught by contrast, and where you write the GRANT statements that SQLite will not run.

4.4 The Eight Dot-Commands

Dot-commands are instructions to the `sqlite3` shell, not SQL. They take no semicolon. There are about sixty; you will use these eight all term.

</> The working set

`.open university.db`	open a database from inside the shell
`.tables`	list the tables
`.schema student`	print the CREATE TABLE text, exactly as stored
`.headers on`	put column names above the output — **set this first, always**
`.mode box`	draw results as a table instead of pipe-separated text
`.read v1_create.sql`	run a file of SQL
`.once out.txt`	send the next result to a file
`.quit`	leave

`.headers on` and `.mode box` together are the difference between output you can read and output you cannot. Appendix A has the full list and the two-line reset procedure for when a lab database gets into a state you do not want to debug.

Common Pitfalls

- **A semicolon after a dot-command.** `.tables;` is an error. SQL takes a semicolon; dot-commands do not.
- **Forgetting the semicolon after SQL.** The shell prints `...>` and waits. It is not frozen — it wants the rest of your statement.
- **Making `roll_no` an `INTEGER`.** `2021-IT-001` is text. This one costs a schema rewrite in week six.
- **Expecting VACUUM to shrink a database.** It reclaims deleted space only. With no deletions it recovers nothing, as measured above.
- **Running `sqlite3` with no filename.** You get a database in memory, you can work in it happily for an hour, and it vanishes on `.quit` with no warning.
- **Judging a database's size by the file alone.** 8,192 bytes held 100 rows and also 1. The file size tells you how many pages were allocated, not how much data there is.

Try It Yourself: Build the first table of the database you will keep all term

This is the beginning of the semester project. Keep `university.db` — every lab from here adds to this one file, and Chapter 23 submits it.

```
# --- 1. Create it, in the folder you will keep all term --------------
sqlite3 university.db
```

```
-- --- 2. Make the output readable before anything else -----------
.headers on
.mode box

-- --- 3. The first table -----------------------------------------
CREATE TABLE student (
  roll_no   TEXT    PRIMARY KEY,
  name      TEXT    NOT NULL,
  programme TEXT    NOT NULL,
  semester  INTEGER NOT NULL
);

-- --- 4. Four rows -----------------------------------------------
INSERT INTO student VALUES
  ('2021-IT-001','Ayesha Khan','BSIT',2),
  ('2021-IT-002','Bilal Ahmed','BSIT',2),
  ('2021-CS-007','Hira Nadeem','BSCS',4),
  ('2021-SE-013','Usman Tariq','BSSE',2);

-- --- 5. Read them back ------------------------------------------
SELECT * FROM student;
SELECT name, programme FROM student WHERE semester = 2;

-- --- 6. Prove the key is enforced -------------------------------
INSERT INTO student VALUES ('2021-IT-001','Someone Else','BSIT',2);
-- Write down the exact error message. This is Chapter 8 arriving early.

-- --- 7. Leave the shell -----------------------------------------
.quit
```

```
# --- 8. Measure your own machine ---------------------------------
# the file size, and the pages it is made of
ls -l university.db
sqlite3 university.db "PRAGMA page_size; PRAGMA page_count;"
python code/m04_firstdatabase.py
```

Record and keep. Your own total milliseconds, your own page size, and your own three growth points. They will not match the measured box exactly — different disk, different cache — and **the ones that should match exactly are the byte counts**. If your page size is not 4,096, say so when you hand in, because every size figure in this handout scales by it.

Checkpoint — Test Yourself

1. An empty database is 0 bytes and one empty table is 8,192. Account for every one of those bytes — what is in them, and why could it not be fewer?
2. The file was 8,192 bytes at 100 rows and 32,768 at 1,000. Predict the size at 2,000 rows, show the arithmetic, then run it and explain any difference between your prediction and the result.
3. Write INSTRUCTOR(emp_id, name, department, designation) as a CREATE TABLE, choosing a type for each attribute, deciding which may be NULL, and justifying both choices for `emp_id` in one sentence each.

Chapter Summary

- `sqlite3 university.db` is the whole of installation, start-up and connection. Three statements — CREATE TABLE, INSERT, SELECT — are a working database.
- A **relation schema** is written STUDENT(roll_no, name, programme, semester): name, attributes, key underlined. Turning it into SQL means adding types, nullability and real key enforcement.
- Measured: **49.75 ms** from nothing to a row read back; an **empty database is 0 bytes**; one CREATE TABLE costs **8,192 bytes** and then holds about 180 rows free; 10,000 rows is **262,144 bytes**, about 26 bytes a row.
- The file grows in **4,096-byte page jumps**, at rows 1, 181 and 351 — not smoothly. Storage is quantised and the unit is the page.
- The 8 KB fixed cost buys the data dictionary and room to grow in place. It is paid once; after that the marginal cost is the data.
- VACUUM reclaims space freed by deletions. With no deletions it recovers nothing.
- Timings vary with the OS disk cache (49.75 vs 70.38 ms on the same machine); byte counts do not vary. Report which kind of number you have.
- "Lightweight" means zero installation steps, processes, ports, accounts and config files — and therefore zero of those to misconfigure, and zero of them learned this term.
- Eight dot-commands cover the course. Set `.headers on` and `.mode box` first, every session.

Review Questions

1. (MCQ) A newly connected, empty SQLite database occupies: (a) 0 bytes; (b) 4,096 bytes; (c) 8,192 bytes; (d) one page per table.

2. (MCQ) VACUUM on a database from which nothing has been deleted: (a) halves the file; (b) compresses the rows; (c) recovers nothing; (d) rebuilds the indexes only.

3. (Short) Why is `roll_no` declared `TEXT` and not `INTEGER`? Give two reasons, one about correctness and one about what you would eventually want to do with the value.

4. (Short) Explain why the file size jumped at row 181 and not at row 150 or row 200.

5. **(Short)** The same harness gave 49.75 ms and 70.38 ms on the same machine with the same data. Is one of them wrong? Answer in two sentences, saying what you would report.

6. **(Applied)** Write the CREATE TABLE for COURSE(course_code, title, credit_hours, department), then predict its empty size in bytes and the row count at which it will first grow. Run it, record both, and explain any difference between prediction and measurement.

7. **(Applied)** A colleague proposes storing the university's data in one table of 40 columns "to avoid joins". Using the page measurements in this chapter, estimate what that costs in bytes at 10,000 students compared with the four-column table above, and then give the argument against it that is *not* about bytes — Chapter 18 will measure both halves of this answer.

Part II

Designing It

Before a database can be built it has to be decided, and the deciding is done on paper. Part II is the design half of the course: what things exist, how they relate, and which of those relationships the database will be made to guarantee. HEC's outline puts this after normalisation. It belongs here, because you cannot normalise a schema you have not designed.

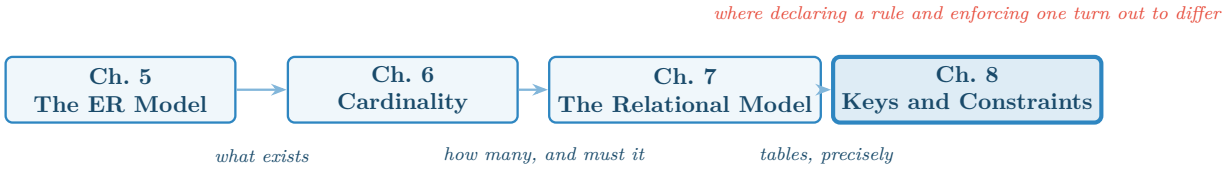


Figure P2.1: Reading path through Part II. Chapter 6 draws the university database this whole course uses; Chapter 8 is the first measured chapter in which the database does something you did not ask for.

Chapter 5

The Entity-Relationship Model

Learning Outcomes

By the end of this chapter you will be able to:

- **Distinguish** an entity, an entity type and an entity set, and **say** which of the three a table corresponds to.
- **Classify** an attribute as simple or composite, single- or multi-valued, stored or derived, and **explain** why each distinction changes what you build.
- **Identify** superkeys, candidate keys and the primary key of an entity type, and **justify** the choice between candidates.
- **Read and draw** a Chen ER diagram, and **state** the degree of a relationship.
- **Decide**, for a given noun in a requirements statement, whether it is an entity, an attribute or a relationship.

Before You Start

Chapter 3: the designer's job — deciding *what is true*. Chapter 4 for the () notation. No SQL is written in this chapter; you will not touch `university.db` until Chapter 9.

Key Terms in This Chapter

entity • entity type • entity set • attribute • simple attribute • composite attribute • single-valued • multi-valued • stored attribute • derived attribute • null value • key attribute • superkey • candidate key • primary key • composite key • surrogate key • relationship • relationship type • degree • binary • ternary • recursive relationship • role name • ER diagram • Chen notation

5.1 Design Happens on Paper

You can create a table in eleven milliseconds (Chapter 4). That is exactly the problem: it is so cheap to make a table that people make the wrong ones, and a wrong table is discovered in week fourteen when four programs already read it.

The entity-relationship model is a way of arguing about a design before any of it is expensive. It was proposed by Peter Chen in 1976 and it has outlived every database product of that decade, for one reason: it is about the *university*, not about the database. Everything in this chapter could be checked by the examinations officer, who has never heard of SQL.

HEC's outline puts this chapter after normalisation. This handout does not.

The published course content lists *normalization, functional dependencies, normal forms* before *entity relationship model, entity sets, . . . , entity-relationship diagrams.* Taken literally, that normalises a schema before designing one, which cannot be done — normalisation (Chapter 18) takes an existing set of tables as its input.

All four of HEC's own recommended textbooks order it the other way. So does this handout: design first (Part II), map it to tables (Part III), and only then check the result for faults

(Part V). Every clause is still covered; the Course Specification records the reordering and this is the one place it is visible.

5.2 Entities: the Things

Entity, entity type, entity set

An **entity** is one thing in the world that can be told apart from other things: the student Ayesha Khan, the course CS-103, Tuesday's section of it.

An **entity type** is the *description* of a kind of thing — the name STUDENT together with the attributes every student has.

An **entity set** is all the entities of that type at one moment — the 4,000 students enrolled today.

This is Chapter 2's schema/instance distinction wearing different words. The entity type is schema; the entity set is instance. A table corresponds to the **entity type**; its rows are the entity set.

Three words people use for one thing, and when it matters

Term	Example	Changes how often
Entity	Ayesha Khan, roll 2021-IT-001	constantly
Entity set	today's 4,000 students	every admission and graduation
Entity type	STUDENT(roll_no, name, programme, semester)	once a year, by argument

In conversation "student" means all three and no harm is done. In an examination answer it must be the right one, and in a design meeting the confusion is expensive: "students change all the time" is true of the set and false of the type, and the two sentences lead to different databases.

5.3 Attributes, and the Four Distinctions That Matter

An **attribute** is a property of an entity type. Students have a name and a roll number. That much is obvious; the four ways attributes differ are not, and each one changes what you have to build.

Four distinctions

Simple or composite. A **simple** (atomic) attribute cannot usefully be divided: `semester`. A **composite** attribute has meaningful parts: `address` = house, street, city, postcode.

Single-valued or multi-valued. `date_of_birth` has exactly one value per student. `phone` may have several. A **multi-valued** attribute is drawn with a double ellipse — and it is the one kind that *cannot become a column*, which is Chapter 9's problem.

Stored or derived. `date_of_birth` is **stored**; `age` is **derived** from it and today's date. A derived attribute is drawn with a dashed ellipse and is normally *not* stored at all.

Key or non-key. A **key attribute** has a different value for every entity in the set. It is underlined.

A fifth property cuts across all four: an attribute may have no value for some entity, which is the **null** of Chapter 14 and a much deeper problem than it looks.

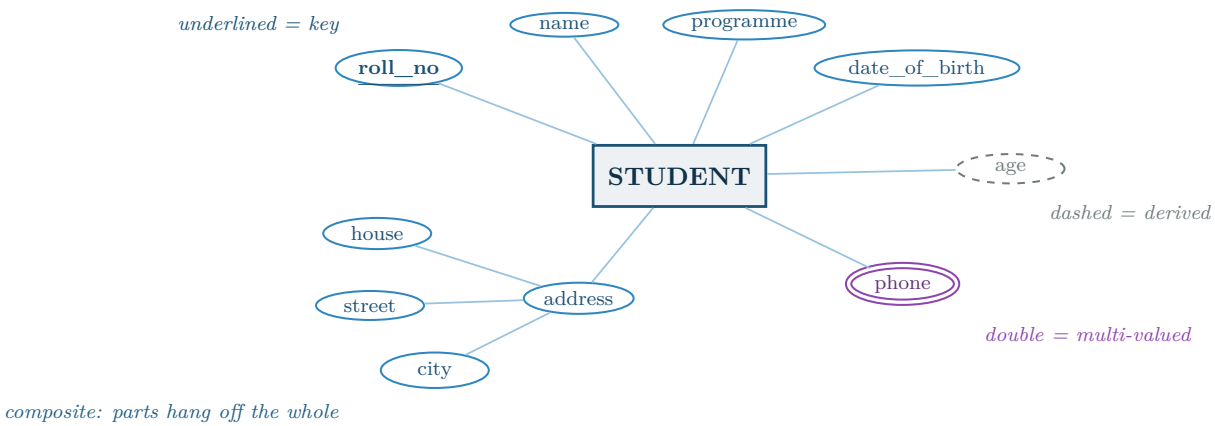


Figure 5.1: STUDENT in Chen notation. Four of the five attribute kinds are present. `phone` and `age` are the two that will not survive contact with a table, and Chapter 9 says what happens to each.

Why the distinctions are not decoration

Each one has a consequence you will meet within three chapters.

- **Composite.** Store the parts, not the whole. A system that stores `address` as one text field can never answer "how many students live in Bahawalpur", and finding out costs a schema change after the data exists — which is Chapter 11.
- **Multi-valued.** A column cannot hold three phone numbers. Not "should not": *cannot*, if you want to search them. It becomes its own table, and that is a rule in Chapter 9.
- **Derived.** Storing `age` means storing a fact that becomes wrong overnight, silently, for about a quarter of the rows each year. Store the birth date and compute.
- **Key.** Everything else in the design hangs off this choice, and the next section is about how to make it.

5.4 Keys: Three Words, One Choice

Superkey, candidate key, primary key

A **superkey** is any set of attributes whose values are different for every entity. {`roll_no`} is one; so is {`roll_no`, `name`}, which is a superkey carrying a passenger.

A **candidate key** is a superkey with no attribute to spare: remove any one attribute and it stops being unique. {`roll_no`} is a candidate key; {`roll_no`, `name`} is not.

The **primary key** is the one candidate key the designer picks to identify entities throughout the database. The others are **alternate keys**.

A key made of more than one attribute is **composite**. A key invented by the designer with no meaning in the world — an auto-numbered id — is a **surrogate key**.

Worked Example: Finding every key of STUDENT, and choosing one

Attributes: `roll_no`, `cnic`, `name`, `programme`, `semester`, `email`.

Step 1 — what the university guarantees. Not what today's data happens to show. Ask, and write the answers down:

- Roll numbers are unique and never reissued. *Guaranteed.*
- CNICs are unique. *Guaranteed* — but a first-semester student may not have one yet.

- Email addresses are issued one per student. *Guaranteed* while enrolled; reused after five years.
- Names are not unique. Two students called Muhammad Ali is normal.

Step 2 — candidate keys. Three: {`roll_no`}, {`cnic`}, {`email`}. {`name`, `programme`, `semester`} is *not* one — two BSIT second-semester students can share a name, and "it has not happened yet" is not a guarantee.

Step 3 — choose, on four grounds.

	always present?	never changes?	short?	verdict
`roll_no`	yes	yes	yes	**primary key**
`cnic`	**no**	yes	yes	alternate, and nullable
`email`	yes	**no**	no	alternate at best

The answer. STUDENT(roll_no, cnic, name, programme, semester, email), with `cnic` and `email` declared unique but not primary.

What decided it. Not uniqueness — all three are unique. The primary key must also be *always present* and *never changing*, because every other table will refer to it (Chapter 8). `cnic` fails the first test and `email` fails the second, and a primary key that can change is a repair job spread across every table that referenced it.

Three things students believe about keys, and what is wrong with each

- **"The data is unique, so it is a key."** A key is a *constraint on all future data*, not an observation about today's. Check the current data by all means — a duplicate proves it is not a key — but no duplicate proves nothing.
- **"Add an auto-number id to every table and the problem goes away."** The surrogate makes rows addressable; it does not make them *distinguishable*. With an id and no other constraint you can insert the same student twice and the database is content. Surrogates are useful — and whenever you use one, the real candidate key still has to be declared unique beside it.
- **"A composite key means bad design."** It usually means the entity genuinely is identified by a combination. An enrolment is identified by *which student* and *which section*, and the pair is the honest key — Chapter 6 and Chapter 9 both rely on it.

5.5 Relationships, and Degree

Relationship, relationship type, degree

A **relationship** is an association among entities: Ayesha is enrolled in Tuesday's section of CS-103.

A **relationship type** is the kind of association — ENROLS, between STUDENT and SECTION. It is drawn as a diamond.

The **degree** of a relationship type is how many entity types take part: **unary** (1), **binary** (2), **ternary** (3). Almost everything is binary.

A **recursive** relationship is a unary one — an entity type related to itself, like a course being a prerequisite for another course. The two ends need **role names** to tell them apart, because "course" appears twice.

A relationship type may have attributes of its own. `grade` belongs to the *enrolment*, not to the student and not to the section.

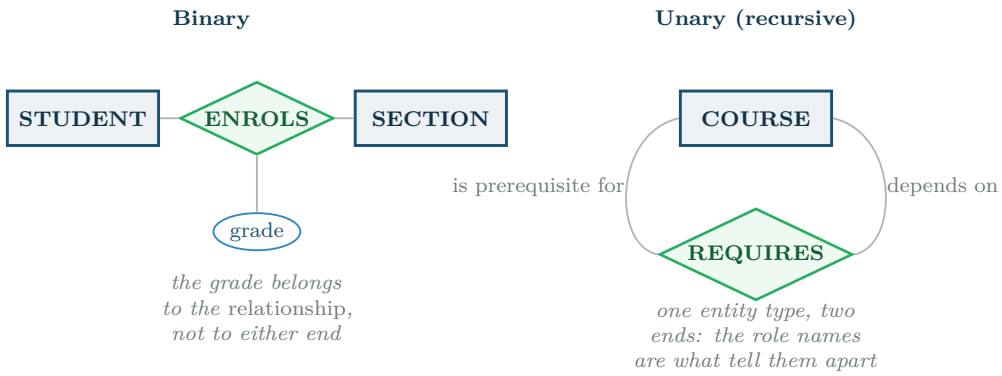


Figure 5.2: Two of the three degrees. ENROLS carries an attribute of its own; REQUIRES joins COURSE to itself and is unreadable without role names. The university database you are building contains both.

</> Entity, attribute or relationship? The question that decides a design

Every noun in a requirements document is one of the three, and getting it wrong is the most common design error there is. Three tests, applied in order:

- **Does it have properties of its own?** If you can list two or three facts *about* it, it is an entity.
- **Is there more than one of it per owner?** If yes, it is not an attribute.
- **Does it only exist because two other things met?** Then it is a relationship.

Noun	Verdict	Why
roll number	attribute	no properties of its own
department	**entity**	it has a name, a head, a building — and fifty courses point at it
grade	**relationship attribute**	exists only because a student met a section
prerequisite	**relationship**	joins course to course
semester	it depends	an attribute of STUDENT if it means "which year they are in"; an *entity* if the university runs Fall 2026 and Spring 2027 as distinct things with dates

The last row is the honest one. The same word is an attribute in one university and an entity in another, and no rule decides it — the *requirement* does. That is why design is a conversation with the examinations officer and not a procedure.

Common Pitfalls

- **Making an entity out of every noun.** "Name" is not an entity. Apply the three tests.
- **Hanging a relationship's attribute on an entity.** Putting `grade` on STUDENT gives each student one grade for the whole degree. It belongs on ENROLS.
- **Choosing a primary key that can change.** Email, phone, name. Every table that referenced it has to be repaired with it.
- **Treating today's data as a guarantee.** No duplicate names today does not make `name` a key.
- **Drawing a recursive relationship without role names.** The diagram is then genuinely ambiguous — nobody can tell which end is the prerequisite.

- **Storing a derived attribute because it is convenient.** `age` is wrong for a quarter of your rows within a year, and nothing reports it.

Try It Yourself: Turn a page of English into an ER diagram

No database this week. Paper, and one argument.

```
# --- 1. The requirement, as the examinations officer states it -------

"Every student has a roll number, a name, a CNIC, one or more phone
 numbers, a postal address, and a date of birth. Students belong to
 one programme. A course has a code, a title and a number of credit
 hours, and belongs to a department. A course may require other
 courses as prerequisites. Each semester a course runs as one or
 more sections; a section has a number, a room and a timetable slot,
 and is taught by one instructor. Students enrol in sections and
 receive a grade. We need to know each student's age, and how many
 students are in each section."
```

```
# --- 2. List every noun ------------------------------------------
    Write them in one column. There are about twenty.

# --- 3. Classify each one ----------------------------------------
    entity / attribute / relationship, using the three tests.
    Write ONE SENTENCE of justification for each. The sentence is
    the exercise; the label is not.

# --- 4. Mark the attribute kinds ---------------------------------
    composite, multi-valued, derived, key. There is at least one
    of each in the paragraph above.

# --- 5. Find the candidate keys ----------------------------------
    For STUDENT, COURSE and SECTION. For each, say what the
    university guarantees -- not what sample data shows.

# --- 6. Draw it, Chen notation -----------------------------------
    Rectangles, ellipses, diamonds. Role names on the recursive
    relationship. Do NOT draw cardinalities yet; that is Chapter 6.

# --- 7. Find the two traps ---------------------------------------
    Two things in the paragraph must NOT become columns, and one
    must not be stored at all. Name all three and say why.

# --- 8. Hand it to somebody else ---------------------------------
    Give your diagram, without the paragraph, to another student.
    Ask them to read it back as English. Every sentence they get
    wrong is a defect in your diagram, not in their reading.
```

What to notice. Step 8 is the one that matters and the one people skip. An ER diagram's whole purpose is to be checked by somebody who cannot read SQL. If it cannot be read back, it has failed at the only job it has — and it is still cheap to fix, which it will not be in Chapter 9.

Keep this diagram. Chapter 6 adds cardinalities to it, Chapter 9 turns it into tables, and it is part of your project submission.

Checkpoint — Test Yourself

1. A colleague proposes `phone` as a column of STUDENT, because "almost every student has one number". Give the argument against in three sentences, and say what you would do instead and what it costs.

2. INSTRUCTOR has `emp_id`, `cnic`, `name`, `email`, `department`, `office_no`. List every candidate key you can justify, state the guarantee each one rests on, choose a primary key and defend the choice on all four grounds in the worked example.
3. Explain, to somebody who has written programs but never designed a database, why `grade` belongs to the *relationship* between a student and a section rather than to either of them. Then say what goes wrong if it is put on STUDENT, and what goes wrong if it is put on SECTION.

Chapter Summary

- An **entity** is one thing; an **entity type** describes a kind of thing; an **entity set** is all of them today. Type is schema, set is instance, and a table corresponds to the type.
- Attributes differ in four ways: simple/**composite**, single-/**multi-valued**, stored/**derived**, key/non-key — and each distinction has a consequence in Part III.
- A **multi-valued** attribute cannot become a column; a **derived** one should not be stored; a **composite** one is stored as its parts.
- A **superkey** is unique; a **candidate key** is unique with nothing to spare; the **primary key** is the candidate the designer picks, and the others are alternate keys.
- Choose a primary key on four grounds: unique, **always present**, **never changing**, short. Uniqueness alone decides nothing when there are three candidates.
- A key is a constraint on *all future data*. A duplicate disproves a key; the absence of one proves nothing.
- A **relationship** associates entities; its **degree** is how many entity types take part. A **recursive** relationship joins a type to itself and needs **role names** to be readable.
- A relationship can carry attributes of its own. `grade` belongs to the enrolment, not to the student and not to the section.
- Every noun is an entity, an attribute or a relationship. The same word can be different things in different universities; the requirement decides, not a rule.
- HEC's clause list puts normalisation before the ER model. This handout reverses the two, because normalisation takes tables as its input and there are none until Part III.

Review Questions

1. (MCQ) A double ellipse in Chen notation marks an attribute that is: (a) derived; (b) composite; (c) multi-valued; (d) a key.

2. (MCQ) {`roll_no`, `name`}, where `roll_no` is unique, is: (a) a candidate key; (b) a superkey but not a candidate key; (c) neither; (d) the primary key.

3. (Short) Give an example of a derived attribute other than age, and say what goes wrong if it is stored.

4. (Short) What is the degree of the REQUIRES relationship between courses, and why does it need role names when a binary relationship usually does not?

5. (Short) A surrogate key is added to a table that already has a natural candidate key. What must still be declared, and what breaks if it is not?

6. (Applied) A library wants to record books, copies of books, members and loans. Decide which of the four are entity types, identify the relationships and their degrees, choose a primary key for each entity type with its justification, and say which attributes of a loan belong to the relationship rather than to either end.

7. **(Applied)** Take the STUDENT diagram in Figure 5.1 and the requirement "we must be able to list every student in a given city, and ring any of a student's numbers". State precisely which parts of the diagram now cannot be implemented as columns, what you would build instead, and what the design would have cost if this requirement had arrived after the database was full.

Chapter 6

Cardinality, Participation and Weak Entities

Learning Outcomes

By the end of this chapter you will be able to:

- **Determine** the cardinality ratio of a relationship — 1:1, 1:N or M:N — by asking two questions in a fixed order.
- **Distinguish** total from partial participation, and **state** what each one forbids.
- **Write** a relationship's constraints in (min, max) notation and **convert** between that and the ratio-plus-participation form.
- **Recognise** a weak entity type, and **identify** its owner, its identifying relationship and its partial key.
- **Read** the same diagram in Chen and crow's-foot notation.

Before You Start

Chapter 5: entities, attributes, keys, relationships and degree. You will extend the diagram you drew in that chapter's laboratory, so have it with you.

Key Terms in This Chapter

cardinality ratio • one-to-one • one-to-many • many-to-many • participation constraint • total participation • partial participation • existence dependency • structural constraint • (min, max) notation • weak entity type • strong entity type • owner entity • identifying relationship • partial key • discriminator • crow's-foot notation

6.1 A Diagram Without Cardinalities Says Almost Nothing

The diagram from Chapter 5 says students enrol in sections. It does not say whether a student may take two sections of the same course, whether a section can run with nobody in it, or whether an instructor must be teaching something. Those are the facts an examinations officer actually argues about, and all three are cardinality or participation.

The two constraints answer different questions and are routinely confused, so they are worth separating before either is defined.

Two different questions

The **cardinality ratio** answers *how many*: given one entity on this side, how many on the other side may it be related to — one, or more than one?

The **participation constraint** answers *must it at all*: can an entity on this side exist without taking part in the relationship?

They are independent. Every combination occurs, and a relationship is not specified until both have been answered for *both* ends — four answers in all.

6.2 Cardinality, by Asking Two Questions

The three ratios

One-to-one (1:1): each entity on either side relates to at most one on the other. A department has one head; a person heads at most one department.

One-to-many (1:N): one entity on the "one" side relates to many on the "many" side, but each of those relates back to at most one. A department offers many courses; each course belongs to one department.

Many-to-many (M:N): entities on both sides may relate to many. A student enrols in many sections; a section holds many students.

The procedure, and it is only two questions

Never guess the ratio. Ask these, in this order, out loud, with a concrete entity in each:

- **"One *student* — how many *sections*?"** One, or many?
- **"One *section* — how many *students*?"** One, or many?

Two answers, four combinations, three ratios:

Q1	Q2	Ratio	Example
one	one	1:1	department — head
many	one	1:N	department — course
one	many	1:N	*the same, read backwards*
many	many	M:N	student — section

Rows 2 and 3 are the same relationship. "1:N" is meaningless until you say which end is the one. Always write it as DEPARTMENT 1—N COURSE, never as a bare "1:N".

The question that catches everybody

"A student enrols in many sections. A section has many students. Therefore M:N." — correct. Now:

*"A student enrols in many **courses**. A course has many students. Therefore M:N."* — also correct, and **it is the wrong relationship**.

A student does not enrol in a course. They enrol in a *section* of a course, in a particular semester, taught by a particular instructor, in a particular room. Model the relationship to COURSE and you can never record which section somebody attended, which instructor taught them, or that they failed in Fall and passed in Spring.

The lesson is not about cardinality. It is that a cardinality question asked about the wrong pair of entity types gives a confident, well-formed, useless answer. Settle what the entities are (Chapter 5) before asking how many.

6.3 Participation: Must It Take Part at All?

Total and partial participation

Total participation of an entity type in a relationship means *every* entity of that type must take part. It is drawn as a double line and is also called an **existence dependency**: a section cannot exist without a course.

Partial participation means some entities may take part and others need not. A single line. A student may be enrolled in nothing — newly admitted, on leave — and is still a student.

Total participation is the stronger claim, and therefore the one to be sceptical about: it forbids something, and somebody will eventually need the thing it forbids.

Total participation is a promise about Tuesday afternoon

"Every section must be taught by an instructor" sounds obviously true, and it is how timetables are published. Then:

- A section is created in June and the instructor is appointed in August. For two months it has none.
- An instructor resigns mid-semester. For a week the section has none.
- A section is cancelled but its enrolment records must be kept.

If participation is total, the database *forbids* all three, and the clerk who meets them invents a fake instructor called "TBA" — which now appears in the staff list, the workload report and the payroll query. A constraint that gets worked around is worse than one that was never there, because the data now lies and nothing says so.

The decision rule. Make participation total when the entity is genuinely meaningless without the relationship — a section without a course is not a thing at all. Make it partial when the relationship is merely *usual*. "Always" and "usually" are different words and the database can tell.

(min, max) notation

Chen's two constraints can be written as one pair on each side. Against each entity type's line, (min, max) — together, the relationship's **structural constraints** — gives the smallest and largest number of relationship instances *one entity of that type* may take part in.

min = 0 is partial participation; min ≥ 1 is total. max = 1 is the "one" side; max = N is the "many" side.

The notation is read against the entity it is nearest, which is the opposite of where the 1 and N labels go in classic Chen notation. This single difference is responsible for most misread diagrams, and is why many examiners insist on one or the other but not both.

Worked Example: The university's four relationships, specified completely

Four answers each: ratio from both sides, participation from both sides.

1. DEPARTMENT **offers** COURSE

- One department — many courses. One course — one department. **1:N.**
- Must every department offer a course? A new department may not yet. **Partial** (0,N).
- Must every course belong to a department? Yes; it is how it is numbered. **Total** (1,1).

⇒ DEPARTMENT (0,N) — OFFERS — (1,1) COURSE

2. INSTRUCTOR **teaches** SECTION

- One instructor — many sections. One section — one instructor (this university's rule; some allow co-teaching, and then it is M:N). **1:N.**
- Must every instructor teach? On sabbatical, no. **Partial** (0,N).
- Must every section have an instructor? See the alert box above: **partial** (0,1), deliberately.

3. STUDENT **enrols in** SECTION

- Many each way. **M:N**, and it carries `grade`.
- Must every student be enrolled in something? No. **Partial** (0,N).
- Must every section have students? A section with nobody in it is cancelled, not impossible. **Partial** (0,N).

4. COURSE **requires** COURSE (recursive)

- A course may require several; a course may be required by several. **M:N**, both ends partial (0,N) — most courses have no prerequisites and are a prerequisite for nothing.

What the four tell you. Three of the four are partial on both ends. Beginners' diagrams are covered in double lines, because every relationship *sounds* compulsory when you describe the normal case. The normal case is not the constraint.

6.4 Weak Entities

Weak entity type, owner, identifying relationship, partial key

A **weak entity type** has no key of its own. Its entities are distinguishable only in combination with the entity that owns them.

The **owner** (or identifying) entity type is the strong one it depends on. The **identifying relationship** joins them, drawn as a double diamond, and the weak entity's participation in it is always *total*.

The **partial key** (or **discriminator**) is the attribute that tells one weak entity from another *belonging to the same owner*. It is underlined with a dashed line.

The full key of a weak entity is **the owner's primary key plus the partial key**.

SECTION is weak, and this is why

Sections are numbered 1, 2, 3 within a course. Section 1 of CS-103 and section 1 of CS-201 are different sections with the same number.

- `section_no` alone is not a key — dozens of sections are numbered 1.
- `section_no` *within one course* is unique. That makes it a **partial key**.
- A section cannot exist without a course. **Total participation** in the identifying relationship.
- Its real key is {`course_code`, `semester`, `section_no`} — the owner's key, plus the semester, plus the discriminator.

SECTION(course_code, semester, section_no, room, slot, emp_id)

The alternative, and why it is not better. You could give SECTION a surrogate `section_id` and make it strong. Many real systems do, and Chapter 9 shows the mechanics. It does not make the entity any less dependent — it only hides the dependency from the key, and the uniqueness of (course, semester, number) must still be declared or the same section can be created twice.

Chen or crow's foot? Both, and here is the translation

Textbooks and examination papers use Chen. Every tool you will meet at work uses crow's foot, because it fits on a screen. They say the same things.

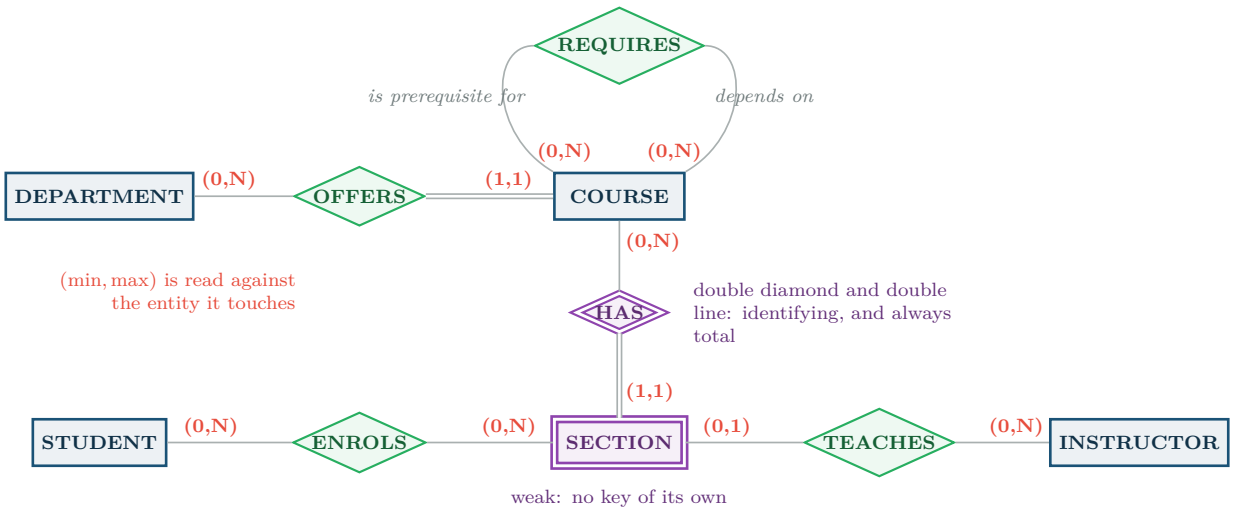


Figure 6.1: The university database, fully constrained. Only two lines in the whole diagram are double — a course must belong to a department and a section must belong to a course. Everything else is partial, which is the usual proportion.

Meaning	Chen	Crow's foot
relationship	a diamond between boxes	a line between boxes
"many" end	N on the line	the three-pronged foot
"one" end	1 on the line	a single cross-stroke
total participation	double line	a cross-stroke nearest the box
partial participation	single line	a circle nearest the box
attributes	ellipses around the box	listed inside the box
weak entity	double rectangle	usually not distinguished
M:N relationship	one diamond	**cannot be drawn** — it is resolved into two 1:N at once

The last row is the real difference. Crow's-foot notation has no way to draw a many-to-many relationship, so a tool forces you to resolve it into a third box before you have decided to. Chen lets you keep the design honest until Chapter 9, where the resolution is a rule with a reason rather than something the drawing program did to you.

Common Pitfalls

- **Writing "1:N" without saying which end is the one.** The label alone is ambiguous; name the entity types.
- **Confusing cardinality with participation.** "Every section has one instructor" states both — one is the ratio, every is the participation — and people hear only the first.
- **Double lines everywhere.** Every relationship sounds compulsory when the normal case is described. Three of this database's four are partial on both ends.
- **Reading** $(\min, \max)$ **against the far entity.** It is read against the entity it touches, which is the opposite of the 1/N labels.
- **Asking the cardinality of the wrong pair.** Student-to-course is M:N and is the wrong relationship; student-to-section is the right one.

- **Giving a weak entity a surrogate key and calling it strong.** The dependency does not go away; it just stops being visible, and the uniqueness of the real key must still be declared.

Try It Yourself: Constrain your diagram, then try to break it

Take the diagram from Chapter 5's laboratory. Still no database.

```
# --- 1. Four answers per relationship -------------------------------
    For EVERY relationship, write all four:
        one <A> -- how many <B>?        one <B> -- how many <A>?
        must every <A> take part?       must every <B> take part?
    Four answers. Not three.

# --- 2. Mark the ratio ----------------------------------------------
    1:1, 1:N or M:N, naming the entity types -- not a bare "1:N".

# --- 3. Mark participation ------------------------------------------
    Double line for total, single for partial.

# --- 4. Rewrite it in (min,max) -------------------------------------
    Every line gets a pair at each end. Check each against step 1;
    they must agree, and if they do not, step 1 was guessed.

# --- 5. Find the weak entity ----------------------------------------
    There is at least one. Name its owner, its identifying
    relationship, its partial key and its full key.

# --- 6. Attack every double line ------------------------------------
    For EACH total participation, invent a real Tuesday on which the
    university needs to break it. If you can, make it partial.
    Write the Tuesday down next to the line; it is the evidence.

# --- 7. Attack every "one" ------------------------------------------
    For each 1 end, ask what happens if the university someday allows
    two. Co-teaching. Joint degrees. Two heads of department.
    Which of these would cost a schema change, and which would not?

# --- 8. Read it back ------------------------------------------------
    Turn the finished diagram into ten English sentences, one per
    constraint. Give them to the person who has the paragraph from
    Chapter 5's lab. Every disagreement is a decision you had not
    actually made.
```

What to notice. Steps 6 and 7 usually remove more constraints than they confirm, and that is the expected result rather than a sign you did it wrong. A design is not improved by forbidding more; it is improved by forbidding exactly what the university actually forbids.

Checkpoint — Test Yourself

1. A relationship is drawn with a double line at the SECTION end of TEACHES. State in one sentence what the database will refuse to do, and give a realistic situation in which somebody needs it to do exactly that.
2. ENROLMENT has $(0, N)$ at the STUDENT end and $(0, N)$ at the SECTION end. Rewrite this as a cardinality ratio plus two participation constraints, and say what would change if the SECTION end became $(1, N)$.
3. Explain why SECTION is a weak entity type, name its partial key and its full key, and then say what you would have to declare instead if you chose to give it a surrogate `section_id` — and what breaks if you forget.

Chapter Summary

- A relationship needs **four** answers: the ratio from each side and the participation from each side. Fewer than four is an unspecified relationship.
- The **cardinality ratio** is found by two questions — "one A, how many B?" and "one B, how many A?" — giving 1:1, 1:N or M:N. Always name which end is the one.
- **Total participation** (double line) means every entity must take part; **partial** means it need not. Total is the stronger claim and deserves the scepticism.
- A constraint that gets worked around is worse than no constraint: the data then contains a fake row and nothing marks it as fake.
- $(\min, \max)$ carries both facts at once: $\min = 0$ is partial, $\min \geq 1$ is total, $\max = 1$ is the one side. **It is read against the entity it touches**, unlike the 1/N labels.
- A **weak entity type** has no key of its own. It has an **owner**, an **identifying relationship** (double diamond, always total) and a **partial key**; its full key is the owner's key plus the partial key.
- SECTION is weak: section 1 exists in every course, and only (course, semester, number) identifies one.
- Crow's-foot notation says the same things more compactly but **cannot draw an M:N relationship**, so a tool resolves it for you before you have decided to.
- Asking a cardinality question about the wrong pair of entity types gives a confident and useless answer. Student-to-*course* is the classic case.

Review Questions

1. (MCQ) A double line between an entity type and a relationship means: (a) many-to-many; (b) total participation; (c) a weak entity; (d) an identifying relationship.

2. (MCQ) In $(\min, \max)$ notation, $(1, 1)$ against COURSE on the OFFERS line means: (a) a department offers exactly one course; (b) a course belongs to exactly one department; (c) there is one department; (d) the relationship is 1:1.

3. (Short) Give an example of a 1:1 relationship in a university and say which end you would make total, with a reason.

4. (Short) Why is the participation of a weak entity type in its identifying relationship always total? Answer from the definition of a weak entity.

5. (Short) A tool that draws crow's-foot diagrams will not let you record a many-to-many relationship. What does it make you do instead, and what is lost by doing it at that moment rather than in Chapter 9?

6. (Applied) A hospital records patients, doctors, appointments and prescriptions. Give all four answers for every relationship you identify, state which entity types are weak and why, and defend each total participation with a specific situation that would break it if you were wrong.

7. (Applied) Your university decides to allow co-teaching: two instructors may share a section. State exactly which constraint in Figure 6.1 changes, what the relationship becomes, what new attributes it may now need, and — looking ahead to Chapter 9 — why this change is cheap on paper and expensive once the database is full.

Chapter 7

The Relational Model

Learning Outcomes

By the end of this chapter you will be able to:

- **Define** relation, tuple, attribute, domain, degree and cardinality, and **use** each word correctly.
- **State** the four properties of a relation and **show** which of them a SQL table does not have.
- **Distinguish** a relation schema from a relation instance, and **write** both in the standard notation.
- **Identify** superkeys, candidate keys and the primary key of a relation from its data *and* its guarantees.
- **Explain** what a null value is, what it is not, and why it is the model's weakest point.

Before You Start

Chapter 5 for keys — the definitions are the same ones, restated for relations. Chapter 4 for the () notation and for `sqlite3`, which you will use at the end to break one of this chapter's rules on purpose.

Key Terms in This Chapter

relational data model • relation • tuple • attribute • domain • atomic value • relation schema • relation instance • degree • arity • cardinality • relational database schema • superkey • candidate key • primary key • foreign key • null value • unknown • inapplicable

7.1 One Idea, and It Is Mathematics

Codd's 1970 paper proposed that all data in a database be represented one way: as relations. Not as files, not as linked records, not as trees. One structure, with a mathematical definition, so that questions about data could be answered by reasoning rather than by knowing how the file was laid out.

That is the whole of the relational model, and the reason every system in Chapter 1's list — Oracle, PostgreSQL, MySQL, SQL Server, SQLite — is a different implementation of the same five pages of theory.

Domain, attribute, tuple, relation

A **domain** is a named set of **atomic** values — values the model treats as indivisible. `semester` might have domain $\{1, 2, \ldots, 8\}$; `roll_no` the set of strings matching the university's pattern.

An **attribute** is a name for the role a domain plays in a relation. Two attributes may share a domain: `start_year` and `end_year` are both years and are not the same attribute.

A **tuple** is an ordered list of values, one from each attribute's domain. Informally, a row.

A **relation** is a *set* of tuples over the same attributes. Informally, a table — and the gap between "informally" and "exactly" is section 7.3.

The vocabulary on one small relation

STUDENT(roll_no, name, programme, semester)

roll_no	name	programme	semester
2021-IT-001	Ayesha Khan	BSIT	2
2021-IT-002	Bilal Ahmed	BSIT	2
2021-CS-007	Hira Nadeem	BSCS	4

- The **relation schema** is the heading: the name plus the four attributes. It is written STUDENT(roll_no, name, programme, semester).
- The **relation instance** is the three tuples. Tomorrow it is different; the schema is not.
- The **degree** (or **arity**) is **4** — the number of attributes. It changes only when the schema changes.
- The **cardinality** is **3** — the number of tuples. It changes every time somebody enrols.
- The **domain** of `semester` is the integers 1 to 8. The value 2 appears twice and that is fine; domains are not keys.

A **relational database schema** is the set of all the relation schemas in one database — *student*, *course*, *section* and the rest, taken together.

Degree and cardinality are easy to swap by accident. Degree counts columns and is a property of the schema; cardinality counts rows and is a property of the instance. "A relation of degree 4 and cardinality 3" names the table above exactly.

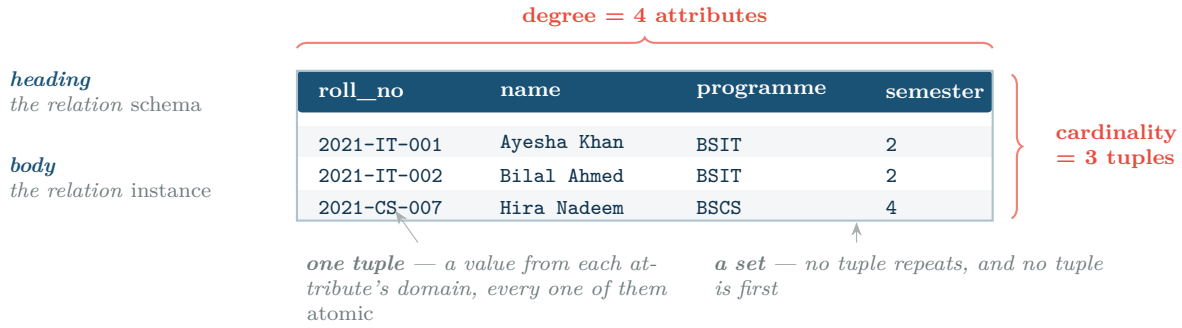


Figure 7.1: One relation, with every word of the vocabulary attached to the thing it names. Degree belongs to the heading and changes only when the schema does; cardinality belongs to the body and changes all day.

7.2 The Four Properties of a Relation

What makes a relation a relation

1. **No duplicate tuples.** A relation is a *set*. The same tuple cannot appear twice — which is another way of saying every relation has a key, if only the whole tuple.
2. **Tuples are unordered.** There is no first row. Two relations with the same tuples in a different sequence are the same relation.
3. **Attributes are unordered.** Attributes are referred to by name, not position. `(name, roll_no)` and `(roll_no, name)` are the same heading.
4. **Every value is atomic.** One value per attribute per tuple: no lists, no nested tables, no "three phone numbers in one field".

Why each property is worth having

These are not arbitrary tidiness. Each one buys something specific.

- **No duplicates** means every fact is stored once and can be addressed. Two identical rows cannot be told apart, so neither can be updated or deleted individually — which is the file-based system's redundancy problem (Chapter 1) reappearing inside the database.
- **Unordered tuples** is what lets the optimiser (Chapter 3) choose any plan it likes. If row order were part of the meaning, no index could ever be used and Chapter 21 would not exist.
- **Unordered attributes** is what lets you add a column without breaking programs — the logical data independence of Chapter 2, and the reason SELECT * is a bad habit: it depends on an order the model says is not there.
- **Atomic values** is what makes the query language finite. If a field could hold a list, every query would need loops, and relational algebra (Chapter 12) could not be written down. This property is also **first normal form**, arrived at from the other direction in Chapter 18.

A SQL table is not a relation, and the difference is measurable

SQL was designed around the relational model and departs from it in three places. You should know all three, because each one is a bug waiting to happen.

Relation	SQL table	What follows
no duplicate tuples	**duplicates allowed** unless a key is declared	a table without a key is not a relation at all
tuples unordered	ORDER BY imposes one, and without it the order is arbitrary	never rely on the order you happen to see
values atomic	mostly enforced, but a text column will hold `"0300-1234567, 0321-9876543"` quite happily	the model cannot stop you; only you can

The first is the serious one and you can demonstrate it in four lines:

```
CREATE TABLE t (a TEXT, b TEXT);        -- no key declared
INSERT INTO t VALUES ('x','y'), ('x','y');
SELECT COUNT(*) FROM t;                 -- 2
DELETE FROM t WHERE a='x' AND b='y';    -- deletes BOTH
```

Two identical rows, and no WHERE clause in the language can address one of them, because there is nothing that distinguishes them. **This is why every table in this course declares a primary key**, and Chapter 8 is about making the declaration real.

7.3 Keys, Restated for Relations

The definitions are Chapter 5's, which is the point: the ER model and the relational model agree about keys, and that agreement is what makes Chapter 9 possible.

Keys of a relation

A **superkey** is a set of attributes K such that no two tuples of any legal instance agree on all of K.

A **candidate key** is a minimal superkey: no proper subset of it is a superkey.

The **primary key** is the chosen candidate key. Its attributes may not be null — an identifier that is missing identifies nothing.

A **foreign key** is a set of attributes in one relation whose values must appear as the primary key of another (or be null). It is how relations are joined to each other, and it is Chapter 8's subject.

Worked Example: Reading keys off an instance, and why you cannot

roll_no	name	prog	sem	email
2021-IT-001	Ayesha Khan	BSIT	2	ayesha@iub.pk
2021-IT-002	Bilal Ahmed	BSIT	2	bilal@iub.pk
2021-CS-007	Hira Nadeem	BSCS	4	hira@iub.pk

Step 1 — what is unique in this instance? All of `roll_no`, `name`, `email`; also {`prog`, `sem`} is *not* — rows 1 and 2 agree.

Step 2 — which of those are superkeys? This is where the exercise stops being mechanical. A superkey is a constraint on *every legal instance*, and three rows cannot establish one.

- `roll_no`: the university guarantees it. **Superkey.**
- `email`: issued one per student. **Superkey** — on that guarantee, not on these rows.
- `name`: unique *here*. The university guarantees nothing. **Not a superkey**, and the data cannot tell you so.

Step 3 — minimality. {`roll_no`} and {`email`} are both minimal, so both are candidate keys. {`roll_no`, `sem`} is a superkey and not minimal.

Step 4 — choose. `roll_no`, on Chapter 5's four grounds: unique, always present, never changing, short. `email` fails "never changing".

The asymmetry worth remembering. An instance can *disprove* a key instantly — one duplicate and it is settled. It can never prove one. So: check the data to find candidates, then ask a human to confirm each, and record the answer. The examination version of this question gives you the guarantees; the real version does not, and you have to go and ask.

7.4 Null, and Why It Is the Awkward Part

Null

A **null** is a marker in a tuple meaning that no value from the domain is recorded for that attribute. It is **not** zero, **not** the empty string, and **not** a value of any kind — it is the absence of one.

Codd distinguished two reasons for a null and most systems, including SQLite, distinguish neither:

- **unknown** — the student has a CNIC, we do not know it;
- **inapplicable** — the student has no CNIC, because they are fifteen.

One marker, two meanings, and no way to tell them apart in a query.

Three nulls in one row, meaning three different things

Attribute	Value	What the null means
`cnic`	NULL	unknown — they have one, it is not on file
`thesis_title`	NULL	inapplicable — second-semester students do not write one
`grade`	NULL	not yet — the semester is not over

All three print as blank and all three behave identically in every comparison. A query that counts "students with no CNIC on file" will silently include anybody whose CNIC was inapplicable, and nothing in the database records that the two cases were different.

What a designer does about it. Where the distinction matters, make it explicit: a separate `cnic_status` attribute, or a different relation for thesis students. Where it does not, allow the null and write the ambiguity down. The one thing not to do is invent a magic value — `'N/A'`, `0000-0000000`, `9999-12-31` — which is a null that the database does not know is a null, and which will be counted, averaged and sorted as though it were real.

Null is ahead of you in three later chapters

This chapter only names the problem. It returns, each time worse:

- Chapter 14: `WHERE grade = NULL` matches nothing, ever — not even the rows where grade is null. Comparison with null is neither true nor false but *unknown*, and SQL's logic has three values.
- Chapter 16: COUNT(*) and COUNT(GRADE) give different answers on the same rows, and AVG silently ignores nulls rather than treating them as zero — which is usually right and occasionally catastrophic.
- Chapter 13: a row whose foreign key is null disappears from an inner join, so the section with no instructor vanishes from the timetable report and nobody notices until a Tuesday.

Nulls are the single largest source of quietly wrong answers in SQL. Every one you allow in a schema is a decision; allow them deliberately.

Common Pitfalls

- **Saying "table" when the question asks about a relation.** In an examination the distinction is the answer: a table may have duplicates and an order.
- **Swapping degree and cardinality.** Degree counts attributes, cardinality counts tuples.
- **Relying on row order.** Without ORDER BY there is none, and the order you see today comes from the plan, which Chapter 21 will change.
- **Reading keys off sample data.** Data disproves keys; it never proves them.
- **Treating null as zero or as the empty string.** It is neither, and the arithmetic will show it.
- **Inventing a magic value instead of a null.** It gets averaged.
- **Creating a table with no primary key.** It is not a relation, and you have given up the ability to address a row.

Try It Yourself: Break three properties of a relation, deliberately

```
# --- 1. A scratch database; keep university.db clean ----------------
sqlite3 relmodel.db
```

```
-- --- 2. A table that is not a relation -------------------------
```

```
.headers on
.mode box
CREATE TABLE nokey (roll_no TEXT, name TEXT);
INSERT INTO nokey VALUES ('2021-IT-001','Ayesha Khan');
INSERT INTO nokey VALUES ('2021-IT-001','Ayesha Khan');
SELECT COUNT(*) FROM nokey;
-- Property 1 broken. Now try to delete exactly one of them.
DELETE FROM nokey WHERE roll_no = '2021-IT-001' AND name = 'Ayesha Khan';
SELECT COUNT(*) FROM nokey;
-- Write down how many rows went, and say why you could not keep one.

-- --- 3. The same table, as a relation ----------------------------
CREATE TABLE withkey (roll_no TEXT PRIMARY KEY, name TEXT NOT NULL);
INSERT INTO withkey VALUES ('2021-IT-001','Ayesha Khan');
INSERT INTO withkey VALUES ('2021-IT-001','Ayesha Khan');
-- Record the error. One keyword is the whole difference.

-- --- 4. Prove tuples are unordered -------------------------------
INSERT INTO withkey VALUES ('2021-CS-007','Hira Nadeem');
INSERT INTO withkey VALUES ('2021-IT-002','Bilal Ahmed');
SELECT * FROM withkey;
CREATE INDEX idx_nm ON withkey(name);
SELECT * FROM withkey;
-- Same relation, same query. Did the order change? Either way,
-- say what you are entitled to assume about it.

-- --- 5. Break atomicity, which nothing will stop you doing ------
CREATE TABLE badphone (roll_no TEXT PRIMARY KEY, phones TEXT);
INSERT INTO badphone VALUES
  ('2021-IT-001','0300-1234567, 0321-9876543');
-- Now answer: which students have a number starting 0321?
SELECT roll_no FROM badphone WHERE phones LIKE '%0321%';
-- It appears to work. Now explain, in one sentence, why
-- '0321' in the MIDDLE of a number would also match, and what
-- you would have to write to do this correctly.

-- --- 6. Three nulls, one appearance ------------------------------
CREATE TABLE nulls (roll_no TEXT PRIMARY KEY, cnic TEXT, grade TEXT);
INSERT INTO nulls VALUES ('2021-IT-001', NULL, NULL);
SELECT * FROM nulls WHERE cnic IS NULL;
SELECT * FROM nulls WHERE cnic = NULL;
-- Two queries, one of them returns nothing. Which, and WHY?
-- This is the whole of Chapter 14 in two lines; write your
-- explanation down now and check it when you get there.
```

What to notice. Step 2 is the one to keep. Two identical rows, and the language has no way to refer to one of them — not because SQLite is deficient, but because there is genuinely nothing that tells them apart. Every primary key you declare for the rest of this course is buying your way out of that position.

Checkpoint — Test Yourself

1. A relation has degree 5 and cardinality 200. Say precisely what each number counts, which of the two can change without a schema change, and what operation changes the other.
2. Give a specific query that returns a different answer depending on whether duplicate tuples are permitted, and say what you would declare to make the answer reliable.

3. `grade` is null for a student who has not sat the examination and also for one whose paper was lost. Explain why the database cannot distinguish them, give two different designs that could, and say what each design costs.

Chapter Summary

- A **domain** is a set of atomic values; an **attribute** names a domain's role; a **tuple** picks one value per attribute; a **relation** is a *set* of tuples.
- **Degree** counts attributes and belongs to the schema; **cardinality** counts tuples and belongs to the instance.
- Four properties: **no duplicate tuples**, **tuples unordered**, **attributes unordered**, **values atomic**.
- Each property buys something: addressability, the optimiser's freedom, logical data independence, and a finite query language.
- **A SQL table is not a relation.** It permits duplicates unless a key is declared, exposes an order, and cannot stop you putting a list in a text column.
- A table with no primary key gives up the ability to address a row: two identical rows cannot be told apart by any WHERE clause.
- **Superkey**, **candidate key** and **primary key** mean in the relational model exactly what they meant in the ER model, which is what makes Chapter 9 mechanical.
- An instance can **disprove** a key but never prove one. Use the data to find candidates and a human to confirm them.
- A **null** marks the absence of a value. It is not zero and not the empty string, it conflates *unknown* with *inapplicable*, and it is the source of quietly wrong answers in Chapters 13, 14 and 16.
- Never substitute a magic value for a null. It will be counted, averaged and sorted as though it were real.

Review Questions

1. (MCQ) The number of attributes in a relation is its: (a) cardinality; (b) degree; (c) domain; (d) arity of its key.

2. (MCQ) Which property of a relation does a SQL table most clearly fail to have? (a) attributes unordered; (b) values atomic; (c) no duplicate tuples; (d) a named schema.

3. (Short) Two attributes share the domain "year". Explain why they are still two different attributes, with an example.

4. (Short) Why can an instance disprove a candidate key but never establish one? Illustrate with `name`.

5. (Short) State the difference between `WHERE cnic IS NULL` and `WHERE cnic = NULL`, and say which returns rows.

6. (Applied) You are given a table of 50,000 rows and told only that "it is the student list". Describe the queries you would run to propose a primary key, say exactly what each result would and would not establish, and state what you would have to ask a person before declaring one.

7. (Applied) A system stores a student's phone numbers in one comma-separated text column. List four distinct questions the university cannot reliably answer in that design, show the relation you would replace it with, and explain which of the four properties of a relation the original violated.

Chapter 8

Keys and Integrity Constraints

Learning Outcomes

By the end of this chapter you will be able to:

- **State** the three model-based integrity constraints — domain, entity and referential — and **give** an example of each being violated.
- **Define** a foreign key and **explain** what referential integrity forbids.
- **Choose** an appropriate referential action (CASCADE, RESTRICT, SET NULL) and **justify** the choice.
- **Demonstrate** that a declared constraint and an enforced constraint are not the same thing.
- **Verify** on any SQLite database whether its foreign keys are actually being enforced.

Before You Start

Chapter 7: relations, keys, nulls. Chapter 6 for participation, which is the constraint this chapter implements. You will need `sqlite3` and `code/m08_constraints.py`.

Key Terms in This Chapter

integrity constraint • domain constraint • entity integrity • referential integrity • foreign key • referencing relation • referenced relation • orphan row • referential action • CASCADE • RESTRICT • SET NULL • semantic constraint • CHECK • PRAGMA • declared versus enforced

8.1 Three Constraints the Model Gives You Free

A schema is not only a list of attributes. It is a list of *promises*, and the front matter of this handout claims that a database is nothing but a set of promises about data. This is the chapter where that is made literal.

The three model-based constraints

A **domain constraint** says every value of an attribute comes from its domain. `semester` is an integer between 1 and 8; `'spring'` is not a semester and neither is −3.

Entity integrity says no attribute of a primary key may be null. An identifier that is missing identifies nothing, so the tuple could not be addressed.

Referential integrity says that if a tuple refers to another relation, the tuple it refers to must exist. A section claiming to belong to course CS-999 when there is no CS-999 is an **orphan**.

These three come from the model itself. A fourth kind, **semantic** constraints, come from the university — "a student may not enrol in more than six sections" — and have to be stated case by case.

One row, three ways to be wrong

Into SECTION(course_code, section_no, room, emp_id):

The row	Violates	Why it matters
`('CS-103','two','R7')`	domain	`section_no` is an integer; sorting and counting now fail
`(NULL,1,'R7')`	entity integrity	part of the primary key is missing, so the row cannot be addressed
`('CS-999',1,'R7')`	referential integrity	there is no course CS-999; the timetable now lists a section of nothing

All three are refusals a database can make *for you*, on every insert, for ever — which is exactly what the application code in Chapter 1 could not do, because each program had to remember to check and one of them always forgot.

8.2 Foreign Keys

Foreign key

A **foreign key** is a set of attributes FK in a **referencing** relation R whose values must either

- appear as the primary key of some tuple in the **referenced** relation S, or
- be entirely null.

The null case is not a loophole — it is Chapter 6's partial participation. A section with no instructor yet has a null `emp_id`, and that is a design decision you already took.

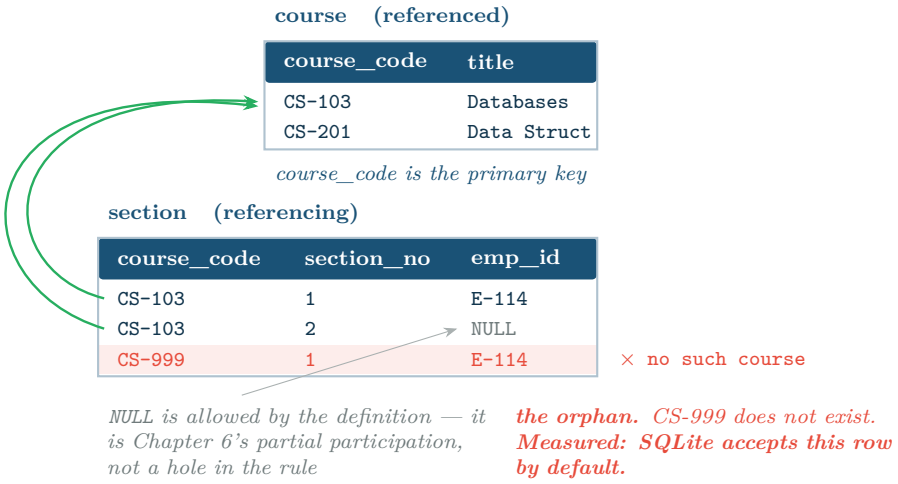


Figure 8.1: Referential integrity. Two of the three child rows point at a course that exists; the null is permitted by the definition; the third is an orphan. The whole of this chapter's measurement is that the third row goes in.

Where foreign keys come from

You do not invent them. Each one is a Chapter 6 constraint written down in a different notation, and Chapter 9 turns one into the other mechanically.

- **A 1:N relationship** puts a foreign key on the *many* side, pointing at the one. COURSE → DEPARTMENT.
- **Total participation** makes that foreign key NOT NULL. Partial participation leaves it nullable.
- **A weak entity** gets a foreign key to its owner, and that foreign key is part of its own primary key.
- **An M:N relationship** has no foreign key, because it has no table yet. It becomes one in Chapter 9, with two.

So the four answers you argued about in Chapter 6 are not an academic exercise: each one lands as a specific line of CREATE TABLE, and you can check your diagram against your schema by reading them back.

Referential actions

Referential integrity can be broken from the *other* end: delete the course and its sections become orphans. A **referential action** says what the database should do about it.

Action	On deleting the parent	Use when
RESTRICT / NO ACTION	refuse the delete	the default, and the right answer when you are unsure
CASCADE	delete the children too	the child is meaningless alone — a weak entity
SET NULL	null the children's foreign key	the child survives without the parent, and the column is nullable

The same three apply ON UPDATE, for when a primary key value changes — which is one more reason Chapter 5 said to choose a key that never changes.

CASCADE is a loaded weapon

ON DELETE CASCADE is attractive because it makes a tidy-up problem disappear. It also means that one DELETE statement can remove rows from tables you did not name, in tables you did not think about, through chains you did not draw.

Delete a department with cascade all the way down and you remove its courses, their sections, every enrolment in them, and therefore the grade history of students who graduated years ago. One statement, no warning, and the transaction commits.

The rule used in this course: cascade only to a *weak* entity — one that genuinely cannot exist without its owner — and use RESTRICT everywhere else. A refused delete is an inconvenience; a successful one is sometimes a disaster, and only one of the two is reversible.

8.3 Declared Is Not Enforced

Everything above is standard and true of every relational system. What follows is specific to the tool on your machine, and it is the most important single fact in this chapter.

Measured: A correct REFERENCES clause that does nothing

`code/m08_constraints.py`, SQLite 3.51.2, Intel i5-6600, 16 GB, Windows 11.

1. The same schema, the same INSERT, two connections

PRAGMA foreign_keys	orphans in	outcome
default (untouched)	1	**accepted**
ON	0	refused: `FOREIGN KEY constraint failed`

The REFERENCES clause is identical in both. Only the connection differs.

2. The setting is per connection, not per database

connection A, after PRAGMA foreign_keys = ON	reports 1
connection A, orphan insert	refused
connection B, *same file*, nothing set	reports 0
connection B, orphan insert	**accepted**

Nothing in the file remembers that enforcement was ever wanted.

3. A PRIMARY KEY column that stores a null

declared as	INSERT NULL	stored	rows
`TEXT PRIMARY KEY`	accepted	**null**	1
`INTEGER PRIMARY KEY`	accepted	integer	1
`TEXT PRIMARY KEY`, STRICT	refused	—	0
`TEXT PRIMARY KEY NOT NULL`	refused	—	0

Read the *stored* column, not the first one. `INTEGER PRIMARY KEY` accepted the null and stored an integer — it is an alias for the rowid, so the null was replaced, and no constraint was broken. `TEXT PRIMARY KEY` accepted it and **stored null**, which breaks entity integrity — and that is the declaration every table in this course uses.

4. What enforcement costs (20,000 valid child rows, 5 runs)

	median	min	max
foreign_keys OFF	58.7 ms	55.1	63.8
foreign_keys ON	73.0 ms	67.3	79.6
cost of enforcement	**14.3 ms**	**(+24%)**	

Read the four results together. The declaration is accepted, stored, and printed back by `.schema` — and by default enforces nothing. It costs 24% on bulk inserts to turn on, which is a real cost and a small one: note that the slowest run with enforcement on (79.6 ms) is only 25% worse than the slowest without it, and that the two ranges do not overlap, so the 24% is a result rather than noise. **A single run of this measurement put the two settings 0.6 ms apart**, which would have been reported as 3%. Repeating it is not a nicety.

⚠ What this means for everything you write from here on

PRAGMA `foreign_keys` has been off by default since foreign keys were added to SQLite in 2009, and it will stay off: turning it on would break existing databases that have been quietly accumulating orphans.

The consequences you have to live with:

- **Set it on every connection**, as the first statement, before any transaction. It is a no-op inside one.
- **Your lab partner's connection is not yours.** A database can be enforced in your program and unenforced in theirs, at the same moment, on the same file.
- **Orphans already in the file are not detected** when you turn it on. Enforcement applies to *new* statements only. `PRAGMA foreign_key_check` is how you find the old ones, and step 7 of the laboratory runs it.

- **A schema that looks correct may never have been enforced.** When you are handed an unfamiliar SQLite database, this is the first thing to check and the last thing anybody remembers.

This is not a criticism of SQLite — the behaviour is documented and the reason is sound. It is a demonstration of something general: **a constraint has two separate lives, one in the schema and one at run time, and only the second one protects anything.**

</> The university schema, with its promises declared

```
PRAGMA foreign_keys = ON;        -- first. every time.

CREATE TABLE department (
  dept_code TEXT PRIMARY KEY NOT NULL,
  name      TEXT NOT NULL
) STRICT;
```

A course belongs to exactly one department, and must:

```
CREATE TABLE course (
  course_code  TEXT PRIMARY KEY NOT NULL,
  title        TEXT NOT NULL,
  credit_hours INTEGER NOT NULL CHECK (credit_hours BETWEEN 1 AND 4),
  dept_code    TEXT NOT NULL              -- total participation
               REFERENCES department(dept_code)
               ON DELETE RESTRICT         -- a department with courses stays
) STRICT;
```

A section is the weak entity of Chapter 6, so its owner's key is part of its own and the cascade is correct here and nowhere else:

```
CREATE TABLE section (
  course_code TEXT    NOT NULL
              REFERENCES course(course_code)
              ON DELETE CASCADE,          -- weak entity: cascade is correct
  semester    TEXT    NOT NULL,
  section_no  INTEGER NOT NULL CHECK (section_no > 0),
  room        TEXT,
  emp_id      TEXT                        -- nullable: partial participation
              REFERENCES instructor(emp_id)
              ON DELETE SET NULL,         -- the section survives; the
  PRIMARY KEY (course_code, semester, section_no)   -- teacher changed
) STRICT;
```

Every line is a decision you have already made. `NOT NULL` on `dept_code` is the total participation from Chapter 6's worked example; the nullable `emp_id` is the partial participation argued for in the same place; the three-part primary key is the weak entity's owner key plus discriminator; and each of the three referential actions follows the rule in this chapter. Nothing here is a matter of taste.

Common Pitfalls

- **Assuming REFERENCES does something.** By default, in SQLite, it does not. Check, every time.
- **Setting the pragma once and assuming it stuck.** It is per connection and lasts until that connection closes.
- **Setting it inside a transaction.** It is silently ignored there.
- **Declaring `TEXT PRIMARY KEY` without NOT NULL.** It will store a null, as measured. Add NOT NULL or STRICT, and this course does both.

- **Cascading beyond a weak entity.** One delete, a chain you did not draw, and a committed transaction.
- **Believing the application will check it.** Three programs, one of them written next year by somebody else. This is Chapter 1's whole argument.
- **Turning enforcement on and assuming the old data is clean.** It applies to new statements only; run `PRAGMA foreign_key_check`.

Try It Yourself: Insert an orphan into a correct schema, then make it impossible

```
# --- 1. A scratch database ------------------------------------------
sqlite3 fk.db
```

```
-- --- 2. A schema that declares referential integrity ------------
.headers on
.mode box
CREATE TABLE course (
  course_code TEXT PRIMARY KEY NOT NULL,
  title       TEXT NOT NULL
) STRICT;
CREATE TABLE section (
  course_code TEXT    NOT NULL REFERENCES course(course_code),
  section_no  INTEGER NOT NULL,
  room        TEXT,
  PRIMARY KEY (course_code, section_no)
) STRICT;
INSERT INTO course VALUES ('CS-103','Database Systems');

-- --- 3. Confirm the declaration is really there -----------------
.schema section
PRAGMA foreign_key_list(section);
-- It is declared. Written down. Reported back to you.

-- --- 4. Now break it --------------------------------------------
PRAGMA foreign_keys;              -- what does it say?
INSERT INTO section VALUES ('CS-999', 1, 'Room 7');
SELECT * FROM section;
-- A section of a course that does not exist. No error.

-- --- 5. Turn enforcement on and try again -----------------------
PRAGMA foreign_keys = ON;
PRAGMA foreign_keys;
INSERT INTO section VALUES ('CS-888', 1, 'Room 8');
-- Record the exact error message.

-- --- 6. The orphan from step 4 is still there -------------------
SELECT * FROM section;
-- Enforcement did not clean up. It never looks backwards.

-- --- 7. Find every orphan already in the file -------------------
PRAGMA foreign_key_check;
-- Learn this one. It is how you audit a database you were handed.

-- --- 8. Prove it is per connection ------------------------------
.quit
```

```
# --- 9. A brand new connection to the SAME file ----------------------
sqlite3 fk.db "PRAGMA foreign_keys;"
sqlite3 fk.db "INSERT INTO section VALUES ('CS-777',1,'R9');"
sqlite3 fk.db "SELECT * FROM section;"
# It went in. Nothing in the file remembered.
```

```
# --- 10. The measured version ---------------------------------------
python code/m08_constraints.py
```

What to notice. Step 3 is the one that makes the point. The constraint is *there* — `PRAGMA foreign_key_list` prints it, the schema stores it, any tool reading the database would report it — and step 4 walks straight past it. From now on, every script you write in this course opens with `PRAGMA foreign_keys = ON;`, and `code/run_all.py` checks that they all do.

Checkpoint — Test Yourself

1. You are handed a SQLite database and told its foreign keys are enforced. List the commands you would run to find out whether that is true *now*, and whether it has ever been true, and say what each result would and would not establish.
2. For each of SECTION → COURSE, SECTION → INSTRUCTOR and ENROLMENT → STUDENT, choose an ON DELETE action and justify it in one sentence. At least one of your three must not be CASCADE.
3. A table is declared `roll_no TEXT PRIMARY KEY`. Explain what value it will accept that it should not, which integrity constraint that violates, and give the two different one-word changes that each fix it.

Chapter Summary

- The model gives three constraints: **domain** (values come from the right set), **entity integrity** (no part of a primary key is null), **referential integrity** (a reference points at something that exists). **Semantic** constraints come from the university.
- A **foreign key** must match a primary key in the referenced relation, *or* be wholly null — and the null case is Chapter 6's partial participation.
- Foreign keys are not invented: a 1:N relationship puts one on the many side, total participation makes it NOT NULL, and a weak entity makes it part of its own key.
- **Referential actions**: RESTRICT by default, CASCADE only to a weak entity, SET NULL when the child survives the parent.
- **Measured:** PRAGMA **foreign_keys is 0 by default.** A correct REFERENCES clause is parsed, stored, reported — and enforces nothing. The orphan went in without an error.
- The setting is **per connection**. Connection B, on the same file, accepted what connection A refused.
- Enforcement **never looks backwards**: orphans already present stay. `PRAGMA foreign_key_check` finds them.
- **Measured: `TEXT PRIMARY KEY` stores a null.** STRICT or NOT NULL fixes it; `INTEGER PRIMARY KEY` only appears to, because it substitutes a rowid.
- **Measured: enforcement costs +24%** on bulk inserts — outside the run-to-run spread, and cheap for what it buys. A single run reported 3%, which is why the harness repeats.
- The general lesson: **a constraint has two lives, one in the schema and one at run time, and only the second protects anything.**

Review Questions

1. (MCQ) Entity integrity is violated when: (a) a foreign key points nowhere; (b) part of a primary key is null; (c) a value is outside its domain; (d) two tables have the same name.

2. **(MCQ)** In SQLite, PRAGMA `foreign_keys` defaults to: (a) ON for STRICT tables; (b) ON; (c) OFF; (d) whatever it was last set to in that file.

3. **(Short)** Give a situation in which ON DELETE SET NULL is the right action and CASCADE would lose data you need.

4. **(Short)** Why is a foreign key allowed to be null at all? Answer in terms of Chapter 6.

5. **(Short)** You turn on foreign key enforcement for a database that has been running for a year. What does it protect you from, and what does it not?

6. **(Applied)** Write the CREATE TABLE statements for INSTRUCTOR and ENROLMENT, with every domain, entity and referential constraint declared and a referential action chosen for each foreign key. Beside each constraint, name the decision from Chapter 5 or Chapter 6 that it implements.

7. **(Applied)** A colleague argues that foreign keys should be left off because they cost 24% on bulk inserts and "the application checks anyway". Write the reply. Use the measured figures, the per-connection result, and Chapter 1's argument, and say under what single condition their position would actually be defensible.

Part III

From Diagram to Schema

A diagram is a drawing until a database agrees to it. Part III is the shortest in the book and the one with the most permanent consequences: the rules for turning a design into tables, the SQL that creates them, and what to do a fortnight later when the design turns out to have been slightly wrong and the tables are already full.

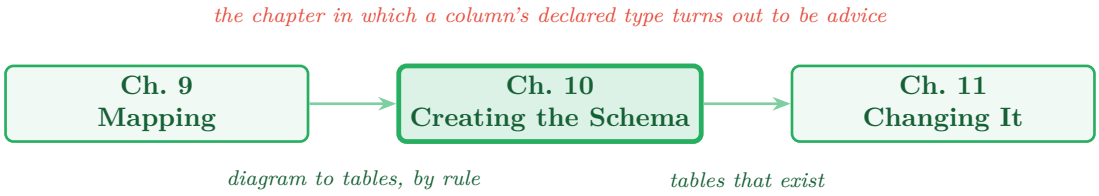


Figure P3.1: Reading path through Part III. Chapter 9 is mechanical and should be: mapping is a set of rules you apply rather than judgements you make. The judgement was Part II and the consequences are Part V.

Chapter 9

Mapping an ER Diagram to Tables

Learning Outcomes

By the end of this chapter you will be able to:

- **Apply** the seven mapping rules, in order, to any ER diagram.
- **Map** a 1:N relationship by placing a foreign key, and **say** which side it goes on and why.
- **Resolve** an M:N relationship into a table, and **determine** its primary key.
- **Map** a weak entity, a multi-valued attribute and a recursive relationship, each of which has its own rule.
- **Check** a finished schema back against the diagram it came from, constraint by constraint.

Before You Start

Chapter 6: the finished, constrained diagram — you will map the one you drew. Chapter 8 for foreign keys and referential actions, which are what the rules produce.

Key Terms in This Chapter

mapping • logical design • strong entity • weak entity • relationship table • junction table • composite primary key • foreign key placement • resolving a many-to-many • surrogate key • recursive relationship • ternary relationship • flattening a composite attribute

9.1 The Part of Design That Is Mechanical

Everything in Part II was judgement: whether a noun is an entity, whether participation is really total, which candidate key to choose. Those decisions were expensive and nobody could make them for you.

This chapter is the opposite. Given a finished diagram, turning it into tables is a *procedure*. Seven rules, applied in order, and two people who follow them get the same schema. That is the point of having spent four chapters on the diagram: the hard thinking is over, and what remains can be checked.

The seven rules, in the order they must be applied

1	Strong entity type	one table; its key becomes the primary key
2	Composite attribute	store the *parts*, not the whole
3	Multi-valued attribute	**a new table**: owner's key + the value, both in the primary key
4	1:1 relationship	a foreign key on the side with *total* participation
5	1:N relationship	a foreign key on the **many** side
6	M:N relationship	**a new table**: both keys, together the primary key, plus the relationship's own attributes
7	Weak entity type	a table whose primary key is the owner's key *plus* the partial key

Order matters. Rules 4–7 place foreign keys, and a foreign key can only point at a primary key that already exists. Do all the entities first.

9.2 Rules 1 to 3: the Entities

Rule 1 — a strong entity becomes a table

STUDENT, with key `roll_no`, becomes

STUDENT(roll_no, name, programme, semester)

```
CREATE TABLE student (
  roll_no   TEXT    PRIMARY KEY NOT NULL,
  name      TEXT    NOT NULL,
  programme TEXT    NOT NULL,
  semester  INTEGER NOT NULL CHECK (semester BETWEEN 1 AND 8)
) STRICT;
```

The CHECK is the domain constraint from Chapter 8, and it came from the ER diagram's domain, not from inspiration.

Rules 2 and 3 — the two attribute kinds that will not fit

Rule 2, composite. `address` = house, street, city. Store the parts:

```
  addr_house  TEXT,
  addr_street TEXT,
  addr_city   TEXT,
```

Three columns, one for each leaf of the ER tree. Never one `address` column — Chapter 5 explained what that costs, and the cost is paid at the moment somebody asks "how many students live in Bahawalpur".

Rule 3, multi-valued. `phone` cannot be a column at all. It becomes its own table:

STUDENT_PHONE(roll_no, phone)

```
CREATE TABLE student_phone (
  roll_no TEXT NOT NULL REFERENCES student(roll_no) ON DELETE CASCADE,
  phone   TEXT NOT NULL,
  PRIMARY KEY (roll_no, phone)
) STRICT;
```

Both attributes are in the primary key, and that is the rule, not a convenience. It says a student may have many numbers and may not have the same number recorded

twice. Make `roll_no` alone the key and each student gets one phone — which is where you started. Leave the key off altogether and Chapter 7's step 2 happens: the same number goes in twice and no DELETE can remove one of them.

The cascade is right here because a phone number belongs to a student in exactly the way Chapter 8 requires: it cannot exist without one.

9.3 Rules 4 to 6: the Relationships

Where a foreign key goes, and why there is no choice

In a **1:N** relationship the foreign key goes on the **many** side, always. The reason is the atomicity of Chapter 7: a course belongs to one department, so one value fits in one column. A department has many courses, and many values do not.

In a **1:1** relationship either side would work, so there is a choice and a rule for making it: **put it on the side with total participation**, so that it can be declared NOT NULL. Put it on the partial side and most of the column is null.

In an **M:N** relationship *neither* side works, and that is why the relationship needs a table of its own.

Worked Example: Rule 6 — resolving ENROLS, and finding its key

STUDENT (0,N) — ENROLS — (0,N) SECTION, carrying `grade`.

Step 1 — why no foreign key will do. Put `section_id` on STUDENT and a student takes one section. Put `roll_no` on SECTION and a section holds one student. Both are wrong, and neither is fixable by choosing the other side.

Step 2 — make a table. It gets the primary key of each participant:

ENROLMENT(roll_no, course_code, semester, section_no, grade)

Four key attributes, not two — because SECTION is weak (Chapter 6) and its own key is already three attributes wide. A weak entity on either end of an M:N relationship makes the junction table wide, and that is the honest consequence of the design rather than a mistake.

Step 3 — the primary key is the combination. All four together. This states exactly one thing, and it is the thing the university means: *a student may appear in a section only once.*

Step 4 — the relationship's attribute comes too. `grade` is not in the key. It is a fact *about* the enrolment, nullable because the semester is not over.

```
CREATE TABLE enrolment (
  roll_no     TEXT    NOT NULL REFERENCES student(roll_no),
  course_code TEXT    NOT NULL,
  semester    TEXT    NOT NULL,
  section_no  INTEGER NOT NULL,
  grade       TEXT    CHECK (grade IN ('A','B','C','D','F')),
  PRIMARY KEY (roll_no, course_code, semester, section_no),
  FOREIGN KEY (course_code, semester, section_no)
    REFERENCES section(course_code, semester, section_no)
) STRICT;
```

Notice the second foreign key is written separately. A foreign key of three columns cannot be declared inline against one column; it needs its own FOREIGN KEY clause naming all three, in the same order as the parent's primary key. This is where a wide weak-entity key is actually felt, and it is the strongest practical argument for giving SECTION a surrogate

`section_id` instead — an argument Chapter 11 makes with a measurement rather than with taste.

The M:N table is not "just a link table"; it is where the facts live

Students meet this table and call it plumbing. It is the most informative table in the database:

- `grade` is there, and nowhere else. Every transcript, every GPA, every pass rate is a query over this table.
- It is the only place that records *when* a student took something.
- It is almost always the largest table — 4,000 students × 5 sections × 8 semesters is 160,000 rows against a few thousand everywhere else, which is why Chapter 21 measures this table and not the others.

If an M:N table has no attributes of its own today, it still will: "when did they enrol", "who approved it", "was it a repeat". Design it as a first class table, because it is one.

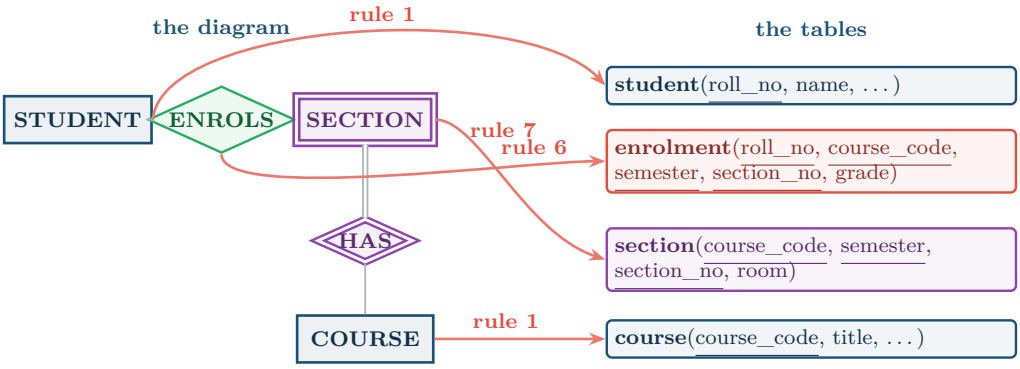


Figure 9.1: Four of the seven rules on the part of the diagram that uses them. Rule 1 makes a table per box; rule 6 turns the diamond into a table of its own; rule 7 gives the weak entity its owner's key. The arrows are the whole of logical design.

9.4 Rule 7, and the Two Awkward Cases

Rule 7 — a weak entity

SECTION is identified by its owner plus its discriminator, so its primary key is the owner's key plus the partial key, and the owner's key is *simultaneously* a foreign key:

```
CREATE TABLE section (
  course_code TEXT    NOT NULL
              REFERENCES course(course_code) ON DELETE CASCADE,
  semester    TEXT    NOT NULL,
  section_no  INTEGER NOT NULL CHECK (section_no > 0),
  room        TEXT,
  emp_id      TEXT REFERENCES instructor(emp_id) ON DELETE SET NULL,
  PRIMARY KEY (course_code, semester, section_no)
) STRICT;
```

Three things are true of `course_code` at once: it is part of the primary key, it is a foreign key, and it is NOT NULL. All three come straight off the diagram — the double line in Figure 6.1 said the participation was total, and CASCADE is correct here for the reason Chapter 8 gave and nowhere else in this schema.

The recursive relationship, and the ternary one

Two cases the seven rules cover, but which look as though they need an eighth.

Recursive (COURSE REQUIRES COURSE, M:N). Rule 6 applies unchanged. The new table takes the key of each participant — and both participants are COURSE, so it takes `course_code` *twice*, under the two role names:

```
CREATE TABLE prerequisite (
  course_code TEXT NOT NULL REFERENCES course(course_code),
  requires    TEXT NOT NULL REFERENCES course(course_code),
  PRIMARY KEY (course_code, requires),
  CHECK (course_code <> requires)          -- nothing requires itself
) STRICT;
```

The role names from Chapter 5 become the column names. Without them you would have two columns called `course_code` and no legal way to name either.

Ternary. A relationship among three entity types becomes one table with three foreign keys, and its primary key is normally all three. Do *not* map it as three binary relationships: "student, section, instructor" as three pairs cannot record which instructor taught which student in which section, only that each pair occurred.

Common Pitfalls

- **Putting the foreign key on the "one" side of a 1:N.** One column cannot hold many values. This is the single most common error.
- **Mapping M:N with a foreign key.** It cannot be done; make the table.
- **Giving the M:N table a surrogate key and nothing else.** An `id` column makes rows addressable; without the composite UNIQUE the same student enrols twice.
- **Making the multi-valued table's key just the owner.** Then the owner has one value again, which is what you were escaping.
- **Storing a composite attribute whole.** Three columns, one per leaf.
- **Declaring a multi-column foreign key inline.** It needs its own FOREIGN KEY clause, with the columns in the parent's key order.
- **Mapping a ternary relationship as three binary ones.** It loses exactly the fact the relationship existed to record.

Try It Yourself: Turn your diagram into the database you will use all term

This lab produces `code/schema/v1_create.sql`, and every later chapter runs against what you build here.

```
# --- 1. Start the file ------------------------------------------------
# Open code/schema/v1_create.sql in an editor. First line, always:
#   PRAGMA foreign_keys = ON;
```

```
-- --- 2. Rule 1: one table per strong entity ----------------------
--     department, course, instructor, student.
--     Primary key first, NOT NULL, STRICT on every table.

-- --- 3. Rules 2 and 3: the awkward attributes --------------------
--     Flatten the composite one into its parts.
--     Give the multi-valued one its own table, with BOTH
--     attributes in the primary key.

-- --- 4. Rule 5: the 1:N foreign keys -----------------------------
--     course -> department.  Which side, and NOT NULL or not?
--     Your Chapter 6 diagram already decided both.
```

```
-- --- 5. Rule 7: the weak entity ---------------------------------
--      section. Owner's key plus discriminator, and the owner's
--      key is a foreign key at the same time.

-- --- 6. Rule 6: the M:N tables ----------------------------------
--      enrolment, and prerequisite (the recursive one).
--      Composite primary keys. Relationship attributes included.

-- --- 7. A referential action for every foreign key --------------
--      There are at least six. CASCADE only to a weak entity.
--      Write a one-line comment on each saying why.
```

```
# --- 8. Create it -----------------------------------------------------
rm -f university.db
sqlite3 university.db ".read code/schema/v1_create.sql"
sqlite3 university.db ".tables"
sqlite3 university.db ".schema section"

# --- 9. Check it against the diagram, constraint by constraint ------
sqlite3 university.db "PRAGMA foreign_key_list(section);"
sqlite3 university.db "PRAGMA foreign_key_list(enrolment);"
sqlite3 university.db "PRAGMA table_info(course);"
# For EVERY line of Figure 6.1, find the line of SQL that implements
# it. Any line of the diagram with no SQL is a mapping you skipped.

# --- 10. Prove the M:N key does its job -------------------------------
sqlite3 university.db "PRAGMA foreign_keys=ON;
  INSERT INTO student (roll_no, name, programme, semester)
    VALUES ('2021-IT-001','Ayesha Khan','BSIT',2);
  INSERT INTO department VALUES ('CS','Computer Science');
  INSERT INTO course VALUES ('CS-103','Database Systems',4,'CS');
  INSERT INTO section VALUES ('CS-103','FALL2026',1,'R7',NULL);
  INSERT INTO enrolment VALUES
    ('2021-IT-001','CS-103','FALL2026',1,NULL);
  INSERT INTO enrolment VALUES
    ('2021-IT-001','CS-103','FALL2026',1,NULL);"
# The second one must fail. Record the message.
```

What to notice. Step 9 is the check that makes mapping a procedure rather than a craft: every line of the diagram should have a line of SQL, and every line of SQL should have a line of the diagram. Lines of SQL with no diagram behind them are decisions you made without noticing, and they are exactly the ones that turn out to be wrong in week twelve.

Checkpoint — Test Yourself

1. A 1:1 relationship joins DEPARTMENT and INSTRUCTOR (the head of department). Say which table gets the foreign key, justify the choice from the participation constraints, and say what the column would look like if you chose the other side.
2. ENROLMENT's primary key has four attributes. Explain what fact about the university each one contributes, and say precisely what becomes possible if you remove `semester` from the key.
3. Map the recursive relationship COURSE REQUIRES COURSE. Give the table, its primary key, its two foreign keys, and one CHECK constraint that the ER diagram does not state but that the university certainly means.

Chapter Summary

- Mapping is **mechanical**. Seven rules, applied in order, and two people get the same schema — which is what four chapters of diagram bought.
- **Entities first** (rules 1–3), then relationships (4–7): a foreign key can only point at a primary key that already exists.
- A **composite** attribute is stored as its parts; a **multi-valued** attribute becomes a table whose primary key is *both* columns.
- **1:N** puts the foreign key on the **many** side, because one column holds one value. **1:1** puts it on the *totally participating* side so it can be NOT NULL.
- **M:N** becomes a table carrying both keys plus the relationship's own attributes; the primary key is the combination, and it is what forbids enrolling twice.
- The M:N table is not plumbing. It holds the grades, it records the time, and it is the biggest table in the database.
- A **weak entity** becomes a table whose key is the owner's key plus the partial key, with the owner's key simultaneously a foreign key and NOT NULL.
- A **recursive** M:N uses rule 6 unchanged, taking the same key twice under the ER diagram's **role names**.
- A **ternary** relationship is one table with three foreign keys, never three binary relationships — that loses the fact it existed to record.
- A multi-column foreign key needs its own FOREIGN KEY clause, with columns in the parent key's order.
- Check the schema back: every line of the diagram should have a line of SQL, and every line of SQL a line of the diagram.

Review Questions

1. (MCQ) In a 1:N relationship the foreign key is placed: (a) on the one side; (b) on the many side; (c) in a new table; (d) on either side.

2. (MCQ) A multi-valued attribute maps to: (a) several columns; (b) a comma-separated column; (c) a new table keyed on owner and value; (d) a CHECK constraint.

3. (Short) Why can an M:N relationship not be mapped with a foreign key? Answer using a property of relations from Chapter 7.

4. (Short) A 1:1 relationship is partial on both sides. Where do you put the foreign key, and what is the consequence either way?

5. (Short) What does the composite primary key of ENROLMENT forbid, and what would be permitted without it?

6. (Applied) Map this diagram completely: a LIBRARY has many BOOKs; each book has many COPYs; MEMBERs BORROW copies and a borrowing records a date out and a date back. Give every table, every key, every foreign key and a referential action for each, and name the rule that produced each table.

7. (Applied) Take your own `v1_create.sql` and go through Figure 6.1 line by line. Produce a two-column table: the diagram constraint, and the SQL that implements it. Then list anything in either column with no partner, and for each one say whether the diagram or the schema is wrong.

Chapter 10

SQL: Creating the Schema

Learning Outcomes

By the end of this chapter you will be able to:

- **Write** a complete CREATE TABLE statement with column types, key, nullability, defaults and CHECK constraints.
- **Explain** what type affinity is, and **predict** what SQLite stores for a given value in a given declared type.
- **Demonstrate** that an `INTEGER` column will hold text, and that STRICT stops it.
- **Trace** a wrong query result back to a value of the wrong type.
- **Choose** between a column constraint and a table constraint, and **name** a case where only the second will do.

Before You Start

Chapter 9: the tables and keys you are now declaring. Chapter 8 for PRAGMA `foreign_keys` and NOT NULL, both of which appear again here for a second reason. You need `code/m10_ddl.py`.

Key Terms in This Chapter

DDL • CREATE TABLE • storage class • type affinity • declared type • STRICT • `typeof()` • column constraint • table constraint • CHECK • DEFAULT • UNIQUE • AUTOINCREMENT • rowid • generated column

10.1 The Statement, in Full

CREATE TABLE is the whole of data definition that this course needs. Everything in it is a promise, and the order of the clauses is fixed.

The anatomy of a CREATE TABLE

```
CREATE TABLE student (
  roll_no   TEXT    PRIMARY KEY NOT NULL,     -- column constraints
  name      TEXT    NOT NULL,
  programme TEXT    NOT NULL DEFAULT 'BSIT',
  semester  INTEGER NOT NULL CHECK (semester BETWEEN 1 AND 8),
  cnic      TEXT    UNIQUE,
  CHECK (length(roll_no) = 11)                -- a table constraint
) STRICT;
```

A **column constraint** follows its column and can mention only that column. A **table constraint** is listed after all the columns and may mention several. PRIMARY KEY on more than one column, and any CHECK comparing two columns, must be table constraints — there is no other way to write them.

The five clauses, and what each one is for

Clause	Promises	Comes from
PRIMARY KEY	these values identify a row	the key chosen in Chapter 5
NOT NULL	a value is always present	total participation (Chapter 6), or entity integrity
UNIQUE	no two rows agree here	an *alternate* key
CHECK	every value satisfies this	the attribute's domain (Chapter 8)
DEFAULT	what to use if none given	a university policy, not a guess

DEFAULT **is the one to be careful with.** It turns a missing value into a real one, so a row that *should* have been rejected is accepted with a plausible value instead. Use it where the university genuinely has a default — `semester` is 1 for a new admission — and not to avoid thinking about nulls.

10.2 Type Affinity, and What It Costs

Every other relational system treats a declared type as a rule. SQLite treats it as a *preference*, and the difference is the subject of this chapter.

Storage class, declared type, affinity

A **storage class** is what a value actually *is* in the file: null, integer, real, text or blob. Every value has one, and `typeof()` reports it.

A **declared type** is what you wrote in CREATE TABLE.

Type affinity is the rule connecting them: on insert, SQLite *tries* to convert the value to the column's declared type. If it can, it does. **If it cannot, it stores the value unchanged.** No error.

STRICT, added to SQLite in 2021, changes the last sentence: the value is refused. A STRICT table may only use the types `INT`, `INTEGER`, `REAL`, `TEXT`, `BLOB` and `ANY`.

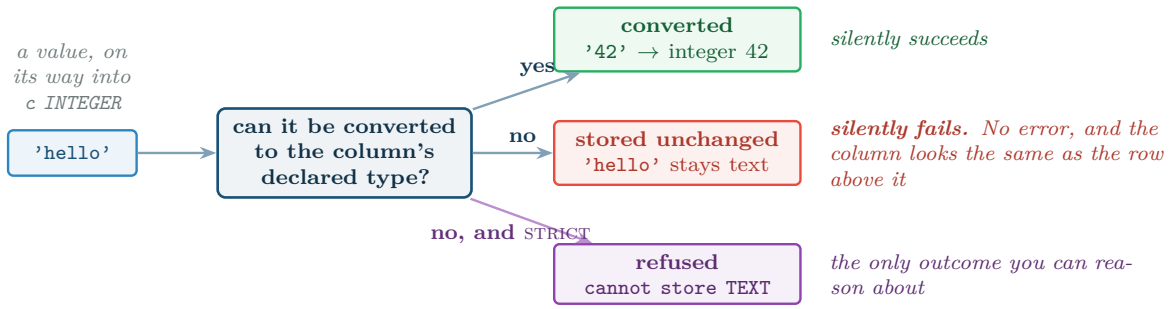


Figure 10.1: Type affinity in one decision. The middle branch is the whole problem: it is not an error, it is not reported, and a column containing both of the top two outcomes looks exactly like a column containing only the first.

Measured: A column declared `INTEGER` that holds the word "absent"

`code/m10_ddl.py`, SQLite 3.51.2, Intel i5-6600, 16 GB, Windows 11.

1. One value, five declared types — what is actually stored

inserted	INTEGER	TEXT	REAL	NUMERIC	(no type)
42	integer	text	real	integer	integer
'42'	**integer**	text	real	integer	text
'hello'	**text**	text	**text**	**text**	text
3.7	**real**	text	real	real	real
NULL	null	null	null	null	null

Read the 'hello' row. Five columns, five different declared types, and every one of them stored the string.

2. The same column, one keyword different

inserted into c INTEGER	plain	STRICT
42	stored as integer	stored as integer
'42'	stored as integer	stored as integer
'hello'	**stored as text**	refused: cannot store TEXT value in INT
3.7	**stored as real**	refused: cannot store REAL value in INT
NULL	stored as null	stored as null

3. What it does to the answers

Marks entered as A=95, B=72, C='absent', D=88, E='100':

ORDER BY score	absent last	text sorts *above* every number
MAX(score)	'absent'	so C is top of the class
AVG(score) as written	**71.00**	'absent' read as 0, still divided by 5
AVG over the numeric rows	88.75	the honest figure
the reported average is	**20% out**	and nothing says so

Note also that '100' *was* converted, to the integer 100. Affinity succeeded silently there and failed silently on 'absent', and the column looks identical either way.

4. What STRICT costs (50,000 rows, 5 runs)

	median	min	max
plain	67.9 ms	66.2	78.5
STRICT	67.9 ms	66.8	72.8
difference	**0.0 ms** (+0%)		

The two ranges overlap, so the difference is noise rather than a cost — which is the opposite of Chapter 8's foreign-key figure, where they did not overlap and the 24% was real. STRICT is free: it is checking a type it already knows.

So the whole argument is one-sided. The protection costs nothing measurable and the absence of it silently moved a class average by twenty per cent. Every table in this course is STRICT.

⚠ Why SQLite is like this, and why that does not help you

This is not a bug and it will not be fixed. SQLite was designed to be embedded, tolerant and backward-compatible for decades; rejecting a value that an older version accepted would break databases in the field. Affinity is the documented, deliberate behaviour, and STRICT was added as an opt-in twenty-one years later.

Knowing the reason does not change what you must do:

- **Write STRICT on every table.** There is no measured reason not to.
- **Use `typeof()` when a result looks wrong.** It is the first diagnostic, and the one nobody reaches for.
- **Be most suspicious of data you imported.** Values typed by hand into INSERT statements are usually the right type; values read from a CSV are all text until something converts them, and Chapter 11 is where that bites.
- **Do not assume other systems behave this way.** PostgreSQL and MySQL reject `'hello'` in an integer column outright. This chapter's behaviour is SQLite's, and Chapter 23 is where the comparison is drawn properly.

10.3 Three Smaller Things That Matter

Rowid, AUTOINCREMENT, and generated columns

Every ordinary SQLite table has a hidden `rowid` — a 64-bit integer key the system maintains. Declaring a column `INTEGER PRIMARY KEY` makes that column *be* the rowid rather than sit beside it, which is why Chapter 8 measured it substituting an integer for a null.

AUTOINCREMENT **is almost never what you want.** Without it, SQLite reuses the largest rowid when rows are deleted; with it, it never does, at the cost of an extra table and a write per insert. Use it only when a reused identifier would be a real problem — an invoice number, not a `section_id`.

A generated column is computed from the others and never stored wrongly, which makes it the right home for Chapter 5's derived attributes:

```
    date_of_birth TEXT NOT NULL,
    age INTEGER GENERATED ALWAYS AS
        (CAST((julianday('now') - julianday(date_of_birth)) / 365.25
              AS INTEGER)) VIRTUAL
```

VIRTUAL computes it on every read and stores nothing; STORED writes it to disk. For `age`, VIRTUAL is the only correct choice: a stored one would be wrong by morning, which was exactly Chapter 5's objection to storing it at all.

Common Pitfalls

- **Trusting a declared type.** Without STRICT it is a preference. Measured above.
- **Using `VARCHAR(50)`.** SQLite accepts the words and ignores the 50 entirely. Write `TEXT`, and enforce a length with CHECK if you mean one.
- **Declaring a composite primary key as two column constraints.** `a INTEGER PRIMARY KEY, b TEXT PRIMARY KEY` is an error; it must be one table constraint.
- **Using DEFAULT to dodge a null.** It replaces a rejection with a plausible wrong value.
- **Reaching for AUTOINCREMENT by habit.** It costs a write per insert and solves a problem you probably do not have.
- **Storing a derived value.** Use a VIRTUAL generated column.
- **Forgetting that CHECK is not re-run on existing rows.** Chapter 11 is about exactly that.

Try It Yourself: Store the wrong type on purpose, then make it impossible

```
# --- 1. A scratch database; keep university.db clean ----------------
sqlite3 types.db
```

```
-- --- 2. A table that looks completely ordinary -----------------
.headers on
.mode box
CREATE TABLE marks (roll TEXT PRIMARY KEY, score INTEGER NOT NULL);

-- --- 3. Put four marks in, one of them the word 'absent' --------
INSERT INTO marks VALUES ('A',95),('B',72),('C','absent'),('D',88);
SELECT roll, score, typeof(score) FROM marks;
-- One row is text. Nothing warned you. The column says INTEGER.

-- --- 4. Ask three ordinary questions ----------------------------
SELECT roll, score FROM marks ORDER BY score DESC;
SELECT MAX(score) FROM marks;
SELECT AVG(score) FROM marks;
-- Write all three answers down. Then work out, by hand, what each
-- one SHOULD be. Two of the three are wrong.

-- --- 5. Find the bad row, which is the real skill ---------------
SELECT roll, score FROM marks
 WHERE typeof(score) NOT IN ('integer','real');
-- Learn this query. It is how you audit a column you inherited.

-- --- 6. The honest average --------------------------------------
SELECT AVG(score) FROM marks
 WHERE typeof(score) IN ('integer','real');
-- Compare with step 4. How far out was it, as a percentage?

-- --- 7. Make it impossible --------------------------------------
CREATE TABLE marks2 (
  roll  TEXT    PRIMARY KEY NOT NULL,
  score INTEGER NOT NULL CHECK (score BETWEEN 0 AND 100)
) STRICT;
INSERT INTO marks2 VALUES ('A',95);
INSERT INTO marks2 VALUES ('C','absent');
INSERT INTO marks2 VALUES ('E',150);
-- Two refusals, two different messages. Record both, and say
-- which constraint produced each.

-- --- 8. Prove VARCHAR(50) is not a length limit -----------------
CREATE TABLE v (s VARCHAR(5));
INSERT INTO v VALUES ('this string is far longer than five');
SELECT length(s) FROM v;
-- Say in one sentence what VARCHAR(5) accomplished.
```

```
# --- 9. The measured version ----------------------------------------
python code/m10_ddl.py

# --- 10. Apply it to your own schema --------------------------------
# Every table in code/schema/v1_create.sql must end in STRICT.
# Check, and then prove it:
sqlite3 university.db "SELECT name, sql FROM sqlite_master
  WHERE type='table' AND sql NOT LIKE '%STRICT%';"
# That query should return nothing.
```

What to notice. Step 4 is the point of the chapter: three ordinary questions, no errors, two wrong answers. Step 5 is the skill — a `typeof()` audit is the only way to find this in a

database you did not build, and it is worth running on any data you import. Step 10 is the protection, and it costs nothing measurable.

Checkpoint — Test Yourself

1. A column is declared `INTEGER` and AVG over it returns 71 when the true average of the real marks is 88.75. Explain exactly how the database arrived at 71, and give the two queries you would run to prove your explanation.
2. State the difference between a column constraint and a table constraint, and give two constraints from your own `v1_create.sql` that *must* be table constraints, saying why each one cannot be written as a column constraint.
3. STRICT costs nothing measurable and prevents a class of silent error. Give the one argument somebody could still make against putting it on every table, and say whether you accept it.

Chapter Summary

- CREATE TABLE carries five kinds of promise: PRIMARY KEY, NOT NULL, UNIQUE, CHECK and DEFAULT, each traceable to a decision in Part II.
- A **column constraint** mentions one column; a **table constraint** may mention several. Composite keys and cross-column CHECKs must be table constraints.
- A **storage class** is what a value is; a **declared type** is what you asked for; **affinity** converts if it can and **stores the value unchanged if it cannot**.
- **Measured:** `'hello'` went into columns declared `INTEGER`, `REAL` and `NUMERIC`, unchanged and without an error.
- **Measured:** on a mixed column, ORDER BY sorts text above every number, MAX returns `'absent'`, and AVG reads it as 0 while still dividing by the full count — a class average reported **20% low**, silently.
- `'100'` *was* converted to 100. Affinity succeeds and fails silently, and the column looks the same either way.
- **Measured:** STRICT **costs nothing** — the two timing ranges overlap, unlike Chapter 8's foreign-key figure. The argument for it is one-sided.
- `typeof()` is the first diagnostic when a result looks wrong, and the one nobody reaches for. Be most suspicious of imported data.
- `VARCHAR(50)` has no length limit in SQLite. Use `TEXT` plus a CHECK.
- A VIRTUAL **generated column** is the right home for a derived attribute: computed on read, never stale.

Review Questions

1. (MCQ) Inserting `'hello'` into a column declared `INTEGER` in a non-STRICT table: (a) raises an error; (b) stores 0; (c) stores the text unchanged; (d) stores null.

2. (MCQ) `VARCHAR(5)` in SQLite limits the column to: (a) 5 characters; (b) 5 bytes; (c) nothing; (d) 5 characters only in STRICT tables.

3. (Short) Why does `'42'` become an integer while `'hello'` stays text? State the rule in one sentence.

4. (Short) Give a constraint that can only be written as a table constraint, and say why.

5. (Short) STRICT showed no measurable cost while foreign key enforcement showed 24%. Explain why the two results differ.

6. **(Applied)** You are given a table of 50,000 imported rows with columns `roll_no`, `score` and `date_sat`. Write the queries that would tell you whether any value is of the wrong type, decide which results you would report as faults, and describe how you would repair the table without losing a row.

7. **(Applied)** Rewrite one table of your `v1_create.sql` to add a DEFAULT, a cross-column CHECK and a VIRTUAL generated column. For each, say what fact about the university it records and what would go wrong if it were omitted — and for the DEFAULT, say what it would wrongly accept.

Chapter 11

Changing a Schema That Is Already Full

Learning Outcomes

By the end of this chapter you will be able to:

- **Classify** a proposed schema change as cheap, expensive or impossible, and **say** what decides which.
- **Use** the four forms of ALTER TABLE SQLite supports, and **state** the limits of each.
- **Perform** the twelve-step rebuild for a change ALTER TABLE cannot make.
- **Write** a migration as a numbered, repeatable script, and **record** the schema version in the database itself.
- **Backfill** a new NOT NULL column on a table that already has rows.

Before You Start

Chapter 10: CREATE TABLE and STRICT. Chapter 9 for the schema you are about to change — this chapter modifies the `university.db` you built there, and produces `v2_constraints.sql`.

Key Terms in This Chapter

schema evolution • migration • ALTER TABLE • ADD COLUMN • RENAME TO • RENAME COLUMN • DROP COLUMN • the twelve-step rebuild • backfill • PRAGMA `user_version` • forward migration • rollback

11.1 The Chapter the Outline Does Not Name

HEC's course content lists nothing about changing a schema — **schema evolution**, in the textbooks' phrase. It is here anyway, for a reason you can check against your own timetable: this course carries one database through sixteen weeks. In week 5 it had seven tables; by Chapter 18 it has more, by Chapter 20 it has views and triggers, and by Chapter 21 it has indexes. Every one of those is a schema change made to a database that already has rows in it.

A student who can only create a schema from scratch has to start again every time the design improves — and in a real system, starting again means losing the data.

Three kinds of change, and what decides the cost

Kind	Examples	Cost
Cheap	add a nullable column; rename; add an index	a change to the *schema only*. The rows are untouched.
Expensive	change a type; add a constraint; split a table	every row must be read and rewritten.
Impossible	recover a fact nobody recorded	no amount of work helps; only a new requirement and future data.

What decides it is whether existing rows must change. Adding a nullable column does not: every existing row's new value is null, and null needs no writing. Adding NOT NULL does, because every existing row needs a value that is not there.

The third row is the one worth remembering. Chapter 5 said a `address` stored whole could never answer "who lives in Bahawalpur". That is not expensive — the city was never recorded separately and splitting the column cannot invent it. Design errors in Part II become *impossible* here, which is why Part II took four chapters.

11.2 What ALTER TABLE Can Do

The four forms, and the limits of each

- ALTER TABLE *t* RENAME TO *u* — renames the table, and since 3.25 updates every reference to it in other tables' foreign keys, views and triggers.
- ALTER TABLE *t* RENAME COLUMN *a* TO *b* — renames a column, updating references.
- ALTER TABLE *t* ADD COLUMN *c* ... — adds a column at the *end*. It may not be PRIMARY KEY or UNIQUE; it may not have a non-constant default; and it may be NOT NULL **only if it has a default**.
- ALTER TABLE *t* DROP COLUMN *c* — removes a column, provided nothing depends on it: not a primary key, not indexed, not mentioned in a constraint, view or trigger.

That is the complete list. **There is no** ALTER COLUMN, so a type cannot be changed, a constraint cannot be added, and a column cannot be moved — not by ALTER TABLE.

The three cheap changes, and the one that catches people

```
-- Cheap: a nullable column. Instant on any number of rows.
ALTER TABLE student ADD COLUMN email TEXT;

-- Cheap: renames.
ALTER TABLE student RENAME COLUMN cnic TO national_id;

-- Refused: NOT NULL with no default, on a table with rows.
ALTER TABLE student ADD COLUMN email TEXT NOT NULL;
--   Cannot add a NOT NULL column with default value NULL
```

The third is not an arbitrary restriction. Every existing row would need a value for `email` and there is none, so the table would be in a state its own schema forbids the instant the statement finished. SQLite refuses rather than produce that.

The two honest ways out are both in the next section, and the dishonest one is DEFAULT '' — which satisfies the constraint with a value that means nothing, and is Chapter 7's magic value wearing a new hat.

11.3 The Twelve-Step Rebuild

For everything else — changing a type, adding a CHECK, adding STRICT, reordering columns — there is one procedure, and it is in SQLite's own documentation. It is worth doing by hand once.

Worked Example: Adding a constraint to a table that is already full

The requirement: `student.semester` must be between 1 and 8. The table exists, has 4,000 rows, and has no such CHECK.

Step 0 — find out whether the data already satisfies it. Before anything else:

```
SELECT roll_no, semester FROM student
 WHERE semester NOT BETWEEN 1 AND 8 OR semester IS NULL;
```

If this returns rows, the migration will fail at step 5 and you need a decision from the university first — not a cleverer script. **This is the step people skip, and it is the one that turns a ten-minute migration into an afternoon.**

Steps 1–12, as a script:

```
PRAGMA foreign_keys = OFF;          -- 1. and NOT inside the transaction
BEGIN;                              -- 2. all of it, or none of it

CREATE TABLE student_new (          -- 3. the table you wish you had
  roll_no   TEXT    PRIMARY KEY NOT NULL,
  name      TEXT    NOT NULL,
  programme TEXT    NOT NULL,
  semester  INTEGER NOT NULL CHECK (semester BETWEEN 1 AND 8),
  cnic      TEXT    UNIQUE
) STRICT;

INSERT INTO student_new             -- 4. copy, naming every column
  SELECT roll_no, name, programme, semester, cnic FROM student;

DROP TABLE student;                 -- 5. the old one
ALTER TABLE student_new RENAME TO student;   -- 6. take its name

CREATE INDEX idx_student_name ON student(name);  -- 7. rebuild indexes
-- 8. recreate views and triggers that referenced it

PRAGMA foreign_key_check;           -- 9. did anything become an orphan?
COMMIT;                             -- 10.
PRAGMA foreign_keys = ON;           -- 11.
PRAGMA integrity_check;             -- 12.
```

Four things about this that are not obvious.

- PRAGMA **foreign_keys goes *outside* the transaction**, both times. Inside one it is silently ignored — which is Chapter 8's pitfall arriving in a place where the consequence is a dropped foreign key rather than a rejected insert.
- **Name every column in the** INSERT**.** SELECT * depends on column order, which is exactly what you may be changing.
- **Indexes, views and triggers do not survive** DROP TABLE**.** Step 7 is not optional and is the most commonly forgotten.
- **It is $O(n)$.** Every row is read and written. On this database that is instant; on a real one it is the difference between a maintenance window and a Tuesday.

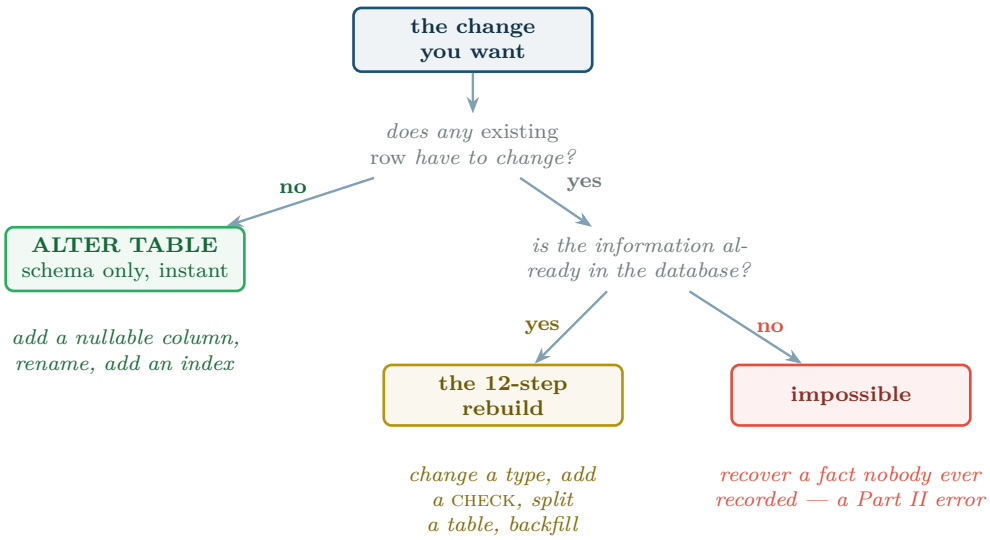


Figure 11.1: Two questions decide what a schema change costs. The right-hand branch is the one Part II existed to prevent: no procedure recovers information that was never stored.

11.4 Migrations as Files

A migration

A **migration** is a numbered script that moves the database from one schema version to the next. It is kept in the repository beside the code, it is run once per database, and it is never edited after it has been run anywhere.

PRAGMA `user_version` is a 32-bit integer SQLite stores in the database file and otherwise ignores completely. It is yours to use, and it is how a script knows which migrations a given file has already had.

The pattern, and it is four lines

```
-- v2_constraints.sql -- run only against a version 1 database
PRAGMA foreign_keys = OFF;
BEGIN;

-- ... the changes ...

PRAGMA user_version = 2;      -- the last thing before COMMIT
COMMIT;
PRAGMA foreign_keys = ON;
```

and the check before running it:

```
sqlite3 university.db "PRAGMA user_version;"
# 1  -> run v2.   2 -> already applied, do nothing.
```

Why the version is set inside the transaction. If the migration fails half-way, the rollback takes the version number with it, and the database is honestly still at version 1. Set it afterwards and a failed migration leaves a database that claims to be something it is not — which is worse than the failure.

Four rules that are about people, not about SQL

- **Never edit a migration that has run anywhere.** Somebody else's database already has the old version applied; changing the file means two databases with the same version number and different schemas, and nothing will ever tell you.
- **Take a backup first, and check that you can restore it.** For SQLite this is `copy university.db university.db.bak` and then actually opening the copy. An untested backup is a hope.
- **Make the change in two deployments when a column is going away.** Stop writing it, ship, confirm nothing broke, then drop it. Dropping and deploying together means the rollback is also a migration.
- **Write the *reverse* migration, even if you never run it.** Writing it is how you find out that the change is irreversible — and finding that out before you run it is the entire point.

Common Pitfalls

- **Adding NOT NULL without a default to a full table.** Refused, and correctly: every existing row would violate it.
- **Satisfying it with DEFAULT ".** A magic value that will be counted and compared as though it meant something.
- **Setting PRAGMA foreign_keys inside the transaction.** Silently ignored, and in a rebuild the consequence is a dropped foreign key.
- **INSERT ... SELECT * in a rebuild.** It depends on the column order you are changing.
- **Forgetting to recreate indexes, views and triggers.** They go with the dropped table and nothing reminds you.
- **Skipping step 0.** Check the data satisfies the new constraint *before* writing the migration.
- **Editing a migration after it has run.** Two databases, one version number, different schemas.

Try It Yourself: Migrate the database you built in Chapter 9

This lab produces `code/schema/v2_constraints.sql`.

```
# --- 1. Back it up, and prove the backup opens ----------------------
cp university.db university.db.bak
sqlite3 university.db.bak ".tables"
sqlite3 university.db "PRAGMA user_version;"     # 0 or 1
```

```
-- --- 2. The cheap change: a nullable column ---------------------
ALTER TABLE student ADD COLUMN email TEXT;
SELECT roll_no, email FROM student LIMIT 3;
-- Instant, whatever the row count. Nothing was rewritten.

-- --- 3. The same column, NOT NULL -------------------------------
ALTER TABLE student ADD COLUMN email2 TEXT NOT NULL;
-- Record the error. Explain it in terms of the existing rows.

-- --- 4. Step 0 of any rebuild: does the data already comply? ----
SELECT COUNT(*) FROM student WHERE semester NOT BETWEEN 1 AND 8;
-- If this is not 0, stop and decide what the right value is.

-- --- 5. Backfill, so NOT NULL becomes possible ------------------
UPDATE student SET email = roll_no || '@iub.edu.pk' WHERE email IS NULL;
SELECT COUNT(*) FROM student WHERE email IS NULL;
-- Now every row has a value, and only now is the constraint addable.
```

```
# --- 6. Write the migration ------------------------------------------
# Put steps 1-12 into code/schema/v2_constraints.sql, adding to the
# student table:  email TEXT NOT NULL, and a CHECK that semester is
# between 1 and 8. Set PRAGMA user_version = 2 before COMMIT.
```

```
-- --- 7. Run it -------------------------------------------------------
.read code/schema/v2_constraints.sql
PRAGMA user_version;
.schema student
PRAGMA integrity_check;
PRAGMA foreign_key_check;

-- --- 8. Prove the new constraint is live -----------------------
INSERT INTO student VALUES
  ('2021-IT-999','Test','BSIT',99,NULL,NULL,NULL,NULL,'x@y.z');
-- Must fail. Which constraint caught it?

-- --- 9. Prove the old data survived ----------------------------
SELECT COUNT(*) FROM student;
-- Compare with the count before step 6. They must match.

-- --- 10. Write the reverse migration ---------------------------
-- Write v2_rollback.sql, which undoes step 6. Do not run it.
-- Then answer: is it a true reverse? What cannot be undone?
```

What to notice. Step 10 is the exercise. The forward migration added a CHECK and a column; the reverse can drop both — but the *backfilled email addresses* are now indistinguishable from real ones, and no rollback can tell them apart. **A migration that writes data is not reversible, whatever the script says**, and knowing which of your migrations are in that class is the whole of the skill.

Checkpoint — Test Yourself

1. Classify each as cheap, expensive or impossible, with the reason: adding a nullable `email`; adding `email` NOT NULL; changing `semester` from `TEXT` to `INTEGER`; splitting `address` into house, street and city on a table where only the whole address was ever stored.
2. A migration adds a NOT NULL column by backfilling it with a computed value. Explain why the rollback script is not a true reverse, and say what you would record at migration time to make it one.
3. PRAGMA `user_version` is set inside the transaction rather than after it. Say what goes wrong if it is set after, and describe the state the database would be left in.

Chapter Summary

- A schema change is **cheap** if no existing row must change, **expensive** if every row must be rewritten, and **impossible** if it needs information nobody recorded.
- The third class is a Part II design error arriving late. No procedure recovers a fact that was never stored.
- SQLite's ALTER TABLE does four things: RENAME TO, RENAME COLUMN, ADD COLUMN, DROP COLUMN. **There is no** ALTER COLUMN, so types and constraints cannot be changed by it.
- ADD COLUMN may be NOT NULL only with a default, because every existing row needs a value — and a default of `''` is a magic value, not a solution.
- Everything else is the **twelve-step rebuild**: new table, copy naming every column, drop, rename, **rebuild indexes, views and triggers**, check, commit.

- PRAGMA `foreign_keys` goes *outside* the transaction. Inside one it is ignored, and in a rebuild that costs you a foreign key.
- A **migration** is a numbered script, kept in the repository, run once, and **never edited after it has run anywhere**.
- PRAGMA `user_version` records which migrations a file has had. Set it *inside* the transaction, so a failed migration leaves an honest version number.
- Back up first and open the backup. Drop a column in a second deployment. Write the reverse migration even if you never run it.
- **A migration that writes data is not reversible**, whatever the rollback script claims — backfilled values cannot be told from real ones afterwards.

Review Questions

1. (MCQ) Which of these can SQLite's ALTER TABLE do? (a) change a column's type; (b) add a CHECK constraint; (c) rename a column; (d) reorder columns.

2. (MCQ) ADD COLUMN `x TEXT NOT NULL` on a table with rows is: (a) instant; (b) refused unless a default is given; (c) refused always; (d) allowed but leaves nulls.

3. (Short) Why must PRAGMA `foreign_keys = OFF` appear before BEGIN rather than after it?

4. (Short) Name two things that are lost by DROP TABLE during a rebuild and must be recreated explicitly.

5. (Short) Give a schema change that is impossible, and say what decision in Part II would have prevented it.

6. (Applied) Write the full twelve-step migration that changes `enrolment.grade` from an unconstrained `TEXT` column to one with CHECK `(grade IN ('A','B','C','D','F'))`. Include step 0, say what you would do if step 0 returned 40 rows with a grade of `'I'`, and state which parts of your script are not reversible.

7. (Applied) Your university decides, in week 12, that an instructor may co-teach: TEACHES becomes M:N. Describe the complete migration — new table, backfill from the existing `section.emp_id`, what to do with the old column, and in how many deployments you would ship it. Then say which single step is irreversible and why.

Part IV

Asking Questions

This is the longest part of the course and the one employers mean when they ask whether you know databases. It begins with the algebra, because SQL is a notation for it and a reader who has seen the algebra first can tell what a query is doing from its shape. Then five chapters of asking the database questions — and one recurring difficulty, which is that the answer to a question about data you do not have is not "none".

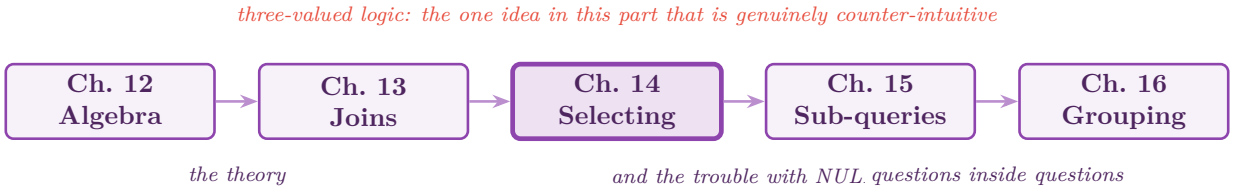


Figure P4.1: Reading path through Part IV. Chapter 12 is the only one here with no SQL in it, and it is the one that makes the other four make sense.

Chapter 12

Relational Algebra

Learning Outcomes

By the end of this chapter you will be able to:

- **Apply** selection, projection, Cartesian product, union, difference and rename to small relations by hand.
- **Write** an algebra expression for a question stated in English.
- **Translate** between an algebra expression and the SELECT statement that computes it.
- **Explain** why the output of every operator is a relation, and what that makes possible.
- **State** which operators need union-compatible inputs, and **check** that two relations are.

Before You Start

Chapter 7: a relation is a set of tuples. Appendix B for set union, intersection and difference — if those three words are not routine, read it first; it is four pages. This chapter is the first in the course with a symbol on most pages.

Key Terms in This Chapter

relational algebra • operator • closure • selection (σ) • projection (π) • Cartesian product ($\times$) • union ($\cup$) • set difference ($-$) • intersection ($\cap$) • rename (ρ) • union-compatible • predicate • query tree • procedural • declarative

12.1 Why a Course on SQL Starts With Algebra

SQL is the language you will use. Relational algebra is the language the *database* uses — the optimiser of Chapter 3 turns your SELECT into an algebra expression, rearranges it, and runs that. Knowing the algebra is knowing what your query becomes.

It also answers a question that SQL hides: why can a query be used inside another query? The answer is one property, and it is the first definition.

Relational algebra, and closure

The **relational algebra** is a set of operators. Each takes one or two relations as input and produces **a relation** as output.

That last fact is **closure**: because every result is a relation, every result can be the input to another operator. Expressions nest without limit, exactly as $(3+4)\times 2$ does in arithmetic — and for the same reason, which is that $3+4$ is a number.

Six operators are fundamental: σ, π, $\times$, $\cup$, $-$ and ρ. Every other operator, including every kind of join, can be written in terms of these six.

12.2 The Two Unary Operators

Selection and projection

Selection $\sigma_p(R)$ returns the tuples of R for which the predicate p is true. It chooses **rows**. The result has the same attributes as R and at most as many tuples.

Projection $\pi_{A_1,\ldots,A_k}(R)$ returns the tuples of R restricted to the attributes $A_1, \ldots, A_k$. It chooses **columns**. Because the result is a set, **duplicates are removed** — so it may have fewer tuples than R.

$\sigma_{programme='BSIT'}(student)$

roll_no	name	programme
IT-001	Ayesha	BSIT
IT-002	Bilal	BSIT
CS-007	Hira	BSCS
IT-014	Saad	BSIT

*chooses **rows**; every column survives*

$\pi_{programme}(student)$

roll_no	name	programme
IT-001	Ayesha	BSIT
IT-002	Bilal	BSIT
CS-007	Hira	BSCS
IT-014	Saad	BSIT

→

programme
BSIT
BSCS

*chooses **columns**, then removes duplicates: four rows in, two out*

Figure 12.1: The two unary operators. Selection cuts horizontally and never loses a column; projection cuts vertically and can lose rows, because three `BSIT` values become one.

Both operators, and their SQL

Algebra	SQL
$\sigma_{semester=2}(student)$	`SELECT * FROM student WHERE semester = 2;`
$\pi_{name,\,programme}(student)$	`SELECT DISTINCT name, programme FROM student;`
$\pi_{name}(\sigma_{semester=2}(student))$	`SELECT DISTINCT name FROM student WHERE semester = 2;`

Look at the word DISTINCT**.** It is in all three projection rows because algebra projection removes duplicates and SQL's does not. SQL returns a *bag* — duplicates kept — unless told otherwise, which is Chapter 7's "a table is not a relation" showing up in the query language. Without DISTINCT, the second query returns 4,000 rows; with it, one per distinct name and programme pair.

And notice the nesting in the third row. σ inside, π outside, and SQL writes them in the opposite order from how they run: the SELECT list is applied last, after WHERE. Reading SQL in execution order — FROM, then WHERE, then SELECT — is the single most useful habit this chapter can give you.

12.3 The Binary Operators

Product, union, difference, intersection

The **Cartesian product** $R \times S$ pairs every tuple of R with every tuple of S. If R has m tuples and S has n, the result has $m \times n$ tuples and the attributes of both.

Union $R \cup S$, **difference** $R - S$ and **intersection** $R \cap S$ are the set operations of Appendix B, applied to tuples. All three require R and S to be **union-compatible**: the same number of attributes, with compatible domains, in the same order.

Intersection is not fundamental: $R \cap S = R - (R - S)$.

Worked Example: How big is a product, and why that matters

On the seeded database that `code/schema/seed.py` builds, *student* has 4,000 tuples and *enrolment* has 60,037.

Step 1 — the size. $|student \times enrolment| = 4{,}000 \times 60{,}037 = 240{,}148{,}000$ tuples.

Step 2 — how many of them mean anything. A pair is meaningful only when the student in it *is* the student the enrolment is about. Each enrolment has exactly one such student, so the meaningful pairs number exactly 60,037.

Step 3 — the proportion. $60{,}037/240{,}148{,}000 = 0.025\%$. One pair in four thousand is true; the other 3,999 pair a student with an enrolment that belongs to somebody else.

Step 4 — the fix, in algebra. Select the meaningful ones:

$$\sigma_{student.roll_no=enrolment.roll_no}(student \times enrolment)$$

That expression *is* a join, and Chapter 13 gives it its own symbol. Every join in SQL is a product followed by a selection — in meaning. In execution it is never computed that way, because 240 million intermediate tuples to keep 60 thousand is exactly the plan an optimiser exists to avoid, and Chapter 13 measures how it avoids it.

Union-compatibility is checked by position, and that is a trap

```
SELECT roll_no, name FROM student
UNION
SELECT emp_id,  name FROM instructor;
```

This runs. Two attributes each side, both text: union-compatible. The result is a list of people — students and staff — and that is what was meant.

```
SELECT name,    roll_no FROM student
UNION
SELECT emp_id,  name    FROM instructor;
```

This *also* runs, and produces a column that holds student names and employee numbers, and a second column holding roll numbers and instructor names. SQL checks that the count and types match *by position*; it does not check that the meanings do, because it cannot.

The rule: in a UNION, EXCEPT or INTERSECT, list the columns in the same order on both sides and give them the same meaning. The database will hold you to the first half only.

12.4 Rename, and a Question Written Three Ways

Rename

Rename $\rho_S(R)$ gives the relation R the name S; $\rho_{S(B_1,\ldots,B_n)}(R)$ also renames its attributes. It is needed whenever a relation appears twice in one expression — the recursive relationship of Chapter 6 — because otherwise *course.course_code* would name two different things. In SQL it is the *alias*: `FROM course AS c1, course AS c2`.

Worked Example: "Which students have taken a course but never received an A?"

Step 1 — in English, the two sets. Students who have taken something, minus students who have an A somewhere.

Step 2 — each set in algebra.

$$T = \pi_{roll_no}(enrolment)$$
$$A = \pi_{roll_no}(\sigma_{grade='A'}(enrolment))$$

Step 3 — combine. $T - A$. Both are projections onto *roll_no*, so they are union-compatible by construction.

Step 4 — the SQL.

```
SELECT roll_no FROM enrolment
EXCEPT
SELECT roll_no FROM enrolment WHERE grade = 'A';
```

Step 5 — check it against a wrong answer. The tempting version is

$$\pi_{roll_no}(\sigma_{grade\neq'A'}(enrolment))$$

— "students with a grade that is not A". That includes every student who has one B, even if they also have five A's. It answers a different question and it runs without complaint. On the seeded database the right expression returns **551** students; the tempting one returns **4,000** — every student in the university, because every one of them has at least one grade that is not an A.

The lesson. "Never" is a difference, not a selection. Questions with *no*, *never*, *none* or *only* in them almost always need $-$ or its SQL equivalents, and Chapter 15 returns to exactly this, with NOT EXISTS and a measurement.

Algebra is procedural; SQL is declarative — and why both exist

An algebra expression says *how*: first select, then project, then take the difference. Its nesting is an order of operations.

SQL says *what*: the rows satisfying this condition, from these tables. It deliberately does not say in what order.

The optimiser sits between them. It turns your SQL into one algebra expression, then uses algebraic laws — σ before $\times$ rather than after, π as early as possible — to rewrite it into a cheaper one that computes the same relation. Those laws are true *only because* every operator is closed and every intermediate result is a relation.

So closure, the first definition in this chapter, is not a curiosity. It is the reason a database can choose a plan at all, and Chapter 21 is about what happens when it chooses badly.

Common Pitfalls

- **Forgetting DISTINCT.** Algebra π removes duplicates; SQL's SELECT list does not.
- **Reading SQL in written order.** It runs FROM, WHERE, SELECT. Read it that way.
- **Writing "never" as a selection.** $\sigma_{\neq}$ answers "at least once something else". "Never" is a difference.
- **Trusting UNION to check meaning.** It checks count and type by position. Order the columns yourself.
- **Forgetting the size of a product.** $m \times n$. Four thousand by sixty thousand is 240 million.
- **Using a relation twice without renaming it.** The attribute names become ambiguous. Use an alias.

Try It Yourself: Compute five algebra expressions, and catch one wrong answer

```
# --- 1. A populated database, the same rows as the handout ---------
python code/schema/seed.py university.db
sqlite3 university.db
```

```
-- --- 2. Selection and projection -------------------------------
.headers on
.mode box
SELECT COUNT(*) FROM student WHERE semester = 2;
SELECT COUNT(*) FROM (SELECT programme FROM student);
SELECT COUNT(*) FROM (SELECT DISTINCT programme FROM student);
-- The last two are both "projection onto programme". Why do they
-- give different numbers, and which one is the algebra's answer?

-- --- 3. The size of a product, without computing it -------------
SELECT (SELECT COUNT(*) FROM student) * (SELECT COUNT(*) FROM enrolment);
-- Do NOT run SELECT COUNT(*) FROM student, enrolment on this
-- database unless you are prepared to wait. Say why.

-- --- 4. A product made meaningful: the selection that is a join -
SELECT COUNT(*) FROM student s, enrolment e
 WHERE s.roll_no = e.roll_no;
-- Compare with the size of enrolment. Explain the match.

-- --- 5. Union, and the positional trap --------------------------
SELECT roll_no AS id, name FROM student
UNION
SELECT emp_id, name FROM instructor
LIMIT 5;
SELECT name, roll_no FROM student
UNION
SELECT emp_id, name FROM instructor
LIMIT 5;
-- Both run. Describe precisely what is wrong with the second.

-- --- 6. "Never an A" -- the right way ---------------------------
SELECT COUNT(*) FROM (
  SELECT roll_no FROM enrolment
  EXCEPT
  SELECT roll_no FROM enrolment WHERE grade = 'A');

-- --- 7. "Never an A" -- the tempting way ------------------------
SELECT COUNT(DISTINCT roll_no) FROM enrolment WHERE grade <> 'A';
-- Write both counts down. One of them is every student in the
-- university. Which, and why? This is the worked example, run.

-- --- 8. Rename: the recursive relationship ----------------------
SELECT c1.title AS course, c2.title AS requires
  FROM prerequisite p
  JOIN course c1 ON c1.course_code = p.course_code
  JOIN course c2 ON c2.course_code = p.requires
 LIMIT 5;
-- course appears twice. Remove the aliases and record what fails.
```

What to notice. Steps 6 and 7 differ by one idea — 551 against 4,000 on the seeded data — and neither one raises an error. Every query you write from here should be checked against a hand-worked expectation, because a confident wrong answer looks identical to a confident right one.

Checkpoint — Test Yourself

1. $\pi_{programme}(student)$ on 4,000 students returns 4 tuples, and `SELECT programme FROM student` returns 4,000 rows. Explain the difference in terms of the definition of a relation, and say which keyword reconciles them.
2. Write, in algebra and in SQL, "students in semester 2 who are enrolled in nothing in FALL2026". Say which operator expresses "nothing", and give the tempting wrong version and what it returns.
3. Every join is a selection over a product. Using the seeded sizes, explain why no database computes a join that way, and say which property of the algebra lets the optimiser compute something else instead and still be correct.

Chapter Summary

- Relational algebra is a set of operators on relations whose every result is **a relation**. That is **closure**, and it is why queries nest.
- Six operators are fundamental: **selection** σ (rows), **projection** π (columns), **product** $\times$, **union** $\cup$, **difference** $-$, **rename** ρ. Intersection and every join are built from them.
- Algebra projection **removes duplicates**; SQL's does not unless you write DISTINCT. A table is a bag; a relation is a set.
- SQL runs FROM, then WHERE, then SELECT — the opposite of the order it is written in.
- $|R \times S| = |R| \times |S|$. On the seeded data that is 240 million tuples, of which 0.025% mean anything. A join is a selection over a product *in meaning*, never in execution.
- UNION, EXCEPT and INTERSECT check union-compatibility **by position and type**, never by meaning.
- "Never", "none" and "only" are **differences**, not selections. The selection version runs and answers a different question.
- **Rename** is the SQL alias, and it is required whenever a relation appears twice in one query.
- Algebra is procedural, SQL declarative; the optimiser turns one into the other and rewrites it using laws that hold only because of closure.

Review Questions

1. (MCQ) Which operator can return fewer tuples than its input even when no condition is given? (a) selection; (b) projection; (c) rename; (d) Cartesian product.

2. (MCQ) $|R| = 30$ and $|S| = 50$. $|R \times S|$ is: (a) 80; (b) 30; (c) 1,500; (d) it depends on the keys.

3. (Short) Define union-compatibility and give two relations from the university schema that are union-compatible but whose union would be meaningless.

4. (Short) Show that intersection can be written using only difference.

5. (Short) Why does SQL need DISTINCT when relational algebra does not?

6. (Applied) Write in algebra and then in SQL: (a) the names of instructors in the CS department; (b) course codes that are a prerequisite for something but require nothing themselves; (c) students who have received both an A and an F. For each, say which operators you used and why.

7. (Applied) Draw the query tree for $\pi_{name}(\sigma_{grade='A'}(student \times enrolment))$ with the join condition added. Then draw an equivalent tree that is cheaper, say which algebraic law justifies each change, and estimate from the seeded table sizes how many intermediate tuples each tree produces.

Chapter 13

Joins: the Algebra and the SQL

Learning Outcomes

By the end of this chapter you will be able to:

- **Define** theta join, equijoin and natural join in algebra, and **write** each in SQL.
- **Distinguish** inner from outer joins, and **predict** which rows an inner join will drop.
- **Explain** why NATURAL JOIN is dangerous, using a case from this course's own schema.
- **Read** an EXPLAIN QUERY PLAN for a join and **say** whether it is using an index.
- **Account for** a hundred-fold difference between two joins that return identical rows.

Before You Start

Chapter 12: a join is a selection over a product. Chapter 7 for nulls, whose consequence for joins this chapter measures. You need `code/m13_joins.py` and a seeded `university.db`.

Key Terms in This Chapter

join • theta join • equijoin • natural join (⋈) • inner join • outer join • left outer join • right outer join • full outer join • self-join • USING • ON • nested-loop join • automatic index • EXPLAIN QUERY PLAN • SCAN • SEARCH

13.1 One Operator, Three Strengths

Chapter 12 ended on the expression $\sigma_{s.roll_no=e.roll_no}(s \times e)$. It occurs so often that it has its own symbol, and three variants that differ only in how much the condition is allowed to say.

Theta join, equijoin, natural join

A **theta join** $R \bowtie_\theta S$ is $\sigma_\theta(R \times S)$ for any condition θ comparing attributes of R with attributes of S using $=, <, \leq, >, \geq$ or $\neq$.

An **equijoin** is a theta join whose condition uses only $=$. Almost every join you will write is one.

A **natural join** $R \bowtie S$ is an equijoin on *every attribute the two relations share by name*, keeping one copy of each. It has no condition at all — the condition is whatever the names happen to be.

The same join, four ways in SQL

```sql
-- 1. the product, then the selection: chapter 12 exactly
SELECT * FROM student s, enrolment e WHERE s.roll_no = e.roll_no;

-- 2. explicit, and the form this course uses
SELECT * FROM student s JOIN enrolment e ON s.roll_no = e.roll_no;

-- 3. when the column has the same name on both sides
SELECT * FROM student JOIN enrolment USING (roll_no);
```

```
-- 4. natural: no condition written at all
SELECT * FROM student NATURAL JOIN enrolment;
```

The first three are one query to the optimiser — the measurement below shows the same plan for each. The fourth is not, on this schema, and the reason is the most useful thing in this chapter.

13.2 What a Join Drops

Inner and outer joins

An **inner join** returns only the pairs that satisfy the condition. A row with no partner on the other side does not appear.

A **left outer join** of R with S returns every row of R; where there is no partner in S, the S attributes are null. A **right outer join** keeps every row of S; a **full outer join** keeps every row of both.

SQLite supports all three. LEFT JOIN is the one you will need, and the question that decides it is always the same: *must a row with no partner still appear?*

section

course	emp_id
CS-103	E-114
CS-201	E-120
CS-305	NULL

JOIN **instructor**

course	emp_id	name
CS-103	E-114	Malik
CS-201	E-120	Raza

*CS-305 is **gone**: its emp_id is null and null equals nothing*

LEFT JOIN **instructor**

course	emp_id	name
CS-103	E-114	Malik
CS-201	E-120	Raza
CS-305	NULL	NULL

every section survives; the missing partner is filled with null

Figure 13.1: The section nobody has been assigned to teach. An inner join drops it without comment; a left join keeps it and marks what is missing. On the seeded database this is 73 sections out of 1,500.

13.3 Measured: Plans, Nulls and the Cost of a Missing Index

Measured: Three joins that return the same rows and one that returns none

`code/m13_joins.py`, seeded database (4,000 students, 60,037 enrolments), SQLite 3.51.2, Intel i5-6600, Windows 11.

1. Four spellings of one inner join

written as	rows	ms	plan
comma + WHERE	60,037	29.1	`SCAN e + SEARCH s`
JOIN ... ON	60,037	30.4	identical
JOIN ... USING	60,037	30.1	identical
NATURAL JOIN	**0**	43.7	

The two tables share **two** attribute names, not one: `roll_no`, and `semester` — which is an `INTEGER` year of study (e.g. `8`) in *student* and a `TEXT` term label (e.g. `'FALL2025'`) in *enrolment*. NATURAL JOIN matched on both, demanded a year equal a term label, and returned nothing. **No error, on the schema exactly as Chapter 9 built it.**

2. Inner against left join

sections in the database	1,500
sections with no instructor yet (`emp_id` null)	73
timetable as an inner join	1,427
timetable as a left join	1,500

The 73 missing from the inner join are exactly the sections nobody has been assigned to — the rows a timetable most needs to show.

3. Students who share a name: a join on an unindexed column

students	pairs found	as written	automatic index off	real index	slowdown
1,000	1,785	3.2 ms	235.1 ms	2.9 ms	82×
2,000	7,109	8.6 ms	1,088.1 ms	11.1 ms	98×
4,000	28,517	25.1 ms	4,170.1 ms	38.7 ms	**108**×

Without an index the join is a **nested-loop join**: for each row of one side, every row of the other is examined — two loops, one inside the other, which is where the n^2 comes from.

Compare across a row, not down a column. The pairs found grow about four-fold per doubling for *every* method, because names come from a fixed pool and the number of real matches is itself quadratic — so growth ratios here mix up how much the database *looked at* with how much it *returned*. Across a row the output is identical, and the difference is pure search: without an index, all $4000 \times 3999/2 \approx 8$ million pairs are examined to find 28 thousand. The slowdown itself rises with size, 82 to 108, which is the real signature of a quadratic method.

4. Why "as written" is fast anyway

automatic index on (default)	`SEARCH b USING AUTOMATIC COVERING INDEX (name=?)`
automatic index off	`SEARCH b USING INDEX ...(roll_no>?)`
`CREATE INDEX` on name	`SEARCH b USING INDEX idx_name (name=?)`

SQLite noticed the missing index, **built one for this single query, used it, and threw it away**. You pay to build an index on every run and never get to keep it.

⚠ What the measurement means for the queries you write

- **Never use** NATURAL JOIN**.** Its condition is decided by column names, and column names change for reasons that have nothing to do with the join. This course's own schema has the trap in it, and it was not put there on purpose — a year of study and a term are both reasonably called "semester". Write ON or USING and say what you mean.
- **Ask, for every join, whether an unmatched row must still appear.** If yes, it is a LEFT JOIN. The inner join's silence about the 73 is not a bug in SQL; it is the definition, applied to a null foreign key that Chapter 6 deliberately allowed.
- **Read the plan.** AUTOMATIC INDEX is the optimiser telling you an index is missing, in plain words. It is the cheapest performance diagnosis there is, and Chapter 21 is about acting on it.

13.4 Joining a Table to Itself

The self-join, and the condition that halves it

"Pairs of students who share a name" joins *student* to itself, which needs two aliases — Chapter 12's rename:

```
SELECT a.roll_no, b.roll_no, a.name
  FROM student a
  JOIN student b ON a.name = b.name
                AND a.roll_no < b.roll_no;
```

The second condition does two jobs. Without it every student matches themselves — Ayesha shares a name with Ayesha — and every real pair appears twice, once each way round. `a.roll_no < b.roll_no` removes both: no row is paired with itself, and each pair appears in exactly one order. Use $<$, never $\neq$, which would keep both orders.

The prerequisite query in Chapter 12's laboratory was a self-join too, of *course*, through *prerequisite*. Every recursive relationship in an ER diagram becomes a self-join in SQL.

Common Pitfalls

- NATURAL JOIN**.** It joined on two columns, not one, and returned nothing. Measured.
- **An inner join where a left join was meant.** The unmatched rows vanish silently — 73 sections from a timetable.
- **A condition on the right table in** WHERE **after a** LEFT JOIN**.** `WHERE i.name = 'Malik'` discards the null-extended rows and turns it back into an inner join. Put such conditions in ON.
- **Forgetting the join condition.** You get the product: 240 million rows on the seeded data.
- **A self-join with $\neq$.** Every pair twice. Use $<$.
- **Ignoring** AUTOMATIC INDEX **in a plan.** It is a missing index announcing itself.

Try It Yourself: Lose 73 sections, find them, and make a join a hundred times faster

```
# --- 1. The seeded database ----------------------------------------
python code/schema/seed.py university.db
sqlite3 university.db
```

```
-- --- 2. The timetable, as an inner join -------------------------
.headers on
.mode box
SELECT COUNT(*) FROM section s JOIN instructor i ON s.emp_id = i.emp_id;
SELECT COUNT(*) FROM section;
-- The two numbers differ. By how much, and which sections are they?

-- --- 3. Find them -----------------------------------------------
SELECT s.course_code, s.section_no, s.semester
  FROM section s LEFT JOIN instructor i ON s.emp_id = i.emp_id
 WHERE i.emp_id IS NULL
 LIMIT 10;
-- This is the anti-join idiom: LEFT JOIN, then keep the rows that
-- found no partner. Learn it; Chapter 15 measures it.

-- --- 4. Break the left join by mistake --------------------------
SELECT COUNT(*) FROM section s LEFT JOIN instructor i
    ON s.emp_id = i.emp_id
 WHERE i.designation = 'Professor';
```

```
SELECT COUNT(*) FROM section s LEFT JOIN instructor i
    ON s.emp_id = i.emp_id AND i.designation = 'Professor';
-- Same condition, two places, two very different counts. Explain.

-- --- 5. The natural join, on the schema you built ---------------
SELECT COUNT(*) FROM student NATURAL JOIN enrolment;
-- Zero. Find out why:
PRAGMA table_info(student);
PRAGMA table_info(enrolment);
-- Name the second shared column, and its two meanings.

-- --- 6. Read a plan ---------------------------------------------
EXPLAIN QUERY PLAN
SELECT COUNT(*) FROM student a JOIN student b
    ON a.name = b.name AND a.roll_no < b.roll_no;
-- Find the word AUTOMATIC. Say what SQLite just did.

-- --- 7. Give it the index it keeps building ---------------------
CREATE INDEX idx_student_name ON student(name);
EXPLAIN QUERY PLAN
SELECT COUNT(*) FROM student a JOIN student b
    ON a.name = b.name AND a.roll_no < b.roll_no;
```

```
# --- 8. The measured version ----------------------------------------
python code/m13_joins.py
```

What to notice. Step 4 is the one that catches people in examinations and in production alike. A condition in WHERE is applied *after* the outer join has produced its null-extended rows, and since a null `designation` is not `'Professor'`, they are all discarded — the outer join has silently become an inner one.

Checkpoint — Test Yourself

1. NATURAL JOIN of *student* and *enrolment* returns 0 rows on this course's schema. Explain exactly why, say what you would change in the query, and say what you would change in the schema if you could change only one column name.
2. An inner join of *section* to *instructor* returns 1,427 rows and the table has 1,500. Identify the missing 73 by a property they share, write the query that lists them, and say which design decision in Chapter 6 made them possible.
3. Two joins return the same 28,517 rows, one in 38.7 ms and one in 4,170 ms. Explain the difference using the number of pairs each one examined, and say why comparing growth ratios down the columns would have given a misleading answer.

Chapter Summary

- A **theta join** is $\sigma_\theta(R \times S)$; an **equijoin** uses only $=$; a **natural join** equijoins on *every* shared attribute name.
- Comma-plus-WHERE, JOIN . . . ON and JOIN . . . USING produce the same plan. Use ON.
- **Measured:** NATURAL JOIN **returned 0 rows** on this course's schema, because *student* and *enrolment* both have a `semester` — a year of study in one, a term label in the other. Never use it.
- An **inner join** drops rows with no partner; a **left join** keeps them with nulls. **Measured: 73 of 1,500 sections** vanished from the inner-join timetable — the ones with no instructor.
- A condition on the right-hand table belongs in ON. In WHERE it turns a left join back into an inner one.

- A **self-join** needs two aliases, and $<$ rather than $\neq$ to avoid both self-pairs and duplicate pairs.
- **Measured: a join on an unindexed column was 108× slower** at 4,000 students than the same join with an index — 8 million pairs examined against 28 thousand found.
- Compare methods **at the same size**. Here the output itself grew quadratically, so growth ratios mixed search cost with output size.
- SQLite builds an **automatic index** for a single query when one is missing, and discards it. AUTOMATIC in a plan is a missing index announcing itself.

Review Questions

1. (MCQ) A natural join of R and S is an equijoin on: (a) their primary keys; (b) their foreign keys; (c) every attribute they share by name; (d) the first column of each.

2. (MCQ) A section whose `emp_id` is null, joined to *instructor* with an inner join, appears: (a) once, with nulls; (b) not at all; (c) once per instructor; (d) with an error.

3. (Short) Why does `a.roll_no < b.roll_no` in a self-join give each pair exactly once, while $\neq$ gives it twice?

4. (Short) What does AUTOMATIC COVERING INDEX in a query plan tell you, and what should you do about it?

5. (Short) Give a query in the university schema that needs a LEFT JOIN, and say what an inner join would silently omit.

6. (Applied) Write the query that lists every course with its number of sections in FALL2026, including courses with none. (On the seeded database every course has one, so first insert a new course with no sections, or there is nothing to omit.) Show the inner-join version that gets it wrong, say which courses it omits, and then give the correct version and explain where its condition on semester must go and why.

7. (Applied) The measured slowdown was 82×, 98× and 108× at 1,000, 2,000 and 4,000 students. Predict it at 8,000, show your reasoning from the number of pairs examined and found, and explain why the slowdown is rising rather than constant.

Chapter 14

Selecting Rows, and the Trouble with NULL

Learning Outcomes

By the end of this chapter you will be able to:

- **Write** a SELECT with WHERE, ORDER BY and LIMIT, and **state** the order in which its clauses run.
- **Use** LIKE, IN, BETWEEN and CASE correctly, including at their boundaries.
- **Evaluate** a condition in three-valued logic, and **predict** which rows a condition involving null will return.
- **Explain** why NOT IN can return nothing, and **replace** it with a form that cannot.
- **Account for** every row of a table when a condition and its negation together do not cover it.

Before You Start

Chapter 7: what a null is. Chapter 13 for the 73 sections a null foreign key dropped. A seeded `university.db`; every number in this chapter is reproduced by its laboratory.

Key Terms in This Chapter

SELECT • WHERE • ORDER BY • LIMIT • OFFSET • LIKE • wildcard • IN • BETWEEN • CASE • three-valued logic • UNKNOWN • IS NULL • COALESCE • NULLIF • NOT IN • NOT EXISTS

14.1 The Statement, and the Order It Runs In

SELECT, in execution order

```
SELECT   name, semester          -- 4. keep these columns
FROM     student                 -- 1. start from this relation
WHERE    programme = 'BSIT'      -- 2. keep these rows
ORDER BY semester DESC, name     -- 5. sort what is left
LIMIT    10 OFFSET 20;           -- 6. then take a slice
```

Step 3, between WHERE and SELECT, is GROUP BY, and Chapter 16 adds it. The order matters in one practical way: a name given in the SELECT list with AS *does not exist yet* when WHERE runs, so it cannot be used there — but it can be used in ORDER BY, which runs after.

The four predicates, and where each one surprises

`name LIKE 'A%'`	the **wildcards**: `%` is any run of characters, `_` exactly one. In SQLite LIKE is **case-insensitive** for ASCII letters, so this also matches `'ayesha'`.
`programme IN ('BSIT','BSCS')`	shorthand for two ORs. Fine — until the list contains a null, below.
`semester BETWEEN 2 AND 4`	**inclusive at both ends**: 2, 3 and 4.
`CASE WHEN g='A' THEN 4 ... END`	evaluated top to bottom; the first true branch wins; with no ELSE, the result is null.

And one that is not a predicate at all. `LIMIT 10` without ORDER BY returns *some* ten rows. Which ten depends on the plan, and Chapter 7 said the plan is allowed to change. A "top ten" with no ORDER BY is ten rows chosen by the optimiser.

14.2 Three-Valued Logic

Every condition so far has been true or false. As soon as a null is involved there is a third answer, and it is the source of most quietly wrong results in SQL.

UNKNOWN

Any comparison with a null — `=`, `<>`, `<`, `>`, even `NULL = NULL` — evaluates to UNKNOWN, not to true or false. It means "the value is missing, so I cannot say".

WHERE keeps a row **only if its condition is true**. False and UNKNOWN are both discarded.

IS NULL and IS NOT NULL are the only tests that return true or false for a null. They exist because `= NULL` cannot work.

The truth tables, and the one row to remember

AND		OR		NOT	
T AND U	= U	T OR U	= **T**	NOT T	= F
F AND U	= **F**	F OR U	= U	NOT F	= T
U AND U	= U	U OR U	= U	NOT U	= **U**

Read the bold entries. AND with a false is false whatever the other side; OR with a true is true whatever the other side — those are the cases where a missing value genuinely does not matter.

The row to remember is the last one: NOT UNKNOWN **is** UNKNOWN**.** Negating a condition does *not* give you the other rows. It gives you the other rows *that are not null*. A condition and its negation together do not cover a table that contains nulls — and the next box counts exactly how many rows fall through the gap.

Worked Example: Grade A, not grade A, and the 19,992 rows that are neither

The seeded *enrolment* table has 60,037 rows. Three queries:

`WHERE grade = 'A'`	7,268
`WHERE grade <> 'A'`	32,777
`WHERE NOT (grade = 'A')`	32,777

Step 1 — the sum. $7{,}268 + 32{,}777 = 40{,}045$. The table has 60,037. **19,992 rows are in neither answer.**

Step 2 — which rows. The FALL2026 enrolments, whose semester is not over: `grade` is null. For those rows `grade = 'A'` is UNKNOWN, so is `grade <> 'A'`, and so — by the last row of the truth table — is NOT `(grade = 'A')`. Every one of the three conditions discards them.

Step 3 — the check.

```
SELECT COUNT(*) FROM enrolment WHERE grade IS NULL;     -- 19992
```

7,268 + 32,777 + 19,992 = 60,037. Every row accounted for.

Step 4 — what goes wrong in practice. A report of "students who did not get an A" built on `grade <> 'A'` omits a third of all enrolments without a word. Whether it *should* include them is a question for the examinations office — an ungraded course is not a failure — but the query decided it silently, and decided it the way most people would not.

The rule. When a column can be null, write the null case explicitly, whichever way you want it:

```
WHERE grade <> 'A' OR grade IS NULL      -- include ungraded
WHERE grade <> 'A'                       -- exclude them, and say so
```

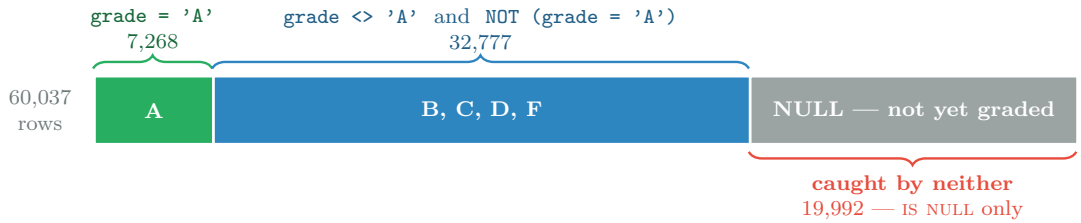


Figure 14.1: The seeded *enrolment* table, drawn to scale. A condition and its negation cover the green and the blue. The grey third is UNKNOWN to both, and WHERE discards it either way.

14.3 NOT IN, Which Can Return Nothing

Three new instructors, and a query that cannot find them

Hire three instructors who are not yet teaching anything. Then ask which instructors teach no section — the obvious way:

```
SELECT COUNT(*) FROM instructor
 WHERE emp_id NOT IN (SELECT emp_id FROM section);
```

On the seeded database, with the three added, this returns **0**.

```
SELECT COUNT(*) FROM instructor i
 WHERE NOT EXISTS (SELECT 1 FROM section s WHERE s.emp_id = i.emp_id);
```

This returns **3**.

Why. `x NOT IN (a, b, c)` means `x <> a AND x <> b AND x <> c`. The sub-query's list contains the 73 nulls from the unassigned sections, so the expansion contains `x <> NULL` — which is UNKNOWN. By the truth table, `T AND U = U`. For every instructor, including the three new ones, the whole condition is at best UNKNOWN, and WHERE discards UNKNOWN. **One null in the list empties the result.**

The fix is either of these:

- use NOT EXISTS, which tests for the *existence* of a row rather than comparing with a value, and is unaffected by nulls; or
- filter the nulls out of the list: `... NOT IN (SELECT emp_id FROM section WHERE emp_id IS NOT NULL)`, which also returns 3.

This course uses NOT EXISTS. The second fix works, but it has to be remembered every time, and it will be forgotten the first time the column becomes nullable — which is a schema change somebody else will make. Chapter 15 measures the two forms against each other.

COALESCE and NULLIF: deciding what null means

COALESCE(a, b, c) returns the first argument that is not null. It is how you *decide*, in one place, what a null should be treated as:

```
SELECT roll_no, COALESCE(cnic, 'not on file') AS cnic FROM student;
SELECT COALESCE(grade, 'in progress') AS grade FROM enrolment;
```

NULLIF(a, b) is the reverse: it returns null if $a = b$, otherwise a. Its main use is cleaning up Chapter 7's magic values — `NULLIF(cnic, 'N/A')` turns an invented marker back into the null it should have been.

Use COALESCE for display, not for arithmetic. Writing `AVG(COALESCE(score, 0))` decides that a missing mark is a zero, which is Chapter 10's 20% error made on purpose.

Where nulls go in ORDER BY

SQLite sorts nulls **first** in ascending order and last in descending, treating null as smaller than every value. Other systems differ: PostgreSQL puts them last in ascending order. SQLite since 3.30 accepts the standard clause that makes it explicit:

```
SELECT roll_no, grade FROM enrolment ORDER BY grade NULLS LAST;
```

Write NULLS FIRST or NULLS LAST whenever the column can be null and the order matters, because "what this database happens to do" is not the same on the next one.

Common Pitfalls

- **`WHERE x = NULL`.** Always UNKNOWN, so always empty. Use IS NULL.
- **Assuming a condition and its negation cover the table.** They miss the nulls — 19,992 of 60,037 enrolments.
- **NOT IN against a nullable column.** One null in the list and the result is empty. Use NOT EXISTS.
- **Using a column alias in WHERE.** It does not exist yet.
- **LIMIT without ORDER BY.** Some rows, chosen by the plan.
- **Forgetting BETWEEN is inclusive.** Both ends.
- **Expecting LIKE to be case-sensitive.** In SQLite it is not, for ASCII.
- **COALESCE to zero inside an average.** A decision that a missing mark is a fail, made silently.

Try It Yourself: Lose 19,992 rows and three instructors, and find them all

```
# --- 1. A fresh seeded database ------------------------------------
python code/schema/seed.py university.db
sqlite3 university.db
```

```
-- --- 2. A condition and its negation ----------------------------
.headers on
```

```
.mode box
SELECT COUNT(*) FROM enrolment WHERE grade = 'A';
SELECT COUNT(*) FROM enrolment WHERE grade <> 'A';
SELECT COUNT(*) FROM enrolment;
-- Add the first two. Compare with the third. Explain the gap.

-- --- 3. Account for every row ------------------------------------
SELECT COUNT(*) FROM enrolment WHERE grade IS NULL;
SELECT COUNT(*) FROM enrolment WHERE grade = NULL;
-- Two queries that look alike. One finds the gap; one finds nothing.

-- --- 4. Hire three people who teach nothing yet ------------------
PRAGMA foreign_keys = ON;
INSERT INTO instructor VALUES
  ('E-9001','New Hire One','Lecturer','CS'),
  ('E-9002','New Hire Two','Lecturer','IT'),
  ('E-9003','New Hire Three','Lecturer','SE');

-- --- 5. Look for them with NOT IN --------------------------------
SELECT COUNT(*) FROM instructor
 WHERE emp_id NOT IN (SELECT emp_id FROM section);
-- There are three. How many did it find?

-- --- 6. Look for them with NOT EXISTS ----------------------------
SELECT name FROM instructor i
 WHERE NOT EXISTS (SELECT 1 FROM section s WHERE s.emp_id = i.emp_id);

-- --- 7. Prove the nulls are to blame -----------------------------
SELECT COUNT(*) FROM section WHERE emp_id IS NULL;
SELECT COUNT(*) FROM instructor
 WHERE emp_id NOT IN (SELECT emp_id FROM section
                      WHERE emp_id IS NOT NULL);

-- --- 8. Decide what null means, once -----------------------------
SELECT COALESCE(grade, 'in progress') AS g, COUNT(*)
  FROM enrolment GROUP BY g ORDER BY g;
SELECT roll_no, grade FROM enrolment ORDER BY grade NULLS LAST LIMIT 3;
```

What to notice. Step 5 against step 6 is the chapter in one comparison: the same question, two spellings, 0 against 3. Neither raised an error, and the wrong one is the one most people write first. If you remember one rule from Part IV, make it this: **never** NOT IN **a column that can be null.**

Checkpoint — Test Yourself

1. `grade = 'A'` returns 7,268 rows and `grade <> 'A'` returns 32,777, on a table of 60,037. Account for the remaining rows using the truth table, and write the single query that returns every row that is *not* an A, including the ungraded ones.
2. Explain, step by step from the expansion of NOT IN into ANDs, why one null in the sub-query's result makes the outer query return no rows at all.
3. A colleague writes `AVG(COALESCE(score, 0))` "so that the average includes everybody". Say what decision this makes about missing marks, show with a three-row example how it changes the average, and say what you would write instead and why.

Chapter Summary

- SELECT runs FROM, WHERE, GROUP BY, SELECT, ORDER BY, LIMIT. An alias does not exist in WHERE; it does in ORDER BY.

- BETWEEN is inclusive; SQLite's LIKE is case-insensitive for ASCII; LIMIT without ORDER BY returns rows the plan chose.
- Any comparison with a null is UNKNOWN. WHERE keeps only *true*, so UNKNOWN rows vanish.
- NOT UNKNOWN **is** UNKNOWN: a condition and its negation do not cover a table with nulls.
- **On the seeded data:** `= 'A'` found 7,268, `<> 'A'` found 32,777, and **19,992 ungraded rows were in neither**.
- `= NULL` is always empty. IS NULL is the only test.
- NOT IN **returned 0 where** NOT EXISTS **returned 3**, because one null in the list makes every comparison UNKNOWN. Never NOT IN a nullable column.
- COALESCE decides what null means; use it for display, never to turn a missing mark into a zero.
- SQLite sorts nulls first ascending; say NULLS LAST when it matters, because other systems differ.

Review Questions

1. (MCQ) `NULL = NULL` evaluates to: (a) true; (b) false; (c) UNKNOWN; (d) an error.

2. (MCQ) `semester BETWEEN 2 AND 4` includes: (a) 3 only; (b) 2 and 3; (c) 3 and 4; (d) 2, 3 and 4.

3. (Short) Why can a column alias be used in ORDER BY but not in WHERE?

4. (Short) Evaluate `F OR U`, `T AND U` and NOT `U`, and say which one causes the most errors in practice.

5. (Short) Give the two fixes for a NOT IN against a nullable column, and say why this course prefers one of them.

6. (Applied) Write a query listing every student with a label computed by CASE: `'senior'` for semesters 7–8, `'mid'` for 3–6, `'new'` for 1–2. Then say what your query does with a null semester, and change it so that a null is labelled `'unknown'` explicitly.

7. (Applied) You are asked for "students who have never failed a course". Write it three ways — with NOT IN, with NOT EXISTS, and with EXCEPT — run all three on the seeded database, and explain any difference between their counts in terms of the null grades of FALL2026.

Chapter 15

Sub-queries, EXISTS and Set Operations

Learning Outcomes

By the end of this chapter you will be able to:

- **Write** a sub-query in the WHERE, FROM and SELECT clauses, and **say** what each one must return.
- **Distinguish** a correlated from an uncorrelated sub-query by reading it.
- **Write** a semi-join and an anti-join in several forms, and **choose** one that is correct whatever the nulls.
- **Explain** what makes a correlated sub-query expensive, and **show** that it is not the correlation.
- **Use** a common table expression to give a sub-query a name.

Before You Start

Chapter 14, and especially its NOT IN alert — this chapter measures the four alternatives. Chapter 13 for the anti-join idiom. You need `code/m15_subqueries.py`.

Key Terms in This Chapter

sub-query • nested query • scalar sub-query • correlated sub-query • uncorrelated sub-query • semi-join • anti-join • IN • EXISTS • ANY • ALL • derived table • common table expression • WITH • UNION • UNION ALL • EXCEPT

15.1 A Query Inside a Query

Chapter 12's closure property — every result is a relation — is what makes this chapter possible. A SELECT produces a relation, so a SELECT can appear wherever a relation, a list or a single value is expected.

Three places a sub-query can go

A sub-query is also called a **nested query**.

In WHERE, after IN or EXISTS: it supplies a list of values, or the fact that some row exists.

In FROM: it supplies a whole relation, called a **derived table**, which must be given an alias.

In the SELECT **list**, or in a comparison: it must be a **scalar sub-query**, returning exactly one column and at most one row. If it returns no rows its value is null; if it returns two, SQLite silently uses the first — other systems raise an error.

Correlated and uncorrelated

An **uncorrelated** sub-query mentions only its own tables. It could be run on its own, and in principle it is run once.

A **correlated** sub-query mentions a column of the *outer* query, so its answer depends on which outer row is being considered. In principle it is run once per outer row.

```sql
-- uncorrelated: the inner query stands alone
WHERE roll_no IN (SELECT roll_no FROM enrolment WHERE grade = 'A')

-- correlated: s.roll_no belongs to the OUTER query
WHERE EXISTS (SELECT 1 FROM enrolment e
              WHERE e.roll_no = s.roll_no AND e.grade = 'A')
```

"In principle" is doing work in both definitions. The optimiser is free to run either one however it likes, and the measurement below shows it doing so.

15.2 Semi-joins and Anti-joins

Semi-join, anti-join

A **semi-join** returns the rows of R that have *at least one* partner in S — each row once, however many partners it has. "Students with at least one A."

An **anti-join** returns the rows of R that have *no* partner in S. "Instructors who teach nothing." It is Chapter 12's difference, and "never", "none" and "no" always mean one.

Measured: Seven spellings, one wrong answer, and correlation that was not the problem

`code/m15_subqueries.py`, seeded database, SQLite 3.51.2, Intel i5-6600, Windows 11. Best of three runs.

1. A semi-join, three ways — students with at least one A

written as	answer	ms	
IN (uncorrelated)	3,449	20.3	
EXISTS (correlated)	3,449	40.1	twice the IN
JOIN + DISTINCT	3,449	21.4	

2. An anti-join, four ways — instructors who teach nothing, after three are hired who teach nothing yet; `section.emp_id` has 73 nulls

written as	answer	ms	right?
NOT IN	**0**	0.76	**no** — one null empties it
NOT EXISTS	3	2.18	yes
LEFT JOIN ... IS NULL	3	2.52	yes
EXCEPT	3	0.44	yes

The fastest wrong answer is no use. EXCEPT is a set operation rather than a comparison, so the null on its right-hand side can only remove a null on its left — and `instructor.emp_id`, a primary key, has none.

3. A correlated sub-query in the SELECT list — enrolments per course

		no index on course_code		with the index	
students	**enrolments**	**correlated**	GROUP BY	**correlated**	GROUP BY
2,000	30,027	191.1	53.8	**2.4**	5.4
4,000	60,037	517.7	121.3	**5.1**	12.2
8,000	120,134	1,101.4	275.8	**10.3**	23.7

All four agree on every count. Times in milliseconds.

Read part 3 carefully, because it contradicts the folklore. Without an index the correlated form is about four times slower than GROUP BY: *enrolment*'s primary key is `(roll_no, course_code, ...)`, so it cannot be searched by `course_code`, and each of the 48 courses scans all 60 thousand enrolments. **Add one index and the correlated form**

becomes the faster of the two — 5.1 against 12.2 ms — because it now does 48 index lookups, while GROUP BY still reads every enrolment. Both grow linearly with the data; neither diverges from the other as the table grows.

So the cost was never the correlation. It was whether the inner query had an index on the column it is correlated on. The plans say so directly: `SCAN e` inside `CORRELATED SCALAR SUBQUERY` without it, `SEARCH e USING ... idx_enr_course (course_code=?)` with it.

What to take from the measurement

- **For anti-joins, use NOT EXISTS.** It is correct whatever the nulls, and its cost is in the same range as the alternatives. EXCEPT is faster when it fits but returns only the compared columns; LEFT JOIN ... IS NULL is correct but easy to break with a condition in the wrong clause (Chapter 13).
- **Do not avoid correlated sub-queries on principle.** Avoid unindexed ones. "Correlated sub-queries are slow" is a rule of thumb learned on databases without the right index, and on this one it was backwards by a factor of two.
- **Read the plan before rewriting a query.** SCAN inside a CORRELATED sub-query is the specific thing to look for; it means the inner query reads a whole table once per outer row.

15.3 ANY, ALL, and Naming a Sub-query

ALL and ANY, and the null trap again

`x > ALL (sub-query)` is true when x exceeds every value returned; `x > ANY (sub-query)` when it exceeds at least one. SQLite does not support either keyword, which is no loss — both are clearer as an aggregate:

```
-- "credit hours greater than every MA course"
WHERE credit_hours > (SELECT MAX(credit_hours) FROM course
                      WHERE dept_code = 'MA')
```

And ALL has NOT IN's trap: `x > ALL (..., NULL)` is UNKNOWN for every x, because one of its comparisons is. The aggregate form ignores nulls, which is usually what was meant.

Common table expression

A **common table expression** (CTE) names a sub-query before the main query uses it:

```
WITH a_students AS (
  SELECT DISTINCT roll_no FROM enrolment WHERE grade = 'A'
)
SELECT s.name
  FROM student s JOIN a_students a ON a.roll_no = s.roll_no
 WHERE s.programme = 'BSIT';
```

It computes nothing a derived table could not. Its value is that the sub-query has a *name*, can be read top to bottom, and can be referred to more than once. WITH RECURSIVE — used in Chapter 3's laboratory to generate 300 rows — is the one form that does something new: it can walk the prerequisite chain to any depth, which no fixed number of joins can.

Worked Example: Every prerequisite of a course, to any depth

The question. Before taking a course, which courses must a student have passed — directly, or because a prerequisite has a prerequisite?

Step 1 — why a join cannot do it. One join finds direct prerequisites. Two joins find their prerequisites. Each extra level needs an extra join, and nobody knows in advance how deep the chain goes.

Step 2 — the recursive CTE. Start from the direct prerequisites (the *anchor*), then repeatedly add the prerequisites of whatever is already found (the *recursive step*), until a step adds nothing:

```
WITH RECURSIVE needs(code) AS (
  SELECT requires FROM prerequisite
   WHERE course_code = 'CS-305'                  -- the anchor
  UNION
  SELECT p.requires FROM prerequisite p
    JOIN needs n ON p.course_code = n.code       -- the step
)
SELECT code FROM needs;
```

Step 3 — the answer, on the seeded database. `CS-305` directly requires two courses, `CS-201` and `CS-258`. The query returns **four**: those two, plus `CS-150` (required by `CS-258`) and `CS-101` (required by `CS-150`). The chain is three levels deep — 305, 258, 150, 101 — and a query with one join would have found half of it.

Step 4 — why UNION and not UNION ALL. UNION discards rows already found, so a course reachable by two paths is listed once — and, more importantly, a cycle in the data (A requires B requires A) terminates instead of running for ever. **Chapter 9's CHECK stops a course requiring itself, but not a two-step cycle.** Only UNION saves you from that one.

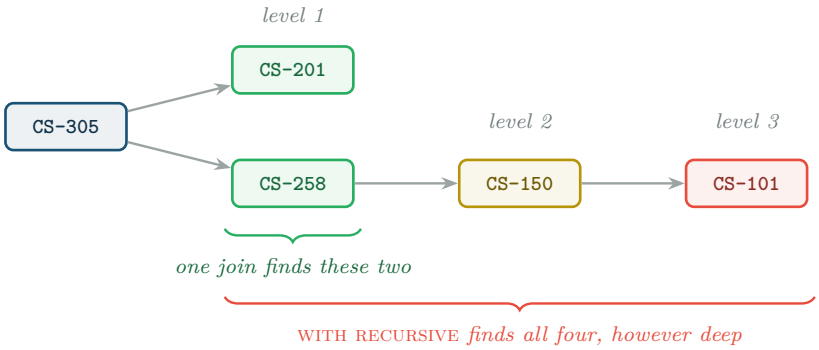


Figure 15.1: The prerequisites of `CS-305` on the seeded database. An arrow means "requires". A fixed query reaches a fixed depth; the recursive one keeps going until a step adds nothing new.

Common Pitfalls

- NOT IN **against a nullable column.** Measured: 0 instead of 3.
- **Assuming correlation is the cost.** The missing index is. Measured: with the index, the correlated form was the faster.
- **A scalar sub-query that returns two rows.** SQLite quietly uses one; other systems refuse.
- **A derived table with no alias.** Required.
- UNION **where** UNION ALL **was meant**, or the reverse. UNION removes duplicates and costs a sort; in a recursive CTE it is also what stops a cycle.
- $x >$ ALL **over a list with a null.** UNKNOWN for every x. Use MAX.

Try It Yourself: Write one question seven ways, and find the one that lies

```
# --- 1. The seeded database ----------------------------------------
python code/schema/seed.py university.db
sqlite3 university.db
```

```
-- --- 2. A semi-join three ways ---------------------------------
.headers on
.mode box
.timer on
SELECT COUNT(*) FROM student WHERE roll_no IN
  (SELECT roll_no FROM enrolment WHERE grade = 'A');
SELECT COUNT(*) FROM student s WHERE EXISTS
  (SELECT 1 FROM enrolment e WHERE e.roll_no = s.roll_no
                               AND e.grade = 'A');
SELECT COUNT(DISTINCT s.roll_no) FROM student s
  JOIN enrolment e ON e.roll_no = s.roll_no WHERE e.grade = 'A';
-- Same answer three times? Record the three timings.

-- --- 3. Hire three people, then an anti-join four ways ----------
PRAGMA foreign_keys = ON;
INSERT INTO instructor VALUES
  ('E-9001','New Hire One','Lecturer','CS'),
  ('E-9002','New Hire Two','Lecturer','IT'),
  ('E-9003','New Hire Three','Lecturer','SE');
SELECT COUNT(*) FROM instructor
 WHERE emp_id NOT IN (SELECT emp_id FROM section);
SELECT COUNT(*) FROM instructor i WHERE NOT EXISTS
  (SELECT 1 FROM section s WHERE s.emp_id = i.emp_id);
SELECT COUNT(*) FROM instructor i LEFT JOIN section s
    ON s.emp_id = i.emp_id WHERE s.emp_id IS NULL;
SELECT COUNT(*) FROM (SELECT emp_id FROM instructor
                      EXCEPT SELECT emp_id FROM section);
-- Three say 3. One says 0. Which, and why?

-- --- 4. A correlated sub-query, and its plan ---------------------
EXPLAIN QUERY PLAN
SELECT c.course_code, (SELECT COUNT(*) FROM enrolment e
                       WHERE e.course_code = c.course_code)
  FROM course c;
-- Find SCAN inside CORRELATED. Then time it:
SELECT c.course_code, (SELECT COUNT(*) FROM enrolment e
                       WHERE e.course_code = c.course_code)
  FROM course c;

-- --- 5. Give the inner query an index, and look again ------------
CREATE INDEX idx_enr_course ON enrolment(course_code);
EXPLAIN QUERY PLAN
SELECT c.course_code, (SELECT COUNT(*) FROM enrolment e
                       WHERE e.course_code = c.course_code)
  FROM course c;
-- SCAN has become SEARCH. Time it again and compare.

-- --- 6. Every prerequisite, to any depth -----------------------
SELECT requires FROM prerequisite WHERE course_code = 'CS-305';
-- The direct prerequisites: two. Now every one, to any depth:
WITH RECURSIVE needs(code) AS (
  SELECT requires FROM prerequisite WHERE course_code = 'CS-305'
  UNION
  SELECT p.requires FROM prerequisite p JOIN needs n
      ON p.course_code = n.code)
SELECT code FROM needs;
```

```
-- Four. Draw the chain on paper and check every one.
```

```
# --- 7. The measured version ----------------------------------------
python code/m15_subqueries.py
```

What to notice. Step 5 is where a rule of thumb meets a measurement. The correlated query you were told to avoid becomes faster than the GROUP BY you were told to prefer, because of one index and nothing else. Rules about query *shape* are usually rules about indexes in disguise, and Chapter 21 takes the disguise off.

Checkpoint — Test Yourself

1. Classify each sub-query in the laboratory as correlated or uncorrelated, and for each correlated one name the outer column it refers to.
2. Of the four anti-join forms, one returned 0 and three returned 3. Say which, explain why in terms of three-valued logic, and say why EXCEPT was not affected by the same nulls.
3. Without an index the correlated sub-query was four times slower than GROUP BY; with one it was twice as fast. Explain both results in terms of what each query reads, and say what this tells you about the advice "avoid correlated sub-queries".

Chapter Summary

- A sub-query can go in WHERE (a list, or existence), FROM (a derived table, which needs an alias) or the SELECT list (a scalar: one column, at most one row).
- A **correlated** sub-query mentions an outer column and, in principle, runs per outer row. The optimiser decides what actually happens.
- A **semi-join** keeps rows with at least one partner; an **anti-join** keeps rows with none.
- **Measured: the three semi-join forms agreed** (3,449); EXISTS took twice as long as IN here.
- **Measured:** NOT IN **returned 0** where NOT EXISTS, LEFT JOIN ... IS NULL and EXCEPT all returned 3. Use NOT EXISTS.
- **Measured: without an index on the correlation column, the correlated form was 4× slower than** GROUP BY**; with one it was 2× faster.** The cost was never the correlation.
- SCAN inside a CORRELATED sub-query in a plan means a whole table read per outer row. Look for it.
- A **CTE** names a sub-query. WITH RECURSIVE walks a chain to any depth, and UNION in it is what stops a cycle.
- ALL over a list containing a null is UNKNOWN; use an aggregate.

Review Questions

1. (MCQ) A sub-query that mentions a column of the outer query is: (a) scalar; (b) derived; (c) correlated; (d) recursive.

2. (MCQ) Which anti-join form can return no rows because of a single null? (a) NOT EXISTS; (b) NOT IN; (c) EXCEPT; (d) LEFT JOIN ... IS NULL.

3. (Short) What must a scalar sub-query return, and what does SQLite do if it returns more?

4. (Short) Why does a recursive CTE over *prerequisite* need UNION rather than UNION ALL?

5. (Short) State in one sentence what decides whether a correlated sub-query is expensive.

6. **(Applied)** Write "courses with no enrolments in FALL2026" as NOT EXISTS, as LEFT JOIN ... IS NULL and as EXCEPT. Run all three on the seeded database (insert one new course first so the answer is not empty), confirm they agree, and read each plan to say which index each one uses.

7. **(Applied)** A student's prerequisite chain is to be checked at enrolment time. Write the recursive query that lists every course a student must have passed before taking a given one, then extend it to list only those the student has *not* passed. Say what happens if somebody inserts a two-step cycle, and how your query behaves.

Chapter 16

Grouping, Aggregation and Window Functions

Learning Outcomes

By the end of this chapter you will be able to:

- **Use** COUNT, SUM, AVG, MIN and MAX, and **predict** what each does with a null.
- **Write** a GROUP BY query and **state** which columns may appear in its SELECT list.
- **Choose** between WHERE and HAVING by what the condition is about.
- **Write** a window function with PARTITION BY and ORDER BY, and **explain** how it differs from grouping.
- **Read** a query plan for TEMP B-TREE, and **say** what index would remove it.

Before You Start

Chapter 14 for three-valued logic, which the aggregates handle in their own way. Chapter 15 for correlated sub-queries, one of which this chapter replaces. You need `code/m16_grouping.py`.

Key Terms in This Chapter

aggregate function • COUNT • SUM • AVG • MIN • MAX • GROUP BY • group • HAVING • bare column • window function • OVER • PARTITION BY • RANK • DENSE_RANK • ROW_NUMBER • running total • TEMP B-TREE

16.1 Collapsing Rows Into One

Every query so far has returned rows that existed — possibly filtered, possibly joined, but each one traceable to a tuple in the database. An **aggregate function** returns a row that existed nowhere: the *average* grade, the *number* of students.

The five aggregates, and what they skip

COUNT(*)	the number of rows. Counts everything, nulls included.
COUNT(x)	the number of rows where x is **not null**.
SUM, AVG	over the non-null values only. AVG divides by the number of non-null values, not by the number of rows.
MIN, MAX	over the non-null values only.

Every aggregate except COUNT(*) **skips nulls**. Over a set that is entirely null, SUM and AVG return null — not zero — and COUNT returns 0.

16.2 GROUP BY

Grouping

GROUP BY *a* partitions the rows into groups, one for each distinct value of *a*, and an aggregate in the SELECT list is then computed *per group*. The result has one row per group.

The rule for the SELECT list: every column in it must either be in the GROUP BY, or be inside an aggregate. A column that is neither — a **bare column** — has no single value per group, so there is nothing correct to print.

Null forms a group of its own. All the nulls are grouped together, even though `NULL = NULL` is UNKNOWN: grouping uses "is the same value" rather than "compares equal".

SQLite allows the bare column, and it should not

```
SELECT programme, name, COUNT(*) FROM student GROUP BY programme;
```

PostgreSQL rejects this: `name` is neither grouped nor aggregated. SQLite runs it and prints, for each programme, the name of *some* student in it — whichever row the engine happened to process last. The output looks like a meaningful answer, and on each run it may name a different student.

There is one documented exception: with exactly one MIN or MAX, SQLite guarantees the bare columns come from the row holding that minimum or maximum. It is a convenience no other system offers, so code that depends on it will not move. **In this course, every non-aggregated column is in the GROUP BY.**

16.3 Measured: Nulls, Plans, and a Rank

Measured: What aggregates skip, what an index removes, and what a window replaces

`code/m16_grouping.py`, seeded database, SQLite 3.51.2, Intel i5-6600, Windows 11. Best of three runs.

1. Aggregates and nulls — *enrolment*, 60,037 rows

COUNT(*)	60,037	every row
COUNT(grade)	40,045	rows where grade is not null
difference	19,992	the ungraded FALL2026 rows
AVG(points)	**2.383**	nulls skipped: ÷ 40,045
AVG(COALESCE(points, 0))	**1.590**	nulls as zero: ÷ 60,037

The second average is **33% lower**, and it has decided that an ungraded course is an F. GROUP BY grade puts the 19,992 nulls in a group of their own, beside A (7,268), B (12,756), C (11,283), D (5,524) and F (3,214).

2. GROUP BY with and without a supporting index — enrolments per course

students	enrolments	no index	with index	speed-up
4,000	60,037	37.0 ms	6.8 ms	5.5×
8,000	120,134	86.3 ms	13.9 ms	6.2×
16,000	240,025	174.2 ms	31.3 ms	5.6×

Without the index the plan says `USE TEMP B-TREE FOR GROUP BY`: the rows were sorted into groups before they could be counted. An index on `course_code` is already in that order, so the sort disappears from the plan — and five-sixths of the cost goes with it. The speed-up is roughly constant as the table grows, because both forms are linear and the sort was a fixed proportion of the work.

3. The same filter in WHERE and in HAVING

	ms	groups	
`WHERE course_code LIKE 'CS-%'`	13.1	6	
`HAVING course_code LIKE 'CS-%'`	13.9	6	same answer

The two compiled to the identical program, 47 bytecode operations each, with the LIKE tested at operation 8, *before* the grouping sort at operation 12. SQLite moved the HAVING condition into WHERE by itself — which it can do only because the condition mentions no aggregate. `HAVING COUNT(*) > 1000` was tested at operation 33, *after* the sort at operation 9, because a count does not exist until the grouping is done.

4. A rank within each programme — by number of A grades

students	window RANK()	correlated	ratio	agree?
1,000	16.9 ms	51.6 ms	3×	yes
2,000	38.4 ms	190.4 ms	5×	yes
4,000	103.2 ms	736.7 ms	**7**×	yes

The ratio grows with the data. The correlated form counts, for each student, how many in the same programme did better — every student against every other in the programme, quadratic. The window function sorts each programme once and reads the rank off the order.

What the four results say about how to write these queries

- **Never COALESCE to zero inside an aggregate unless a missing value really is a zero.** It moved the average by a third. That is the same fact Chapter 10 found with a text value in a numeric column, approached from the other side.
- **TEMP B-TREE in a plan is a sort you could have avoided.** An index on the grouping column supplies the order for free. Chapter 21 shows where the same idea stops working.
- **Put a condition on a column in WHERE because that is what it means, not because it is faster here.** SQLite fixed the HAVING version itself; that is an optimisation, not a promise, and the next system you use may not make it.
- **For rankings, running totals and "top n per group", use a window function.** The correlated alternative was seven times slower at 4,000 students and getting worse.

16.4 Window Functions

A window function

A **window function** computes a value for each row from a set of related rows — its *window* — **without collapsing them**. Grouping turns many rows into one; a window function keeps every row and adds a column.

```
SELECT roll_no, programme, n_a,
       RANK() OVER (PARTITION BY programme ORDER BY n_a DESC) AS pos
  FROM per_student;
```

PARTITION BY divides the rows into windows, like GROUP BY but without merging them. ORDER BY inside OVER orders each window. The function then runs along the ordered window.

ROW_NUMBER, RANK and DENSE_RANK on a tie

Four students in one programme with 9, 7, 7 and 5 A grades:

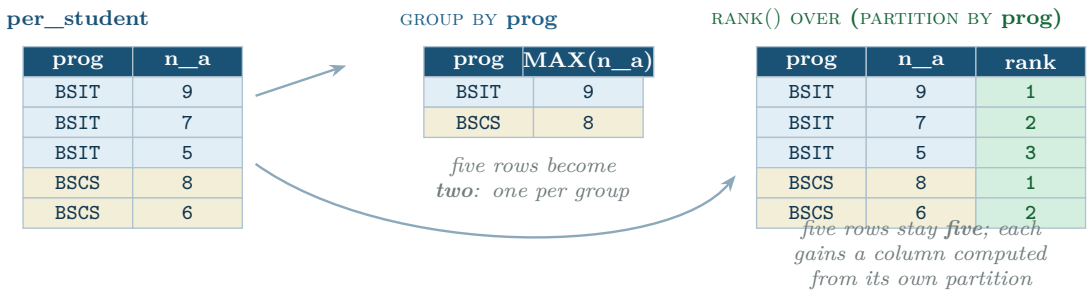


Figure 16.1: The difference in one picture. Grouping collapses each partition into a single row; a window function keeps every row and adds a value computed from the partition it belongs to. The colours mark the two partitions.

A grades	ROW_NUMBER	RANK	DENSE_RANK
9	1	1	1
7	2	2	2
7	3	2	2
5	4	**4**	**3**

ROW_NUMBER never ties, so it numbers the two sevens arbitrarily — use it only when the order is fully determined. RANK gives ties the same number and *skips* the next, as a sports table does. DENSE_RANK does not skip. The university's merit list is a policy decision between the last two, and the query has to make it explicitly.

Worked Example: A running total of credit hours, per student, in term order

The question. For each enrolment of one student, how many credit hours had they completed up to and including that term?

Step 1 — why grouping cannot do it. GROUP BY roll_no gives one total per student. The question wants every row kept, each with its own total so far.

Step 2 — the window.

```
SELECT e.semester, c.course_code, c.credit_hours,
       SUM(c.credit_hours) OVER (
         PARTITION BY e.roll_no
         ORDER BY e.semester
         ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
       ) AS hours_so_far
  FROM enrolment e JOIN course c ON c.course_code = e.course_code
 WHERE e.roll_no = (SELECT MIN(roll_no) FROM student);
```

Step 3 — the frame. ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW says "from the first row of the window to this one". That is what makes it a running total rather than a grand total repeated on every row.

Step 4 — a trap in the ordering. `ORDER BY e.semester` sorts the text `'FALL2025'`, `'FALL2026'`, `'SPRING2026'` *alphabetically* — and so puts Fall 2026 before Spring 2026, which is wrong. A term label that sorts correctly as text has to be designed to, `'2025-2'` rather than `'FALL2025'`. This is the same column that broke NATURAL JOIN in Chapter 13, and the second reason to revisit it when Chapter 18 reconsiders the schema.

Common Pitfalls

- **A bare column in a grouped query.** SQLite prints some row's value; other systems refuse.
- COALESCE **to zero inside** AVG**.** Measured: a third off.
- **Expecting** SUM **of nothing to be 0.** It is null. COALESCE(SUM(x), 0) is the honest fix there.
- HAVING **for a condition on a column.** It works here, because SQLite rewrites it. Write WHERE.
- **A correlated sub-query for a rank.** Measured: seven times slower and growing. Use RANK OVER.
- ROW_NUMBER **where ties are possible.** It breaks them arbitrarily.
- **Ordering by a term label stored as text.** It sorts alphabetically, which is not chronologically.

Try It Yourself: Move an average by a third, remove a sort, and rank a programme

```
# --- 1. The seeded database ----------------------------------------
python code/schema/seed.py university.db
sqlite3 university.db
```

```
-- --- 2. Count three ways -------------------------------------------
.headers on
.mode box
.timer on
SELECT COUNT(*), COUNT(grade), COUNT(DISTINCT grade) FROM enrolment;
-- Three numbers. Say what each one counts.

-- --- 3. Two averages, one of them a decision about failure ------
SELECT AVG(CASE grade WHEN 'A' THEN 4 WHEN 'B' THEN 3 WHEN 'C' THEN 2
                      WHEN 'D' THEN 1 WHEN 'F' THEN 0 END) FROM enrolment;
SELECT AVG(COALESCE(CASE grade WHEN 'A' THEN 4 WHEN 'B' THEN 3
               WHEN 'C' THEN 2 WHEN 'D' THEN 1 WHEN 'F' THEN 0 END, 0))
  FROM enrolment;
-- By what percentage do they differ, and what did the second decide?

-- --- 4. The null group ---------------------------------------------
SELECT grade, COUNT(*) FROM enrolment GROUP BY grade ORDER BY grade;

-- --- 5. A sort you can see -----------------------------------------
EXPLAIN QUERY PLAN
SELECT course_code, COUNT(*) FROM enrolment GROUP BY course_code;
SELECT course_code, COUNT(*) FROM enrolment GROUP BY course_code;
-- Find TEMP B-TREE. Record the time.

-- --- 6. Remove it --------------------------------------------------
CREATE INDEX idx_enr_course ON enrolment(course_code);
EXPLAIN QUERY PLAN
SELECT course_code, COUNT(*) FROM enrolment GROUP BY course_code;
SELECT course_code, COUNT(*) FROM enrolment GROUP BY course_code;
-- What disappeared from the plan, and what happened to the time?

-- --- 7. WHERE against HAVING, and a condition that cannot move ---
SELECT course_code, COUNT(*) FROM enrolment
 GROUP BY course_code HAVING COUNT(*) > 1300;
-- Try to write that one with WHERE instead. Why can you not?

-- --- 8. A rank, with a window --------------------------------------
```

```
WITH a AS (SELECT s.roll_no, s.programme, SUM(e.grade = 'A') AS n_a
             FROM student s JOIN enrolment e ON e.roll_no = s.roll_no
            GROUP BY s.roll_no)
SELECT programme, roll_no, n_a,
       RANK()       OVER w AS rnk,
       DENSE_RANK() OVER w AS dense
  FROM a WINDOW w AS (PARTITION BY programme ORDER BY n_a DESC)
 ORDER BY programme, rnk LIMIT 12;
-- Find a tie. Show where RANK and DENSE_RANK disagree.
```

```
# --- 9. The measured version ----------------------------------------
python code/m16_grouping.py
```

What to notice. Step 7 is the real difference between the two clauses. A condition on `course_code` could go in either, and SQLite quietly makes them identical. A condition on COUNT(*) can only go in HAVING, because when WHERE runs the count has not been computed. Choose by *what the condition is about*, and the clause chooses itself.

Checkpoint — Test Yourself

1. On the seeded data AVG(points) is 2.383 and AVG(COALESCE(points, 0)) is 1.590. Explain both numbers in terms of what each one divides by, and say which one the examinations office should publish and why.
2. The WHERE and HAVING versions of the same filter compiled to identical programs. Explain why SQLite was able to do that for `course_code LIKE 'CS-%'` and not for `COUNT(*) > 1000`, and say why you should write WHERE anyway.
3. Give a question that needs a window function rather than GROUP BY, write the query, and say what GROUP BY would have lost.

Chapter Summary

- Every aggregate except COUNT(*) **skips nulls**; AVG divides by the non-null count; SUM of nothing is null, not 0.
- **Measured:** AVG skipping nulls gave 2.383; COALESCE to zero gave 1.590 — **a third lower**, and a decision that ungraded means failed.
- GROUP BY makes one row per group. Every SELECT column must be grouped or aggregated; SQLite permits a bare column and prints an arbitrary row's value.
- Null forms its own group.
- **Measured:** an index on the grouping column removed TEMP B-TREE FOR GROUP BY from the plan and made the query **5.5–6.2**× faster.
- **Measured:** the WHERE and HAVING forms of a column condition **compiled to the same program** — SQLite moved it. An aggregate condition cannot move. Choose by what the condition is about.
- A **window function** adds a value to every row without collapsing them. PARTITION BY defines the windows, ORDER BY orders them, the frame bounds them.
- **Measured:** a window RANK() was **7**× faster than the correlated equivalent at 4,000 students, and the gap was widening.
- RANK skips after a tie, DENSE_RANK does not, ROW_NUMBER breaks ties arbitrarily.
- A term label stored as `'FALL2025'` sorts alphabetically, not chronologically — the second fault found in the `semester` column.

Review Questions

1. **(MCQ)** On a column with 10 rows of which 4 are null, COUNT(x) returns: (a) 10; (b) 6; (c) 4; (d) null.
2. **(MCQ)** A condition on COUNT(*) must go in: (a) WHERE; (b) HAVING; (c) ORDER BY; (d) either.
3. **(Short)** Why can a bare column in a grouped query not have a correct value, and what does SQLite print instead?
4. **(Short)** What does TEMP B-TREE FOR GROUP BY in a plan mean, and what removes it?
5. **(Short)** Give four rows with a tie and show how RANK, DENSE_RANK and ROW_NUMBER number them.
6. **(Applied)** Write the query giving each programme's average grade points, number of graded enrolments and number of ungraded ones, only for programmes with more than 5,000 graded enrolments. State for each condition which clause it belongs in, and why.
7. **(Applied)** The university wants a merit list: the top three students by grade points in each programme, ties included. Write it with a window function, say whether you chose RANK or DENSE_RANK and what the choice means for a student tied in third place, and estimate from the measurement how the correlated alternative would behave at 16,000 students.

Part V

Getting the Design Right

Part V is the chapter this course exists for, and the three around it. Normalisation has a reputation for being the dull, formal part of databases; it is in fact the only part that tells you your design is wrong before *the data proves it. The whole of it follows from one idea, met in Chapter 17, which is simpler than its notation suggests.*

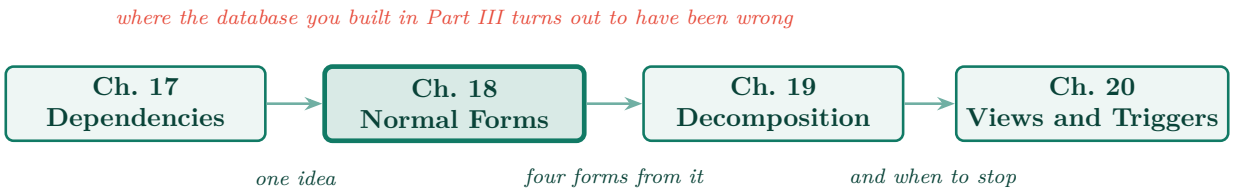


Figure P5.1: Reading path through Part V. Chapter 18 breaks the running database and rebuilds it, and Chapter 19 measures what that cost — because normalising is a trade and a course that presents it as a pure good has not finished the argument.

Chapter 17

Functional Dependencies

Learning Outcomes

By the end of this chapter you will be able to:

- **Define** a functional dependency and **read** one written as X → Y.
- **State** Armstrong's three axioms and **derive** new dependencies from given ones.
- **Compute** the closure X^+ of a set of attributes, step by step.
- **Use** attribute closure to test whether a set of attributes is a superkey or a candidate key.
- **Reduce** a set of dependencies to a minimal cover.

Before You Start

Chapter 7 for keys. Appendix B for sets and the meaning of "function" — the Prerequisite Self-Check's question 12 is the whole foundation of this chapter, and if it was uncomfortable, read Appendix B first, not during.

Key Terms in This Chapter

functional dependency • determinant • dependent • trivial dependency • Armstrong's axioms • reflexivity • augmentation • transitivity • union rule • decomposition rule • attribute closure • X^+ • superkey • candidate key • minimal cover • extraneous attribute • redundant dependency

17.1 One Table That Everyone Has Built

Every university has a spreadsheet like this one. It is the first thing a department builds when it is asked for "all the results in one place", and it is the table this Part is about:

REPORT(roll_no, name, programme, course_code, title, credit_hours, dept_code, dept_name, semester, section_no, room, emp_id, instructor, grade)

Fourteen columns, one row per enrolment, and on the seeded data 60,037 rows. It answers every question in Part IV without a single join, which is why people build it. Chapter 18 measures what it costs. This chapter supplies the vocabulary for saying *why* it costs anything — and the vocabulary is one idea.

Functional dependency

A **functional dependency** X → Y, read "X determines Y", holds in a relation when any two tuples that agree on X must also agree on Y.

X is the **determinant**; Y is the **dependent**. Both are sets of attributes.

It is the word "function" in its mathematical sense, from Appendix B: give me the value of X and there is exactly one value of Y it can be paired with. roll_no → name says a roll number has exactly one name.

</> The dependencies in *report*, found by asking

As with keys in Chapter 5, these come from the university, not from the data. Each one is a rule somebody can confirm:

Dependency	Because
roll_no → name, programme	a student has one name and is in one programme
course_code → title, credit_hours, dept_code	a course is defined once
dept_code → dept_name	a department has one name
course_code, semester, section_no → room, emp_id	a section meets in one room with one instructor
emp_id → instructor	an employee has one name
roll_no, course_code, semester, section_no → grade	one grade per enrolment

Six rules. Notice that each one is a sentence the examinations officer would agree with immediately — and that each one says the dependent is stored in *report once per enrolment*, when it is one fact. Ayesha's name is in the table once for every section she has ever taken.

⚠ An instance can disprove a dependency, never prove one

Chapter 7 said this about keys, and it is the same fact, because a key is a dependency (below). On the seeded data, *name* happens to determine *programme* for some names. That is coincidence, not a rule — two students called Hira Nadeem can be in different programmes, and the next admission may be the one that proves it.

A query *can* disprove a dependency, and it is worth knowing:

```
-- Does course_code -> title hold in this data?
SELECT course_code FROM report
 GROUP BY course_code HAVING COUNT(DISTINCT title) > 1;
-- Any row returned is a counter-example. None returned proves nothing.
```

17.2 Deriving New Dependencies

Armstrong's axioms

For sets of attributes X, Y, Z:

Reflexivity. If $Y \subseteq X$ then X → Y. (A set determines its own parts. Such a dependency is **trivial**.)

Augmentation. If X → Y then XZ → YZ. (Adding the same attributes to both sides keeps it true.)

Transitivity. If X → Y and Y → Z then X → Z.

These three are *sound* (they never derive anything false) and *complete* (they derive everything that follows). Two derived rules save time: **union** — X → Y and X → Z give X → YZ; and **decomposition** — X → YZ gives X → Y and X → Z.

</> One transitive dependency, and why it is the important one

From the table:

- course_code → dept_code (given, by decomposition)
- dept_code → dept_name (given)

- therefore course_code → dept_name (transitivity)

Nobody wrote that third rule down, and it is true. It is also the one that causes trouble: *dept_name* depends on *course_code* only *through dept_code*. Renaming the department means changing every row of every course it offers — Chapter 18 calls that a transitive dependency and third normal form forbids it.

17.3 Attribute Closure

Deriving dependencies one axiom at a time is how proofs are written. The practical question is narrower: *given X, what does it determine?* There is an algorithm, and it is the most useful thing in Part V.

Attribute closure X^+

The **closure** X^+ of a set of attributes X under a set of dependencies F is the set of every attribute that X determines.

Algorithm.

1. Start with $X^+ = X$.
2. Repeat: for each dependency A → B in F, if $A \subseteq X^+$, add B to X^+.
3. Stop when a full pass adds nothing.

It always terminates — there are finitely many attributes and X^+ only grows.

Worked Example: $\{course_code\}^+$, then the key of *report*

Using the six dependencies from the examplebox, abbreviated F.

Part A — what does *course_code* determine?

pass	dependency used	X^+ is now
start	—	{course_code}
1	course_code → title, credit_hours, dept_code	{course_code, title, credit_hours, dept_code}
1	dept_code → dept_name — its left side is now inside	… plus dept_name
2	nothing new applies	**stop**

$\{course_code\}^+ = \{$course_code, title, credit_hours, dept_code, dept_name$\}$. Five of the fourteen. *course_code* is not a key of *report*.

Part B — the key. Try $K = \{$roll_no, course_code, semester, section_no$\}$:

- roll_no → name, programme adds 2;
- course_code → title, credit_hours, dept_code adds 3, and then dept_code → dept_name adds 1;
- course_code, semester, section_no → room, emp_id adds 2, and then emp_id → instructor adds 1;
- roll_no, course_code, semester, section_no → grade adds 1.

$4 + 2 + 3 + 1 + 2 + 1 + 1 = 14$. K^+ is every attribute: K **is a superkey**.

Part C — is it a candidate key? Remove each attribute in turn and recompute. Without *roll_no*, nothing determines *name*; without *course_code*, nothing determines *title*; without *semester* or *section_no*, nothing determines *room*. No proper subset is a superkey, so K **is a candidate key** — and it is exactly Chapter 9's primary key of *enrolment*, found here from the dependencies alone.

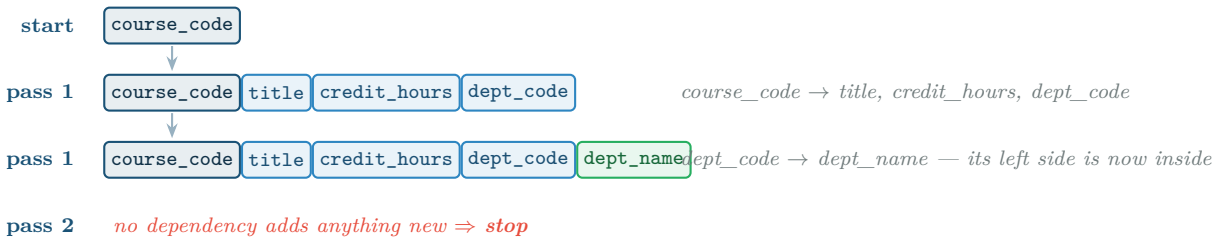


Figure 17.1: $\{course_code\}^+$, one step per row. Each step applies a dependency whose left side is already inside the set; the second only becomes applicable because the first added *dept_code*. The closure stops at five attributes — fewer than fourteen, so *course_code* is not a key of *report*.

Why closure is the tool for everything in Part V

Three questions, one algorithm:

- **Is X a superkey?** Compute X^+; it is if X^+ is every attribute.
- **Does X → Y follow from F?** Compute X^+; it does if $Y \subseteq X^+$. No axioms needed.
- **Is an attribute in a dependency redundant?** Remove it and recompute; if the closure is unchanged, it was. That is the minimal cover, next.

Every normal form in Chapter 18 is a test you run with closures, and Chapter 19's lossless-join test is one too.

17.4 Minimal Cover

Minimal cover

A **minimal cover** of F is an equivalent set of dependencies — one with the same closure — in which:

1. every right-hand side is a single attribute;
2. no left-hand side has an **extraneous attribute** (one that could be removed without changing what the dependency implies);
3. no dependency is **redundant** (derivable from the others).

It is the smallest statement of the same rules. Chapter 18's 3NF synthesis algorithm starts from one.

Worked Example: A minimal cover, in three steps

$F = \{$ A → BC, B → C, AB → D $\}$.

Step 1 — single right-hand sides. Decompose: A → B, A → C, B → C, AB → D.

Step 2 — extraneous left-hand attributes. Only AB → D has two. Is B extraneous? Compute $\{A\}^+$: $A \to B$ gives B, so $\{A\}^+ \ni B$, and then AB → D applies. $\{A\}^+ = \{A, B, C, D\}$, which contains D. So A → D already holds: B **is extraneous**, and AB → D becomes A → D.

Step 3 — redundant dependencies. Is A → C redundant? Remove it and compute $\{A\}^+$ from the rest: $A \to B$, $B \to C$. It contains C. **Redundant** — it follows by transitivity. Remove it.

Result. $\{$ A → B, B → C, A → D $\}$. Three dependencies instead of four, each as small as it can be, and the same closure as the original. Check: $\{A\}^+ = \{A, B, C, D\}$ in both.

Common Pitfalls

- **Reading a dependency off sample data.** Data disproves; it never proves. Ask.
- **Reading X → Y backwards.** roll_no → name does not give name → roll_no; two students share a name.
- **Stopping closure after one pass.** A dependency whose left side becomes available in pass 1 must still be applied. Repeat until a pass adds nothing.
- **Calling a superkey a candidate key.** Test every proper subset.
- **Splitting a left-hand side.** AB → C does *not* give A → C. Only right-hand sides decompose.
- **Forgetting the transitive dependencies nobody wrote down.** Closure finds them; that is what it is for.

Try It Yourself: Build the report table, then find its dependencies and its key

```
# --- 1. The seeded database ----------------------------------------
python code/schema/seed.py university.db
sqlite3 university.db
```

```
-- --- 2. Build the table everybody builds ------------------------
CREATE TABLE report AS
SELECT s.roll_no, s.name, s.programme,
       c.course_code, c.title, c.credit_hours,
       d.dept_code, d.name AS dept_name,
       e.semester, e.section_no, x.room, x.emp_id,
       i.name AS instructor, e.grade
  FROM enrolment e
  JOIN student    s ON s.roll_no     = e.roll_no
  JOIN course     c ON c.course_code = e.course_code
  JOIN department d ON d.dept_code   = c.dept_code
  JOIN section    x ON x.course_code = e.course_code
                   AND x.semester    = e.semester
                   AND x.section_no  = e.section_no
  LEFT JOIN instructor i ON i.emp_id = x.emp_id;
SELECT COUNT(*) FROM report;

-- --- 3. Test a dependency that should hold ----------------------
SELECT course_code FROM report
 GROUP BY course_code HAVING COUNT(DISTINCT title) > 1;
-- No rows: no counter-example. Does that PROVE course_code -> title?

-- --- 4. Test one that should not --------------------------------
SELECT name FROM report
 GROUP BY name HAVING COUNT(DISTINCT roll_no) > 1 LIMIT 5;
-- Counter-examples: name does not determine roll_no. Settled.

-- --- 5. Count the copies of one fact ----------------------------
SELECT roll_no, name, COUNT(*) AS copies FROM report
 GROUP BY roll_no ORDER BY copies DESC LIMIT 3;
-- One student's name, stored this many times. Why?

-- --- 6. The transitive one --------------------------------------
SELECT dept_code, COUNT(*) AS rows_holding_its_name
  FROM report GROUP BY dept_code;
-- To rename one department, how many rows must change?
```

```
# --- 7. On paper ------------------------------------------------------
# Compute {course_code}+ , {roll_no}+ and {emp_id}+ by the algorithm,
# one pass per line, from the six dependencies in this chapter.
```

```
# Then prove {roll_no, course_code, semester, section_no} is a
# candidate key: superkey first, then every proper subset.
```

What to notice. Steps 5 and 6 are the whole motivation for Part V. One fact — a name, a department name — is stored once for every enrolment it touches, because the dependency that determines it has a smaller left side than the table's key. Keep *report*: Chapter 18 measures it against the normalised schema, and Chapter 19 measures it again.

Checkpoint — Test Yourself

1. Compute $\{emp_id\}^+$ and $\{course_code, semester, section_no\}^+$ under the six dependencies of *report*, showing each pass.
2. Using closure only — no axioms — decide whether roll_no, course_code → dept_name follows from the six dependencies, and show the closure that settles it.
3. Find a minimal cover of { A → B, A → C, B → C, AC → D }, showing all three steps and checking that the result has the same closure of A.

Chapter Summary

- X → Y holds when tuples agreeing on X must agree on Y. It is a function, in Appendix B's sense, from X-values to Y-values.
- Dependencies come from the university. An instance can disprove one with a GROUP BY ... HAVING COUNT(DISTINCT) query; it can never prove one.
- **Armstrong's axioms** — reflexivity, augmentation, transitivity — are sound and complete. Union and decomposition follow from them.
- Only right-hand sides decompose. AB → C does not give A → C.
- The **closure** X^+ is everything X determines: start with X, apply every dependency whose left side is inside, repeat until a pass adds nothing.
- Closure answers three questions: is X a superkey, does a dependency follow, is an attribute redundant.
- The key of *report* is {roll_no, course_code, semester, section_no} — the primary key of *enrolment*, found from the dependencies alone.
- A **minimal cover** has single-attribute right sides, no extraneous left-side attributes, and no redundant dependencies.
- In *report*, every fact whose determinant is smaller than the key is stored once per enrolment. That is what Chapter 18 removes, and measures.

Review Questions

1. (MCQ) AB → C and C → D together imply: (a) A → D; (b) AB → D; (c) D → AB; (d) C → AB.

2. (MCQ) A dependency X → Y with $Y \subseteq X$ is: (a) transitive; (b) trivial; (c) redundant; (d) a key.

3. (Short) Why can a query find a counter-example to a dependency but never prove one holds?

4. (Short) State the closure algorithm in three lines, and say why it must terminate.

5. (Short) Show by a two-row example that AB → C does not imply A → C.

6. (Applied) A relation BOOKING(room, date, slot, course_code, emp_id, capacity) has the rules: a room has a fixed capacity; a room, date and slot is booked for one course; a course has one

instructor per booking. Write the dependencies, compute the closure of {room, date, slot}, and find every candidate key.

7. **(Applied)** Using the six dependencies of *report*, list every attribute whose determinant is a proper subset of the key, and for each one compute from the seeded data (step 5 of the laboratory) how many copies of a single fact it causes. Which one costs the most rows?

Chapter 18

Normal Forms: 1NF to BCNF

Learning Outcomes

By the end of this chapter you will be able to:

- **Name** the three anomalies — update, insertion, deletion — and **show** each one in a real table.
- **State** the conditions for 1NF, 2NF, 3NF and BCNF, and **test** a relation against each using its dependencies.
- **Identify** a partial and a transitive dependency, and **say** which normal form each one violates.
- **Decompose** a relation into BCNF by splitting on a violating dependency.
- **Weigh** what normalisation costs in reads against what it saves in correctness, using measured figures.

Before You Start

Chapter 17: functional dependencies, attribute closure and candidate keys — every test in this chapter is a closure. The *report* table from that chapter's laboratory. You need `code/m18_normalforms.py`.

Key Terms in This Chapter

normalisation • anomaly • update anomaly • insertion anomaly • deletion anomaly • first normal form • atomic • partial dependency • second normal form • transitive dependency • third normal form • prime attribute • Boyce–Codd normal form • decomposition • redundancy

18.1 Three Ways a Wide Table Goes Wrong

Chapter 17 built *report*: fourteen columns, one row per enrolment, every fact the university holds about it in one place. It answers questions without joins. This chapter is about what it does when the data *changes*, which is the thing it was never designed for.

The three anomalies

An **update anomaly**: one fact is stored in many rows, so changing it means changing all of them — and changing some of them leaves the table holding two versions of the truth.

An **insertion anomaly**: a fact cannot be recorded without also recording an unrelated one that does not exist yet.

A **deletion anomaly**: deleting one fact removes another that had nowhere else to live.

All three have one cause. **A fact whose determinant is smaller than the key is stored once per key value**, so the table is holding several relations at once and can only change them together.

Measured: The same information, held wide and held normalised

`code/m18_normalforms.py`, seeded database (4,000 students), SQLite 3.51.2, Intel i5-6600, Windows 11. Best of three runs where timed.

1. Bytes, and copies of one fact

	rows	bytes	
report, as CREATE TABLE AS made it	60,037	6,860,800	no index
report, with its key indexed	60,037	9,289,728	like for like
normalised, 6 tables	65,693	5,574,656	key indexes

Like for like, the wide table is 1.7× the size. Compared as created it looks like 1.2×, because CREATE TABLE AS copies rows and nothing else — no key, no index — while every normalised table carries the index its PRIMARY KEY implies. The bytes are not the real cost; the copies are:

copies of one fact	*report*	normalised
a student's name	18	1
an instructor's name	949	1
a course's title	1,382	1
a department's name	**7,936**	1

2. The update anomaly — rename one department

	rows changed	ms
report	7,869	378.9
department	1	25.4

Most of the 25.4 ms is the commit, not the change: one row costs about as much to make durable as it does to write.

3. A half-finished update — the IT department renamed in the FALL2025 rows only, as a crashed program or a wrong WHERE would leave it

`'Information Technology'`	5,273 rows
`'Information Tech.'`	2,663 rows

Two names for one department, in one table, and no constraint anywhere that could have refused it. In *department* the same mistake is impossible: there is only one place to write the name.

4. Insertion and deletion

a new course, nobody enrolled yet: rows in *report* that can hold it	**0**
the same course in *course*	one row, accepted
report after deleting the 1,122 enrolments of PH-350: rows still describing the course	**0**
the same course in *course*	still there

5. Read cost — where the wide table is supposed to win

(a) one student's transcript	ms	
report, no index on roll_no	15.35	**162× slower**
normalised, one join	0.09	
report, with an index on roll_no	0.08	a tie
(b) enrolments per department		
report, no joins	66.51	**1.5× faster**
normalised, two joins	98.34	

The wide table lost the lookup by two orders of magnitude, and not because of a join: the normalised *enrolment* table's primary key begins with `roll_no`, so the transcript was an index search, while *report* had no index and was read end to end. Given one index the two tied. It won only the full-scan report, by 1.5× — the case where reading one table instead of joining three is a real saving.

So the trade is not "fast reads against slow writes"

That is how it is usually described, and the measurement does not support it:

- **Lookups were not faster wide.** They were 162× slower until an index was added, and equal after. "No joins" is not an advantage when the join is an index search.
- **Full scans were faster wide, by 1.5×.** Real, and modest.
- **Writes were 7,869 times the work**, and one half-finished write left the table permanently self-contradictory.

The honest summary: normalisation costs a little on reports over every row, and buys correctness that no amount of care in the application can supply. Chapter 19 asks when the 1.5× is worth paying for, and the answer is a measurement, not a preference.

18.2 The Normal Forms

Each normal form forbids one kind of dependency. They are nested: every BCNF relation is in 3NF, every 3NF relation in 2NF, and every 2NF relation in 1NF.

1NF and 2NF

First normal form (1NF): every attribute value is atomic — no lists, no repeating groups, no "three phone numbers in one field". It is Chapter 7's fourth property of a relation, and Chapter 9's rule for multi-valued attributes.

A **partial dependency** is one whose determinant is a *proper subset* of a candidate key: roll_no → name, when the key is {roll_no, course_code, semester, section_no}.

Second normal form (2NF): 1NF, and no non-prime attribute is partially dependent on any candidate key. (A **prime attribute** is one that belongs to some candidate key.)

3NF and BCNF

A **transitive dependency** is K → Y holding only because K → X and X → Y, where X is not a superkey: course_code → dept_name through *dept_code*.

Third normal form (3NF): for every non-trivial X → A, either X is a superkey, or A is a prime attribute.

Boyce–Codd normal form (BCNF): for every non-trivial X → A, X is a superkey. Full stop — the prime-attribute exception is gone.

The one sentence to remember: in BCNF, every determinant is a superkey. Every fact is about the key, the whole key, and nothing but the key.

Worked Example: *report*, from nowhere to BCNF

Key K = {roll_no, course_code, semester, section_no} (Chapter 17). Test each dependency: is its determinant a superkey?

dependency	superkey?	violates
roll_no → name, programme	no	2NF — partial
course_code → title, credit_hours, dept_code	no	2NF — partial
dept_code → dept_name	no	3NF — transitive
course_code, semester, section_no → room, emp_id	no	2NF — partial
emp_id → instructor	no	3NF — transitive
K → grade	**yes**	nothing

Five violations, one clean dependency. **The decomposition procedure:** take a violating X → Y; split R into $R_1 = X \cup Y$ and $R_2 = R - Y$; repeat on whatever still violates. **Split on the transitive dependencies first** — the box below shows what goes wrong otherwise.

1. On dept_code → dept_name: DEPARTMENT(dept_code, dept_name). Remove dept_name from the rest.
2. On emp_id → instructor: INSTRUCTOR(emp_id, instructor). Remove instructor.
3. On roll_no → name, programme: STUDENT(roll_no, name, programme). Remove name and programme.
4. On course_code → title, credit_hours, dept_code: COURSE(course_code, title, credit_hours, dept_code). Remove title, credit_hours and dept_code.
5. On course_code, semester, section_no → room, emp_id: SECTION(course_code, semester, section_no, room, emp_id). Remove room and emp_id.
6. What remains: ENROLMENT(roll_no, course_code, semester, section_no, grade) — whose only dependency has the key as its determinant.

Why the order matters. Split on course_code → title, credit_hours, dept_code *first* and $R_2 = R - Y$ removes *dept_code* from the remainder — but leaves *dept_name* there, with nothing beside it that determines it except, transitively, *course_code*. The next split is then on course_code → dept_name, and produces (course_code, dept_name): a department's name stored once *per course*, which is the redundancy you were removing. Every relation is still in BCNF, so no normal-form test fails — but the department has no table of its own, and dept_code → dept_name now lives in *no* single table, so nothing can enforce it: two CS courses could carry two different department names. Chapter 19 calls that losing dependency preservation. Handle the deepest dependency first and each fact lands with its own determinant.

Six relations, every one in BCNF — and they are exactly the six tables Chapter 9 produced from the ER diagram. Two routes, one from design and one from dependencies, arrived at the same schema. That agreement is the best evidence a design is right.

3NF against BCNF: the one case where they differ

They differ only when a relation has two *overlapping* composite candidate keys — rare, but it exists. A classic case: TEACHES(student, course, instructor), where each instructor teaches one course (instructor → course) and each student has one instructor per course (student, course → instructor).

instructor → course: *instructor* is not a superkey, so BCNF is violated. But *course* is prime — it is in the key {student, course} — so 3NF is satisfied. Decomposing to BCNF gives

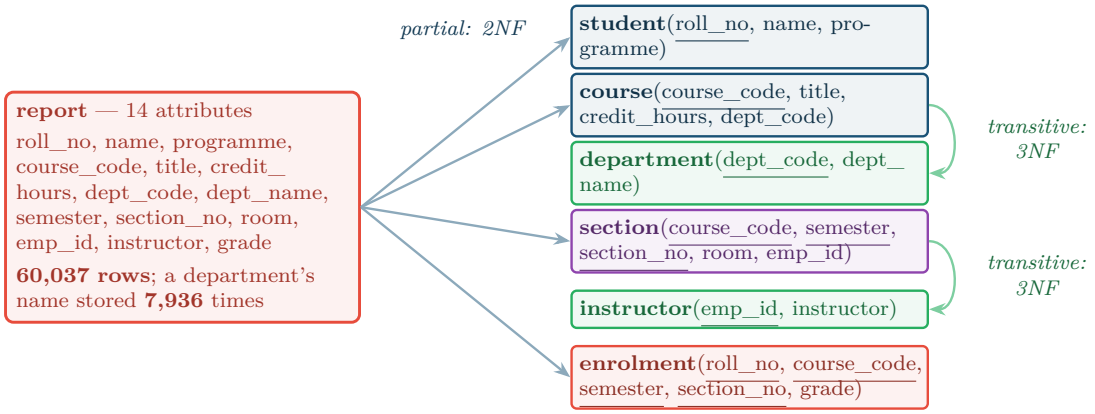


Figure 18.1: *report* decomposed to BCNF. Splitting on each partial dependency gives four relations; splitting on the two transitive ones gives two more. The result is the six tables Chapter 9 derived from the ER diagram by an entirely different route.

(instructor, course) and (student, instructor), and **loses** the dependency student, course → instructor: no single table can now enforce it.

That trade — BCNF, or keeping every dependency enforceable — is Chapter 19's subject. For the university schema it never arises.

Common Pitfalls

- **Thinking normalisation is about saving bytes.** It saved a factor of 1.7. It removed 7,935 copies of one department name. The copies are the point.
- **Believing a wide table reads faster.** Measured: 162× slower on a lookup without an index; a tie with one.
- **Comparing a** CREATE TABLE AS **copy with indexed tables.** The copy has no index. Measure like for like.
- **Testing 2NF against one key when there are several.** A partial dependency on *any* candidate key violates it.
- **Stopping at 3NF without checking BCNF.** They usually coincide; check rather than assume.
- **Splitting a relation and losing the join.** Every decomposition must keep the determinant in both halves — Chapter 19 explains why.

Try It Yourself: Break the wide table three ways, then decompose it

```
# --- 1. Seed, and rebuild report from Chapter 17 --------------------
python code/schema/seed.py university.db
sqlite3 university.db
```

```
-- --- 2. Recreate report (Chapter 17's lab, step 2), then ---------
SELECT COUNT(*) FROM report;

-- --- 3. The update anomaly ----------------------------------------
.timer on
UPDATE report SET dept_name = 'Computing' WHERE dept_code = 'CS';
-- Rows changed? (.changes on, or SELECT changes();)
UPDATE department SET name = 'Computing' WHERE dept_code = 'CS';

-- --- 4. Leave it half done ----------------------------------------
```

```
UPDATE report SET dept_name = 'Information Tech.'
 WHERE dept_code = 'IT' AND semester = 'FALL2025';
SELECT dept_name, COUNT(*) FROM report
 WHERE dept_code = 'IT' GROUP BY dept_name;
-- Two answers. Write the query a programmer would need to find out
-- which name is current. Can it be written at all?

-- --- 5. The insertion anomaly ------------------------------------
-- Add a course nobody is enrolled in yet, to report.
-- Try it. What would you have to put in roll_no and grade?

-- --- 6. The deletion anomaly -------------------------------------
SELECT course_code, COUNT(*) FROM report
 GROUP BY course_code ORDER BY 2 LIMIT 1;
-- Delete every row for that course. Then ask report its title.

-- --- 7. Read cost, like for like ---------------------------------
SELECT course_code, title FROM report
 WHERE roll_no = (SELECT MIN(roll_no) FROM student);
CREATE INDEX idx_report_roll ON report(roll_no);
SELECT course_code, title FROM report
 WHERE roll_no = (SELECT MIN(roll_no) FROM student);
-- Two timings. Which part of the difference was the join?
```

```
# --- 8. On paper: decompose report to BCNF ----------------------------
# One violating dependency at a time. Name each new relation, its key,
# and which normal form the split repaired. Compare your six relations
# with Figure 9.1. Then run the measured version:
python code/m18_normalforms.py
```

What to notice. Step 4 is the one with no repair. Once a table holds two versions of a fact and nothing records which came later, no query can tell you which is true. That is not a performance problem, and no index fixes it. It is the reason normal forms exist.

Checkpoint — Test Yourself

1. For each of the three anomalies, give the specific operation on *report* that exhibits it, the measured consequence, and the single dependency responsible.
2. R(A, B, C, D) has A → C and C → D. State its normal form, identify each violation by kind, and decompose it to BCNF.
3. The wide table read a transcript 162× slower than the normalised schema and a department report 1.5× faster. Explain both results, and say which one would change if *report* were given an index on `roll_no`, and to what.

Chapter Summary

- Three anomalies — **update**, **insertion**, **deletion** — share one cause: a fact whose determinant is smaller than the key is stored once per key value.
- **Measured:** one department's name was held **7,936** times; renaming it changed **7,869** rows against 1; a half-finished update left **two names** for one department.
- **Measured:** a new course could not be inserted into the wide table at all, and deleting a course's enrolments deleted the course.
- **Measured:** like for like, the wide table was **1.7**× the size — 1.2× only when compared without its index.
- **Measured:** the wide table was **162× slower** on a lookup with no index and tied with one; it was **1.5× faster** only on a full-scan report.

- **1NF**: atomic values. **2NF**: no partial dependency. **3NF**: no transitive dependency on a non-prime attribute. **BCNF**: every determinant is a superkey.
- Decompose on a violating X $\rightarrow$ Y into $X \cup Y$ and $R - Y$, and repeat.
- *report* decomposes to exactly the six tables Chapter 9 derived from the ER diagram — two independent routes, one schema.
- 3NF and BCNF differ only with overlapping composite keys, where BCNF can cost an enforceable dependency.

Review Questions

1. **(MCQ)** A non-prime attribute depending on part of a composite key violates: (a) 1NF; (b) 2NF; (c) 3NF but not 2NF; (d) BCNF only.

2. **(MCQ)** In BCNF, for every non-trivial X $\rightarrow$ A: (a) A is prime; (b) X is a superkey; (c) X is a candidate key; (d) A is in X.

3. **(Short)** Why can no query repair a table holding two versions of the same fact?

4. **(Short)** Give the dependency in *report* that is transitive, and say which normal form it violates.

5. **(Short)** Why was the wide table 1.2× the size as created but 1.7× like for like?

6. **(Applied)** A spreadsheet records LOAN(member_id, member_name, book_id, title, author, date_out, date_back), one row per loan. Write its dependencies, identify its key, classify every violation, and decompose it to BCNF, naming each relation's key.

7. **(Applied)** A manager proposes keeping *report* as the only table "because reports are faster". Using every figure in the measured box, write a reply that concedes what is true, refutes what is not, and proposes a design that keeps the 1.5× for reports without keeping the anomalies — a design Chapter 19 measures.

Chapter 19

Decomposition, and Denormalising on Purpose

Learning Outcomes

By the end of this chapter you will be able to:

- **Define** a lossless-join decomposition and **apply** the one-line test for it.
- **Show** what a lossy decomposition does to a query, by counting its spurious tuples.
- **Define** dependency preservation and **identify** a decomposition that loses a dependency.
- **Decide** between BCNF and 3NF when they conflict.
- **Measure** a denormalisation — what it saves per read, what it costs per write — and **compute** where it breaks even.

Before You Start

Chapter 18: the normal forms, and its measurement of the wide table. Chapter 17 for closure, which the lossless test uses. You need `code/m19_decomposition.py`.

Key Terms in This Chapter

decomposition • lossless join • lossy join • spurious tuple • dependency preservation • 3NF synthesis • denormalisation • summary table • trigger • break-even • read–write ratio

19.1 Two Things a Split Must Not Lose

Chapter 18 decomposed *report* into six tables by splitting on violating dependencies. Any split *can* be made; not every split is correct. A decomposition must keep two things, and they are independent.

Lossless join

A decomposition of R into R_1 and R_2 is **lossless** if joining them back gives exactly R — no tuple missing, **no tuple added**.

The test. It is lossless if and only if the attributes they share determine all of one side:

$$(R_1 \cap R_2) \to R_1 \quad \text{or} \quad (R_1 \cap R_2) \to R_2.$$

In words: the shared attributes are a superkey of one of the halves. Compute $(R_1 \cap R_2)^+$, as in Chapter 17, and look.

A **lossy** decomposition never loses tuples — it *adds* **spurious tuples**, rows that look like facts and were never true.

Dependency preservation

A decomposition **preserves dependencies** if every dependency of R can be checked within a single one of the R_i — or follows from dependencies that can.

It matters because a database enforces rules one table at a time. A dependency that spans two tables can only be checked by joining them, and nothing in the schema does that on every insert.

Measured: A split that invents facts, a split that loses a rule, and a summary table priced

`code/m19_decomposition.py`, seeded database (4,000 students), SQLite 3.51.2, Intel i5-6600, Windows 11.

1. Lossy against lossless

lossy: R_1(roll_no, course_code) and R_2(course_code, grade)	
graded (roll_no, course_code, grade) facts	39,152
$R_1 \bowtie R_2$ on course_code	182,920
spurious tuples — never true	**143,768**
lossless: S_1(roll_no, name, programme) and S_2(roll_no, semester, cnic)	
student rows, and $S_1 \bowtie S_2$	4,000 and 4,000

The lossy join returned **4.7**× the real facts: every student now appears to hold every grade that anybody got in the course. The test explains it in one line — $R_1 \cap R_2 = \{\text{course_code}\}$, a key of neither half; $S_1 \cap S_2 = \{\text{roll_no}\}$, the key of both.

2. Lossless, but a dependency is lost

Two tables, *course_dept*(course_code, dept_code) and *course_dname*(course_code, dept_name), both keyed on course_code — so the join is lossless. Two courses, both CS. A second name for CS:

insert `('CS-150', 'Computing')`	**accepted**
the join: CS →	`'Computer Science'`
the join: CS →	`'Computing'`

dept_code → dept_name is violated, and nothing could refuse it: the rule spans two tables, and a CHECK sees one row of one table. This is exactly the decomposition Chapter 18 warned the wrong splitting order produces.

3. Denormalisation, priced — enrolments and credit hours per department

read: computed by two joins	69.79 ms
read: from a summary table	0.060 ms
saved per read	**69.73 ms**
write: one enrolment, no trigger	28.464 ms
write: one enrolment, trigger maintaining the summary	32.143 ms
added per write	**3.679 ms**
summary agreed with the joins, before and after 400 writes	yes, yes

Break-even: one read every 19 writes. Above that ratio the summary table is cheaper overall; below it, computing on demand is. A department report is read far more often than once per nineteen enrolments, so for this report the summary table wins — by a margin that is not close.

Most of the 28 ms per write is the COMMIT — making one enrolment durable — and the trigger's two UPDATEs ride inside that same transaction. That is why the trigger adds 13%, not 200%.

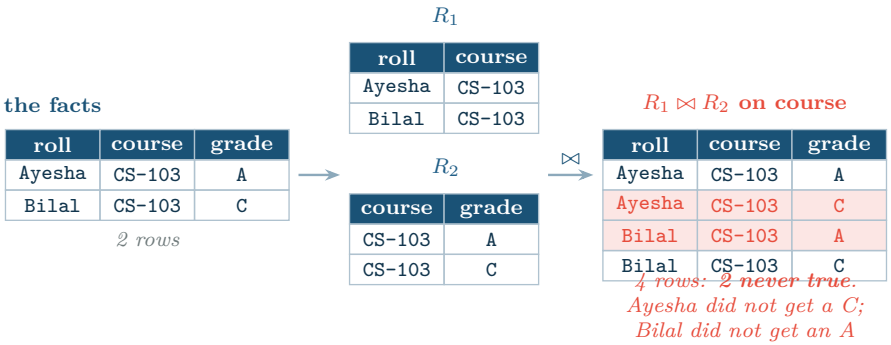


Figure 19.1: A lossy split on the smallest possible scale. The two halves share only *course*, which is a key of neither, so the join pairs every student in the course with every grade in it. On the seeded data this is 143,768 invented facts.

What this does and does not license

- **Always test a decomposition for losslessness.** A lossy one produces 4.7 false facts for every true one here, and no query afterwards can tell them apart.
- **The summary table is a denormalisation, and it is correct only while the trigger runs.** Drop the trigger, insert through a path that bypasses it, or restore an old backup of one table, and the summary is silently wrong. That risk is what the 69 ms buys.
- **Denormalise by adding, never by replacing.** The normalised tables stayed. The summary is derived from them and can be rebuilt from them at any moment — which is the difference between this and Chapter 18's *report*, where the wide table was the only copy.
- **Decide with a ratio, not a feeling.** One read per nineteen writes is a number; "reports are slow" is not.

19.2 When BCNF and Dependency Preservation Conflict

The trade, and the rule for making it

Every relation has a **lossless, dependency-preserving decomposition into 3NF**. The **3NF synthesis** algorithm finds one: take a minimal cover (Chapter 17), make one relation per dependency, add one relation holding a candidate key if none already does.

A lossless BCNF decomposition always exists too — but it is *not* always dependency-preserving. Chapter 18's TEACHES(student, course, instructor) is the case: BCNF forces student, course → instructor out of every single table.

The rule: prefer BCNF; when it would lose a dependency you need enforced, stop at 3NF and accept the small redundancy that remains. A redundancy you can see is safer than a rule nothing enforces. For the university schema the question never arises — the six tables are in BCNF *and* preserve every dependency.

Worked Example: Testing two decompositions of *section*

SECTION(course_code, semester, section_no, room, emp_id), with course_code, semester, section_no → room, emp_id.

Decomposition A: A_1(course_code, semester, section_no, room) and A_2(course_code, semester, section_no, emp_id).

$A_1 \cap A_2 = \{\text{course_code, semester, section_no}\}$, whose closure is everything: a superkey of both. **Lossless.** The dependency splits into $\ldots \to$ room in A_1 and $\ldots \to$ emp_id in A_2 — each checkable in one table. **Preserving.** Correct, and pointless: it is two tables where one would do, and every split is a join somebody pays for on every read.

Decomposition B: B_1(course_code, room) and B_2(room, semester, section_no, emp_id). $B_1 \cap B_2 = \{\text{room}\}$. Does *room* determine all of either half? No: one room hosts many courses and many sections. **Lossy.** Joined back on room, every course would appear in every section that ever met in the same room — the pattern of part 1 of the measurement, in a different table.

The lesson. The test took one closure each time. B "looks reasonable" — room is a natural thing to split on — and is wrong in a way the data would never report.

19.3 Triggers, and the Summary Table Written Out

</> The summary table that was measured

```
CREATE TABLE dept_stats (
  dept_code   TEXT PRIMARY KEY NOT NULL REFERENCES department(dept_code),
  n_enrol     INTEGER NOT NULL,
  sum_credits INTEGER NOT NULL
) STRICT;

CREATE TRIGGER stats_ins AFTER INSERT ON enrolment BEGIN
  UPDATE dept_stats
     SET n_enrol     = n_enrol + 1,
         sum_credits = sum_credits +
           (SELECT credit_hours FROM course
             WHERE course_code = NEW.course_code)
   WHERE dept_code =
           (SELECT dept_code FROM course
             WHERE course_code = NEW.course_code);
END;
```

`NEW` is the row just inserted. A matching `stats_del` trigger on DELETE uses `OLD`. **An UPDATE trigger is needed too**, and the harness does not have one: an enrolment moved from one course to another would leave both departments' figures wrong. That gap is Chapter 20's starting point, and the reason a view is often the better answer.

Common Pitfalls

- **Splitting on an attribute that is not a key of either half.** Lossy: 143,768 spurious tuples here.
- **Expecting a lossy join to lose rows.** It adds them. The answer is too big, not too small, and looks plausible.
- **A rule that spans two tables.** No CHECK can enforce it; it needs a trigger, or a design that keeps it in one table.
- **Denormalising by replacing the normalised tables.** Add the summary; keep the source of truth.
- **Forgetting the UPDATE trigger.** Insert and delete are maintained; a changed row silently is not.
- **Deciding to denormalise without the ratio.** Measure reads and writes; compute the break-even.

Try It Yourself: Invent 143,768 facts, then price a summary table

```
# --- 1. The seeded database ----------------------------------------
python code/schema/seed.py university.db
sqlite3 university.db
```

```
-- --- 2. A lossy split ---------------------------------------------
CREATE TEMP TABLE r1 AS SELECT DISTINCT roll_no, course_code
                          FROM enrolment WHERE grade IS NOT NULL;
CREATE TEMP TABLE r2 AS SELECT DISTINCT course_code, grade
                          FROM enrolment WHERE grade IS NOT NULL;
SELECT COUNT(*) FROM (SELECT DISTINCT roll_no, course_code, grade
                        FROM enrolment WHERE grade IS NOT NULL);
SELECT COUNT(*) FROM r1 JOIN r2 USING (course_code);
-- How many facts were invented? Apply the one-line test to explain.

-- --- 3. Pick one student and see what the lossy join says ------
SELECT grade FROM r1 JOIN r2 USING (course_code)
 WHERE roll_no = (SELECT MIN(roll_no) FROM r1) LIMIT 10;
SELECT grade FROM enrolment
 WHERE roll_no = (SELECT MIN(roll_no) FROM r1) AND grade IS NOT NULL;

-- --- 4. The summary table and its trigger -----------------------
-- Type in dept_stats and stats_ins from this chapter, then fill it:
INSERT INTO dept_stats
  SELECT d.dept_code, COUNT(e.roll_no), COALESCE(SUM(c.credit_hours),0)
    FROM department d
    LEFT JOIN course c    ON c.dept_code   = d.dept_code
    LEFT JOIN enrolment e ON e.course_code = c.course_code
   GROUP BY d.dept_code;

-- --- 5. Time both reads -----------------------------------------
.timer on
SELECT * FROM dept_stats;
SELECT c.dept_code, COUNT(*), SUM(c.credit_hours)
  FROM enrolment e JOIN course c ON c.course_code = e.course_code
 GROUP BY c.dept_code;

-- --- 6. Break it ------------------------------------------------
UPDATE enrolment SET course_code =
       (SELECT course_code FROM course WHERE dept_code = 'MA' LIMIT 1)
 WHERE rowid = (SELECT MIN(rowid) FROM enrolment);
-- Re-run both reads from step 5. Do they still agree? Why not?
```

```
# --- 7. The measured version ----------------------------------------
python code/m19_decomposition.py
```

What to notice. Step 6 is the price of the 69 ms. The summary table is correct exactly as long as every change to *enrolment* passes through a trigger that knows about it — and the harness, like most first attempts, forgot UPDATE. (Step 6 may also be refused if the move would duplicate an enrolment's key; pick another row if so.)

Checkpoint — Test Yourself

1. Apply the lossless test to R_1(roll_no, course_code) and R_2(course_code, grade), and explain from the result why the join produced 4.7 times the real facts.
2. Give a decomposition of the university schema that is lossless but not dependency-preserving, name the dependency it loses, and show an insert that violates the dependency without any table refusing it.

3. The summary table saves 69.73 ms per read and costs 3.679 ms per write. Show the break-even calculation, and say what would change it if the database moved to a fast SSD where a commit took 1 ms.

Chapter Summary

- A decomposition must be **lossless** (the join gives back exactly R) and should be **dependency-preserving** (every rule checkable in one table). The two are independent.
- **Lossless test:** the shared attributes must be a superkey of one half. One closure decides it.
- **Measured:** a lossy split of *enrolment* produced **143,768 spurious tuples** — 4.7× the real facts. A lossy join adds rows; it does not lose them.
- **Measured:** a lossless split that lost dept_code → dept_name **accepted two names for CS**. A rule spanning two tables cannot be enforced by a CHECK.
- 3NF synthesis is always lossless and preserving; BCNF is always lossless but not always preserving. Prefer BCNF; stop at 3NF when BCNF would cost a rule you need.
- **Measured:** a trigger-maintained summary table saved **69.73 ms per read** for **3.679 ms per write** — break even at **one read per 19 writes**.
- Most of a write's cost is the COMMIT; a trigger rides inside it, which is why it added 13%.
- Denormalise by *adding* derived data beside the normalised tables, never by replacing them — and maintain it on insert, delete *and* update.

Review Questions

1. (MCQ) A lossy decomposition, when joined back, produces: (a) fewer tuples; (b) the same tuples; (c) extra, spurious tuples; (d) an error.

2. (MCQ) $R_1 \cap R_2 = \{A\}$ is lossless if: (a) A is a superkey of R_1 or of R_2; (b) A is in both; (c) A is indexed; (d) R_1 and R_2 have the same size.

3. (Short) Why can a CHECK constraint not enforce a dependency that spans two tables?

4. (Short) State the rule for choosing between BCNF and 3NF.

5. (Short) What three triggers does a summary table need, and what goes wrong without the third?

6. (Applied) Decompose BOOKING(room, date, slot, course_code, emp_id, capacity) (Chapter 17's review question) into BCNF, test each split for losslessness by closure, and say whether every dependency is preserved.

7. (Applied) The university asks for a "students per programme per semester" figure on its home page, read 5,000 times a day; about 300 enrolments change a day. Design the summary table and its triggers, compute the break-even using this chapter's measured costs, and say whether you would build it.

Chapter 20

Views, Triggers and Keeping Data Honest

Learning Outcomes

By the end of this chapter you will be able to:

- **Create** a view and **explain** what is stored when you do.
- **Show**, from a compiled program, that a view costs nothing at run time.
- **Use** a view to provide an external schema and to hide a schema change from the programs that read it.
- **Write** BEFORE, AFTER and INSTEAD OF triggers, and **say** when each is the right one.
- **Choose** between a view and a trigger-maintained table by what each guarantees.

Before You Start

Chapter 2: the external level, which a view implements. Chapter 19: the summary table and its missing UPDATE trigger. You need `code/m20_views.py`.

Key Terms in This Chapter

view • virtual table • base table • view expansion • flattening • co-routine • updatable view • trigger • BEFORE • AFTER • INSTEAD OF • `NEW` • `OLD` • RAISE • audit table • business rule • materialised view

20.1 A Query With a Name

View

A **view** is a named query. CREATE VIEW *v* AS SELECT ... stores the *text* of the SELECT in the data dictionary (Chapter 3) and nothing else — no rows.

A query that names *v* is rewritten, before it is compiled, with *v*'s SELECT in its place — **view expansion**. The view is a **virtual table**; the tables it reads are **base tables**.

So a view is always **current**: it is the query, run now, against whatever the base tables hold now. It cannot be stale, because there is nothing stored to go stale.

The three jobs a view does in this course

1. An external schema. Chapter 2's `fees_view` — roll number and name, nothing else — is the external level, built:

```
CREATE VIEW fees_view AS SELECT roll_no, name FROM student;
```

2. Logical data independence. When Chapter 11's migration split a column, a view with the old shape let old programs keep working:

```
CREATE VIEW student_v1 AS
SELECT roll_no, name, programme, semester,
       addr_house || ', ' || addr_street || ', ' || addr_city AS address
  FROM student;
```

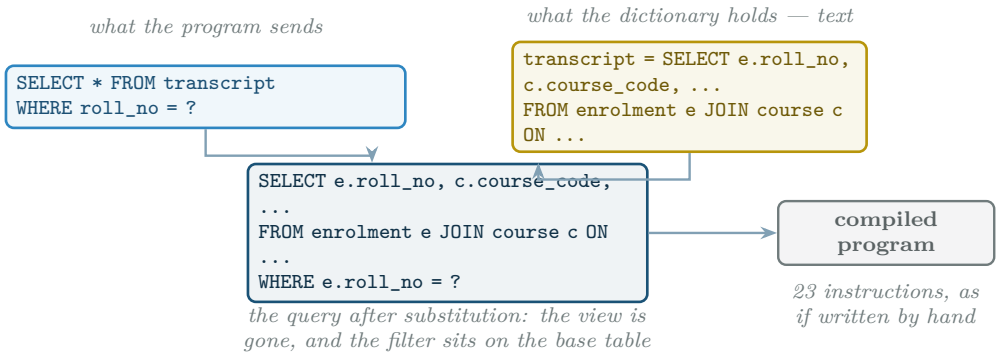


Figure 20.1: What a view is, in one diagram. The dictionary holds only the view's text; SQLite substitutes it into the query that names it, then compiles. By the time anything runs there is no view left to cost anything, or to be out of date.

3. A query written once. The transcript is a join every report needs. Write it once and every query that names `transcript` gets it right.

What a view does *not* do in SQLite is restrict anyone to it. There are no users (Chapter 3), so a person who can open the file can read *student* directly. The external level exists here; its enforcement waits for Chapter 23.

20.2 Measured: What a View Costs, and What a Trigger Costs

Measured: A view costs nothing; a trigger costs less than the same work by hand

`code/m20_views.py`, seeded database (4,000 students), SQLite 3.51.2, Intel i5-6600, Windows 11, mechanical disk.

1. A view against the query it wraps — one student's transcript

	ms	rows	instructions
the query written out	0.091	15	23
the same through a view	0.104	15	23

The same 23 instructions in the same order. Seventeen of them differ at all, and every one differs only in which *cursor number* it uses: the view's program numbers its tables one higher. That is the whole difference — the view was substituted and compiled away before it could cost anything.

2. Does a filter on the view reach inside it?

`transcript WHERE roll_no = ?`	`SEARCH e USING INDEX ... (roll_no=?)`
grouped view, no index on course_code	`CO-ROUTINE` + `SCAN enrolment`
grouped view, with that index	`CO-ROUTINE` + `SEARCH ... (course_code=?)`

A grouped view cannot be flattened, so it runs as a co-routine — but the filter is still pushed into it. Without an index the plan shows a `SCAN`, which looks like no push-down and is not: there was simply nothing to search with. With the index, only one course's rows were ever grouped (0.277 ms through the view, 0.268 ms written out).

3. What a trigger costs — an audit trigger recording every grade change. Commit per row: 200 changes, median of 3. One commit: 30,000 changes, median of 5. Per row.

	1 per row	(range)	1 for all	(range)
UPDATE only	27.58 ms	27.4–34.7	0.0494 ms	0.041–0.064
+ audit trigger	31.29 ms	28.2–49.7	0.0622 ms	0.052–0.168
+ the same INSERT by hand	30.15 ms	29.6–34.9	0.0634 ms	0.051–0.089

The trigger costs what the work inside it costs. Against the same INSERT written by hand, the ranges overlap in both modes — neither is reliably cheaper — so the mechanism itself adds nothing measurable. What the trigger adds to an update is real but small: about a hundredth of a millisecond per row in bulk. With a commit per row it is lost in the disk sync, whose run-to-run range is wider than the trigger's whole cost.

A note on how this was established, because it took three tries. The first version ran each case once and blamed the cost on "the second table"; a hand-written control cost *more*, so that was wrong. The second version, on 200 rows, concluded the trigger was *cheaper* than by hand — and `code/run_all.py` caught that conclusion failing on a re-run: at 200 rows a bulk run takes 30 ms in total, and the trigger's "cost" came out *negative* on two runs in three. A second full run of this version also moved the trigger's share of an update from 26% to 49%. The one result that held on every run is the one stated above. Small effects need many rows, several runs, ranges and a control — and a harness that re-checks the handout's claims, because the second error would otherwise have been published.

4. The view and Chapter 19's summary table, after one enrolment moves from a CS course to an MA course

	truth	view	summary table
CS	7,868	7,868	**7,869**
MA	7,174	7,174	**7,173**
read cost		53.69 ms	0.076 ms

The view is right, because it is the query run now. The table is wrong in both rows, because it has insert and delete triggers and no update trigger — the gap Chapter 19 left. It is 700× faster to read and exactly as correct as its triggers are complete.

⚠ Choosing between a view and a maintained table

- **Default to the view.** It costs nothing beyond the query it names, and it cannot be wrong. Measured.
- **Replace it with a maintained table only when the measured read cost forces you to** — and then with triggers for INSERT, DELETE *and* UPDATE, plus a query that checks the table against the view, run on a schedule.
- **Do not fear triggers for their cost.** An audit trigger cost what the same INSERT written by the program cost — about a hundredth of a millisecond a row in bulk. What they really cost is the attention needed to keep them complete.
- **Other systems offer a materialised view** — a view whose result is stored and refreshed by the database. SQLite does not. The summary table with triggers is what that feature does, written out.

20.3 Triggers

Trigger

A **trigger** is a block of SQL the database runs automatically when a named event happens to a table: INSERT, UPDATE (optionally OF named columns) or DELETE.

It runs BEFORE or AFTER the change, once for each row affected. Inside it, `NEW` is the row as it will be and `OLD` the row as it was — `NEW` has no meaning in a DELETE trigger, nor `OLD` in an INSERT trigger.

WHEN limits it to rows meeting a condition. `RAISE(ABORT, 'message')` inside a trigger cancels the statement with an error.

Worked Example: A business rule no CHECK can express

The rule. A student may not enrol in more than six sections in one semester.

Step 1 — why a CHECK cannot do it. A CHECK sees one row of one table. "More than six" is a fact about *other* rows — the student's other enrolments — so it is out of reach, exactly as Chapter 19's two-table dependency was.

Step 2 — BEFORE INSERT, so the bad row never arrives.

```
CREATE TRIGGER max_six BEFORE INSERT ON enrolment
WHEN (SELECT COUNT(*) FROM enrolment
       WHERE roll_no = NEW.roll_no AND semester = NEW.semester) >= 6
BEGIN
  SELECT RAISE(ABORT, 'a student may take at most six sections a term');
END;
```

Step 3 — the gap. This guards INSERT. An UPDATE that moves an enrolment from Fall into a Spring that is already full is not guarded. **Every rule implemented by trigger needs a trigger for every event that can break it** — the lesson of the summary table, again, and the single most common defect in trigger code.

Step 4 — why BEFORE and not AFTER. Either would work: `RAISE(ABORT)` in an AFTER trigger also undoes the statement. BEFORE states the intent — *check, then allow* — and does no work on a row that will be refused.

INSTEAD OF: making a view accept writes

A view that accepts writes is called **updatable**. In SQLite **no** view is: none can be written to directly — not even `fees_view`, which reads a single table: `cannot modify fees_view because it is a view`. Other systems accept writes to simple one-table views; SQLite accepts none, and an INSTEAD OF trigger is the only way to say what a write to a view should do:

```
CREATE TRIGGER fees_view_update INSTEAD OF UPDATE OF name ON fees_view
BEGIN
  UPDATE student SET name = NEW.name WHERE roll_no = OLD.roll_no;
END;

UPDATE fees_view SET name = 'Ayesha K. Khan'
 WHERE roll_no = '2021-IT-001';      -- now works
```

This completes the external level: a group of users can read *and* correct the part of the database they see, through the view, without ever naming the base table.

Common Pitfalls

- **Believing a view stores rows.** It stores a SELECT. It cannot be stale.
- **Avoiding views for speed.** Measured: compiled away; same instructions as the query written out.
- **Reading `SCAN` in a grouped view's plan as "no push-down".** It may mean there was no index to search with.
- **A trigger for one event when the rule can be broken by three.** The summary table had no UPDATE trigger and was wrong after one UPDATE.
- **Using `NEW` in a DELETE trigger.** There is no new row.
- **Treating a view as security in SQLite.** No users, so no restriction.
- **Concluding from one timing run.** The first version of this chapter's harness did, and its explanation was wrong.

Try It Yourself: Prove a view is free, catch a summary table lying, and enforce a rule

```
# --- 1. The seeded database ------------------------------------------
python code/schema/seed.py university.db
sqlite3 university.db
```

```
-- --- 2. A view, and the query it wraps --------------------------
CREATE VIEW transcript AS
SELECT e.roll_no, c.course_code, c.title, e.semester, e.grade
  FROM enrolment e JOIN course c ON c.course_code = e.course_code;
EXPLAIN QUERY PLAN SELECT * FROM transcript
 WHERE roll_no = (SELECT MIN(roll_no) FROM student);
-- The filter reached the base table. Where can you see that?

-- --- 3. What was stored -----------------------------------------
SELECT type, name, sql FROM sqlite_master WHERE name = 'transcript';
-- Text. No rows. That is the whole of a view.

-- --- 4. An external schema --------------------------------------
CREATE VIEW fees_view AS SELECT roll_no, name FROM student;
SELECT * FROM fees_view LIMIT 3;
SELECT * FROM student LIMIT 3;
-- The second still works. What would a server DBMS do instead?

-- --- 5. A rule no CHECK can express -----------------------------
CREATE TRIGGER max_six BEFORE INSERT ON enrolment
WHEN (SELECT COUNT(*) FROM enrolment WHERE roll_no = NEW.roll_no
                                       AND semester = NEW.semester) >= 6
BEGIN SELECT RAISE(ABORT, 'at most six sections a term'); END;
SELECT roll_no, COUNT(*) FROM enrolment WHERE semester = 'FALL2026'
 GROUP BY roll_no HAVING COUNT(*) = 6 LIMIT 1;
-- Try to enrol that student in one more FALL2026 section. Record
-- the message. Then find a way around the trigger with an UPDATE.

-- --- 6. An audit trail ------------------------------------------
CREATE TABLE grade_audit (roll_no TEXT, course_code TEXT,
  old_grade TEXT, new_grade TEXT,
  changed_at TEXT DEFAULT (datetime('now')));
CREATE TRIGGER audit_grade AFTER UPDATE OF grade ON enrolment
WHEN OLD.grade IS NOT NEW.grade BEGIN
  INSERT INTO grade_audit (roll_no, course_code, old_grade, new_grade)
  VALUES (OLD.roll_no, OLD.course_code, OLD.grade, NEW.grade);
END;
UPDATE enrolment SET grade = 'A'
 WHERE rowid = (SELECT MIN(rowid) FROM enrolment WHERE grade = 'B');
```

```
SELECT * FROM grade_audit;
```

```
# --- 7. The measured version ----------------------------------------
python code/m20_views.py
```

What to notice. Step 5's second half is the chapter's warning, found by hand: a rule enforced on INSERT can be broken by UPDATE. Writing the BEFORE UPDATE trigger that closes the gap is part of the project in Chapter 23.

Checkpoint — Test Yourself

1. A view's compiled program had the same 23 instructions as the query it wraps, differing only in cursor numbers. Explain what SQLite did with the view to produce that, and why it means the view cannot be stale.
2. The summary table read in 0.076 ms and the view in 53.69 ms, and after one UPDATE only the view was right. State the condition under which you would choose the table, and list everything you would have to build alongside it.
3. Write the trigger that closes the gap in the six-section rule for UPDATE, and say which of `NEW` and `OLD` it must use, and why.

Chapter Summary

- A **view** stores a SELECT, not rows. It is substituted into each query that names it, so it is always current.
- **Measured:** a view compiled to **the same 23 instructions** as the query it wraps, differing only in cursor numbers. It costs nothing.
- **Measured:** a filter on a view reaches the base table, even through a grouped view run as a co-routine — visible once there is an index to search with.
- Views provide the **external level**, **logical data independence**, and a query written once. In SQLite they do not restrict access.
- A **trigger** runs per row, BEFORE or AFTER an event, with `NEW` and `OLD`; RAISE(ABORT) refuses the change.
- **Measured:** an audit trigger cost **what the same** INSERT **written by hand cost** — the ranges overlapped on every run — about a hundredth of a millisecond a row in bulk. With a commit per row its cost was inside the disk-sync noise.
- Triggers enforce rules a CHECK cannot reach — but only for the events they are written for. Every event that can break the rule needs one.
- **Measured:** after one UPDATE, the view was right and the trigger-maintained table was wrong. Default to the view.
- INSTEAD OF triggers make a view writable.

Review Questions

1. (MCQ) CREATE VIEW stores: (a) a copy of the rows; (b) the text of the query; (c) an index; (d) a trigger.

2. (MCQ) In a DELETE trigger, which row reference is available? (a) `NEW`; (b) `OLD`; (c) both; (d) neither.

3. (Short) Why can a view never be stale, while a summary table can?

4. (Short) Give a rule that needs a trigger because no CHECK can express it, and say why.

5. (Short) When is INSTEAD OF the only kind of trigger that will do?

6. **(Applied)** Write the views that give the examinations office, the fees office and the student portal their external schemas from Chapter 2's review question, an INSTEAD OF trigger letting the portal correct a student's phone number, and say what SQLite cannot enforce about any of it.

7. **(Applied)** The audit trigger added about 0.013 ms per row in bulk. Estimate what auditing every grade change costs per semester on the seeded data, decide whether that matters, and list the three trigger events an audit of *enrolment* would need to be complete.

Part VI

Making It Work, and What Comes Next

Everything so far has been one person, one database, one question at a time. Part VI is what changes when that stops being true: when the table is large enough that how you look matters, when two people write at once, and when somebody who should not be reading the data is. It closes with the two questions a database course owes its students — what a real server does that this one does not, and when the relational model is the wrong answer.

the rule everybody repeats, and the row count at which it stops being true

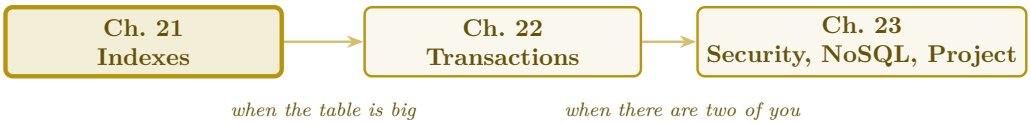


Figure P6.1: Reading path through Part VI. Chapter 21 is the measured chapter students remember, because it contradicts something they were told in their first week and shows them exactly where the contradiction begins.

Chapter 21

Indexes, and Where They Stop Helping

Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** what an index is, using the B-tree, without assuming any course in data structures.
- **Predict** whether a query can use a given index, including a composite one, from the order of its columns.
- **Find** the share of a table above which an index is slower than reading everything, by measurement.
- **Explain** why SQLite's optimiser chose a twenty-times slower plan, and **override** it.
- **State** what an index costs, and **decide** whether a proposed one is worth it.

Before You Start

Chapter 3: the optimiser and the buffer manager. Chapter 4: the database is a file of 4,096-byte pages. Chapter 13, Chapter 15 and Chapter 16, each of which ended with an index making something faster — this chapter is about why, and about where it stops. **No data-structures course is assumed.** You need `code/m21_indexes.py`.

Key Terms in This Chapter

index • B-tree • node • fan-out • depth • leaf • rowid • covering index • composite index • leftmost prefix • selectivity • crossover • sequential read • random read • ANALYZE • `sqlite_stat1` • STAT4 • INDEXED BY • NOT INDEXED

21.1 What an Index Is

A table in SQLite is stored in order of its rowid. To find the student with roll number `2024-IT-00317` in a table with no index on roll number, the only method is to look at every row — a SCAN. Four chapters have shown plans that said SEARCH instead, and every one of them was using the same idea.

Index

An **index** on a column is a separate structure, kept in the same file, that holds every value of that column **in sorted order**, each paired with the rowid of the row it came from.

Because it is sorted, a value can be found without looking at most of them — exactly as a word is found in a dictionary without reading the dictionary. Because each entry carries a rowid, the full row can then be fetched directly.

SQLite keeps every index as a **B-tree**. You do not need to know how one is built to use it; you need to know one number about it, below.

The B-tree, in one paragraph and one number

A B-tree is a tree of pages — its **nodes**. The top page holds a few hundred keys that divide the whole range of values into a few hundred parts, each with a pointer to a page below; that page divides its part into a few hundred more, and so on, until the bottom pages — the **leaves** — hold the actual entries.

The number that matters is the **fan-out**: how many children each page points to. A 4,096-byte page holding short integer keys has a fan-out of a few hundred. So:

1 level	a few hundred entries
2 levels	tens of thousands
3 levels	millions
4 levels	hundreds of millions

Finding any value means reading one page per level — three or four reads, whether the table has a thousand rows or a million. That is the whole reason part 1 of the measurement below shows a lookup costing the same at every size.

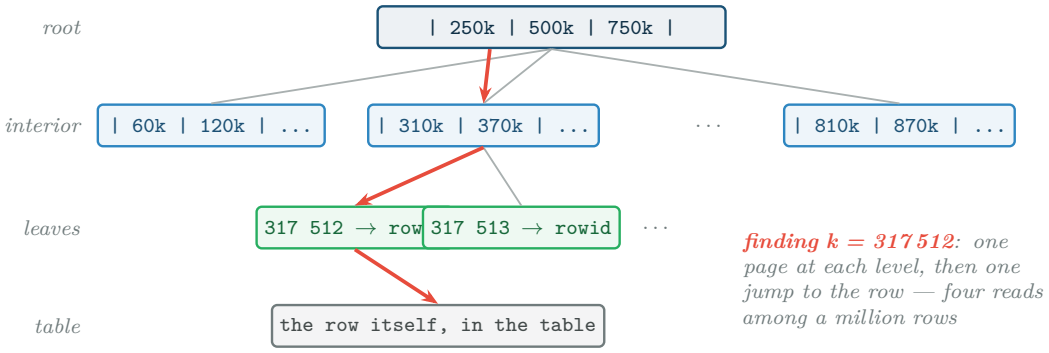


Figure 21.1: An index on k over a million rows, three levels deep. Each page divides its range among a few hundred children, so the path from the root to any key is three pages long, and one more read fetches the row. The scan it replaces reads every page of the table.

21.2 Measured: Where an Index Wins, and Where It Loses

Measured: A 3,501× win, a 21× loss, and an optimiser that could not tell the difference

`code/m21_indexes.py`, a table of up to 1,000,000 rows of about 80 bytes, column k a random permutation (so index order and table order are unrelated). SQLite 3.51.2, Intel i5-6600, 16 GB, Windows 11, mechanical disk. Best of three.

1. A point lookup — `SELECT v FROM t WHERE k = ?`

rows	scan	index	speed-up
10,000	1.37 ms	0.053 ms	26×
100,000	19.03 ms	0.053 ms	360×
1,000,000	199.90 ms	0.057 ms	**3,501×**

The scan grows in proportion to the table. The index barely moves: a hundred times the rows added one level, at most, to a three-level tree.

2. The crossover — `SUM(length(v)) WHERE k BETWEEN 0 AND` x, each plan forced, over a growing share of a million rows

share	rows	via index	full scan	faster	optimiser chose
0.1%	1,000	7.1 ms	178.1 ms	index	index
1%	10,000	90.6 ms	207.0 ms	index	index
2%	20,000	182.0 ms	210.0 ms	index	index
5%	50,000	436.4 ms	190.3 ms	scan	**index**
10%	100,000	784.3 ms	202.5 ms	scan	**index**
20%	200,000	1,549.9 ms	220.1 ms	scan	**index**
50%	500,000	3,955.3 ms	287.5 ms	scan	**index**
100%	1,000,000	7,934.8 ms	381.0 ms	scan	**index**

The index stops winning at about 2% of the table. Below that it reads only the rows it needs. Above it, every matching row is a separate jump to a different page of the table, a **random read** — and a scan, which reads each page once and in order, a **sequential read**, wins by more as more rows are wanted. At the whole table the index plan was **21× slower**.

3. Why the optimiser chose the index every time

STAT4 histograms compiled into this SQLite	**no**
what ANALYZE recorded in `sqlite_stat1`	`'1000000 1'`
the whole table, as written	7,952.4 ms (`SEARCH ... ik`)
the whole table, with `+k`	407.2 ms (`SCAN t`)

`'1000000 1'` is everything the optimiser knew: a million rows, one row per value of k. It knew k was unique. It did *not* know how the values were spread, so it could not tell `BETWEEN 0 AND 999` from `BETWEEN 0 AND 999999`, and treated every range on an indexed column as selective. The histograms that would have told it — STAT4 — are not compiled into the SQLite that Python ships.

4. What indexes cost — 100,000 rows; insert 10,000 more; median of 3

	file bytes	insert	(range)
0 indexes	8,232,960	71.5 ms	47.6–79.0
1 on k	9,383,936	64.8 ms	62.0–69.2
+ 1 on v (68-byte text)	17,272,832	137.9 ms	123.8–150.2
+ 1 on (k, v)	25,485,312	141.5 ms	135.2–168.9

Read the ranges first. Zero and one index overlap: the new rows arrived in key order, so each new entry went on the right-hand end of the index — the cheapest case, as for roll numbers issued in sequence. The index on the wide text column is the one that cost: it doubled the insert time and added more bytes than the table had.

⚠ What the crossover means in practice

- **An index is for finding a few rows, and "a few" must be measured.** About 2% for this million-row table. On the seeded *enrolment* table — 60,037 narrower rows — an index on *grade* selecting 21% ran in 8.94 ms against a scan's 9.40 ms: a tie, so the crossover there sits near a fifth. The share moves with table size, row width and cache; the shape — a share past which the index loses — does not.
- **The optimiser can be wrong, and here it was wrong every time past 2%** — not through a bug, but because it was never told how the data was spread. "The database will choose the best plan" assumes it has the information to.

- **Two ways to overrule it.** Write `+k` instead of `k` in the condition, which hides the column from the index for that one query; or name the table `t NOT INDEXED`. The opposite, `t INDEXED BY ik`, insists on a named index — the harness used both to measure each plan at every share. Use them only on a query you have measured.
- **Low-selectivity columns rarely deserve an index.** An index on *programme* (four values) or *grade* (six) selects a quarter or a fifth of the table for every value — at or past the crossover. Measured above, the grade index only tied a scan: it bought nothing, and it still costs bytes and a write on every change.

21.3 Using an Index Well

Composite index, leftmost prefix, covering index

A **composite index** on (a, b) is sorted by a, and by b within each a — like a telephone directory sorted by surname, then first name.

It can serve a condition on a, or on a and b together, but **not on b alone**: the b values are scattered through every a. This is the **leftmost-prefix** rule, and Chapter 15's measurement turned on it — *enrolment*'s primary key is (roll_no, course_code, ...), so it could find a student's enrolments and could not find a course's.

A **covering index** contains every column a query needs, so the table itself is never read. `USING COVERING INDEX` in a plan means exactly that, and it removes the per-row jump that loses past the crossover.

Ordering: what an index gives for free, and what it cannot

From part 3 of the measurement, on 100,000 rows with an index on k:

`ORDER BY k LIMIT 10`	0.08 ms	reads in index order, stops at 10
`ORDER BY v LIMIT 10`	23.87 ms	`TEMP B-TREE`: sorts every row
`ORDER BY k % 7, k LIMIT 10`	24.74 ms	an expression: sorts too

An index is a sorted copy of *one* ordering. A query that wants that order gets it for nothing — and, with LIMIT, stops after ten rows instead of reading a hundred thousand. Any other order, **including an expression of the indexed column itself**, means sorting everything first. The third row is the one people do not expect.

Common Pitfalls

- **Indexing every column.** Each is another B-tree kept in step on every write. Measured: the text index doubled insert time.
- **Indexing a column with few values.** Every lookup selects a large share — past the crossover.
- **Expecting an index on (a, b) to serve a condition on b.** Leftmost prefix only.
- **Wrapping the indexed column in an expression.** `WHERE lower(name) = 'ali'` cannot use an index on *name*. (SQLite allows an index on the expression itself.)
- **Trusting the optimiser's choice without measuring.** It picked a 21× slower plan here, after ANALYZE.
- **Measuring an index on a table whose order matches the column.** Then "random" lookups are sequential and the crossover vanishes. The harness shuffles k for exactly this reason.

Try It Yourself: Find the crossover on your own machine

```
# --- 1. The measured version first: it builds a million rows --------
python code/m21_indexes.py --quick
```

```
-- --- 2. On the university database, the indexes you already have -
.open university.db
SELECT name, tbl_name, sql FROM sqlite_master WHERE type = 'index';
-- Some have no sql: they were made by PRIMARY KEY and UNIQUE.

-- --- 3. Leftmost prefix, seen in a plan ---------------------------
EXPLAIN QUERY PLAN SELECT * FROM enrolment WHERE roll_no = 'x';
EXPLAIN QUERY PLAN SELECT * FROM enrolment WHERE course_code = 'x';
-- Same primary-key index. One SEARCH, one SCAN. Why?

-- --- 4. A covering index ------------------------------------------
CREATE INDEX idx_enr_cg ON enrolment(course_code, grade);
EXPLAIN QUERY PLAN
SELECT grade, COUNT(*) FROM enrolment WHERE course_code = 'CS-101'
 GROUP BY grade;
-- Find COVERING. Which table was never read?

-- --- 5. An index on a column with six values ----------------------
CREATE INDEX idx_enr_grade ON enrolment(grade);
.timer on
SELECT COUNT(*), AVG(section_no) FROM enrolment WHERE grade = 'B';
SELECT COUNT(*), AVG(section_no) FROM enrolment WHERE +grade = 'B';
-- Which was faster? What share of the table did 'B' select?

-- --- 6. What ANALYZE knows ----------------------------------------
ANALYZE;
SELECT * FROM sqlite_stat1;
-- Read each stat string: rows, then rows per key value.

-- --- 7. An order the index cannot give ----------------------------
EXPLAIN QUERY PLAN SELECT * FROM student ORDER BY name LIMIT 10;
CREATE INDEX idx_student_name ON student(name);
EXPLAIN QUERY PLAN SELECT * FROM student ORDER BY name LIMIT 10;
EXPLAIN QUERY PLAN SELECT * FROM student ORDER BY lower(name) LIMIT 10;
```

```
# --- 8. The full version, a million rows (about a minute) -----------
python code/m21_indexes.py
```

What to notice. Step 5 is the crossover in your own schema. Grade B selects 21% of enrolments, and on the seeded table the two forms come out level — 8.94 against 9.40 ms on the measuring machine — while the optimiser chooses the index without hesitation. Compare that with the 2% the million-row table gave: the same index, the same optimiser, a crossover ten times further out, because a small table is cheap to scan. **The rule is not "2%". The rule is that the share exists and has to be measured.**

Checkpoint — Test Yourself

1. A lookup took 0.053 ms at 10,000 rows and 0.057 ms at a million, while a scan went from 1.37 to 199.90 ms. Explain both using the fan-out and depth of a B-tree, without assuming any data-structures course.
2. The optimiser chose the index for a query that selected the whole table, after ANALYZE, and it was 21× slower. Explain what information it lacked, where that information would have come from, and two ways to make it scan instead.

3. An index on *enrolment*(roll_no, course_code) exists. For each of `WHERE roll_no = ?`, `WHERE course_code = ?` and `WHERE roll_no = ? AND course_code = ?`, say whether it can be used and why.

Chapter Summary

- An **index** is every value of a column, in sorted order, each with the rowid of its row — kept by SQLite as a **B-tree**.
- A B-tree's **fan-out** is a few hundred, so a million entries fit in three levels and a lookup reads one page per level.
- **Measured:** a point lookup was **3,501**× faster with an index at a million rows, and cost the same at every size.
- **Measured: the index stopped winning at about 2%** of the table. Past that, a jump per row loses to reading every page once, in order; at the whole table the index was **21× slower**.
- **Measured: the optimiser chose the index at every share**, after ANALYZE, because `sqlite_stat1` records only row counts and STAT4 histograms are not compiled into Python's SQLite.
- Override a bad plan with `+column` or NOT INDEXED — on a query you have measured.
- A **composite** index serves only its **leftmost prefix**; a **covering** index means the table is never read.
- An index gives its own order free, including for LIMIT; any other order, even an expression of the indexed column, costs a sort.
- **Measured:** indexes cost bytes and writes; a wide text index doubled insert time, while an integer index on in-order keys cost nothing measurable.

Review Questions

1. (MCQ) An index on (a, b) can be used for a condition on: (a) b alone; (b) a alone; (c) either; (d) neither.

2. (MCQ) `USING COVERING INDEX` in a plan means: (a) the index is unique; (b) the table was not read; (c) the index covers the whole table; (d) a temporary index was built.

3. (Short) Why does a query that wants half a table run faster as a scan than through an index?

4. (Short) What does `sqlite_stat1`'s string `'1000000 1'` tell the optimiser, and what does it not?

5. (Short) Why is an index on *grade* unlikely to help?

6. (Applied) For the university schema, propose the three indexes you would add, justify each with a query from Part IV that it serves and the plan it would change, and name one index you would refuse to add, with the reason.

7. (Applied) Re-run part 2 of the harness with k in the same order as the rowid instead of shuffled (change one line), and report what happens to the crossover. Explain the result in terms of sequential and random reads, and say why the shuffled version is the honest measurement.

Chapter 22

Transactions, Concurrency and Recovery

Learning Outcomes

By the end of this chapter you will be able to:

- **State** the four ACID properties and **give** a failure that each one prevents.
- **Show** a lost update happening, and **prevent** it with the right kind of transaction.
- **Explain** SQLite's locks and why it allows one writer at a time.
- **Distinguish** the deadlock SQLite can have from the one a server DBMS has, and **say** how each is resolved.
- **Back up** a live database safely, and **state** what the journal does after a crash.

Before You Start

Chapter 1: atomicity and concurrent access were two of the six problems. Chapter 3: the transaction, lock and recovery managers. Chapter 11: BEGIN and COMMIT in a migration. No operating-systems course is assumed. You need `code/m22_transactions.py`.

Key Terms in This Chapter

transaction • BEGIN • COMMIT • ROLLBACK • SAVEPOINT • ACID • atomicity • consistency • isolation • durability • autocommit • lost update • dirty read • serialisable • lock • SHARED • RESERVED • EXCLUSIVE • DEFERRED • IMMEDIATE • deadlock • `SQLITE_BUSY` • `busy_timeout` • rollback journal • WAL • `synchronous`

22.1 Promises About Groups of Changes

Every constraint in Part II was a promise about a row. A transaction is a promise about a *group* of changes: that they happen together, or not at all, and that nothing anybody else does at the same moment can see them half done. Chapter 1 listed atomicity and concurrent access among the six problems of files. This chapter is where the database keeps those promises — and measures what keeping them costs.

Transaction, and ACID

A **transaction** is a sequence of operations treated as one unit, begun with BEGIN and ended with COMMIT (keep it) or ROLLBACK (undo it all). Outside one, SQLite is in **autocommit**: every statement is its own transaction.

Atomicity: all of it happens, or none. **Consistency**: it takes the database from one state satisfying every constraint to another. **Isolation**: concurrent transactions do not see each other's unfinished work. **Durability**: once committed, it survives a crash.

Measured: A crash, a lost update, a deadlock, and the price of durability

`code/m22_transactions.py`; two real connections to one file, real failures. SQLite 3.51.2, Intel i5-6600, Windows 11, mechanical disk.

1. Atomicity — move a student between sections, a DELETE then an INSERT, and fail between them

	enrolled in	
autocommit	**nothing**	the DELETE was already permanent
BEGIN ... ROLLBACK	section 1	the two were one unit

2. The lost update — two clerks each add one to a seat count of 40: A reads, B reads, A writes, B writes

	what happened	**final**
no transaction	both read 40, both wrote 41	**41**
BEGIN DEFERRED	B's write refused: `database is locked`; then A's COMMIT refused too; B rolled back; A committed	41
BEGIN IMMEDIATE	B could not begin until A had finished; then read 41 and wrote 42	**42**

With no transaction an enrolment simply vanished, and nobody was told. With DEFERRED the two clerks **deadlocked**: B held a read lock and wanted to write; A held the write lock and needed B's read lock released before it could commit. Neither could move until B gave up — so B's +1 was lost again, *but B was told*, and can retry. IMMEDIATE counted both.

3. `SQLITE_BUSY` — A holds the write lock for 300 ms; B tries to write

B's busy_timeout	**outcome**	**B waited**
0 ms (the default)	failed at once: `database is locked`	0 ms
1,000 ms	succeeded after A committed	455 ms

B waited for A's 300 ms, A's commit, and the interval at which SQLite's busy handler retries — and then went ahead.

4. Durability, priced — 200 inserts

	total	**per row**
a commit each, `synchronous = FULL`	4,590.9 ms	22.954 ms
one commit for all, `synchronous = FULL`	27.8 ms	0.139 ms
a commit each, `synchronous = OFF`	239.2 ms	1.196 ms

One commit for all 200 was 165× faster than one each. A commit waits until the disk confirms the data is really on it; on a mechanical disk that wait is 23 ms and is almost the whole cost. `synchronous = OFF` skips the wait — 19× faster, still 9× slower than batching — and gives up durability: a power cut can lose transactions that were reported as committed, or corrupt the file. Batching keeps the guarantee *and* is faster.

Four rules from the measurement

- **Anything that is two statements and must be one fact goes in a transaction.** Measured: autocommit left a student enrolled in nothing.
- **A read-then-write goes in BEGIN IMMEDIATE.** It takes the write lock before the read, so nobody else can read the old value in between. DEFERRED (the default) deadlocked.
- **Set `busy_timeout` on every connection.** The default is 0, so a second writer is refused instead of waiting. It is per connection, like Chapter 8's `foreign_keys`, and for the same reason has to be set every time.
- **Group writes into transactions; never turn durability off to go faster.** The guarantee and the speed are not in conflict here.

22.2 How SQLite Isolates: One Lock for the Whole File

SQLite's locks

SQLite locks the *database file*, not rows or tables. A connection progresses through:

SHARED	may read. Any number of connections may hold it.
RESERVED	intends to write. Only one at a time; readers continue.
EXCLUSIVE	is committing. Needed to write to the file; requires every SHARED lock to be released first.

BEGIN DEFERRED takes no lock until the first read (then SHARED) or write (then RESERVED). BEGIN IMMEDIATE takes RESERVED at once. BEGIN EXCLUSIVE takes EXCLUSIVE at once.

The consequence: **one writer at a time, for the whole database**, and transactions that are **serialisable** — the result is always as if they had run one after another. There are no weaker isolation levels to choose from, because there is nothing to relax — and so no **dirty read**: no connection can ever see another's uncommitted change.

A **savepoint** marks a point inside a transaction to roll back to without abandoning the whole of it: `SAVEPOINT s1; ... ROLLBACK TO s1;` undoes only the work since `s1`.

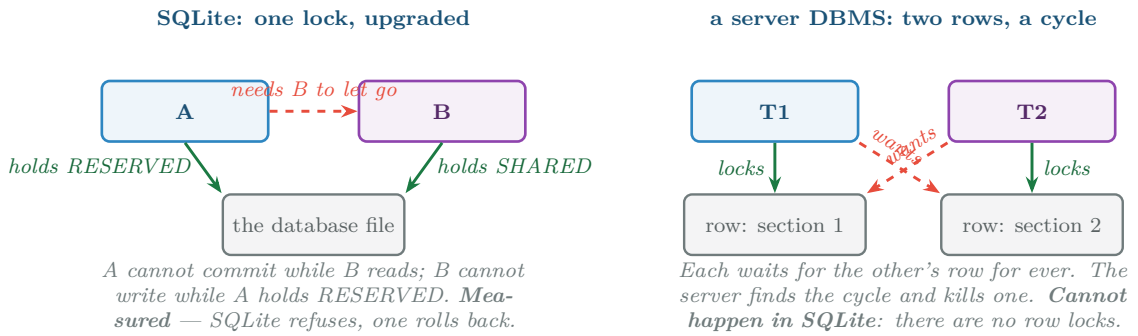


Figure 22.1: Two kinds of deadlock. The left one was produced by the harness with two DEFERRED transactions. The right one needs row-level locks, which SQLite does not have, so it is drawn here rather than run — Chapter 23 is where a server's locking is compared properly.

What SQLite does not have, and what you would get instead

A server DBMS — PostgreSQL, MySQL, Oracle — locks rows, and most use **multi-version concurrency control**: a writer creates a new version of a row while readers keep seeing the old one, so readers and writers never block each other. It offers several *isolation levels*, from READ COMMITTED to SERIALIZABLE, trading correctness for concurrency.

SQLite has none of that, and for one database file on one machine it does not need it. What it costs is concurrency: many readers, **one** writer. For a university's results system with a few clerks it is plenty; for a booking site with a thousand simultaneous purchases it is the reason to move to a server, and Chapter 23 makes that comparison.

22.3 Recovery, and Backup

The journal, and what happens after a crash

Before changing a page of the database file, SQLite writes the page's *original* contents to a **rollback journal**, `university.db-journal`. Only then does it change the file. At COMMIT it deletes the journal.

After a crash, the next connection finds a journal still present, knows a transaction was interrupted, and copies the original pages back. The database returns to its state before the transaction began — atomicity and durability, kept by one extra file.

WAL mode (`PRAGMA journal_mode = WAL`) reverses the idea: changes are appended to a write-ahead log and copied into the database later. Readers then never block the writer, which is the nearest SQLite comes to a server's concurrency.

Backing up a database that is in use

The wrong way: `copy university.db backup.db` while a program is writing. If the copy happens mid-transaction it captures a half-written file without its journal — a backup that may not open, discovered on the day it is needed.

Two right ways, both of which take a consistent snapshot:

```
.backup backup.db                    -- in the sqlite3 shell
VACUUM INTO 'backup.db';             -- as SQL, from any program
```

And Chapter 11's rule applies: **open the backup** and run `PRAGMA integrity_check;` on it. A backup that has never been restored is a hope.

Common Pitfalls

- **Two statements that must be one fact, in autocommit.** Measured: a student enrolled in nothing.
- **Read-then-write in BEGIN DEFERRED.** Measured: a deadlock. Use IMMEDIATE.
- **Leaving `busy_timeout` at 0.** A second writer is refused instead of waiting.
- **A commit per row in a bulk load.** Measured: 165× slower.
- **`synchronous = OFF` for speed.** It gives up durability and is still slower than batching.
- **Copying a live database file as a backup.** Use `.backup` or VACUUM INTO, then open the result.
- **Python's default `isolation_level`.** The `sqlite3` module silently issues BEGIN for you before writes. The harness sets `isolation_level=None` so every BEGIN is one you wrote.

Try It Yourself: Lose an enrolment, deadlock two clerks, and back up safely

```
# --- 1. Two shells on ONE file: open two terminals ------------------
sqlite3 txn.db          # terminal A
sqlite3 txn.db          # terminal B
```

```
-- --- 2. In A only: a table to fight over -------------------------
CREATE TABLE section (id TEXT PRIMARY KEY, seats_taken INTEGER) STRICT;
INSERT INTO section VALUES ('CS-103/1', 40);

-- --- 3. The lost update, by hand: type in this order -------------
-- A:  SELECT seats_taken FROM section;          -- 40
-- B:  SELECT seats_taken FROM section;          -- 40
-- A:  UPDATE section SET seats_taken = 41;
-- B:  UPDATE section SET seats_taken = 41;
-- A:  SELECT seats_taken FROM section;          -- 41. Two clerks, +1.
```

```
-- --- 4. The deadlock: reset to 40, then type in this order -------
-- A:  BEGIN DEFERRED;  SELECT seats_taken FROM section;
-- B:  BEGIN DEFERRED;  SELECT seats_taken FROM section;
-- A:  UPDATE section SET seats_taken = 41;      -- works
-- B:  UPDATE section SET seats_taken = 41;      -- database is locked
-- A:  COMMIT;                                   -- also locked. Why?
-- B:  ROLLBACK;
-- A:  COMMIT;                                   -- now it works

-- --- 5. The fix: reset to 40, then -----------------------------
-- A:  BEGIN IMMEDIATE;  SELECT seats_taken FROM section;
-- B:  BEGIN IMMEDIATE;                          -- refused: wait
-- A:  UPDATE section SET seats_taken = 41;  COMMIT;
-- B:  BEGIN IMMEDIATE;  SELECT seats_taken FROM section;   -- 41
-- B:  UPDATE section SET seats_taken = 42;  COMMIT;

-- --- 6. Waiting instead of failing -----------------------------
-- B:  .timeout 5000
-- Repeat step 5. What does B do now instead of refusing?

-- --- 7. A safe backup ------------------------------------------
.backup txn_backup.db
```

```
# --- 8. Open the backup and check it ---------------------------------
sqlite3 txn_backup.db "PRAGMA integrity_check; SELECT * FROM section;"

# --- 9. The measured version -----------------------------------------
python code/m22_transactions.py
```

What to notice. Step 4's second COMMIT is the one to explain. A already holds the write lock; to commit it needs the EXCLUSIVE lock, which requires every reader to leave — and B is a reader, waiting to become a writer. Neither can move. SQLite cannot wait its way out of that, so it refuses, and one of the two must roll back.

Checkpoint — Test Yourself

1. For each ACID property, give the measured result in this chapter that shows it being kept or broken, and say what would have happened without it.
2. Two clerks deadlocked under BEGIN DEFERRED and both increments counted under BEGIN IMMEDIATE. Explain both outcomes in terms of which lock each connection held and wanted at each step.
3. A bulk load of 200 rows took 4.6 seconds with a commit per row and 28 ms in one transaction. Explain the difference, say why `synchronous = OFF` is not the fix, and say what it would cost the university if it were used.

Chapter Summary

- A **transaction** groups changes into one unit with BEGIN, COMMIT and ROLLBACK. Outside one, every statement is its own (**autocommit**).
- **ACID**: atomicity, consistency, isolation, durability.
- **Measured:** a failure between a DELETE and an INSERT left a student **enrolled in nothing** in autocommit; inside a transaction, nothing changed.
- **Measured:** two read-then-write clerks **lost an update** with no transaction, **deadlocked** under DEFERRED, and were both counted under IMMEDIATE.

- SQLite locks the whole file — SHARED, RESERVED, EXCLUSIVE — so it has one writer at a time and is always serialisable.
- **Measured:** with `busy_timeout` 0 a second writer was refused; with 1,000 ms it waited 455 ms and succeeded. Set it on every connection.
- A server's two-row deadlock cycle cannot occur in SQLite, which has no row locks; SQLite's deadlock is two connections on the one file lock.
- **Measured:** one commit for 200 inserts was **165**× faster than one each. `synchronous = OFF` gave up durability and was still 9× slower than batching.
- The **rollback journal** holds original pages until commit, and is replayed after a crash. Back up with `.backup` or VACUUM INTO, never by copying a live file — then open the backup.

Review Questions

1. **(MCQ)** In autocommit mode, each statement is: (a) never committed; (b) its own transaction; (c) part of one transaction per connection; (d) rolled back on exit.
2. **(MCQ)** BEGIN IMMEDIATE acquires: (a) no lock; (b) SHARED; (c) RESERVED; (d) a row lock.
3. **(Short)** Why does a read-then-write need its write lock before the read?
4. **(Short)** Why can SQLite never have a two-row deadlock cycle?
5. **(Short)** What does the rollback journal contain, and what is done with it after a crash?
6. **(Applied)** Write the transaction that moves a student from one section to another and adjusts both sections' seat counts. Say which kind of BEGIN you would use and why, what happens if it fails after the second statement, and how a second clerk doing the same move at the same moment would be affected.
7. **(Applied)** The university's results are entered by six clerks at once during the last week of term. Using the measured lock behaviour, explain what each clerk would experience with `busy_timeout` 0 and with 5,000 ms, estimate how long a clerk might wait if each commit took 23 ms, and say at what number of clerks you would recommend moving to a server DBMS — and why the answer is not a fixed number.

Chapter 23

Security, NoSQL, and the Semester Project

Learning Outcomes

By the end of this chapter you will be able to:

- **Explain** SQL injection, **demonstrate** it against your own database, and **prevent** it with parameters.
- **Write** the GRANT and REVOKE statements a server would need for the university's external schemas, and **say** what SQLite offers instead.
- **Describe** the four families of NoSQL database and **state** what each gives up to get what it offers.
- **Store and query** a document inside SQLite using its JSON functions, and **compare** it with the relational form.
- **Decide** whether a given system needs a server DBMS, using the limits measured in this course.

Before You Start

Appendix A, section A.6 — **the scope statement, which governs this chapter's laboratory**. Chapter 3 for the authorisation manager SQLite lacks; Chapter 20 for external schemas; Chapter 22 for the one-writer limit. The finished `university.db`.

Key Terms in This Chapter

SQL injection • parameterised query • placeholder • least privilege • GRANT • REVOKE • role • encryption at rest • NoSQL • key–value store • document store • wide-column store • graph database • schema-on-read • horizontal scaling • eventual consistency • CAP • JSON

23.1 Security: the Component SQLite Does Not Have

Chapter 3 drew six components and said SQLite was missing one: the authorisation manager. HEC's 2017 and 2023 outlines do not mention security at all; HEC 2025's fifth learning outcome requires it, which is why it is here. It has two halves — keeping out people who should not be in, and making sure that people who *are* in cannot make the database do something it should not — and SQLite can help with only the second.

Scope — this governs the laboratory below

Everything in this chapter runs against **a database file you created, on a machine you own**: your `university.db`, with invented data, which you can delete. **Running any of it against a system you do not own and have not been authorised in writing to test is a criminal offence in Pakistan** under the **Prevention of Electronic Crimes Act 2016**, sections 3 and 4. The purpose of the demonstration is that you never write the vulnerable code, not that you can find it in somebody else's.

SQL injection

SQL injection happens when a program builds a SQL statement by pasting in text that a user supplied. If the text contains SQL, the database runs it — as part of the statement the programmer wrote, with the programmer's permissions.

A **parameterised query** sends the statement and the values separately. The statement has **placeholders** (`?`); the values are bound to them afterwards and are *never* parsed as SQL, whatever they contain.

Worked Example: One login check, built two ways

A student portal checks a roll number before showing marks.

The vulnerable version — text pasted into SQL:

```
roll = input("Roll number: ")
sql = f"SELECT name FROM student WHERE roll_no = '{roll}'"
rows = cx.execute(sql).fetchall()
```

Step 1 — an ordinary input. The roll number `2023-IT-00003` produces the statement

```
SELECT name FROM student WHERE roll_no = '2023-IT-00003'
```

and one row comes back. Correct.

Step 2 — an input that is SQL. `x' OR '1'='1` gives

```
SELECT name FROM student WHERE roll_no = 'x' OR '1'='1'
```

The quote in the input closed the string the programmer opened; `OR '1'='1'` is true for every row. On the seeded database it returns **all 4,000 students**.

Step 3 — the parameterised version.

```
rows = cx.execute("SELECT name FROM student WHERE roll_no = ?",
                  (roll,)).fetchall()
```

The same input now looks for a student whose roll number is, literally, `x' OR '1'='1`. There is none: **0 rows**. The quote is a character in a value, not the end of a string, because the value was never parsed.

The rule. Never build SQL from input with `+`, `%` or an f-string. Every value goes through a placeholder. There is no escaping function that is as safe, and no input validation that replaces it.

The external schemas of Chapter 2, as a server would enforce them

SQLite cannot run any of this. It is written out because it is what the external level of Chapter 2 *means* once a database has users:

```
CREATE ROLE fees_office;
CREATE ROLE exams_office;
CREATE ROLE portal;

GRANT SELECT ON fees_view TO fees_office;          -- names and rolls only
GRANT SELECT, UPDATE (grade) ON enrolment TO exams_office;
GRANT SELECT ON transcript TO portal;              -- a view, never a table

REVOKE ALL ON student FROM portal;                 -- least privilege
```

Least privilege: each user gets exactly what its job needs and no more. Then even a successful SQL injection through the portal can only read what `portal` may read — the damage is bounded by the grant, not by the programmer's care.

What SQLite offers instead is the operating system: whoever can read the file can read all of it. File permissions, an encrypted disk, or an encrypted SQLite build (SQLCipher) — **encryption at rest** — protect the file; nothing protects one table from another.

23.2 NoSQL: What It Gives Up to Get What It Offers

Four families

NoSQL names databases that do not use the relational model. The four common families:

Family	Stores	Good at
key–value	a value per key, opaque	caching, sessions: get and put by key
document	a nested document (JSON) per key	records read whole, with varying shape
wide-column	rows with many sparse columns	huge write volumes across many machines
graph	nodes and edges	paths: friends of friends, prerequisites of prerequisites

What they usually give up, to different degrees: **joins**, **a declared schema** (the data's shape is checked when read, not when written — **schema-on-read**), and **multi-record ACID transactions**. What they gain is the ability to spread one database across many machines (**horizontal scaling**). Spreading it costs something too: the **CAP** result says that when the network between machines fails, a system must choose between staying consistent and staying available, and many choose **eventual consistency** — copies may disagree for a while, and agree once updates have spread.

Worked Example: One student's record as a document, inside SQLite

SQLite has JSON functions, so a document store can be shown without installing one.

Step 1 — build the document. One student, with their enrolments nested inside:

```
CREATE TABLE student_doc (roll_no TEXT PRIMARY KEY, doc TEXT NOT NULL);
INSERT INTO student_doc
SELECT s.roll_no,
       json_object('name', s.name, 'programme', s.programme,
         'enrolments', (SELECT json_group_array(
             json_object('course', e.course_code, 'term', e.semester,
                         'grade', e.grade))
           FROM enrolment e WHERE e.roll_no = s.roll_no))
  FROM student s;
```

Step 2 — what it is good at. One student's whole record is one row, read in one lookup, with no join:

```
SELECT doc FROM student_doc WHERE roll_no = '2023-IT-00003';
```

Step 3 — what it is bad at. "How many A grades were given in CS-101?" was one GROUP BY in Chapter 16. Now every document must be opened and unpacked:

```
SELECT COUNT(*) FROM student_doc, json_each(doc, '$.enrolments') j
 WHERE json_extract(j.value, '$.course') = 'CS-101'
   AND json_extract(j.value, '$.grade')  = 'A';
```

On the seeded database both forms find **161**. The relational query took 9.8 ms; the document form took **78.4 ms**, eight times as long, because it parsed every one of 4,000 documents to look inside each.

Step 4 — what it has given up. Nothing stops a document with `"grade": "Z"`, or `"grde"`, or a course that does not exist — Chapter 8's three model-based constraints are gone, and a course's title, if stored, is copied into every document that mentions it: Chapter 18's update anomaly, by design.

The lesson. A document store is the right model when records are read and written *whole* and rarely questioned *across*. A university's results are the opposite — nearly every question in Part IV crosses students — which is why this course is relational. That is a decision about the *questions*, not about which technology is newer.

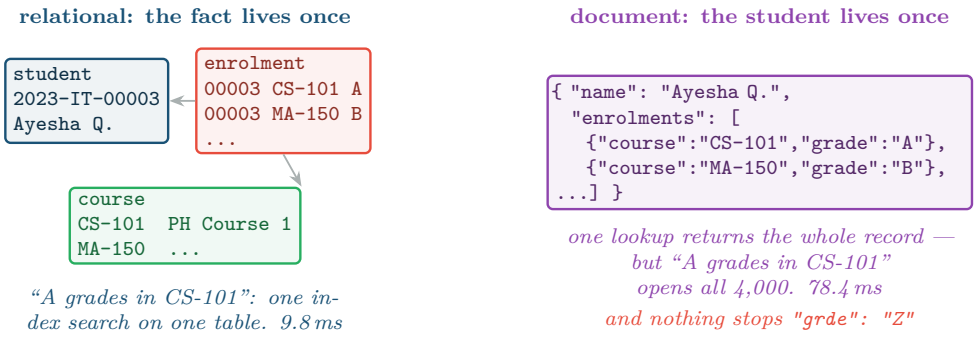


Figure 23.1: One student, held two ways. The relational form stores each fact once and answers questions across students with a search; the document form stores each student once and answers questions about one student in a single read. Which is right depends on which kind of question the system is asked.

When to leave SQLite — using what this course measured

Limit	Measured in	Move to a server when
one writer at a time	Chapter 22	many users write at the same moment, all day
no users or privileges	Chapter 3, this chapter	different people must see different parts
on one machine	Chapter 4	the data must be reached over a network by many programs
optimiser without STAT4	Chapter 21	large, skewed tables where plan choice matters

And stay with it when none of those apply: SQLite is the most widely deployed database in the world — in every phone, every browser — precisely because for one program and one file it has nothing to administer. Chapter 4 counted that as zero installation steps, zero ports and zero accounts; this table is the other side of the same trade.

23.3 The Semester Project

What you submit

The project is the `university.db` you have built since Chapter 4, finished. Submit one folder containing:

1. `design.pdf` — your ER diagram with every cardinality and participation constraint (Chapter 6), and the dependencies of every table (Chapter 17).
2. `v1_create.sql`, `v2_constraints.sql` and `v3_final.sql` — the schema and each migration (Chapter 11), each runnable from an empty file.
3. `queries.sql` — twelve queries answering the questions on the next page, each preceded by a comment stating the question.
4. `triggers.sql` — the six-section rule for INSERT *and* UPDATE (Chapter 20), and an audit trigger.
5. `report.pdf` — six pages: three decisions you made and the evidence for each, including at least one measurement of your own.
6. `university.db` itself, after `PRAGMA integrity_check` and `PRAGMA foreign_key_check` both pass.

The twelve questions

1. Every student's transcript, with course titles and credit hours.
2. Each student's grade-point average over graded courses only.
3. The top three students in each programme, ties included.
4. Sections with no instructor — the 73 of Chapter 13.
5. Instructors teaching nothing, written with NOT EXISTS.
6. Students who have never received an A.
7. Every prerequisite of a course, to any depth.
8. Students enrolled in a course without having passed its direct prerequisites.
9. Enrolments and average credit hours per department.
10. Each course's pass rate, ungraded enrolments excluded and counted.
11. A running total of credit hours per student, in correct term order.
12. One question of your own that needs a window function.

For each one, `queries.sql` must also contain its EXPLAIN QUERY PLAN, and your report must name any query whose plan you changed with an index, with both timings.

How it is marked

Component	%	What earns the marks
design: ER diagram and dependencies	20	every constraint argued, not just drawn
schema and migrations	20	runs from empty; every constraint enforced, checked
the twelve queries	30	correct with nulls; NOT IN on a nullable column scores 0
triggers	10	every event that can break the rule is covered
report and measurement	20	claims backed by numbers you produced

A query that returns the right answer on the seeded data and the wrong one when a null appears is wrong. Test every one of them with a null.

Common Pitfalls

- **Building SQL from input.** Parameters, always.
- **Escaping instead of parameterising.** Escaping is a list of characters somebody remembered; a parameter is never parsed.
- **Treating a view as access control in SQLite.** No users, so no restriction.
- **Choosing NoSQL because it is newer.** Choose by the questions: across records, relational; whole records, document.
- **Submitting a schema that only runs on your machine.** Run every script from an empty file before handing in.
- **Testing queries only on data without nulls.** The FALL2026 grades are null for a reason.

Try It Yourself: Inject your own database, then make it impossible

Read Appendix A, section A.6, first. Every step uses `university.db` on your own machine.

```
# --- 1. The seeded database, which you can delete -------------------
python code/schema/seed.py university.db
```

```
# --- 2. The vulnerable query, in a file called inject.py ------------
import sqlite3
cx = sqlite3.connect("university.db")
for roll in ["2023-IT-00003", "x' OR '1'='1"]:
    sql = f"SELECT COUNT(*) FROM student WHERE roll_no = '{roll}'"
    print(repr(roll), "->", cx.execute(sql).fetchone()[0], "rows")

# --- 3. The same two inputs, parameterised --------------------------
for roll in ["2023-IT-00003", "x' OR '1'='1"]:
    n = cx.execute("SELECT COUNT(*) FROM student WHERE roll_no = ?",
                   (roll,)).fetchone()[0]
    print(repr(roll), "->", n, "rows (parameterised)")
```

```
-- --- 4. A document, and a question it is bad at -----------------
.open university.db
CREATE TABLE student_doc (roll_no TEXT PRIMARY KEY, doc TEXT NOT NULL);
-- Type in Step 1 of the worked example, then:
SELECT json_extract(doc, '$.name'),
       json_array_length(doc, '$.enrolments')
  FROM student_doc LIMIT 3;

-- --- 5. Compare one cross-record question both ways -------------
.timer on
SELECT COUNT(*) FROM enrolment
 WHERE course_code = 'CS-101' AND grade = 'A';
SELECT COUNT(*) FROM student_doc, json_each(doc, '$.enrolments') j
 WHERE json_extract(j.value, '$.course') = 'CS-101'
   AND json_extract(j.value, '$.grade') = 'A';
-- Same count? Record both timings.

-- --- 6. Put in a document no relational table would accept --------
INSERT INTO student_doc VALUES ('X-1',
  '{"name":"Nobody","enrolments":[{"course":"NONE","grde":"Z"}]}');
-- Accepted. List the constraints from Chapter 8 it breaks.
```

```
# --- 7. Run your project's checks before submitting -----------------
sqlite3 university.db "PRAGMA integrity_check; PRAGMA foreign_key_check;"
```

What to notice. Step 2's second line and step 3's second line are the same input, giving 4,000 rows and 0. Nothing about the input changed; only whether it was ever parsed as SQL. And step 6 is the other half of the chapter: a model that accepts anything checks nothing.

Checkpoint — Test Yourself

1. Explain exactly why `x' OR '1'='1` returned every student when pasted into the query and none when bound as a parameter, naming the character that made the difference and what happened to it in each case.
2. Write the GRANT statements giving the examinations office permission to read every enrolment and change only grades, and say what SQLite would let that office do with the same file.
3. For the question "all of one student's grades" and the question "the number of A grades in CS-101", say which model — relational or document — answers each more naturally, and why.

Chapter Summary

- **SQL injection**: text pasted into SQL is parsed as SQL. `x' OR '1'='1` returned **4,000 students** pasted and **0** as a parameter.
- A **parameterised query** sends values separately; they are never parsed. There is no escaping function as safe.
- A server enforces external schemas with roles, GRANT and REVOKE, and **least privilege** bounds the damage of any flaw. SQLite has no users: the file's permissions are the only control.
- **NoSQL** families — key–value, document, wide-column, graph — usually give up joins, a declared schema and multi-record ACID to gain horizontal scaling.
- A document is read whole in one lookup and questioned across records only by unpacking every one; it accepts any shape, so none of Chapter 8's constraints hold.
- Choose the model by the questions. A university's questions cross records, which is why this course is relational.
- Leave SQLite when the measured limits bite: many simultaneous writers, per-user access, network access, large skewed tables. Otherwise stay.
- The project is the `university.db` built all term, with its design, migrations, twelve queries, triggers and a report backed by your own measurements.

Review Questions

1. (MCQ) A parameterised query prevents SQL injection because the value is: (a) escaped; (b) validated; (c) never parsed as SQL; (d) encrypted.

2. (MCQ) Which NoSQL family suits "friends of friends"? (a) key–value; (b) document; (c) wide-column; (d) graph.

3. (Short) State the principle of least privilege and say how it limits the damage of an injection that does succeed.

4. (Short) What does "schema-on-read" mean, and what does a document store therefore not check on write?

5. (Short) Give two limits measured in this course that would justify moving the university's database to a server.

6. **(Applied)** Rewrite a program of your own from the term that builds any SQL with string formatting so that every value is a parameter. Show one input that changed the old version's behaviour and does not change the new one's.

7. **(Applied)** A startup proposes storing the university's records as one JSON document per student "because it is simpler". Using this chapter's worked example and the twelve project questions, say which questions become harder and why, what integrity is lost, and under what changed requirements the startup would be right.

Appendices

Chapter A

The Toolkit: Setting It Up and Keeping It

This appendix is the whole of the course's setup. It is short, and that is the point: the longest-running complaint about database courses is that the first fortnight goes on installing something. Here there is nothing to install.

A.1 What You Need

Thing	Version used here	How to get it
SQLite	3.51.2	**You already have it.** It ships inside Python, inside most operating systems, and inside a great deal of software you already run
A terminal	any	Already present
Python	3.13	Only for the measurement harnesses in `code/`. No chapter's *content* needs it
A text editor	any	For writing `.sql` files

Check, in one line, before you read any further

```
python -c "import sqlite3; print(sqlite3.sqlite_version)"
```

If that prints a version number — any version from 3.37 onwards — you have everything this course needs. If you also have the `sqlite3` command on your path, so much the better; if not, every lab works through Python's built-in shell instead, and Chapter 4 shows both.

A.2 Why SQLite, and What It Costs You

This is a curriculum decision, not a shortcut

HEC names no database product in any edition of this curriculum. The 2017 and 2023 outlines say a student should use SQL "in any DBMS"; the 2025 outcomes say "popular database management systems". There is no edition in which choosing SQLite departs from the published syllabus.

What it buys is the first fortnight. There is no server to start, no port, no password, no user account, no configuration file and nothing to go wrong before the subject begins. A database is one file. You can copy it, email it, put it in version control, delete it and start again — and in Chapter 18 you will do exactly that.

What it costs is stated here and again in Chapter 23, because a tool's limits are part of what you are learning:

- **No users, no GRANT, no REVOKE.** SQLite has no access control of any kind: whoever can open the file can do anything to it. Chapter 23 writes the SQL a server DBMS would use and runs it nowhere.
- **No stored procedures.** Triggers (Chapter 20) carry the argument about putting logic in the database.

- **No row-level locking and no isolation levels.** SQLite lets one writer at a time and makes the others wait. Chapter 22 can therefore show you a lost update prevented, a busy timeout and a failed upgrade from a deferred transaction — but a genuine two-row deadlock is drawn rather than run.

Every one of those three is a thing a *server* adds, and knowing which three they are is more useful than having had them switched on all semester without noticing.

A.3 The Ten Commands You Will Actually Use

SQLite's shell has two kinds of input: SQL, which ends with a semicolon, and *dot-commands*, which do not and which are the shell's own.

Command	What it does
`sqlite3 university.db`	Open (or create) a database and enter the shell
`.tables`	List the tables
`.schema student`	Show the `CREATE` statement for one table — the fastest way to find out what a table actually promises
`.headers on`	Print column names above results. Turn this on first, every time
`.mode box`	Draw results as a table instead of pipe-separated text
`.read setup.sql`	Run a file of SQL. How every lab in this book starts
`.output results.txt`	Send what follows to a file
`.import data.csv student`	Load a CSV into a table
`.backup backup.db`	Take a consistent copy *while the database is open* — Chapter 22 explains why that is not the same as copying the file
`.quit`	Leave

</> The two lines to type at the start of every session

```
sqlite3 university.db
```

then, inside the shell:

```
.headers on
.mode box
PRAGMA foreign_keys = ON;
```

The first two make results readable. **The third is not cosmetic.** SQLite does not enforce foreign keys unless you ask it to, *every time you connect*, and Chapter 8 is largely about what happens when you forget. Put all three in a file called `init.sql` and start every session with `.read init.sql` until it is a habit.

A.4 Resetting to a Known State

A database is one file, which makes this the simplest reset in the series:

```
# --- 1. Keep a copy before anything risky ------------------------
cp university.db university.db.bak

# --- 2. Start again from nothing ---------------------------------
rm university.db
sqlite3 university.db ".read code/schema/v1_create.sql"

# --- 3. And refill it --------------------------------------------
python code/schema/seed.py 1000
```

Do this deliberately once, in Week 2

Delete your database on purpose, rebuild it from the script, and confirm it is the same. A student who has done that once is a student who will keep their schema in a file rather than in the shell's history — which is the single habit that decides whether the Week 16 project can be handed to somebody else and run.

A.5 The Machine Every Timing Was Taken On

Eleven chapters quote a measured figure. Every one was produced by running the named file in `code/` on this machine:

Component	Specification
Processor	Intel Core i5-6600, 4 cores, 3.30 GHz
Memory	16 GB
Operating system	Windows 11, 64-bit
SQLite	3.51.2
Python	3.13.13
Storage	Mechanical disk, not a solid-state drive — which matters for Chapter 21 and is the reason its figures are larger than yours may be

Your numbers will differ, and the ratios should not

A measurement is a fact about the machine it ran on. If your laptop has a solid-state disk it will beat these figures, perhaps by a factor of ten on the chapters that touch storage.

What should survive the move is the shape: the row count at which an index starts to pay (Chapter 21), the direction normalisation moves query time (Chapter 18), the fact that one writer blocks another (Chapter 22). If a *ratio* in this book does not reproduce on your machine, that is worth investigating and worth telling your lecturer about. If an absolute number does not, that is expected.

A.6 Scope: What You May Point Any of This At

Read this before Chapter 23's laboratory

Everything in this course runs against **a database file you created, on a machine you own**. Chapter 23 discusses SQL injection and shows what a badly built query permits — against your own university database, with invented data, in a file you can delete.

Running the same thing against any system you do not own and have not been authorised in writing to test is a criminal offence in Pakistan under the **Prevention of Electronic Crimes Act 2016**, sections 3 and 4, which cover unauthorised access to an information system and to data.

There is no educational exemption, and a university assignment is not authorisation. "I was learning", "I did no harm" and "the site was insecure" are not defences.

In practice this is an easy rule to follow, because the only file named anywhere in this book is `university.db` and it is yours.

Chapter B

Just Enough Sets, Logic and Logarithms

This appendix is the mathematics the course uses, and nothing more. It is short because the course needs little: sets for the relational model and the algebra, logic for WHERE and for nulls, one idea about logarithms for indexes, and the definition of a function for Part V. Each section names the chapter that depends on it. Questions 6–9, 11 and 12 of the Prerequisite Self-Check are answered here.

B.1 Sets

Set, element, subset

A **set** is a collection of distinct things, its **elements**, with no order. $\{1, 2, 3\}$ and $\{3, 1, 2\}$ are the same set; $\{1, 1, 2\}$ is just $\{1, 2\}$.

$x \in A$ means x is an element of A. $A \subseteq B$ (A is a **subset** of B) means every element of A is in B. The empty set is $\varnothing$. $|A|$ is the number of elements.

Why it matters: a relation is a *set* of tuples (Chapter 7). "No duplicates" and "no order" are not rules somebody chose for databases; they are what the word "set" means.

The three operations — Self-Check question 6

$A = \{1, 2, 3\}$, $B = \{2, 3, 4\}$.

Operation	Result	Contains	SQL
union $A \cup B$	$\{1, 2, 3, 4\}$	in A or in B (or both)	UNION
intersection $A \cap B$	$\{2, 3\}$	in both	INTERSECT
difference $A - B$	$\{1\}$	in A and not in B	EXCEPT

Difference is not symmetric: $B - A = \{4\}$. And $A \cap B = A - (A - B)$: take away from A everything not in B, and what is left is in both. Chapter 12 uses exactly this to show intersection is not a fundamental operator.

Ordered pair, Cartesian product — Self-Check question 7

An **ordered pair** (a, b) has a first element and a second; unlike a set, $(a, b) \neq (b, a)$ when $a \neq b$.

The **Cartesian product** $A \times B$ is the set of every pair (a, b) with $a \in A$ and $b \in B$. It has $|A| \times |B|$ elements.

$\{1, 2\} \times \{x, y, z\} = \{(1, x), (1, y), (1, z), (2, x), (2, y), (2, z)\}$ — six pairs. In Chapter 12 the product of 4,000 students and 60,037 enrolments is 240,148,000 pairs, which is why no database ever computes a join that way.

B.2 Relations and Functions

Relation, in the mathematical sense

A **relation** between sets A and B is any subset of $A \times B$ — a chosen collection of pairs. "Is enrolled in", between students and sections, is the set of (student, section) pairs that are true.

With more sets the pairs become tuples: a relation over A, B and C is a subset of $A \times B \times C$. That is precisely Codd's definition, and why a database table is called a relation. The *domains* of Chapter 7 are the sets A, B, C.

Function — Self-Check question 12

A **function** f from A to B is a relation in which **every element of A is paired with exactly one element of B**. One input, one output. $f(x) = x^2$ is a function: each x has one square. "The square roots of x" is not: 4 has two.

Many inputs may share an output — $(-2)^2 = 2^2$ — that is allowed. What is not allowed is one input with two outputs.

This is the whole foundation of Chapter 17. The functional dependency roll_no → name says "name is a function of roll_no": a roll number has exactly one name. Two students sharing a name is fine (many inputs, one output). One roll number with two names is the violation.

B.3 Logic

Propositions, and the three connectives

A **proposition** is a statement that is true or false: "$x > 5$".

P	Q	P AND Q	P OR Q	NOT P
T	T	T	T	F
T	F	F	T	F
F	T	F	T	T
F	F	F	F	T

AND is true only when both are; OR when at least one is. OR is *inclusive*: true when both are.

Two Self-Check questions, answered

Question 8 — is "$x > 5$ and $x < 3$" ever true? No. No number is both greater than 5 and less than 3, so the statement is a **contradiction**: false for every x. A WHERE clause that is a contradiction returns no rows, and no error — a common bug when a condition was meant to be OR.

Question 9 — "not (P and Q)" with P true, Q false. P AND Q is false, so its negation is **true**. **De Morgan's laws** make this mechanical:

$$\neg(P \wedge Q) \equiv \neg P \vee \neg Q \qquad \neg(P \vee Q) \equiv \neg P \wedge \neg Q$$

"Not both" means "at least one is not". Chapter 14 uses the second law to explain NOT IN: `x NOT IN (a, b)` is `NOT (x = a OR x = b)`, which is `x <> a AND x <> b`.

A third truth value — what SQL adds

Two-valued logic has no room for "I do not know". SQL needs one, because a comparison with a null cannot be true or false. It adds UNKNOWN, and the tables extend (Chapter 14 has them in full). The two facts that matter:

- `F AND` anything is F, and `T OR` anything is T — a missing value cannot change those.
- `NOT` UNKNOWN is UNKNOWN. So De Morgan's laws still hold, but a condition and its negation no longer cover every row — 19,992 enrolments fall through the gap on the seeded data.

Quantifiers, and their SQL

$\exists x\, P(x)$, "there exists", is true if P holds for *at least one* x. $\forall x\, P(x)$, "for all", is true if P holds for *every* x.

They are linked by negation: $\neg\forall x\, P(x) \equiv \exists x\, \neg P(x)$ — "not every student passed" means "some student did not".

SQL has $\exists$ directly, as EXISTS (Chapter 15). It has no FOR ALL; "every" is written as "there is no counter-example": NOT EXISTS (... NOT ...). "Students who passed every course they took" becomes "students for whom there is no course they took and failed".

B.4 Logarithms: How Many Halvings

The logarithm as a count — Self-Check question 11

How many times must 1,000,000 be halved to reach 1? Halving 20 times gives $1{,}000{,}000/2^{20} = 1{,}000{,}000/1{,}048{,}576 < 1$, and 19 times is not quite enough. So about **20**.

That count is the **logarithm**: $\log_2 1{,}000{,}000 \approx 20$. In general $\log_b n$ is how many times you divide n by b to reach 1 — or, the other way round, the power of b that gives n.

The property that matters: **multiplying n adds only a little to its logarithm.** $\log_2$ of a thousand is about 10; of a million, 20; of a billion, 30. A thousand times more data costs ten more halvings.

Why it is the whole story of Chapter 21

A B-tree page divides its range into a few hundred parts — divides by, say, $b = 300$ rather than by 2. Its depth is $\log_{300} n$:

rows n	$\log_2 n$	$\log_{300} n$
1,000	10	1.2
1,000,000	20	2.4
1,000,000,000	30	3.6

Rounding up, a million rows need a tree three levels deep, and a billion only four. That is why Chapter 21 measured a lookup at 0.053 ms for 10,000 rows and 0.057 ms for a million: a hundredfold more data added at most one level. A scan, which reads everything, has no logarithm in it — a hundred times the rows is a hundred times the work.

Three to try, with answers

1. With $A = \{a, b, c, d\}$ and $B = \{c, d, e\}$, write $A \cup B$, $A \cap B$, $A - B$ and $B - A$, and give $|A \times B|$.

2. Is "each student has exactly one date of birth" a function from students to dates? Is "each date has exactly one student born on it"? Say which, if either, could be a functional dependency.
3. Write "every section has at least one student" using only NOT EXISTS, by first rewriting it as "there is no section such that ...".

Answers. (1) $\{a, b, c, d, e\}$; $\{c, d\}$; $\{a, b\}$; $\{e\}$; $4 \times 3 = 12$. (2) The first is a function, and is roll_no $\to$ date_of_birth. The second is not — one date has many students — so date_of_birth $\to$ roll_no does not hold. (3) "There is no section with no enrolment":

```
NOT EXISTS (
  SELECT 1 FROM section s
   WHERE NOT EXISTS (
     SELECT 1 FROM enrolment e
      WHERE e.course_code = s.course_code
        AND e.semester    = s.semester
        AND e.section_no  = s.section_no))
```

Chapter C

Answers to the Checkpoints

Three answers per chapter, in order. Attempt each checkpoint from memory before reading these. Several are judgement calls rather than facts, and say so; where an answer quotes a number, the number comes from the chapter's harness or from the seeded database, and you can reproduce it.

Chapter 1 — Why Not Just Files?

1. **It escapes most of them, for now.** With one clerk per file and no shared data there is no redundancy, no inconsistency between copies and no concurrent access. It still has *isolation* ("which borrowers are also in arrears?" needs both files), *integrity* (rules live in each clerk's head) and *atomicity* (a half-saved file). The answer changes the moment any fact — a student's name — is recorded in both files, or a second clerk edits either one.

2. A **database** is the data together with its description; a **DBMS** is the software that manages it. "We bought a database" might mean a product (SQLite, Oracle) with no data in it, or a populated set of tables, or a whole system — three purchases with different costs, owners and risks. You cannot install, back up or migrate it until you know which.

3. Disk is cheap; **disagreement** is not. Redundancy's real cost is *inconsistency*: two copies of one fact that can diverge. Example: a student's name changes and is corrected in `students.csv` but not `fees.csv`. Each file is internally correct; the system now holds two names for one person, and the fee receipt goes to someone who no longer exists.

Chapter 2 — Three Levels, and Why They Are Separate

1. Adding an index: **internal**, physical independence. Adding a column: **conceptual**, logical independence (programs that never named it carry on). Renaming a column: **conceptual**, and logical independence *fails* unless a view keeps the old name. A faster disk: **internal**, physical independence. A view with no home addresses: **external**; no independence is involved — it is the external level being used.

2. A schema is the *form*; an instance is what is *written on it*. A blank admission form is the schema: it says there will be a name, a date of birth, a programme. This year's 4,000 filled-in forms are the instance. Reprinting the form with a new box is a schema change and is rare; filling one in is an instance change and happens every minute.

3. The **external level** is affected: views can define it, but nothing can confine a user to one, because SQLite has no users. You can still write the views and have programs use only them. You cannot stop anyone who can open the file from reading the base tables. In an application the program itself becomes the gatekeeper — it holds the file, authenticates people, and only ever queries the views — which is enforcement by convention, not by the database.

Chapter 3 — What Is Inside a DBMS, and Who Does What

1. The **optimiser** (a different plan — perhaps statistics were refreshed, or an index dropped) and the **buffer manager** (Monday the pages were in memory; Friday they were not). Tell them apart with EXPLAIN QUERY PLAN: if the plan changed, it is the optimiser; if the plan is identical, run the query twice — a fast second run means the first was paying for disk reads.

2. **Advantage:** any tool can discover a database it has never seen with an ordinary SELECT, and the schema is backed up with the data. **Problem:** the dictionary can be damaged by the

same failures as the data, and in SQLite anyone who can write the file can in principle corrupt `sqlite_master` and with it every table.

3. DBA: when to back up. Designer: what the keys are. Programmer: how a query is parameterised. End user: what question is asked. In this course the designer, programmer and DBA decisions are all yours; the DBA decisions that need users and privileges simply cannot be made, and backup becomes `.backup` run by you.

Chapter 4 — Your First Database, in One Command

1. Page 1 (4,096 bytes) holds the 100-byte database header and the schema table `sqlite_master` — the data dictionary. Page 2 (4,096 bytes) is the root page of the new table's B-tree, empty. Fewer is impossible: storage is allocated in whole pages, the dictionary must exist for the file to describe itself, and the table must have a root.

2. From 1,000 to 10,000 rows the file grew $262{,}144 - 32{,}768 = 229{,}376$ bytes, about 25.5 bytes a row, so 2,000 rows predicts $32{,}768 + 25{,}500 \approx 58{,}300$, rounded up to a whole page: 61,440 (15 pages). **Measured: 57,344 bytes, 14 pages.** The naive guess — double 32,768 — gives 65,536 and is further off, because part of the file is fixed overhead (the header page and the tree's interior page) that does not grow with the rows.

3
```sql
CREATE TABLE instructor (
  emp_id      TEXT PRIMARY KEY NOT NULL,
  name        TEXT NOT NULL,
  department  TEXT NOT NULL,
  designation TEXT
) STRICT;
```

`emp_id` is **text** because `E-0114` has a letter and a leading zero and is never used in arithmetic; it is **not null** because an identifier that is missing identifies nothing. `designation` may be null: a new appointment may not have one yet.

Chapter 5 — The Entity-Relationship Model

1. "Almost every" means some have two, and one column cannot hold two values that can be searched. The first student with two numbers forces either a lost number or a comma-separated list, which no query can search correctly. Instead, a `student_phone(roll_no, phone)` table with both columns in the key; it costs one join whenever a number is shown.

2. Candidate keys: {emp_id} (issued uniquely by the university), {cnic} (national identity, unique by law), {email} (issued one per employee). **Primary key: emp_id** — unique, always present (CNIC may be missing for foreign staff), never changing (email changes with a name or a reorganisation), short. `office_no`, `name` and `department` are not keys: offices are shared, names repeat, departments hold many people.

3. A grade is a fact about *a student in a particular section* — neither alone determines it. On STUDENT there is room for one grade per student for their whole degree. On SECTION there is room for one grade per section, the same for every student in it. Only the pairing of the two has a grade, and the pairing is the relationship.

Chapter 6 — Cardinality, Participation and Weak Entities

1. **It will refuse a section with no instructor.** A section is created in June for the autumn timetable; the lecturer is appointed in August. For two months the database will not accept the section — and somebody will invent an instructor called "TBA".

2. **M:N**, with **partial** participation of STUDENT (a student may be enrolled in nothing) and **partial** participation of SECTION (a section may have no students). With $(1, N)$ at the section

end, participation of SECTION becomes **total**: a section with no students could not exist, so a new section could not be created until its first enrolment arrived.

3. Section numbers repeat across courses — every course has a section 1 — so `section_no` identifies a section only within its owner. Partial key: `section_no`. Full key: {course_code, semester, section_no}. With a surrogate `section_id` you must still declare UNIQUE (course_code, semester, section_no); forget it and the same section can be created twice, with students split between the copies.

Chapter 7 — The Relational Model

1. **Degree 5** counts attributes and belongs to the schema; **cardinality 200** counts tuples and belongs to the instance. Cardinality changes without a schema change, with every INSERT or DELETE. Degree changes only with a schema change: ALTER TABLE ... ADD or DROP COLUMN.

2. `SELECT COUNT(*) FROM enrolment WHERE roll_no = ?` — if the same enrolment can be inserted twice, a student's course count is inflated. Declare the composite PRIMARY KEY (roll_no, course_code, semester, section_no), which makes the duplicate impossible.

3. One null marker carries both meanings, and SQLite has no second kind. **Design 1:** a `grade_status` column (`'not sat'`, `'lost'`, `'graded'`) beside a nullable grade — one column more, and a rule that the two agree. **Design 2:** a separate `exam_incident` table recording lost papers — no nulls overloaded, but a join to tell the two cases apart.

Chapter 8 — Keys and Integrity Constraints

1. `PRAGMA foreign_keys;` tells you whether *this connection* enforces — not whether anyone else's does, or ever did. `PRAGMA foreign_key_list(t);` shows the declarations exist. `PRAGMA foreign_key_check;` lists every orphan already present: rows means enforcement was off at some point; none means no violation survives, *not* that enforcement was on. Nothing in the file records whether it ever was.

2. SECTION→COURSE: CASCADE — a section is a weak entity and means nothing without its course. SECTION→INSTRUCTOR: SET NULL — the section survives a resignation; participation is partial. ENROLMENT→STUDENT: RESTRICT — a student with a grade history must never be silently deleted.

3. It will accept a **null** primary key and store it, violating **entity integrity**. Fix it with NOT NULL on the column, or STRICT on the table — measured in Chapter 8, each one alone refuses the null.

Chapter 9 — Mapping an ER Diagram to Tables

1. On DEPARTMENT, as `head_emp_id`: every department must have a head (total), but most instructors head nothing (partial), so the totally-participating side can declare the column NOT NULL. On INSTRUCTOR instead it would be `heads_dept_code`, null for all but eight of a hundred instructors.

2. `roll_no`: who. `course_code`: which course. `semester`: when. `section_no`: which group within that offering. Without `semester` a student could not appear in the same course and section number in two terms — so repeating a failed course becomes impossible, or overwrites the old grade.

3. The table *prerequisite*, whose primary key is the pair and both of whose columns reference *course*; the unstated rule is the CHECK:

```
CREATE TABLE prerequisite (
```

```
  course_code TEXT NOT NULL REFERENCES course(course_code),
  requires    TEXT NOT NULL REFERENCES course(course_code),
  PRIMARY KEY (course_code, requires),
  CHECK (course_code <> requires)    -- nothing requires itself
) STRICT;
```

(A two-step cycle needs a trigger; Chapter 15 shows why it also needs UNION.)

Chapter 10 — SQL: Creating the Schema

1. The marks were 95, 72, `'absent'`, 88 and 100. Affinity left `'absent'` as text; AVG read it as 0, so the sum was $95 + 72 + 0 + 88 + 100 = 355$ and, because it is not null, the count was 5: $355/5 = 71$. Proof: `SELECT typeof(score), COUNT(*) FROM marks GROUP BY 1;` shows the text row, and `SELECT SUM(score), COUNT(score) FROM marks;` returns 355 and 5.

2. A column constraint follows one column and can name only it; a table constraint follows all the columns and can name several. In `v1_create.sql`: the composite PRIMARY KEY of *section* (three columns), and the three-column FOREIGN KEY of *enrolment* — neither belongs to one column, so neither can be written beside one.

3. **Compatibility**: other tools and older SQLite versions (before 3.37) cannot read a STRICT table, and it forbids type names like `VARCHAR` or `DATETIME` that ported SQL uses. That is a real cost when the file is shared with old software; for this course's database it does not apply, and the measured cost was zero.

Chapter 11 — Changing a Schema That Is Already Full

1. Nullable `email`: **cheap** — no existing row changes. `email` NOT NULL: **expensive** — every row needs a value, so backfill and rebuild. `semester` text to integer: **expensive** — a rebuild, and step 0 must find any value that will not convert. Splitting a whole address: **impossible** to do exactly — the parts were never recorded separately, and parsing free text guesses.

2. After the migration, backfilled values are indistinguishable from values entered since; a rollback that drops the column cannot tell which were invented, and one that keeps them keeps invented data. To make it reversible, record at migration time which rows were backfilled — a `migration_backfill(roll_no)` table, or a flag column — so the reverse can remove exactly those.

3. If the migration fails after COMMIT but the version is set afterwards, that is merely late. The real danger is the reverse: set after a migration that failed and was rolled back, and the file claims to be version 2 while holding the version 1 schema. Every later migration then assumes columns that are not there.

Chapter 12 — Relational Algebra

1. A relation is a *set*, so π removes duplicates: four programmes, four tuples. A SQL result is a *bag* and keeps every row. DISTINCT reconciles them: `SELECT DISTINCT programme FROM student` returns 4.

2. $\pi_{roll}(\sigma_{semester=2}(student)) - \pi_{roll}(\sigma_{semester='FALL2026'}(enrolment))$, that is:

```
SELECT roll_no FROM student   WHERE semester = 2
EXCEPT
SELECT roll_no FROM enrolment WHERE semester = 'FALL2026';
```

"Nothing" is the difference. The tempting wrong version, a join with `e.semester <> 'FALL2026'`, returns second-year students enrolled in *some other* term — including those also enrolled in FALL2026. Note the two meanings of `semester`: Chapter 13 returns to them.

3. 4,000×60,037 = 240,148,000 intermediate tuples, of which 60,037 survive the selection. **Closure** — every intermediate result is a relation — is what makes algebraic laws such as "select before you multiply" valid, so the optimiser may compute the join by index lookups and still return exactly the same relation.

Chapter 13 — Joins: the Algebra and the SQL

1. The two tables share **two** column names: `roll_no` and `semester`. NATURAL JOIN required both to be equal, and a year of study (`8`) never equals a term label (`'FALL2025'`). In the query: `JOIN ... USING (roll_no)` or ON. In the schema: rename `student.semester` to `year_of_study` — it is the one whose name is misleading.

2. They are the 73 sections whose `emp_id` is null. `SELECT s.* FROM section s LEFT JOIN instructor i ON s.emp_id = i.emp_id WHERE i.emp_id IS NULL;` The decision was making SECTION's participation in TEACHES **partial** in Chapter 6 — deliberately, so a section can exist before its lecturer is appointed.

3. Without an index, every pair of 4,000 students was compared — about 8 million — to find 28,517 matches; with one, only the matches were visited: 108× faster. Growth ratios mislead because the number of *matches* itself grew four-fold per doubling (names come from a fixed pool), so even the indexed join looked quadratic down its column. Only a comparison at equal size isolates search cost.

Chapter 14 — Selecting Rows, and the Trouble with NULL

1. 60,037 − 7,268 − 32,777 = 19,992 rows: the ungraded FALL2026 enrolments, whose grade is null. For them both `= 'A'` and `<> 'A'` are UNKNOWN, and WHERE keeps only *true*. `WHERE grade <> 'A' OR grade IS NULL` returns all 52,769 that are not an A (in SQLite `WHERE grade IS NOT 'A'` says the same).

2. `x NOT IN (a, NULL)` means `x <> a AND x <> NULL`. The second comparison is UNKNOWN. If the first is false the whole is false; if the first is true, `T AND U = U`. So for every x the condition is false or UNKNOWN — never true — and WHERE discards every row.

3. It decides that **a missing mark is a zero** — an absent student failed. Scores 80, 90, null: AVG gives $(80+90)/2 = 85$; AVG(COALESCE) gives $(80+90+0)/3 = 56.7$. Write AVG(score), and report the count of missing marks beside it, so the decision about them is made by the examinations office rather than by a function.

Chapter 15 — Sub-queries, EXISTS and Set Operations

1. The IN sub-query of step 2 is **uncorrelated**. Its EXISTS is **correlated**, on `s.roll_no`. In step 3, NOT IN and the two EXCEPT branches are uncorrelated; NOT EXISTS is correlated, on `i.emp_id`. Steps 4 and 5 are correlated, on `c.course_code`. The recursive CTE's step refers to itself, not to an outer query.

2. NOT IN returned 0. Its list contained the 73 null `emp_id`s, so each instructor's test included `x <> NULL`, which is UNKNOWN — and no row's condition could be true. EXCEPT is a set operation, not a comparison: it removes a left-hand row only if an equal row appears on the right, and a null on the right could only remove a null on the left. `instructor.emp_id` has none.

3. Without an index, each of 48 courses scanned all 60,037 enrolments — 2.9 million row visits — while GROUP BY read them once. With an index, each course became one index search and the correlated form read only what it counted, while GROUP BY still read everything. The advice is really "avoid *unindexed* correlated sub-queries"; correlation itself costs nothing.

Chapter 16 — Grouping, Aggregation and Window Functions

1. 2.383 divides the points by the **40,045** graded enrolments; 1.590 divides the same points by all **60,037**, counting each ungraded enrolment as a zero. Publish 2.383, labelled as the average over graded courses, with the 19,992 in progress reported separately: an unfinished course is not an F.

2. `course_code LIKE 'CS-%'` mentions only a grouping column, so it gives the same answer whether rows are removed before grouping or whole groups after — and SQLite moved it into WHERE itself. `COUNT(*) > 1000` needs the count, which does not exist until grouping is finished, so it can only be tested after. Write WHERE because it states what you mean; the rewrite is an optimisation, not a promise.

3. "Each student's rank within their programme." `SELECT roll_no, programme, gpa, RANK() OVER (PARTITION BY programme ORDER BY gpa DESC) FROM per_student;` GROUP BY programme would collapse each programme to one row and lose every student; the window keeps all of them and adds the rank.

Chapter 17 — Functional Dependencies

1. {emp_id}$^+$: start {emp_id}; emp_id → instructor adds instructor; nothing else applies: **{emp_id, instructor}**. {course_code, semester, section_no}$^+$: pass 1 adds room and emp_id (the section rule) and title, credit_hours, dept_code (the course rule); pass 2 adds instructor (via emp_id) and dept_name (via dept_code); pass 3 adds nothing. Ten attributes; not a superkey — roll_no, name, programme and grade are missing.

2. {roll_no, course_code}$^+$: pass 1 adds name, programme (from roll_no) and title, credit_hours, dept_code (from course_code); pass 2 adds dept_name. **dept_name is in the closure, so the dependency follows.**

3. Step 1: already single right sides. Step 2: in AC → D, is C extraneous? $\{A\}^+ = \{A, B, C\}$ by A → B and A → C, then AC → D gives D — so A → D holds and C is extraneous. Step 3: A → C is redundant (A → B, B → C). Result {A → B, B → C, A → D}; $\{A\}^+ = \{A, B, C, D\}$ in both.

Chapter 18 — Normal Forms: 1NF to BCNF

1. **Update**: renaming department CS changed 7,869 rows (one in *department*), and a half-done rename left two names — caused by dept_code → dept_name. **Insertion**: a new course with no enrolments had no row to live in — course_code → title, ... with a key that needs a student. **Deletion**: removing PH-350's 1,122 enrolments removed the course — the same dependency.

2. Key $\{A, B\}$. A → C is **partial** (violates 2NF); C → D is **transitive** (violates 3NF). So R is in 1NF only. Deepest first: on C → D, $R_1(\underline{C}, D)$, remainder (A, B, C); on A → C, $R_2(\underline{A}, C)$, remainder $R_3(\underline{A}, \underline{B})$. All three are in BCNF.

3. The transcript was a *lookup*: the normalised *enrolment* could search its primary key by roll_no, while *report* had no index at all and was read end to end. The department report was a *full scan*, cheaper over one table than over three. With an index on roll_no the transcript result changes to **a tie** (0.08 against 0.09 ms); the full-scan result does not change.

Chapter 19 — Decomposition, and Denormalising on Purpose

1. $R_1 \cap R_2 = \{\text{course_code}\}$. Its closure (under the rules these two tables can express) does not contain roll_no or grade, so it is a superkey of neither: **lossy**. Joining on course_code pairs every student who took a course with every grade given in it — 182,920 rows from 39,152 true facts.

2. Split *course* into `course_dept(course_code, dept_code)` and `course_dname(course_code, dept_name)`: lossless (course_code is the key of both), but dept_code → dept_name lies in neither. Inserting `('CS-150', 'Computing')` when another CS course says `'Computer Science'` is accepted — measured in Chapter 19.

3. Pays off when reads per write exceed $3.679/69.73 = 0.053$: one read in every 19 writes. On a 1 ms-commit SSD, writes get cheaper in general, and the part of the 3.679 ms that came from making a second table's page durable at each commit shrinks; Chapter 20 measured the trigger's own work at about a hundredth of a millisecond. The added cost per write falls, so the break-even falls too — the summary table pays off at even fewer reads.

Chapter 20 — Views, Triggers and Keeping Data Honest

1. SQLite stored only the view's SELECT text, substituted it into the query that named it, and compiled the result. What runs is the hand-written query with renumbered cursors. Nothing was stored, so nothing can be out of date: each use is the query run now.

2. Choose the table when measured read cost matters — here 700× — and reads greatly outnumber writes. Alongside it: triggers on INSERT, DELETE *and* UPDATE of every table it summarises; a query comparing it with the equivalent view, run on a schedule; and a script that rebuilds it from the base tables when they disagree.

3
```
CREATE TRIGGER max_six_upd
BEFORE UPDATE OF semester, roll_no ON enrolment
WHEN (SELECT COUNT(*) FROM enrolment
       WHERE roll_no = NEW.roll_no
         AND semester = NEW.semester) >= 6
BEGIN SELECT RAISE(ABORT, 'at most six sections a term'); END;
```

   It uses `NEW`, because the limit concerns the term the enrolment is moving *into*; `OLD` is the term it is leaving.

Chapter 21 — Indexes, and Where They Stop Helping

1. Each page of a B-tree divides its range among a few hundred children, so three levels reach millions of entries and a lookup reads one page per level — three or four reads at 10,000 rows or a million. Hence 0.053 against 0.057 ms. A scan reads every page, and a hundred times the rows is roughly a hundred times the pages: 1.37 to 199.90 ms.

2. It lacked the **distribution** of k: `sqlite_stat1` recorded only the row count and one row per key, so every range looked selective. That would come from **STAT4** histograms, which the SQLite shipped with Python does not include. To force the scan: write `+k` in the condition, or `FROM t NOT INDEXED`.

3. `roll_no = ?`: **yes** — the leftmost column. `course_code = ?`: **no** — course_code values are scattered through every roll_no. Both: **yes** — the full prefix, the most selective use.

Chapter 22 — Transactions, Concurrency and Recovery

1. **Atomicity**: the failed move left a student in nothing under autocommit, unchanged inside a transaction. **Consistency**: constraints from Chapter 8 refuse any end state that breaks them. **Isolation**: IMMEDIATE counted both clerks where no transaction lost one. **Durability**: `synchronous = FULL` waited 23 ms per commit for the disk; without it a power cut can lose a reported commit.

2. DEFERRED: both took SHARED to read; A's write took RESERVED; B's write wanted RESERVED and was refused; A's COMMIT wanted EXCLUSIVE, which needs B's SHARED gone — deadlock until B rolled back. IMMEDIATE: A took RESERVED at BEGIN, so B could not begin; after A committed, B began, read 41 and wrote 42.

3. Each commit waited about 23 ms for the disk to confirm; 200 waits is 4.6 s, and one transaction waits once. `synchronous = OFF` removes the wait by abandoning the guarantee — and was still 9× slower than batching. Used for results entry, a power cut in the last week of term could lose grades the clerks saw saved, or corrupt the file.

Chapter 23 — Security, NoSQL, and the Semester Project

1. The **single quote**. Pasted, it closed the string the programmer had opened, so `OR '1'='1'` became SQL and was true for all 4,000 rows. Bound as a parameter, the value was never parsed: the quote was just a character in a roll number that no student has, so 0 rows.

2. ```
   GRANT SELECT         ON enrolment TO exams_office;
   GRANT UPDATE (grade) ON enrolment TO exams_office;
   ```

   In SQLite the same office, if it can open the file, can read and change every table, including fees and addresses — there are no users to grant to.

3. "All of one student's grades": the **document** — one lookup returns the whole record. "A grades in CS-101": **relational** — one indexed condition on one table, against unpacking all 4,000 documents (9.8 against 78.4 ms on the seeded data). The model should follow the questions the system is asked most.
   ```

Chapter D

Glossary

Every key term in the handout, in one line, with the chapter that defines it. The definition here is a reminder; the chapter's is the one to learn.

Term	Meaning	Ch.
ACID	atomicity, consistency, isolation, durability: what a transaction promises	22
aggregate function	computes one value from many rows: COUNT, SUM, AVG, MIN, MAX	16
anomaly	a change that a redundant table makes wrong: update, insertion or deletion	18
ANSI/SPARC architecture	one database described three times: external, conceptual, internal	2
anti-join	the rows of R with *no* partner in S	15
Armstrong's axioms	reflexivity, augmentation, transitivity: derive every dependency that follows	17
atomic value	one indivisible value per attribute per tuple	7
atomicity	a transaction happens wholly or not at all	1, 22
attribute	a named property of an entity type, or a column of a relation	5, 7
attribute closure (X^+)	every attribute that X determines	17
automatic index	an index SQLite builds for one query and then discards	13
autocommit	every statement is its own transaction	22
B-tree	the sorted tree of pages SQLite stores every table and index as	21
backfill	filling a new column for rows that already exist	11
bare column	a column in a grouped query that is neither grouped nor aggregated	16
BCNF	every determinant is a superkey	18
break-even	the read–write ratio at which a denormalisation pays for itself	19
buffer manager	decides which pages are held in memory	3
busy_timeout	how long a connection waits for a lock before failing	22
candidate key	a minimal superkey	5, 7
cardinality (of a relation)	the number of tuples	7
cardinality ratio	1:1, 1:N or M:N — how many on each side	6
Cartesian product	every pairing of a tuple of R with a tuple of S	12
cascade	a referential action: delete the children with the parent	8
CHECK	a constraint every value must satisfy, within one row	8, 10

Term	Meaning	Ch.
closure (algebra)	every operator's result is a relation, so expressions nest	12
column / table constraint	a constraint on one column / on several	10
common table expression	a named sub-query, written with WITH	15
composite attribute	an attribute with meaningful parts	5
composite key	a key of more than one attribute	5
conceptual schema	the whole database described once, without storage	2
concurrent access	more than one user changing data at the same time	1, 22
correlated sub-query	one that refers to a column of the outer query	15
covering index	an index holding every column a query needs	21
crossover	the share of a table past which an index is slower than a scan	21
data dictionary	the tables in which the DBMS stores the schema	3
data independence	changing one level without changing those above it	2
database	data together with its description	1
DBA	database administrator: decides how the database runs	3
DBMS	the software that creates, maintains and controls a database	1
deadlock	two transactions each waiting for a lock the other holds	22
decomposition	splitting a relation into smaller ones	18, 19
degree (of a relation)	the number of attributes	7
degree (of a relationship)	how many entity types take part	5
denormalisation	storing derived or repeated data on purpose, for speed	19
dependency preservation	every dependency checkable within one table	19
derived attribute	one computed from others, normally not stored	5
determinant	the left side of a functional dependency	17
difference ($-$)	tuples in R and not in S	12
dirty read	reading another transaction's uncommitted change	22
domain	the set of values an attribute may take	7
domain constraint	every value comes from its attribute's domain	8
durability	a committed transaction survives a crash	22
entity / type / set	a thing; a kind of thing; all of them at one moment	5
entity integrity	no part of a primary key is null	8
equijoin	a theta join using only equality	13
EXPLAIN QUERY PLAN	shows the plan the optimiser chose	3, 13
external schema	what one group of users sees: a view	2, 20
fan-out	how many children a B-tree page points to	21

Term	Meaning	Ch.
first normal form	every value atomic	18
foreign key	attributes whose values must match a primary key elsewhere, or be null	7, 8
functional dependency	X → Y: tuples agreeing on X agree on Y	17
generated column	a column computed from others; VIRTUAL or STORED	10
identifying relationship	joins a weak entity to its owner	6
index	a column's values in sorted order, each with its row's rowid	21
inner / outer join	drops / keeps rows with no partner	13
instance	the data in a database at one moment	2
internal schema	how the data is actually stored	2
isolation	concurrent transactions do not see each other's unfinished work	22
leftmost prefix	a composite index serves only conditions on its first columns	21
lossless join	a decomposition that joins back to exactly the original	19
lost update	two read-then-writes, one overwriting the other	1, 22
mapping	translating between adjacent levels, or an ER diagram to tables	2, 9
migration	a numbered script moving a schema from one version to the next	11
minimal cover	the smallest equivalent set of dependencies	17
multi-valued attribute	one with several values per entity; becomes a table	5, 9
natural join	an equijoin on every shared attribute name	13
NoSQL	databases outside the relational model	23
null	the absence of a value; neither zero nor empty	7, 14
optimiser	chooses the cheapest plan for a statement	3
page	the fixed-size unit of storage: 4,096 bytes here	4
parameterised query	values bound separately and never parsed as SQL	23
partial / total participation	an entity need not / must take part	6
partial dependency	on part of a candidate key: violates 2NF	18
partial key	distinguishes weak entities of one owner	6
primary key	the candidate key chosen to identify rows	5, 7
projection (π)	chooses columns and removes duplicates	12
recursive relationship	an entity type related to itself	5
referential action	what happens to children when a parent changes	8
referential integrity	a reference points at something that exists	8
relation	a set of tuples over the same attributes	7
relation schema	a relation's name and attributes, key underlined	4, 7
relational algebra	operators that take and return relations	12

Term	Meaning	Ch.
role name	names an end of a relationship, essential when recursive	5
rollback journal	original pages saved until commit, replayed after a crash	22
rowid	the hidden 64-bit key of every ordinary SQLite table	10
savepoint	a point inside a transaction to roll back to	22
schema	the description of a database; changes rarely	2
second normal form	1NF with no partial dependency	18
selection (σ)	chooses rows	12
self-join	a table joined to itself under two aliases	13
semi-join	the rows of R with at least one partner in S	15
serialisable	the result is as if transactions ran one after another	22
SQL injection	input pasted into SQL and run as SQL	23
STAT4	histograms that let the optimiser estimate a range; not in Python's SQLite	21
storage class	what a value actually is: null, integer, real, text, blob	10
STRICT	a table whose declared types are enforced	10
sub-query	a query inside another	15
superkey	attributes no two tuples agree on	5, 7
surrogate key	an invented identifier with no meaning in the world	5
third normal form	no transitive dependency on a non-prime attribute	18
three-valued logic	true, false and UNKNOWN	14
transaction	a group of changes treated as one	22
transitive dependency	$K \to X \to Y$, with X not a key: violates 3NF	18
trigger	SQL run automatically on an insert, update or delete	19, 20
tuple	one row: a value from each attribute's domain	7
type affinity	a declared type is a preference; unconvertible values are kept	10
union-compatible	same number of attributes, compatible domains, same order	12
UNKNOWN	the truth value of any comparison with a null	14
view	a named query, stored as text and expanded when used	20
WAL	write-ahead log: readers never block the writer	22
weak entity type	one with no key of its own	6
window function	a value per row from its partition, without collapsing rows	16

Chapter E

Resources

E.1 The Four Books HEC Names, and Which Editions to Use

HEC's 2017 and 2023 outlines list the same four reference books, in the same words, with an edition for each and **no publication year for any of them**. An edition number without a year cannot be checked against a library catalogue or a bookshop without first finding out when it appeared — and it does not say whether a newer one has replaced it. The 2023 booklet adds "(or use any other standard and latest books)", which licenses exactly what this appendix does: name each book at the edition to use, with its year.

Book, as HEC lists it	HEC names	Use
Connolly and Begg, *Database Systems: A Practical Approach to Design, Implementation, and Management*	6th	6th edition, Pearson, 2015 — still the latest
Garcia-Molina, Ullman and Widom, *Database Systems: The Complete Book*	2nd	2nd edition, Pearson Prentice Hall, 2009 — still the latest
Silberschatz, Korth and Sudarshan, *Database System Concepts*	6th	**7th edition**, McGraw-Hill, 2019 — HEC names a superseded edition
Ramakrishnan and Gehrke, *Database Management Systems*	3rd	3rd edition, McGraw-Hill, 2003 — still the latest, and the oldest of the four

Which to read for what

- **Silberschatz** is the one to own. Its chapters map most closely onto this course's order — relational model, SQL, ER design, normalisation, indexing, transactions — and the 7th edition covers the NoSQL material of Chapter 23.
- **Connolly and Begg** is the most patient with design: read it beside Part II if Chapter 5 and Chapter 6 felt fast.
- **Ramakrishnan and Gehrke** is the strongest on what is inside a DBMS — buffer management, B-trees, query evaluation — and is the book for Chapter 21 and Chapter 22 if you want the mechanism behind the measurement.
- **Garcia-Molina, Ullman and Widom** is the most mathematical; its treatment of functional dependencies and decomposition is the one to read for Chapter 17 to Chapter 19.

None of the four is about SQLite. Where this handout describes SQLite's behaviour — affinity, PRAGMA `foreign_keys`, file locking — the textbooks describe the standard, and SQLite's own documentation (below) is the authority.

E.2 The Tool's Own Documentation

Every SQLite behaviour this handout measures is documented by its authors, and the documentation is the place to settle any disagreement with a textbook. All are at `sqlite.org`; the page names are given so they can be found from the site's search.

Page	Settles
Datatypes In SQLite	type affinity and storage classes — Chapter 10's `'hello'` in an `INTEGER` column
STRICT Tables	what STRICT enforces, and since which version
SQLite Foreign Key Support	why `foreign_keys` is off by default, and that it is per connection — Chapter 8
The SQLite Query Optimizer Overview	automatic indexes, push-down, and the `+column` trick — Chapter 13 to Chapter 21
EXPLAIN QUERY PLAN	how to read SCAN, SEARCH and TEMP B-TREE
ANALYZE	`sqlite_stat1`, and STAT4 when it is compiled in
File Locking And Concurrency In SQLite Version 3	SHARED, RESERVED, EXCLUSIVE — Chapter 22
Atomic Commit In SQLite	the rollback journal and recovery after a crash
Write-Ahead Logging	WAL mode and what it changes for readers
ALTER TABLE	the four forms and the twelve-step procedure — Chapter 11
JSON Functions And Operators	`json_object`, `json_each` — Chapter 23

For the Python side, the `sqlite3` module's documentation at `docs.python.org` explains `isolation_level` — the setting Chapter 22' harness changes so that every BEGIN is one you wrote.

E.3 The Original Papers

Three papers founded the subject. All three are short, and the first is still the clearest statement of why a database should be relational.

- E. F. Codd, "A Relational Model of Data for Large Shared Data Banks", *Communications of the ACM* 13(6), 1970 — Chapter 7 and Chapter 12.
- P. P. Chen, "The Entity-Relationship Model — Toward a Unified View of Data", *ACM Transactions on Database Systems* 1(1), 1976 — Chapter 5.
- ANSI/X3/SPARC Study Group on Database Management Systems, *Interim Report*, 1975 — the three-level architecture of Chapter 2.

E.4 Law

- *Prevention of Electronic Crimes Act 2016*, Pakistan, sections 3 and 4 — unauthorised access to an information system and to data. The scope statement of Appendix A, section A.6, and of Chapter 23 rests on it.

E.5 This Handout's Own Code

Every measured figure was produced by a file in `code/`, on the machine described in Appendix A, section A.5:

File	Produces
`schema/v1_create.sql`	the university schema, from Chapter 9
`schema/v2_constraints.sql`	the migration of Chapter 11
`schema/seed.py`	reproducible data at any scale: 4,000 students, 60,037 enrolments by default
`m04_firstdatabase.py`	Chapter 4: time to a first row, bytes per page
`m08_constraints.py`	Chapter 8: foreign keys off by default
`m10_ddl.py`	Chapter 10: affinity against STRICT
`m13_joins.py`	Chapter 13: NATURAL JOIN, missing sections, automatic indexes
`m15_subqueries.py`	Chapter 15: anti-joins and correlation
`m16_grouping.py`	Chapter 16: nulls, sorts, windows
`m18_normalforms.py`	Chapter 18: the wide table against BCNF
`m19_decomposition.py`	Chapter 19: lossy joins and a priced summary table
`m20_views.py`	Chapter 20: what views and triggers cost
`m21_indexes.py`	Chapter 21: the crossover
`m22_transactions.py`	Chapter 22: a lost update, a deadlock, the price of a commit
`run_all.py`	runs every harness and checks each result's *shape* still holds
`sync_labs.py`	checks that every file and command a laboratory names exists

Chapter F

The Boundary With Other Courses

This course sits early in the degree — second semester — and three later courses depend on it. It therefore assumes little and promises a great deal, and both are recorded here: what is assumed, what is delivered, and to whom, so that a lecturer can see what may be taken for granted on each side and a student can see why a familiar topic reappears.

F.1 The Edition History

Edition	Status	Degrees	Semester	Content published?
2017	Computing Core, 3+1	BSCS, BSSE, BSIT	IV, IV, **V**	yes
2023	Computing Core, 4 (3-3)	all ten computing BS, and ADCS	**2**	yes — the 2017 text
2025	Major Compulsory, 3+1	BSCS and ADCS only — **no BSIT**	III	**no**

The department teaches CS-103 at **BSIT, second semester, 4 credit hours (3+1)**, which matches HEC 2023. HEC 2017 placed it at BSIT semester V. The 2023 course content is the 2017 text word for word; 2025 removed it. The Course Specification at the front records all six defects found in these outlines.

One booklet, two prerequisites

HEC 2017's course description page gives **"Prerequisites: None"**. The same booklet's scheme-of-studies tables, on every page that lists the course, give **"Data Structures & Algorithms"**. HEC 2023 gives "None".

At second semester Data Structures has not been taken, so this handout **assumes no data-structures course**. The two chapters that would most naturally lean on one are written to stand alone: Chapter 21 explains a B-tree by its fan-out and depth, with Appendix B's logarithm, and never by an algorithm; Chapter 22 explains locks without operating-systems vocabulary. Recorded here so that a reviewer comparing the two documents finds the answer already written.

F.2 Programming Fundamentals — the Real Prerequisite

The rule

Programming Fundamentals owns the program. This course owns what the program keeps.

A student arriving here should be able to write a loop, read a file, and call a function. Nothing more is used: Python appears in the harnesses, which students run and read rather than write, and in two laboratories — Chapter 1's file-based system and Chapter 23's injection demonstration — where the programs are a dozen lines.

F.3 Data Structures — Not Assumed

Topic	Owned by	Here
B-trees: insertion, splitting, balance	Data Structures	*not taught*; Chapter 21 needs only fan-out and depth
hashing	Data Structures	mentioned, not used
sorting algorithms	Data Structures	Chapter 16 measures what a sort *costs* — TEMP B-TREE — not how it works
logarithms	Appendix B	taught here, as "how many halvings"

The useful repetition: a student who takes Data Structures later will meet the B-tree's insertion algorithm, and will already know why it matters — Chapter 21 measured a lookup costing 0.057 ms in a million rows, and a scan 199.90 ms.

F.4 E-Commerce Application Development — Downstream

The rule, as that course states it

"Database Systems owns the model. This course owns the consequences of somebody else's model."

E-Commerce Application Development's own Appendix F says that, and lists what it assumes: *"SQL selects and joins, primary and foreign keys, normalisation, what an index is."* This course must deliver all four, and does.

E-Commerce assumes	Delivered in	And measured
SQL selects and joins	Chapter 14, Chapter 13, Chapter 15, Chapter 16	NOT IN returning 0 where NOT EXISTS returned 3
primary and foreign keys	Chapter 7, Chapter 8	foreign keys off by default; a `TEXT PRIMARY KEY` storing null
normalisation	Chapter 17 to Chapter 19	a wide table 162× slower on a lookup; 143,768 spurious tuples
what an index is	Chapter 21	the crossover — which E-Commerce then finds again, in a product catalogue

The useful repetition, from the downstream side: E-Commerce tells its students that "every Database Systems student has been told that indexes make queries faster", and makes that a measurement with a crossover in it. In this course they have already measured it — so the E-Commerce chapter becomes a second instance of a known result, on a schema they did not design, rather than a surprise.

F.5 Information Security — Later

The rule

Information Security owns the principles. This course owns one database, and what can be done to it from inside.

Chapter 23 shows SQL injection against the student's own file, under the scope statement of Appendix A, and stops there. Threat modelling, authentication, cryptography and the law are Information Security's; this course shows the one defence that belongs to the person writing the query — the parameter — and the access control a server adds that SQLite lacks.

F.6 Software Engineering — Later

- **Assumed there, delivered here:** that a schema is designed before it is built (Chapter 5 to Chapter 9), and changed by numbered, reviewed migrations (Chapter 11).
- **Taught properly there, used informally here:** version control. Chapter 11's first rule — never edit a migration that has run anywhere — is a version-control rule as much as a database one, and Software Engineering is where the tooling behind it is taught.

Chapter G

The Sixteen-Week Schedule

This appendix is the **source of record** for the teaching schedule. The one-page `DBS_Fall2026_Lecture_Schedule.pdf` distributed to students is generated from it, and the slide pipeline parses the table below. Change this first; regenerate the one-page version afterwards.

Twenty-three chapters are taught across sixteen weeks, so seven weeks carry two chapters. The pairings are not arbitrary: in every case the two are either one topic the printed book splits for length — the two halves of the ER model, the two halves of decomposition — or a pair in which the second is unusable without the first.

Two weeks are deliberately left alone. **Chapter 18, Normal Forms, has a week to itself**, although it would pair naturally with Chapter 17. Its laboratory breaks the running database and rebuilds it, and that does not fit in half a week alongside anything else. **Chapter 21, Indexes**, likewise: the measurement at a million rows takes most of a laboratory session to run and the rest of it to argue about.

G.1 The Schedule

Wk	Lecture topic	Chapters	Laboratory	Assessment
1	Why not files; the three-level architecture	1, 2	Break a file-based system on purpose; find the three levels in SQLite	—
2	Who does what; your first database	3, 4	Install nothing; create a database and measure what it cost	—
3	The entity-relationship model	5	Entities and attributes for the university, on paper	—
4	Cardinality, participation, weak entities	6	The university ER diagram, finished and argued over	—
5	The relational model; keys and constraints	7, 8	Orphan rows in a correctly declared schema	**Quiz 1 (Ch 1–6)**
6	Mapping a diagram to tables	9	Your diagram becomes eight tables, by rule	**Assignment 1 set**
7	Creating and changing a schema	10, 11	`CREATE TABLE`; then change one that is already full	—
8	Relational algebra	12	Midterm, then algebra by hand on small relations	**Midterm (Ch 1–11)**
9	Joins; selecting rows and NULL	13, 14	Lose 73 sections and 19,992 rows to joins and NULL, then find them all	**Assignment 1 due**

Wk	Lecture topic	Chapters	Laboratory	Assessment
10	Sub-queries; grouping and aggregation	15, 16	One question seven ways; an average moved by a third	**Project proposal**
11	Functional dependencies	17	The wide report table: its dependencies, its closures and its key	—
12	Normal forms: 1NF to BCNF	18	Break the running database, then normalise it and measure both	**Quiz 2 (Ch 12–17)**
13	Decomposition; views and triggers	19, 20	A lossy split that invents facts; a summary table caught lying	**Assignment 2 set**
14	Indexes, and where they stop helping	21	One million rows, with and without an index	—
15	Transactions, concurrency and recovery	22	Two writers, one database, and a lost update	**Assignment 2 due**
16	Security, NoSQL and the project	23	Project presentations	**Project due; presentations**

G.2 How the Assessment Adds Up

HEC publishes the assessment instruments for this course — sessional examination, home assignments, quizzes, project, presentations, final examination — without weights. The weights below are the department's, and are stated here so that a student can see the whole shape of the semester on one page.

Instrument	Weight	What it examines
Two quizzes	10%	Weeks 5 and 12. Short, closed-book, on the chapters named in the schedule
Two assignments	10%	Set in weeks 6 and 13. Both are done on your own database and submitted as a change to it plus a short write-up
Laboratory	**15%**	Assessed weekly from Week 2. This course carries a **credited laboratory** — one of its four credit hours — and the marks follow the credit
Midterm examination	20%	Week 8, Chapters 1–11
Semester project	20%	The database itself, built across the whole semester (Chapter 23), submitted in week 16 with its presentation
Final examination	25%	All chapters, with emphasis after the midterm

The laboratory is a quarter of the course, not a formality

This is the only course in the series with a *credited* laboratory: HEC specifies 4 credit hours as 3+1 in 2017 and 2025, and as 4 (3–3) in 2023. One of those four hours is the lab, and 15% of the marks follow it.

That is also why the laboratories in this handout are longer than those in its sibling courses. They are not homework to be fitted around the lectures; they are a timetabled hour with a demonstrator in the room, and every one of them adds something to the database you will submit in Week 16.

Chapter 23 states what is required and how it is marked. Read it in Week 3, not in Week 15.